Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

STUDY NOTES · BASIC TO ADVANCED

Arduino
Sense, Decide, Act: Build Real Electronics

Complete notes for the Learn Being Forward Arduino syllabus: electronics, boards, programming, sensors, displays, motors, communication, IoT and product design, explained with circuit diagrams and tested sketches, followed by basic, intermediate and advanced questions and programs with fully worked solutions.

27	3	46	675
modules	levels	minimum days	questions and programs

How to use these notes

These notes follow the **Learn Being Forward Arduino syllabus** exactly: the same 27 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **9.2.1**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 9	10 days	Understand the Arduino board and IDE, apply basic electronics, write sketches in Arduino C/C++, use digital and analog I/O, read buttons and sensors and use the Serial Monitor
2. Intermediate	10 to 19	18 days	Use timing without delay, interrupts, PWM and motors, I2C, SPI and UART devices, displays, libraries, data storage, power management and structured multi-file code
3. Advanced	20 to 27	18 days	Build IoT projects with ESP32 and Wi-Fi, use Bluetooth and wireless links, program at register level, use RTOS, design custom PCBs, debug and test firmware and complete a capstone product
Total	1 to 27	46 days	About 8 weeks at 6 study days per week

1 · Understand
Read the concept and study the circuit diagram until you can explain every connection.

2 · Build
Wire the circuit on a breadboard or in a simulator such as Wokwi, then type the sketch yourself.

3 · Practise
Solve the basic, intermediate and advanced questions and programs before looking at the solutions.

4 · Check and extend
Compare with the solutions, change one thing at a time, and log every problem you solved.

Every topic follows the same pattern: the concept in plain words, circuit diagrams, annotated sketches and **shortcut tricks** and safety notes. Each module then gives about 25 practice questions in three levels, **basic**, **intermediate** and **advanced**: concept and calculation questions plus programming tasks. Every sketch in this book was compiled and run on a simulated Arduino core, and the Serial Monitor output shown beside it is the real output of that run; all calculations were checked by program.

HOW TO READ THE BOXES

Gold-bordered boxes hold **shortcut tricks**, worked examples and notes, red boxes warn about common mistakes and hardware dangers, and navy boxes hold the key rule to remember. Grey panels under a sketch show its Serial Monitor output or a simulated run.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to Arduino	Beginner	1
02	Electronics essentials for Arduino	Beginner	1
03	Arduino boards and hardware	Beginner	1
04	Setup and development environment	Beginner	1

Module	Topic	Level	Min days
05	Arduino programming basics	Beginner	2
06	Digital output	Beginner	1
07	Digital input	Beginner	1
08	Analog input and the ADC	Beginner	1
09	PWM and analog output	Beginner	1
10	Serial communication and debugging	Intermediate	1
11	Functions, arrays, strings and structures	Intermediate	2
12	Timing and non-blocking code	Intermediate	1
13	Sensors	Intermediate	3
14	Displays	Intermediate	2
15	Sound and user-input devices	Intermediate	2
16	Motors and motion control	Intermediate	2
17	Libraries	Intermediate	1
18	Communication protocols in depth	Intermediate	2
19	Data storage and timekeeping	Intermediate	2
20	Interrupts and timers	Advanced	2
21	Wireless communication	Advanced	2
22	IoT with Wi-Fi boards	Advanced	3
23	Power management and battery projects	Advanced	1
24	Advanced Arduino programming	Advanced	2
25	Beyond the Arduino IDE	Advanced	2
26	From prototype to product: PCB design and enclosures	Advanced	2
27	Projects and careers	Advanced	4

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 9

Module 01 · Introduction to Arduino 9

1.1 What is Arduino? 9
1.2 History of Arduino 9
1.3 Why do we need Arduino? 9
1.4 Where is Arduino used? 9
1.5 Microcontroller vs microprocessor 9
1.6 What you can build 9
1.7 Practice questions 9
1.8 Solutions 9

Module 02 · Electronics essentials for Arduino 17

2.1 Electrical basics 17
2.2 Components 17
2.3 Prototyping tools 17
2.4 Reading schematics and pinout diagrams 17
2.5 Safety 17
2.6 Practice questions 17
2.7 Solutions 17

Module 03 · Arduino boards and hardware 27

3.1 Arduino Uno 27
3.2 Other Arduino boards 27
3.3 Compatible boards 27
3.4 Shields and modules 27
3.5 Powering the board 27
3.6 Practice questions 27
3.7 Solutions 27

Module 04 · Setup and development environment 36

4.1 Arduino IDE 2 36
4.2 Arduino Cloud Editor 36
4.3 Simulators 36
4.4 First sketch 36
4.5 Common upload problems and fixes 36
4.6 Practice questions 36
4.7 Solutions 36

Module 05 · Arduino programming basics 47

5.1 Sketch structure 47
5.2 Variables and data types 47
5.3 Operators 47
5.4 Control structures 47
5.5 Built-in functions 47
5.6 Practice questions 47
5.7 Solutions 47

Module 06 · Digital output 61
6.1 Digital output basics 61
6.2 LED projects 61
6.3 Driving bigger loads 61
6.4 Shift registers 61
6.5 Practice questions 61
6.6 Solutions 61

Module 07 · Digital input 74
7.1 Digital input basics 74
7.2 Buttons 74
7.3 Debouncing 74
7.4 State change detection (edge detection) 78
7.5 Other digital input devices 74
7.6 Practice questions 74
7.7 Solutions 74

Module 08 · Analog input and the ADC 88
8.1 Analog-to-digital conversion 88
8.2 Converting readings 88
8.3 Analog sensors 88
8.4 Noise and smoothing 88
8.5 Practice questions 88
8.6 Solutions 88

Module 09 · PWM and analog output 100
9.1 Pulse-width modulation 100
9.2 PWM projects 100
9.3 True analog output 100
9.4 Practice questions 100
9.5 Solutions 100

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 10 TO 19

Module 10 · Serial communication and debugging 112
10.1 Serial basics 112
10.2 Reading serial input 112
10.3 Serial Plotter 112
10.4 Debugging techniques 112
10.5 Practice questions 112
10.6 Solutions 112

Module 11 · Functions, arrays, strings and structures 124
11.1 Functions 124
11.2 Arrays 124
11.3 Strings 124
11.4 Structures and enums 124
11.5 Bit manipulation 124
11.6 Practice questions 124
11.7 Solutions 124

Module 12 · Timing and non-blocking code 137

12.1 Problems with delay() 137
12.2 millis()-based timing 137
12.3 micros() for short intervals 137
12.4 Simple task scheduling 137
12.5 State machines (introduction) 137
12.6 Practice questions 137
12.7 Solutions 137

Module 13 · Sensors **150**

13.1 Distance sensors 150
13.2 Environment sensors 150
13.3 Motion and position sensors 150
13.4 Other sensors 150
13.5 Calibration and accuracy 150
13.6 Practice questions 150
13.7 Solutions 150

Module 14 · Displays **163**

14.1 7-segment displays 163
14.2 Character LCDs 163
14.3 OLED displays 163
14.4 LED matrices and addressable LEDs 163
14.5 TFT colour displays and touchscreens (overview) 163
14.6 Practice questions 163
14.7 Solutions 163

Module 15 · Sound and user-input devices **175**

15.1 Sound output 175
15.2 Input devices 175
15.3 Simple user interfaces 175
15.4 Practice questions 175
15.5 Solutions 175

Module 16 · Motors and motion control **187**

16.1 DC motors 187
16.2 Servo motors 187
16.3 Stepper motors 187
16.4 Power for motors 187
16.5 Motor projects 187
16.6 Practice questions 187
16.7 Solutions 187

Module 17 · Libraries **200**

17.1 Using libraries 200
17.2 How libraries work 200
17.3 Writing your own library 200
17.4 Library conflicts and version issues 200
17.5 Practice questions 200
17.6 Solutions 200

Module 18 · Communication protocols in depth **212**

18.1 UART 212
18.2 I2C 212

18.3 SPI 212
18.4 One-Wire protocol 212
18.5 Choosing a protocol 212
18.6 Practice questions 212
18.7 Solutions 212

Module 19 · Data storage and timekeeping 224

19.1 EEPROM 224
19.2 SD cards 224
19.3 Real-time clocks 224
19.4 Data logger project 224
19.5 Practice questions 224
19.6 Solutions 224

LEVEL 3 · ADVANCED LEVEL · MODULES 20 TO 27

Module 20 · Interrupts and timers 237

20.1 External interrupts 237
20.2 Pin-change interrupts (overview) 237
20.3 Hardware timers 237
20.4 Applications 237
20.5 Watchdog timer 237
20.6 Practice questions 237
20.7 Solutions 237

Module 21 · Wireless communication 250

21.1 Bluetooth 250
21.2 Radio 250
21.3 Infrared communication 250
21.4 Choosing a wireless technology 250
21.5 Practice questions 250
21.6 Solutions 250

Module 22 · IoT with Wi-Fi boards 260

22.1 Wi-Fi boards 260
22.2 Wi-Fi basics 260
22.3 Web servers 260
22.4 Calling web APIs 260
22.5 MQTT 260
22.6 IoT platforms 260
22.7 Over-the-air (OTA) updates 260
22.8 IoT security basics 260
22.9 Practice questions 260
22.10 Solutions 260

Module 23 · Power management and battery projects 275

23.1 Power requirements 275
23.2 Batteries 275
23.3 Voltage regulation 275
23.4 Low-power techniques 275
23.5 Solar-powered projects 275
23.6 Practice questions 275

23.7 Solutions 275

Module 24 · Advanced Arduino programming 287

24.1 Object-oriented programming 287
24.2 State machines 287
24.3 Memory optimisation 287
24.4 Speed optimisation 287
24.5 Multitasking 287
24.6 Code quality 287
24.7 Practice questions 287
24.8 Solutions 287

Module 25 · Beyond the Arduino IDE 300

25.1 PlatformIO 300
25.2 Debugging 300
25.3 Other microcontroller families 300
25.4 MicroPython and CircuitPython (overview) 300
25.5 Introduction to bare-metal programming (overview) 300
25.6 Practice questions 300
25.7 Solutions 300

Module 26 · From prototype to product: PCB design and enclosures 310

26.1 Moving off the breadboard 310
26.2 PCB design 310
26.3 Using a bare microcontroller 310
26.4 Enclosures 310
26.5 Reliability and testing 310
26.6 Practice questions 310
26.7 Solutions 310

Module 27 · Projects and careers 319

27.1 Capstone projects 319
27.2 Documenting projects 319
27.3 Competitions and hackathons 319
27.4 Careers 319
27.5 Next steps 319
27.6 Final test 319
27.7 Solutions 319

LEVEL 1

Beginner level

Understand the Arduino board and IDE, apply basic electronics, write sketches in Arduino C/C++, use digital and analog I/O, read buttons and sensors and use the Serial Monitor

9 MODULES · 10 DAYS MINIMUM

Module 01 · Introduction to Arduino	1 day
Module 02 · Electronics essentials for Arduino	1 day
Module 03 · Arduino boards and hardware	1 day
Module 04 · Setup and development environment	1 day
Module 05 · Arduino programming basics	2 days
Module 06 · Digital output	1 day
Module 07 · Digital input	1 day
Module 08 · Analog input and the ADC	1 day
Module 09 · PWM and analog output	1 day

MODULE 01 · BEGINNER

Introduction to Arduino

Minimum duration **1 day** | Level **Beginner**

This module explains what Arduino is, where it came from, why it became the most popular platform for learning electronics and building prototypes, where it is used, how a microcontroller differs from a microprocessor (and Arduino from Raspberry Pi), and what you will be able to build by the end of this course, followed by 25 solved questions.

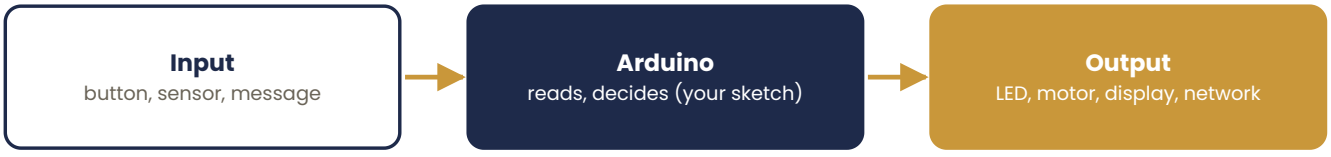
IN THIS MODULE

1.1	What is Arduino?	1.2	History of Arduino
1.3	Why do we need Arduino?	1.4	Where is Arduino used?
1.5	Microcontroller vs microprocessor	1.6	What you can build
1.7	Practice questions	1.8	Solutions

1.1 What is Arduino?

1.1.1 Definition

Arduino is an open-source electronics platform made of easy-to-use hardware boards and software. A board reads **inputs** (a button press, light on a sensor, a message) and turns them into **outputs** (lighting an LED, turning a motor, sending data online). You tell the board what to do by writing a program, called a **sketch**, and uploading it from your computer.



EVERY ARDUINO PROJECT: SENSE → DECIDE → ACT

1.1.2 Microcontroller board + IDE + libraries + community

Part	What it is
Board	a circuit board built around a microcontroller, with pins for connecting components
IDE	the Arduino software where you write, compile and upload sketches
Libraries	ready-made code for sensors, displays, motors and protocols
Community	forums, tutorials and open designs shared by millions of users

1.1.3 Open-source hardware and clones

Arduino board designs are published openly, so other manufacturers may legally make compatible boards ("clones"). Clones are cheaper and work with the same IDE, though some use a different USB-to-serial chip that needs its own driver.

1.2 History of Arduino

1.2.1 Interaction Design Institute Ivrea, Italy (2005)

Arduino began in 2005 at the Interaction Design Institute Ivrea in Italy, as a cheap, simple tool for design students who were not engineers.

1.2.2 Founders

The project was led by Massimo Banzi with David Cuartielles, Tom Igoe, Gianluca Martino and David Mellis.

1.2.3 Arduino Uno and the maker movement

The Arduino Uno (2010) became the standard beginner board. Together with cheap sensors and 3D printers, it powered the "maker movement" of hobbyists, students and inventors building their own devices.

1.2.4 Newer boards

Recent additions include the Uno R4 (Minima and WiFi), the Nano ESP32, the professional Portenta family and the Arduino Cloud for connected projects.

1.3 Why do we need Arduino?

Reason	Explanation
Low cost and beginner-friendly	boards are inexpensive and need no special programmer
Simple programming	a simplified C/C++ with functions like digitalWrite() and a huge library ecosystem
Fast prototyping	ideas can be tested in hours instead of weeks
Large community	tutorials, shields and answers for almost every problem

1.4 Where is Arduino used?

Field	Example
Education and STEM labs	teaching programming and electronics
Robotics	line followers, robotic arms, obstacle-avoiding cars

Field	Example
Home automation and IoT	smart lights, door sensors, Wi-Fi weather stations
Wearables and art	LED costumes, interactive installations
Industrial prototypes and data loggers	test rigs, machine monitoring
Agriculture and environment	soil moisture irrigation, air quality monitors

1.5 Microcontroller vs microprocessor

	Microcontroller (Arduino)	Microprocessor (Raspberry Pi)
What it is	CPU + memory + I/O on one chip	CPU only; memory and I/O are separate chips
Operating system	none: runs one sketch directly	full OS (Linux)
Start-up	instant	tens of seconds
Timing	precise, real-time	less predictable (OS scheduling)
Power	milliwatts; runs on batteries for months	watts
Best for	reading sensors, controlling hardware	cameras, video, web servers, AI

1.5.1 Arduino vs Raspberry Pi

Use Arduino when the job is controlling hardware with reliable timing and low power; use Raspberry Pi when you need a full computer. Many projects use both: the Pi for the "brain", the Arduino for the "muscles".

1.6 What you can build

Level	Projects in this course
Beginner	blinking LEDs, traffic lights, button-controlled lights, light and temperature meters
Intermediate	digital thermometer with LCD, servo-controlled arm, ultrasonic distance meter, data logger
Advanced	Wi-Fi weather station, Bluetooth-controlled car, battery-powered sensor node, custom PCB product

1.7 Practice questions

1.7.1 Basic level

Q1 A program written for Arduino is called a

(a) script (b) drawing (c) sketch (d) module

Q2 Arduino was created in

(a) 2005 (b) 1995 (c) 2015 (d) 2010

Q3 Arduino was developed at the Interaction Design Institute in

(a) Boston, USA (b) Munich, Germany
(c) Bengaluru, India (d) Ivrea, Italy

Q4 Which of these is an output device?

(a) push button (b) LED
(c) temperature sensor (d) potentiometer

Q5 Which of these is an input device?

(a) buzzer (b) light sensor (LDR)
(c) motor (d) LCD display

Q6 What does IDE stand for?

(a) Internal Device Engine (b) Integrated Digital Electronics
(c) Input Data Editor (d) Integrated Development Environment

Q7 Arduino hardware designs are

(a) open source (b) secret and patented
(c) available only to universities (d) licensed only to one manufacturer

Q8 Which person is one of the founders of Arduino?

(a) Linus Torvalds (b) Eben Upton (c) Massimo Banzi (d) Steve Wozniak

1.7.2 Intermediate level

Q9 A microcontroller contains

(a) only a CPU (b) only memory
(c) a full operating system (d) CPU, memory and I/O on one chip

Q10 Which board runs a full operating system such as Linux?

(a) Arduino Uno (b) Raspberry Pi (c) Arduino Nano (d) Arduino Mega

Q11 Which is a major advantage of Arduino over Raspberry Pi for a battery-powered sensor?

(a) much lower power consumption (b) faster video processing
(c) built-in desktop interface (d) more RAM

Q12 Which task is better suited to a Raspberry Pi than to an Arduino Uno?

(a) blinking an LED (b) reading a push button

(c) streaming video from a camera (d) controlling a servo

Q13 Compatible boards made by other companies are commonly called

(a) clones (b) shields (c) modules (d) sketches

Q14 Which Arduino board became the standard beginner board around 2010?

(a) Portenta (b) Uno (c) Mega (d) Leonardo

Q15 Ready-made code for sensors and displays is supplied as

(a) shields (b) firmware updates
(c) drivers only (d) libraries

Q16 Which statement about an Arduino starting up is correct?

(a) It boots an operating system for 30 seconds.
(b) It needs a keyboard to start.
(c) It starts running the sketch almost instantly after power-on.
(d) It must be connected to the internet.

1.7.3 Advanced level

Q17 A soil-moisture sensor that switches on a pump is an example of Arduino use in

(a) wearables (b) art installations
(c) agriculture (d) video editing

Q18 Which describes the sense–decide–act cycle?

(a) write code, delete code, rewrite code
(b) power on, power off, reset
(c) compile, crash, restart
(d) read inputs, process them in the sketch, drive outputs

Q19 Why can a timing-critical task (such as generating precise pulses) be easier on Arduino than on a Pi?

(a) Arduino has a faster processor.
(b) Arduino runs only the sketch, without an operating system interrupting it.
(c) Arduino has more memory.
(d) Arduino uses Python.

Q20 Which newer board combines the Nano form factor with Wi-Fi and Bluetooth?

(a) Nano ESP32 (b) Uno R3 (c) Mega 2560 (d) Leonardo

Q21 Which of these best describes the "maker movement"?

(a) factories mass-producing phones
(b) people building their own devices with cheap tools like Arduino and 3D printers
(c) a programming language
(d) a type of microcontroller

Q22 A project needs to read a temperature sensor, show the value on a display and run on batteries for months. The best choice is

(a) a microcontroller board such as Arduino
(b) a desktop PC
(c) a Raspberry Pi running a web browser
(d) a gaming console

Q23 What is the Arduino Cloud used for?

(a) storing 3D printer files only
(b) designing PCBs only
(c) connecting, monitoring and controlling boards over the internet
(d) replacing the microcontroller

Q24 In a smart door alarm, the magnetic door switch, the Arduino and the buzzer act respectively as

(a) output, input, controller
(b) controller, output, input
(c) input, output, controller
(d) input, controller, output

Q25 Why do some clone boards need an extra driver on the computer?

(a) They use a different programming language.
(b) They have no microcontroller.
(c) They work only on Linux.
(d) They use a different USB-to-serial chip (such as CH340).

1.8 Solutions

Q1 **Answer: (c) sketch**

Arduino programs are called **sketches**.

Q2 **Answer: (a) 2005**

It began at Ivrea, Italy, in **2005**.

Q3 **Answer: (d) Ivrea, Italy**

The **Interaction Design Institute Ivrea, Italy**.

Q4 **Answer: (b) LED**

An **LED** is driven by the board; the others provide input.

Q5 **Answer: (b) light sensor (LDR)**

A **light sensor** sends information to the board.

Q6 **Answer: (d) Integrated Development Environment**

Integrated Development Environment: where you write and upload code.

Q7 **Answer: (a) open source**

Designs are **open source**, which is why compatible clones exist.

Q8 **Answer: (c) Massimo Banzi**

Massimo Banzi led the project. Eben Upton founded Raspberry Pi.

Q9 **Answer: (d) CPU, memory and I/O on one chip**

Microcontroller = **CPU + memory + I/O** on a single chip.

Q10 **Answer: (b) Raspberry Pi**

Raspberry Pi is a single-board computer with an OS.

Q11 **Answer: (a) much lower power consumption**

Microcontrollers draw **milliwatts**, so batteries last far longer.

Q12 **Answer: (c) streaming video from a camera**

Video needs a full computer: **Raspberry Pi**.

Q13 **Answer: (a) clones**

They are **clones**; shields are add-on boards.

Q14 **Answer: (b) Uno**

The **Uno**.

Q15 **Answer: (d) libraries**

Libraries provide tested code you can reuse.

Q16 **Answer: (c) It starts running the sketch almost instantly after power-on.**

With no OS, the sketch runs **almost instantly**.

Q17 **Answer: (c) agriculture**

Automatic irrigation: **agriculture**.

Q18 **Answer: (d) read inputs, process them in the sketch, drive outputs**

Inputs → decision in code → outputs.

Q19 **Answer: (b) Arduino runs only the sketch, without an operating system interrupting it.**

No OS scheduling, so timing is **predictable**.

Q20 **Answer: (a) Nano ESP32**

The **Nano ESP32**.

Q21 **Answer: (b) people building their own devices with cheap tools like Arduino and 3D printers**

Hobbyists and students making their own devices.

Q22 **Answer: (a) a microcontroller board such as Arduino**

Low power and simple hardware control: **a microcontroller board**.

Q23 **Answer: (c) connecting, monitoring and controlling boards over the internet**

It is a platform for **connected (IoT) projects**.

Q24 **Answer: (d) input, controller, output**

Switch senses (input), Arduino decides (controller), buzzer acts (output).

Q25 **Answer: (d) They use a different USB-to-serial chip (such as CH340).**

Many clones use a **CH340** or similar USB-serial chip with its own driver.

Arduino = open-source boards + IDE + libraries + community. A sketch reads inputs, decides and drives outputs. Microcontrollers (Arduino) suit real-time hardware control at low power; microprocessors (Raspberry Pi) suit full-computer tasks.

MODULE 02 · BEGINNER

Electronics essentials for Arduino

Minimum duration **1 day** | Level **Beginner**

Every Arduino project is an electrical circuit. This module covers voltage, current, resistance and power, Ohm's law, series and parallel circuits, voltage dividers, resistors and colour codes, LEDs and current-limiting resistors, capacitors, diodes, transistors as switches, potentiometers, breadboards and multimeters, reading schematics, and the safety rules that protect you and your board, followed by 25 solved questions with every calculation checked.

IN THIS MODULE

2.1 Electrical basics
2.2 Components
2.3 Prototyping tools
2.4 Reading schematics and pinout diagrams
2.5 Safety
2.6 Practice questions
2.7 Solutions

2.1 Electrical basics

2.1.1 Voltage, current, resistance and power

Quantity	Symbol	Unit	Meaning (water analogy)
Voltage	V	volt (V)	electrical "pressure" pushing charge (water pressure)
Current	I	ampere (A); 1 mA = 0.001 A	rate of flow of charge (water flow)
Resistance	R	ohm (Ω); 1 kΩ = 1000 Ω	opposition to current (a narrow pipe)
Power	P	watt (W)	rate of energy use: P = V × I

2.1.2 Ohm's law

V = I × R. Rearranged: I = V / R and R = V / I. Power: P = V × I = I² × R = V² / R.

WORKED EXAMPLE

Ohm's law

A 5 V supply across 1 kΩ: I = 5 / 1000 = 0.005 A = **5 mA**; P = 5 × 0.005 = 0.025 W = 25 mW.

SHORTCUT TRICK

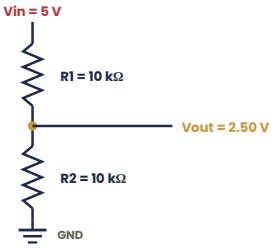
Use volts, milliamps and kilohms together: V = mA × kΩ. 5 V across 1 kΩ is 5 mA; 3.3 V across 330 Ω (0.33 kΩ) is 10 mA.

2.1.3 Series and parallel circuits

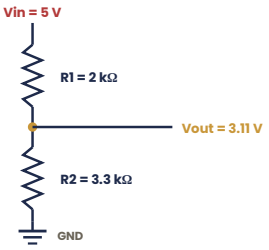
	Series	Parallel
Current	same through every part	splits between branches
Voltage	shared between parts	same across every branch
Total resistance	R = R1 + R2 + …	1/R = 1/R1 + 1/R2 + …; two resistors: R1R2 / (R1 + R2)
Example	220 Ω + 330 Ω = 550 Ω	1 kΩ ‖ 1 kΩ = 500 Ω

2.1.4 Voltage dividers

Two resistors in series divide a voltage. The output at their junction is **Vout = Vin × R2 / (R1 + R2)**. Dividers convert resistance changes (light sensors, thermistors) into voltages an Arduino can read, and can step a 5 V signal down to 3.3 V.



EQUAL RESISTORS HALVE THE VOLTAGE: 2.50 V



STEPPING 5 V DOWN TO ABOUT 3.3 V (2 KΩ AND 3.3 KΩ)

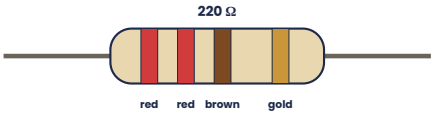
2.2 Components

2.2.1 Resistors and colour codes

A four-band resistor shows two digits, a multiplier (number of zeros) and a tolerance band (gold ±5 %, silver ±10 %).

Colour	Digit	Colour	Digit
black	0	green	5
brown	1	blue	6
red	2	violet	7
orange	3	grey	8

Colour	Digit	Colour	Digit
yellow	4	white	9



220 Ω = RED (2), RED (2), BROWN (×10), GOLD



10 KΩ = BROWN (1), BLACK (0), ORANGE (×1000), GOLD

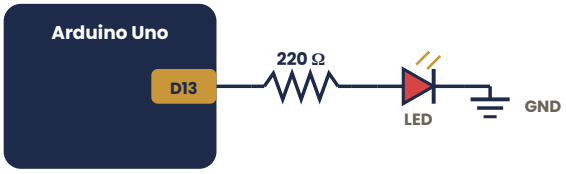
SHORTCUT TRICK

Mnemonic for the colour order 0–9: "**B**etter **B**e **R**ight **O**r **Y**our **G**reat **B**ig **V**enture **G**oes **W**rong" (black, brown, red, orange, yellow, green, blue, violet, grey, white).

2.2.2 LEDs and current-limiting resistors

An LED (light-emitting diode) lights when current flows from its **anode** (longer leg, +) to its **cathode** (shorter leg, flat side, -). It drops a roughly fixed **forward voltage** (about 2 V for red, 3 V for blue/white) and must have a series resistor to limit current.

R = (Vsupply - Vforward) / I. For a red LED (2 V) at 15 mA from 5 V: R = (5 - 2) / 0.015 = 200 Ω → use the next standard value, 220 Ω.



AN LED WITH ITS CURRENT-LIMITING RESISTOR

2.2.3 Capacitors

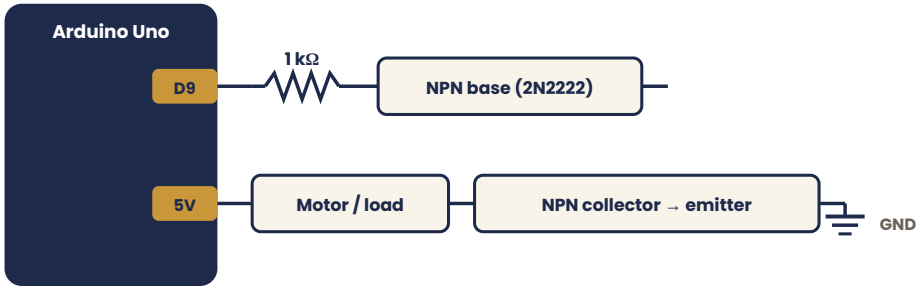
A capacitor stores charge. Small ceramic capacitors (0.1 µF) placed across the power pins of a chip smooth out tiny voltage dips ("decoupling"); larger electrolytic capacitors (10–1000 µF, polarised: note the - stripe) smooth power supplies. An RC circuit has time constant $\tau = R \times C$.

2.2.4 Diodes

A diode lets current flow in one direction only (anode → cathode). Uses: protecting against reversed power, and the **flyback diode** across a motor or relay coil, which absorbs the voltage spike when the coil is switched off.

2.2.5 Transistors as switches

An Arduino pin can supply only about 20 mA. To switch a motor, relay or LED strip, use a transistor as a switch: a small base (or gate) current from the pin controls a large current through the load.



NPN TRANSISTOR SWITCHING A LOAD (FLYBACK DIODE ACROSS THE MOTOR NOT SHOWN)

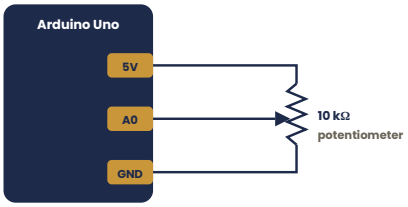
WORKED EXAMPLE

Base resistor

Load current 200 mA, transistor gain $\beta \approx 100$ → base current ≈ 2 mA; use 5–10× more to be sure the transistor saturates. $R = (5 - 0.7) / 0.01 \approx 430\ \Omega$ → 470 Ω. A logic-level MOSFET needs almost no gate current and is better for loads above about 500 mA.

2.2.6 Potentiometers

A potentiometer is a variable resistor with three pins: the outer pins go to 5 V and GND and the middle pin (wiper) gives a voltage between 0 and 5 V as the knob turns: an adjustable voltage divider.



POTENTIOMETER WIRED AS A VOLTAGE DIVIDER TO A0

2.3 Prototyping tools

2.3.1 Breadboards and jumper wires

In a breadboard, each short column of 5 holes is connected inside; the long rails along the edges carry power (red +) and ground (blue -). The central gap separates the two halves so that a chip can straddle it. Many full-size boards split the power rails in the middle: bridge them with a wire.

2.3.2 Multimeter use

Measure	Setting and connection
Voltage	V⎓ range, probes across the component (in parallel)
Current	A or mA range, meter in series with the circuit (break the circuit)

Measure	Setting and connection
Resistance	Ω range, component out of circuit, power off
Continuity	beep mode: checks that a wire or joint is connected

WATCH OUT
Never measure voltage with the meter set to current: in current mode the meter is almost a short circuit and may blow its fuse.

2.4 Reading schematics and pinout diagrams

A schematic shows connections, not physical layout. Lines are wires; a dot where lines cross means they are joined; the ground symbol means "connected to GND". Pinout diagrams show which physical pin has which function (digital, analog, PWM ~, power).

2.5 Safety

2.5.1 Pin current limits

Keep each Uno R3 I/O pin at or below about 20 mA (40 mA is the absolute maximum), and the total for all pins under about 200 mA. Drive bigger loads through transistors, MOSFETs, relays or driver chips.

2.5.2 Never short 5 V to GND

A direct wire from 5 V to GND draws a huge current, can damage the USB port or regulator, and makes components hot. Check wiring with power off.

2.5.3 5 V vs 3.3 V logic

Uno R3 and Mega use 5 V logic; ESP32, Nano 33 IoT and most modern sensors use 3.3 V. Feeding 5 V into a 3.3 V input can destroy it: use a voltage divider or a level shifter.

2.6 Practice questions

2.6.1 Basic level

Q1 A 5 V supply is connected across a 1 kΩ resistor. What current flows?

(a) 0.5 mA **(b)** 50 mA **(c)** 5 mA **(d)** 5 A

Q2 What is the total resistance of 220 Ω and 330 Ω in series?

(a) 550 Ω **(b)** 132 Ω **(c)** 110 Ω **(d)** 660 Ω

Q3 What is the value of a resistor with bands red, red, brown, gold?

(a) 22 Ω **(b)** 2.2 kΩ **(c)** 221 Ω **(d)** 220 Ω

Q4 Which leg of an LED is the anode (+)?

(a) the shorter leg **(b)** the longer leg
(c) the leg next to the flat side **(d)** either leg

Q5 What is the unit of electrical resistance?

(a) volt **(b)** ohm **(c)** ampere **(d)** watt

Q6 To measure current with a multimeter, the meter must be connected

(a) in parallel with the component **(b)** across the battery terminals
(c) with the power off **(d)** in series with the circuit

Q7 What is the colour code of a 10 kΩ resistor?

(a) brown, black, orange **(b)** brown, black, red
(c) red, black, orange **(d)** brown, black, yellow

Q8 In a breadboard, the long rails along the edges are usually used for

(a) mounting chips **(b)** analog signals only
(c) power and ground **(d)** nothing; they are decorative

2.6.2 Intermediate level

Q9 What resistor is needed for a red LED (forward voltage 2 V) at 15 mA from a 5 V pin?

(a) 333 Ω **(b)** 133 Ω
(c) 20 Ω **(d)** 200 Ω (use 220 Ω)

Q10 Two 1 kΩ resistors are connected in parallel. The total resistance is

(a) 2 kΩ **(b)** 500 Ω **(c)** 1 kΩ **(d)** 250 Ω

Q11 A voltage divider has R1 = 10 kΩ (top) and R2 = 10 kΩ (bottom) with Vin = 5 V. What is Vout?

(a) 2.5 V **(b)** 5 V **(c)** 1.25 V **(d)** 3.3 V

Q12 What power is dissipated in a 220 Ω resistor carrying 20 mA?

(a) 4.4 mW **(b)** 0.88 W **(c)** 88 mW **(d)** 8.8 W

Q13 A divider steps 5 V down for a 3.3 V input with R1 = 2 kΩ and R2 = 3.3 kΩ. Vout is about

(a) 3.11 V **(b)** 3.30 V **(c)** 1.89 V **(d)** 2.50 V

Q14 Why is a flyback diode placed across a relay coil?

(a) to make the relay click louder
(b) to absorb the voltage spike when the coil switches off
(c) to increase the coil current
(d) to convert AC to DC

Q15 About how much current can one Uno R3 I/O pin safely supply?

(a) 2 A **(b)** 500 mA **(c)** 0.2 mA **(d)** 20 mA

Q16 What happens if 5 V is wired directly to GND?

(a) nothing, because Arduino is protected
(b) the LED becomes brighter
(c) a short circuit draws a large current and can damage the board or USB port
(d) the voltage doubles

2.6.3 Advanced level

Q17 A blue LED (forward voltage 3.0 V) is driven from 5 V through a 100 Ω resistor. What is the current?

(a) 50 mA **(b)** 30 mA **(c)** 20 mA **(d)** 10 mA

Q18 A 4.7 kΩ resistor is coloured

(a) yellow, violet, orange **(b)** yellow, violet, brown
(c) orange, violet, red **(d)** yellow, violet, red

Q19 A thermistor (R2) and a fixed 10 kΩ resistor (R1, top) form a divider from 5 V. When the thermistor reads 5 kΩ, Vout is

(a) 3.33 V **(b)** 1.67 V **(c)** 2.50 V **(d)** 0.83 V

Q20 An RC circuit has R = 10 kΩ and C = 100 µF. Its time constant τ is

(a) 1 s **(b)** 0.1 s **(c)** 10 s **(d)** 1 ms

Q21 A transistor with β = 100 must switch a 300 mA motor. What is the minimum base current, before adding a safety margin?

(a) 30 mA **(b)** 3 mA **(c)** 0.3 mA **(d)** 300 mA

Q22 Three resistors of 100 Ω, 200 Ω and 300 Ω are connected in parallel. The total is about

(a) 54.5 Ω **(b)** 600 Ω **(c)** 200 Ω **(d)** 100 Ω

Q23 Why must a 5 V Arduino output not be connected directly to an ESP32 input?

(a) the ESP32 needs 12 V
(b) the signals are always compatible
(c) the ESP32 uses 3.3 V logic and can be damaged by 5 V
(d) the ESP32 has no input pins

Q24 What is the maximum power rating needed for a resistor dropping 7 V at 50 mA?

(a) 0.035 W **(b)** 3.5 W
(c) 0.125 W is enough **(d)** at least 0.35 W (use a 0.5 W resistor)

Q25 Six red LEDs, each drawing 20 mA, are connected to six Uno R3 pins at once. What is the total pin

current, and is it acceptable?

(a) 120 mA; too high for one pin

(b) 20 mA; always acceptable

(c) 1.2 A; dangerous

(d) 120 mA; acceptable (under about 200 mA total)

2.7 Solutions

Q1 **Answer: (c) 5 mA**

I = V / R = 5 / 1000 = 0.005 A = **5 mA**.

Q2 **Answer: (a) 550 Ω**

Series: 220 + 330 = **550 Ω**.

Q3 **Answer: (d) 220 Ω**

Red 2, red 2, brown ×10: 22 × 10 = **220 Ω** ±5 %.

Q4 **Answer: (b) the longer leg**

The **longer leg** is the anode; the flat side marks the cathode.

Q5 **Answer: (b) ohm**

Resistance is measured in **ohms (Ω)**.

Q6 **Answer: (d) in series with the circuit**

Current flows **through** the meter, so it goes in series.

Q7 **Answer: (a) brown, black, orange**

1, 0, ×1000: **brown, black, orange**.

Q8 **Answer: (c) power and ground**

The rails carry **power (+) and ground (-)**.

Q9 **Answer: (d) 200 Ω (use 220 Ω)**

R = (5 - 2) / 0.015 = **200 Ω**; the nearest safe standard value is 220 Ω.

Q10 **Answer: (b) 500 Ω**

Two equal resistors in parallel: R/2 = **500 Ω**.

Q11 **Answer: (a) 2.5 V**

Vout = 5 × 10 / (10 + 10) = **2.5 V**.

Q12 **Answer: (c) 88 mW**

P = I²R = 0.02² × 220 = 0.088 W = **88 mW**.

Q13 **Answer: (a) 3.11 V**

Vout = 5 × 3.3 / 5.3 = **3.11 V**, safely below 3.3 V.

Q14 **Answer: (b) to absorb the voltage spike when the coil switches off**

Switching off a coil produces a large reverse spike; the diode **absorbs** it and protects the transistor.

Q15 **Answer: (d) 20 mA**

Keep each pin at about **20 mA** (40 mA absolute maximum).

Q16 **Answer: (c) a short circuit draws a large current and can damage the board or USB port**

A **short circuit**: very high current and possible damage.

Q17 **Answer: (c) 20 mA**

I = (5 - 3) / 100 = 0.02 A = **20 mA**.

Q18 **Answer: (d) yellow, violet, red**

4, 7, ×100: **yellow, violet, red**.

Q19 **Answer: (b) 1.67 V**

Vout = 5 × 5 / (10 + 5) = **1.67 V**.

Q20 **Answer: (a) 1 s**

τ = RC = 10 000 × 0.0001 = **1 s**.

Q21 **Answer: (b) 3 mA**

Ib = Ic / β = 300 / 100 = **3 mA**; in practice use several times more to saturate the transistor.

Q22 **Answer: (a) 54.5 Ω**

1/R = 1/100 + 1/200 + 1/300 = 0.01833 → R ≈ **54.5 Ω**.

Q23 **Answer: (c) the ESP32 uses 3.3 V logic and can be damaged by 5 V**

ESP32 pins are **3.3 V** and not 5 V tolerant: use a divider or level shifter.

Q24 **Answer: (d) at least 0.35 W (use a 0.5 W resistor)**

P = 7 × 0.05 = 0.35 W, so a standard 0.25 W resistor would overheat: **use 0.5 W**.

Q25 **Answer: (d) 120 mA; acceptable (under about 200 mA total)**

6 × 20 = **120 mA**, under the roughly 200 mA total limit, and each pin is at 20 mA.

V = I × R and P = V × I. Series resistances add; parallel ones combine as 1/R = Σ1/Rn. Vout = Vin × R2/(R1 + R2). LED resistor = (Vs − Vf)/I. Keep pins at 20 mA, use transistors for big loads, never short 5 V to GND, and respect 3.3 V logic.

MODULE 03 · BEGINNER

Arduino boards and hardware

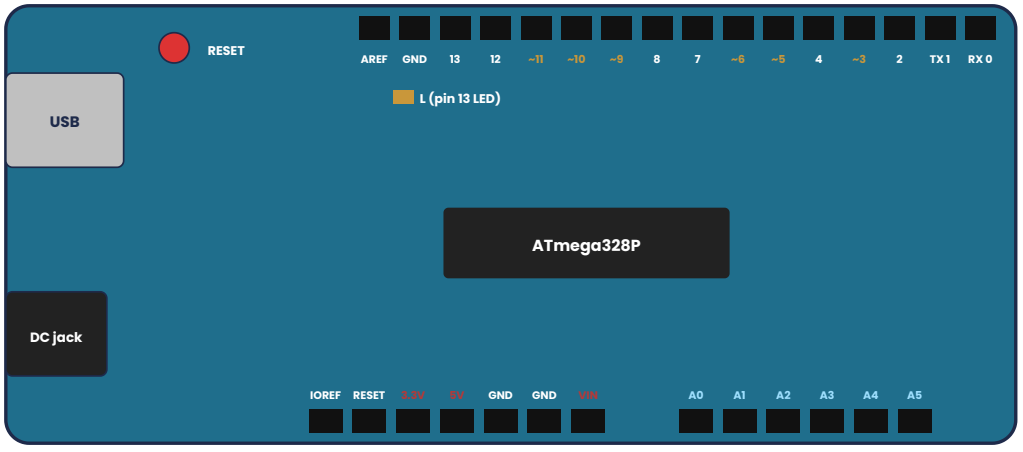
Minimum duration **1 day** | Level **Beginner**

This module tours the Arduino Uno pin by pin (microcontroller, digital, PWM and analog pins, power pins, USB, reset and built-in LED, and the three kinds of memory), compares the other Arduino boards and compatible boards such as ESP32 and Raspberry Pi Pico, introduces shields and breakout modules, and explains every way of powering a board, followed by 25 solved questions.

IN THIS MODULE

3.1 Arduino Uno
3.2 Other Arduino boards
3.3 Compatible boards
3.4 Shields and modules
3.5 Powering the board
3.6 Practice questions
3.7 Solutions

3.1 Arduino Uno



Digital pins 0–13 on top (~ = PWM) · power and analog A0–A5 at the bottom

ARDUINO UNO R3: THE MAIN FEATURES

3.1.1 Microcontroller

Board	Microcontroller	Clock	Flash	SRAM	Logic
Uno R3	ATmega328P (8-bit AVR)	16 MHz	32 KB	2 KB	5 V
Uno R4 Minima / WiFi	Renesas RA4M1 (32-bit Arm Cortex-M4)	48 MHz	256 KB	32 KB	5 V
Mega 2560	ATmega2560	16 MHz	256 KB	8 KB	5 V
Nano ESP32	ESP32-S3 (dual-core, Wi-Fi/BLE)	240 MHz	16 MB	512 KB	3.3 V

3.1.2 Digital pins, PWM pins, analog pins

The Uno R3 has 14 digital pins (0–13), each usable as input or output. Six of them (3, 5, 6, 9, 10, 11, marked ~) can produce PWM. Six analog inputs (A0–A5) read voltages from 0 to 5 V with 10-bit resolution (0–1023); they can also be used as digital pins. Pins 0 and 1 are shared with the USB serial link, so avoid them while using the Serial Monitor.

3.1.3 Power pins: 5 V, 3.3 V, VIN, GND

Pin	Function
5V	regulated 5 V output (from USB or the on-board regulator)
3.3V	regulated 3.3 V output, about 50 mA maximum on Uno R3
VIN	input for an external 7–12 V supply (or output of the jack voltage)
GND	ground: the 0 V reference; all grounds in a project must be connected
IOREF	tells shields the board's logic voltage

3.1.4 USB port, power jack, reset button, built-in LED

The USB port powers the board, uploads sketches and carries Serial Monitor data. The DC jack accepts 7–12 V. The reset button restarts the sketch from the beginning. The built-in LED ("L") is connected to pin 13 (LED_BUILTIN).

3.1.5 Flash, SRAM and EEPROM memory

Memory	Uno R3 size	Keeps data without power?	Holds
Flash	32 KB (0.5 KB used by the bootloader)	yes	your compiled sketch
SRAM	2 KB	no	variables while the sketch runs
EEPROM	1 KB	yes	settings you save from the sketch

WATCH OUT
SRAM is the scarcest resource on an Uno R3: 2 KB is only 2048 bytes. Long strings and big arrays can exhaust it and cause strange crashes. Keep constant text in flash with F("...").

3.2 Other Arduino boards

Board	Key feature	Typical use
Nano / Nano Every	Uno features in a breadboard-friendly size	compact projects, wearables
Mega 2560	54 digital pins, 16 analog inputs, 4 serial ports, 8 KB SRAM	3D printers, large robots

Board	Key feature	Typical use
Leonardo / Micro	native USB: can act as a keyboard or mouse (HID)	custom controllers, macro pads
Nano 33 IoT / Nano ESP32	Wi-Fi and Bluetooth, 3.3 V logic	IoT devices
MKR / Portenta	low-power IoT; industrial-grade performance	professional products

3.3 Compatible boards

3.3.1 ESP32, ESP8266, Raspberry Pi Pico with the Arduino IDE

By installing a "board package" in the Arduino IDE, you can program boards from other makers with the same language: the **ESP32** (dual-core, Wi-Fi, Bluetooth, cheap), the **ESP8266** (Wi-Fi), and the **Raspberry Pi Pico** (RP2040). All three use 3.3 V logic.

3.4 Shields and modules

3.4.1 Shields

Shields are boards that plug directly on top of an Uno or Mega: motor shields (drive DC and stepper motors), sensor shields (easy connectors), Ethernet and Wi-Fi shields, LCD keypad shields.

3.4.2 Breakout modules

Breakout modules put one chip (a sensor, a display driver, a real-time clock) on a small board with labelled pins, ready to wire to any Arduino with jumper wires.

3.5 Powering the board

3.5.1 USB, barrel jack, VIN and battery options

Source	Voltage	Notes
USB	5 V	easiest; limited to about 500 mA from a computer port
Barrel jack	7–12 V recommended	passes through the on-board regulator
VIN pin	7–12 V	same as the jack, but via a wire
5V pin	exactly 5 V (regulated)	bypasses the regulator: never exceed 5 V
Batteries	9 V battery (short life), 4 × AA (6 V) to VIN, Li-ion with a boost converter to 5 V	see Module 23 for battery projects

WATCH OUT

Below about 7 V on VIN the regulator cannot hold 5 V; above about 12 V it overheats. Never feed more than 5 V into the 5V pin.

3.6 Practice questions

3.6.1 Basic level

Q1 Which microcontroller is used on the Arduino Uno R3?

(a) ESP32 (b) RP2040 (c) ATmega328P (d) ATmega2560

Q2 How many digital I/O pins does the Uno R3 have?

(a) 14 (b) 6 (c) 20 (d) 54

Q3 Which symbol marks PWM-capable pins on the Uno?

(a) * (b) # (c) + (d) ~

Q4 The built-in LED on the Uno is connected to pin

(a) 0 (b) 13 (c) 1 (d) A0

Q5 How many analog input pins does the Uno have?

(a) 4 (b) 6 (c) 8 (d) 16

Q6 What is the clock speed of the Uno R3?

(a) 8 MHz (b) 48 MHz (c) 240 MHz (d) 16 MHz

Q7 Which pressing restarts the sketch from the beginning?

(a) the reset button (b) the power LED
(c) the USB chip (d) pin 13

Q8 Which memory holds your compiled sketch?

(a) SRAM (b) EEPROM (c) flash (d) cache

3.6.2 Intermediate level

Q9 Which memory loses its contents when power is removed?

(a) flash (b) EEPROM (c) none of them (d) SRAM

Q10 What are the PWM pins on the Uno R3?

(a) 2, 4, 7, 8, 12, 13 (b) 3, 5, 6, 9, 10, 11
(c) 0, 1, 2, 3, 4, 5 (d) A0–A5

Q11 Why should pins 0 and 1 be avoided when using the Serial Monitor?

(a) they are shared with the USB serial connection
(b) they are analog only
(c) they are power pins
(d) they are not connected

Q12 What is the recommended voltage range for the barrel jack or VIN?

(a) 3.3–5 V (b) 12–24 V (c) 7–12 V (d) 0–5 V

Q13 Which board can act as a USB keyboard or mouse (HID) natively?

(a) Leonardo (b) Uno R3 (c) Mega 2560 (d) Nano

Q14 Which board would you choose for a project needing 40 digital pins and 3 extra serial ports?

(a) Uno R3 **(b)** Mega 2560 **(c)** Nano **(d)** Leonardo

Q15 How much SRAM does the Uno R3 have?

(a) 32 KB **(b)** 1 KB **(c)** 256 KB **(d)** 2 KB

Q16 Which logic voltage does the ESP32 use?

(a) 5 V **(b)** 12 V **(c)** 3.3 V **(d)** 1.8 V

3.6.3 Advanced level

Q17 What is the resolution of the Uno R3 analog inputs?

(a) 8 bits (0–255) **(b)** 12 bits (0–4095)
(c) 10 bits (0–1023) **(d)** 16 bits (0–65535)

Q18 A sketch with many long Serial.print messages crashes randomly on an Uno R3. The most likely cause is

(a) running out of flash **(b)** a broken USB cable
(c) the clock speed **(d)** running out of SRAM

Q19 Which board family is aimed at professional and industrial applications?

(a) Uno **(b)** Portenta **(c)** Nano **(d)** Leonardo

Q20 What happens if 9 V is applied to the 5V pin?

(a) the board can be damaged because the 5V pin bypasses the regulator
(b) the regulator reduces it to 5 V
(c) nothing; the 5V pin accepts 7–12 V
(d) the board runs faster

Q21 Which 32-bit microcontroller is used on the Uno R4 Minima and WiFi?

(a) ATmega328P **(b)** Renesas RA4M1 **(c)** ESP8266 **(d)** RP2040

Q22 What is a "shield" in the Arduino world?

(a) a board that plugs on top of an Arduino to add features
(b) a protective plastic case
(c) a type of resistor
(d) a software library

Q23 Which non-volatile memory can a sketch use to save settings, and how big is it on the Uno R3?

(a) SRAM, 2 KB **(b)** flash, 32 KB, freely writable
(c) EEPROM, 1 KB **(d)** EEPROM, 32 KB

Q24 A project needs Wi-Fi, Bluetooth and the Arduino IDE at the lowest cost. Which board is most suitable?

(a) Uno R3 **(b)** Mega 2560 **(c)** Leonardo **(d)** ESP32

Q25 Why must the grounds of an Arduino and an external motor supply be connected together?

(a) to increase the motor speed
(b) to reduce the motor voltage

(c) grounds should never be connected

(d) so that both circuits share the same 0 V reference for signals

3.7 Solutions

Q1 **Answer: (c) ATmega328P**

The Uno R3 uses the **ATmega328P**.

Q2 **Answer: (a) 14**

Digital pins **0 to 13**: 14 pins.

Q3 **Answer: (d) ~**

PWM pins are marked with a tilde **~**.

Q4 **Answer: (b) 13**

Pin 13 (LED_BUILTIN).

Q5 **Answer: (b) 6**

A0–A5: six.

Q6 **Answer: (d) 16 MHz**

The ATmega328P on the Uno runs at **16 MHz**.

Q7 **Answer: (a) the reset button**

The **reset button** restarts the sketch.

Q8 **Answer: (c) flash**

The program is stored in **flash**.

Q9 **Answer: (d) SRAM**

SRAM is volatile; flash and EEPROM keep data.

Q10 **Answer: (b) 3, 5, 6, 9, 10, 11**

Pins **3, 5, 6, 9, 10, 11**.

Q11 **Answer: (a) they are shared with the USB serial connection**

Pins 0 (RX) and 1 (TX) carry **USB serial** data.

Q12 **Answer: (c) 7–12 V**

The regulator needs **7–12 V**.

Q13 **Answer: (a) Leonardo**

The **Leonardo** (and Micro) has native USB.

Q14 **Answer: (b) Mega 2560**

The **Mega 2560** has 54 digital pins and 4 serial ports.

Q15 **Answer: (d) 2 KB**

2 KB of SRAM.

Q16 **Answer: (c) 3.3 V**

ESP32 is a **3.3 V** device.

Q17 **Answer: (c) 10 bits (0–1023)**

10-bit: 1024 levels, 0–1023.

Q18 **Answer: (d) running out of SRAM**

String literals are copied into the tiny **2 KB SRAM**; use F("...") to keep them in flash.

Q19 **Answer: (b) Portenta**

Portenta is the professional family.

Q20 **Answer: (a) the board can be damaged because the 5V pin bypasses the regulator**

The 5V pin has **no regulator**; over-voltage damages the board.

Q21 **Answer: (b) Renesas RA4M1**

The Uno R4 uses the **Renesas RA4M1** (Arm Cortex-M4).

Q22 **Answer: (a) a board that plugs on top of an Arduino to add features**

Shields stack on the headers to add **motor drivers, Ethernet, displays** and more.

Q23 **Answer: (c) EEPROM, 1 KB**

The Uno R3 has **1 KB of EEPROM** for saved settings.

Q24 **Answer: (d) ESP32**

The **ESP32** is cheap and has Wi-Fi and Bluetooth.

Q25 **Answer: (d) so that both circuits share the same 0 V reference for signals**

Signals are measured relative to ground; without a **common ground** the control signal has no reference.

Uno R3: ATmega328P, 16 MHz, 14 digital pins (6 PWM: 3, 5, 6, 9, 10, 11), 6 analog inputs (10-bit), 32 KB flash, 2 KB SRAM, 1 KB EEPROM, 5 V logic. Power by USB (5 V) or 7–12 V on the jack/VIN; never more than 5 V on the 5V pin. ESP32, Nano ESP32 and Pico use 3.3 V logic.

MODULE 04 · BEGINNER

Setup and development environment

Minimum duration **1 day** | Level **Beginner**

This module gets you from an empty laptop to a blinking LED. It covers installing Arduino IDE 2 and USB drivers, choosing the board and port, verifying, uploading and the Serial Monitor, the Boards and Library Managers, auto-completion and debugging, the Arduino Cloud Editor, the Tinkercad and Wokwi simulators, your first sketch, and fixes for the common upload problems, followed by 25 solved questions and programs.

IN THIS MODULE

4.1 Arduino IDE 2
4.2 Arduino Cloud Editor
4.3 Simulators
4.4 First sketch
4.5 Common upload problems and fixes
4.6 Practice questions
4.7 Solutions

4.1 Arduino IDE 2

4.1.1 Installing the IDE

Download Arduino IDE 2 from arduino.cc for Windows, macOS or Linux and install it with the default options. On first launch it installs the core for AVR boards (Uno, Nano, Mega) automatically.

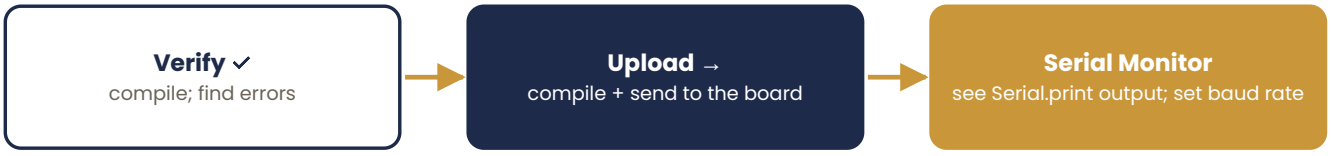
4.1.2 Installing USB drivers

Genuine boards are recognised automatically. Many clones use a **CH340** (or CP2102) USB-to-serial chip: install its driver if the board does not appear as a port.

4.1.3 Selecting board and port

Use the board selector at the top of the IDE (or Tools → Board and Tools → Port). Choose the exact board (for example "Arduino Uno") and the port it appears on: COM3, COM4… on Windows, /dev/ttyUSB0 or /dev/ttyACM0 on Linux, /dev/cu.usbmodem… on macOS.

4.1.4 Verify, upload and Serial Monitor



THE THREE BUTTONS YOU USE MOST

Action	Shortcut
Verify / compile	Ctrl + R
Upload	Ctrl + U
Serial Monitor	Ctrl + Shift + M
Auto-format code	Ctrl + T

WATCH OUT
The baud rate selected in the Serial Monitor must match Serial.begin() in the sketch, or you will see garbage characters.

4.1.5 Boards Manager and Library Manager

Boards Manager installs support for other boards (ESP32, Pico, Uno R4). **Library Manager** searches and installs libraries (Servo, DHT sensor library, LiquidCrystal I2C) with one click; they are then added with #include.

4.1.6 Auto-completion and debugging features

IDE 2 offers auto-completion, "go to definition", and error highlighting. Boards with debug support (such as the Zero or Portenta, or with an external probe) can use the built-in debugger with breakpoints; on an Uno, debugging is done mostly with Serial.print.

4.2 Arduino Cloud Editor

The Cloud Editor runs in a browser, keeps sketches online and uploads through a small agent program. It is useful on shared computers and for Arduino Cloud IoT projects.

4.3 Simulators

4.3.1 Tinkercad Circuits

Tinkercad Circuits (free, in a browser) simulates an Uno with breadboards, LEDs, buttons, sensors and motors. Ideal for beginners and classrooms.

4.3.2 Wokwi

Wokwi simulates Uno, Mega, ESP32 and Pico boards and many modules (displays, sensors, Wi-Fi on ESP32). Projects can be shared by link: a convenient way to test the sketches in this book without hardware.

4.4 First sketch

4.4.1 Blink the built-in LED

BLINK.INO

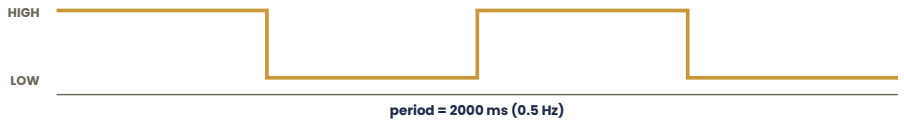
```
// Blink: turns the built-in LED on for one second, then off for one second
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);   // pin 13 on the Uno
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);  // LED on
  delay(1000);                      // wait 1000 ms
  digitalWrite(LED_BUILTIN, LOW);   // LED off
  delay(1000);
}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] pin 13 -> HIGH
[  1000 ms] pin 13 -> LOW
[  2000 ms] pin 13 -> HIGH
[  3000 ms] pin 13 -> LOW
```

setup() runs once when the board powers up or resets; **loop()** then repeats for ever. The simulated run above shows the pin changing every 1000 ms.



PIN 13 OVER TIME

4.5 Common upload problems and fixes

Problem	Likely cause and fix
Port greyed out / no port	charge-only USB cable (use a data cable); missing CH340 driver; try another USB port
"avrdude: stk500_recv(): programmer is not responding"	wrong board or port selected; something connected to pins 0/1; try another cable
"Access denied" / port busy	Serial Monitor or another program has the port open: close it
Upload works but nothing happens	wrong pin in the sketch; LED reversed; missing resistor or ground
Garbage in Serial Monitor	baud rate mismatch
"was not declared in this scope"	misspelt name, missing #include, or variable used outside its scope

SHORTCUT TRICK

When an upload fails, check in this order: cable → port → board type → anything on pins 0 and 1 → close other serial programs.

4.6 Practice questions

4.6.1 Basic level

Q1 Which IDE button compiles the sketch without uploading it?

(a) Upload **(b)** Serial Monitor **(c)** Verify **(d)** Save

Q2 Which function runs only once when the board starts?

(a) setup() **(b)** loop() **(c)** main() **(d)** start()

Q3 What does delay(1000) do?

(a) pauses for 1000 seconds
(b) repeats the loop 1000 times
(c) sets the clock to 1000 Hz
(d) pauses the sketch for 1000 milliseconds (1 second)

Q4 Which driver is commonly needed for cheap clone boards?

(a) NVIDIA **(b)** CH340 **(c)** Bluetooth **(d)** HDMI

Q5 Where do you install support for ESP32 boards in the Arduino IDE?

(a) Library Manager **(b)** Boards Manager
(c) Serial Monitor **(d)** Preferences only

Q6 The Serial Monitor shows random characters. The most likely cause is

(a) a broken LED **(b)** too much flash
(c) the reset button **(d)** a baud-rate mismatch

Q7 What is the keyboard shortcut to upload a sketch in Arduino IDE?

(a) Ctrl + U **(b)** Ctrl + R **(c)** Ctrl + S **(d)** Ctrl + T

Q8 Which free browser-based simulator supports ESP32 as well as Uno?

(a) Notepad **(b)** Excel **(c)** Wokwi **(d)** Paint

4.6.2 Intermediate level

Q9 Modify Blink so that the built-in LED is on for 200 ms and off for 800 ms.

Q10 Write a sketch that prints "Hello, Arduino!" once in setup() and then prints the number of seconds since start-up every second.

Q11 Upload fails with "programmer is not responding". Which should you check first?

(a) the LED colour
(b) the delay value
(c) the comment style
(d) that the correct board and port are selected

Q12 Your board does not appear in the port list even after installing drivers. A likely cause is

(a) the sketch has a syntax error
(b) a charge-only USB cable without data wires

(c) the LED is broken
(d) the baud rate is wrong

Q13 Uploading fails with "port busy / access denied". What is the fix?

(a) close the Serial Monitor or any other program using the port
(b) press the reset button 10 times
(c) remove the LED
(d) change the delay

Q14 Where are libraries such as "Servo" installed from?

(a) Boards Manager
(b) Serial Plotter
(c) Library Manager
(d) Upload menu

Q15 Which IDE shortcut auto-formats (indents) your code?

(a) Ctrl + T
(b) Ctrl + U
(c) Ctrl + M
(d) Ctrl + F

Q16 Write a sketch that blinks the built-in LED three times quickly (100 ms on, 100 ms off) and then waits 1 second, repeating for ever.

4.6.3 Advanced level

Q17 Error: "'ledPin' was not declared in this scope". What is the most likely cause?

(a) the board is not connected
(b) the variable was misspelt or declared inside another function
(c) the baud rate is wrong
(d) the LED is reversed

Q18 On Linux, an Uno usually appears as which port?

(a) COM3
(b) C:\Arduino
(c) /home/arduino
(d) /dev/ttyACM0

Q19 Write a sketch that prints the numbers 1 to 5 on separate lines once, in setup().

Q20 A sketch uploads successfully, but the external LED on pin 8 does not light. Which is NOT a likely cause?

(a) the LED is inserted the wrong way round
(b) the sketch uses pin 7 instead of pin 8
(c) the baud rate is set to 9600
(d) the LED circuit has no connection to GND

Q21 Which IDE 2 feature lets you jump to where a function or variable is defined?

(a) Serial Plotter
(b) Burn Bootloader
(c) Go to Definition
(d) Archive Sketch

Q22 Blink an LED on pin 8 instead of the built-in LED, with 500 ms on and 500 ms off. Use a named constant for the pin.

Q23 Why are sketches often tested in Wokwi or Tinkercad first?

(a) because simulators are faster than real boards for all tasks
(b) because real boards cannot run sketches

(c) to avoid installing the IDE ever

(d) to check wiring and code without risking real components

Q24 What file extension do Arduino sketches use?

(a) .py (b) .ino (c) .exe (d) .ard

Q25 In Blink, what is the frequency of the LED flashing with delay(1000) after both HIGH and LOW?

(a) 0.5 Hz (b) 1 Hz (c) 2 Hz (d) 1000 Hz

4.7 Solutions

Q1 **Answer: (c) Verify**

Verify compiles and reports errors.

Q2 **Answer: (a) setup()**

setup() runs once; loop() repeats.

Q3 **Answer: (d) pauses the sketch for 1000 milliseconds (1 second)**

delay() takes milliseconds: **1 second**.

Q4 **Answer: (b) CH340**

Many clones use the **CH340** USB-serial chip.

Q5 **Answer: (b) Boards Manager**

Board packages are installed in the **Boards Manager**.

Q6 **Answer: (d) a baud-rate mismatch**

The monitor's baud rate must match **Serial.begin()**.

Q7 **Answer: (a) Ctrl + U**

Ctrl + U uploads; Ctrl + R verifies.

Q8 **Answer: (c) Wokwi**

Wokwi simulates ESP32, Uno, Mega and Pico.

Q9 **Answer: Solution program**

Change only the two delay() values. The simulated run shows the on and off times.

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(200);
  digitalWrite(LED_BUILTIN, LOW);
  delay(800);
}
```

SIMULATED RUN

```
[     0 ms] pin 13 -> HIGH
[   200 ms] pin 13 -> LOW
[  1000 ms] pin 13 -> HIGH
[  1200 ms] pin 13 -> LOW
```

Q10 **Answer: Solution program**

millis() returns the milliseconds since start-up; dividing by 1000 gives seconds. Serial.begin(9600) must match the Serial Monitor setting.

```
void setup() {
  Serial.begin(9600);
  Serial.println("Hello, Arduino!");
}
void loop() {
  delay(1000);
  Serial.print("Seconds: ");
  Serial.println(millis() / 1000);
}
```

SERIAL MONITOR

```
Hello, Arduino!
Seconds: 1
Seconds: 2
Seconds: 3
Seconds: 4
```

Q11 **Answer: (d) that the correct board and port are selected**

The commonest cause is the **wrong board or port** (or a bad cable).

Q12 **Answer: (b) a charge-only USB cable without data wires**

Some USB cables have only power wires: use a **data cable**.

Q13 **Answer: (a) close the Serial Monitor or any other program using the port**

Only one program can use the port: **close the other one**.

Q14 **Answer: (c) Library Manager**

Libraries come from the **Library Manager**.

Q15 **Answer: (a) Ctrl + T**

Ctrl + T auto-formats.

Q16 **Answer: Solution program**

A for loop repeats the blink three times; the final delay creates the pause. The run shows three pulses, a 1-second gap, then the pattern again.

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}
void loop() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(LED_BUILTIN, HIGH);
    delay(100);
    digitalWrite(LED_BUILTIN, LOW);
    delay(100);
  }
  delay(1000);
}
```

SIMULATED RUN

```
[     0 ms] pin 13 -> HIGH
[   100 ms] pin 13 -> LOW
[   200 ms] pin 13 -> HIGH
[   300 ms] pin 13 -> LOW
[   400 ms] pin 13 -> HIGH
[   500 ms] pin 13 -> LOW
[  1600 ms] pin 13 -> HIGH
[  1700 ms] pin 13 -> LOW
[  1800 ms] pin 13 -> HIGH
[  1900 ms] pin 13 -> LOW
[  2000 ms] pin 13 -> HIGH
[  2100 ms] pin 13 -> LOW
```

Q17 **Answer: (b) the variable was misspelt or declared inside another function**

It is a compile error: the name is **unknown at that point**.

Q18 **Answer: (d) /dev/ttyACM0**

Genuine Unos appear as **/dev/ttyACM0** (clones as /dev/ttyUSB0).

Q19 **Answer: Solution program**

Code that should run only once goes in setup(); loop() can be left empty.

```
void setup() {
  Serial.begin(9600);
  for (int n = 1; n <= 5; n++) {
    Serial.println(n);
  }
}
void loop() {
}
```

SERIAL MONITOR

```
1
2
3
4
5
```

Q20 **Answer: (c) the baud rate is set to 9600**

Baud rate affects only serial communication, not the LED.

Q21 **Answer: (c) Go to Definition**

Go to Definition (right-click menu).

Q22 **Answer: Solution program**

Using a named constant means the pin number appears in one place only, so it is easy to change. Wire pin 8 → 220 Ω → LED → GND.

```
const int LED_PIN = 8;
void setup() {
  pinMode(LED_PIN, OUTPUT);
}
void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(500);
  digitalWrite(LED_PIN, LOW);
  delay(500);
}
```

SIMULATED RUN

```
[     0 ms] pin 8 -> HIGH
[   500 ms] pin 8 -> LOW
[  1000 ms] pin 8 -> HIGH
[  1500 ms] pin 8 -> LOW
```

Q23 **Answer: (d) to check wiring and code without risking real components**

Simulators let you **test safely**, then build the real circuit.

Q24 **Answer: (b) .ino**

Sketches are saved as **.ino** files inside a folder of the same name.

Q25 **Answer: (a) 0.5 Hz**

One full on-off cycle takes 2 s, so f = 1/2 = **0.5 Hz**.

Install IDE 2 (and the CH340 driver for clones), select board and port, Verify (Ctrl + R), Upload (Ctrl + U), and match the Serial Monitor baud rate to Serial.begin(). setup() runs once, loop() repeats. For upload problems check cable, port, board, pins 0/1 and other programs using the port.

MODULE 05 · BEGINNER

Arduino programming basics

Minimum duration **2 days** | Level **Beginner**

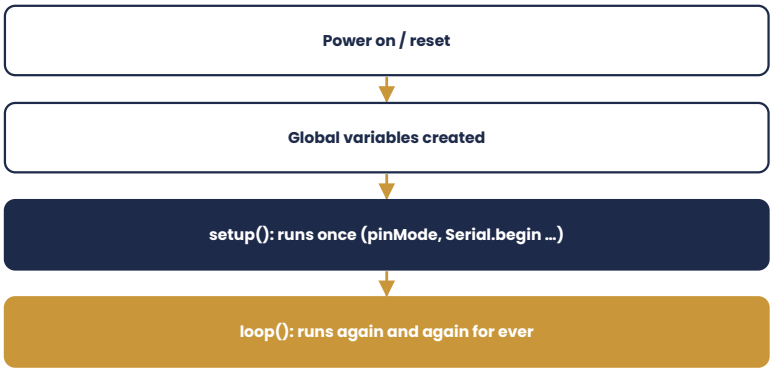
Arduino sketches are written in C/C++ with a set of built-in functions. This module covers sketch structure, comments and syntax, variables and data types (and their sizes on 8-bit and 32-bit boards), constants and scope, operators including integer division and overflow, if/else, switch and the three loops with break and continue, and the essential built-in functions for pins, time and numbers, followed by 25 solved questions and programs, all compiled and run.

IN THIS MODULE

5.1 Sketch structure
5.2 Variables and data types
5.3 Operators
5.4 Control structures
5.5 Built-in functions
5.6 Practice questions
5.7 Solutions

5.1 Sketch structure

5.1.1 setup() and loop()



HOW A SKETCH RUNS

5.1.2 Comments

`// single-line comment` and `/* multi-line comment */` are ignored by the compiler. Use them to explain *why* the code does something.

5.1.3 Semicolons and braces

Every statement ends with a semicolon `;`. Braces `{ }` group statements into a block (a function body, an if-block, a loop body). C/C++ is case-sensitive: `digitalWrite` is not `digitalwrite`.

WATCH OUT
The commonest beginner errors are a missing semicolon ("expected ';' before …") and unmatched braces. Auto-format (Ctrl + T) makes brace mistakes easy to see.

5.2 Variables and data types

5.2.1 Data types

Type	Uno R3 size	Range / use
bool	1 byte	true or false
byte (uint8_t)	1 byte	0 to 255
char	1 byte	a character such as 'A' (-128 to 127)
int	2 bytes	-32 768 to 32 767
unsigned int	2 bytes	0 to 65 535
long	4 bytes	-2 147 483 648 to 2 147 483 647
unsigned long	4 bytes	0 to 4 294 967 295: use for millis()
float / double	4 bytes	decimals, about 6–7 significant digits (double = float on Uno R3)

5.2.2 Data type sizes on 8-bit vs 32-bit boards

Type	Uno R3, Mega (8-bit AVR)	Uno R4, ESP32, Nano 33 (32-bit)
int	2 bytes (max 32 767)	4 bytes (max 2 147 483 647)
double	4 bytes (same as float)	8 bytes (more precise)
long	4 bytes	4 bytes

SHORTCUT TRICK
Use fixed-size types when the size matters: int16_t, uint16_t, int32_t, uint32_t are the same on every board.

5.2.3 Constants: const and #define

TWO WAYS TO NAME A CONSTANT

```
const int LED_PIN = 9;        // typed constant: preferred
#define BUTTON_PIN 2          // text substitution by the preprocessor (no semicolon!)
```

5.2.4 Scope: global and local variables

SCOPE.INO

```
int counter = 0;            // global: visible everywhere, keeps its value

void setup() {
  Serial.begin(9600);
}

void loop() {
  int local = 0;            // local: created fresh on every pass of loop()
  static int kept = 0;      // static local: created once, keeps its value
  counter++;
  local++;
  kept++;
  Serial.print("counter=");  Serial.print(counter);
  Serial.print("  local=");  Serial.print(local);
  Serial.print("  kept=");   Serial.println(kept);
}
```

SERIAL MONITOR

```
counter=1  local=1  kept=1
counter=2  local=1  kept=2
counter=3  local=1  kept=3
```

5.3 Operators

5.3.1 Arithmetic, comparison, logical, compound

Kind	Operators	Example
Arithmetic	+ - * / % (remainder)	17 % 5 is 2
Comparison	== != < > <= >=	if (temp >= 30)
Logical	&& (and) || (or) ! (not)	if (dark && motion)
Compound	+= -= *= /= ++ --	count += 2; i++;

WATCH OUT

= assigns, == compares. "if (x = 5)" sets x to 5 and is always true: a classic bug.

5.3.2 Integer division and overflow

DIVISION.INO

```
void setup() {
  Serial.begin(9600);
  int a = 7, b = 2;
  Serial.println(a / b);          // integer division drops the fraction
  Serial.println(a / 2.0);        // use a float to keep it
  Serial.println(a % b);          // remainder
  int big = 32767;
  big = big + 1;                  // int overflows on an 8-bit board
  Serial.println(big);
  long product = 300L * 200;      // L makes the arithmetic long
  Serial.println(product);
}
void loop() {}
```

SERIAL MONITOR

```
3
3.50
1
32768
60000
```

The run above used a 64-bit host, so `big` printed 32768; **on an Uno R3 it would print -32768**, because a 2-byte int wraps around after 32 767. Always choose a type large enough for the result.

5.4 Control structures

5.4.1 if, else if, else

IFELSE.INO

```
void setup() {
  Serial.begin(9600);
  int temps[] = {12, 24, 33};
  for (int i = 0; i < 3; i++) {
    int t = temps[i];
    Serial.print(t);
    if (t < 18) Serial.println(" C: cold");
    else if (t < 30) Serial.println(" C: comfortable");
    else Serial.println(" C: hot");
  }
}
void loop() {}
```

SERIAL MONITOR

```
12 C: cold
24 C: comfortable
33 C: hot
```

5.4.2 switch

SWITCH.INO

```
void setup() {
  Serial.begin(9600);
  char cmds[] = {'r', 'g', 'x'};
  for (int i = 0; i < 3; i++) {
    switch (cmds[i]) {
      case 'r': Serial.println("red on");   break;
      case 'g': Serial.println("green on"); break;
      default:  Serial.println("unknown command");
    }
  }
}
void loop() {}
```

SERIAL MONITOR

```
red on
green on
unknown command
```

WATCH OUT

Forgetting break makes execution "fall through" into the next case.

5.4.3 for, while, do-while

LOOPS.INO

```
void setup() {
  Serial.begin(9600);
  for (int i = 1; i <= 3; i++) Serial.print(i);   // counts 1 2 3
  Serial.println();
  int n = 5;
  while (n > 0) { Serial.print(n); n -= 2; }      // 5 3 1
  Serial.println();
  int k = 10;
  do { Serial.print(k); } while (k < 5);          // body runs once even though k >= 5
  Serial.println();
}
void loop() {}
```

SERIAL MONITOR

```
123
531
10
```

5.4.4 break and continue

BREAKCONTINUE.INO

```
void setup() {
  Serial.begin(9600);
  for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;   // skip even numbers
    if (i > 7) break;           // stop the loop completely
    Serial.print(i); Serial.print(' ');
  }
  Serial.println();
}
void loop() {}
```

SERIAL MONITOR

```
1 3 5 7
```

5.5 Built-in functions

5.5.1 Pin functions

Function	Purpose
pinMode(pin, mode)	set a pin as INPUT, OUTPUT or INPUT_PULLUP
digitalWrite(pin, value)	set an output pin HIGH (5 V) or LOW (0 V)
digitalRead(pin)	read HIGH or LOW from an input pin
analogRead(pin)	read 0–1023 from an analog pin (Uno R3)
analogWrite(pin, value)	PWM output 0–255 on a ~ pin

5.5.2 Time functions

Function	Returns / does
delay(ms)	pauses the sketch (nothing else runs)
millis()	milliseconds since start-up (unsigned long; wraps after about 49.7 days)
micros()	microseconds since start-up (wraps after about 70 minutes)

5.5.3 Math functions

MATHFUNCTIONS.INO

```
void setup() {
  Serial.begin(9600);
  Serial.println(map(512, 0, 1023, 0, 100));   // re-scale a range
  Serial.println(constrain(150, 0, 100));      // clamp to limits
  Serial.println(min(4, 9));
  Serial.println(max(4, 9));
  Serial.println(abs(-7));
  randomSeed(42);
  Serial.println(random(1, 7));                // dice roll: 1 to 6
}
void loop() {}
```

SERIAL MONITOR

```
50
100
4
9
7
1
```

`map()` uses integer arithmetic, so map(512, 0, 1023, 0, 100) gives 50, not 50.05. `random(a, b)` returns a value from a up to b - 1.

5.6 Practice questions

5.6.1 Basic level

Q1 How many times does setup() run?

(a) once every second
(b) once, at power-up or reset
(c) continuously
(d) never

Q2 Which data type should store the value returned by millis()?

(a) unsigned long (b) int (c) byte (d) float

Q3 What is the result of 17 % 5?

(a) 3 (b) 3.4 (c) 2 (d) 0

Q4 What is the size of an int on an Uno R3?

(a) 1 byte (b) 4 bytes (c) 8 bytes (d) 2 bytes

Q5 Which symbol starts a single-line comment?

(a) # (b) -- (c) /* (d) //

Q6 What does 7 / 2 evaluate to when both are int?

(a) 3.5 (b) 4 (c) 3 (d) 3.0

Q7 Which operator means logical AND?

(a) && (b) & (c) || (d) and

Q8 What does constrain(150, 0, 100) return?

(a) 150 (b) 0 (c) 50 (d) 100

5.6.2 Intermediate level

Q9 Write a sketch that prints the 7 times table (7 × 1 to 7 × 10) once.

Q10 What does map(256, 0, 1023, 0, 255) return?

(a) 64 (b) 63 (c) 256 (d) 127

Q11 What value does an int hold after 32767 + 1 on an Uno R3?

(a) 32768 (b) -32768 (c) 0 (d) 65535

Q12 What does random(1, 7) return?

(a) a whole number from 1 to 7
(b) a decimal from 1 to 7
(c) always 7
(d) a whole number from 1 to 6

Q13 Write a sketch that prints the sum of the numbers 1 to 100 using a while loop.

Q14 What is wrong with: if (x = 10) { … }?

(a) it assigns 10 to x instead of comparing, so the condition is always true
(b) nothing
(c) it compares x with 10 correctly
(d) it causes a compile error always

Q15 Using switch, print the name of the day for the numbers 1, 3 and 7 (1 = Monday … 7 = Sunday).

Q16 A variable declared inside loop() without "static"

(a) keeps its value between passes
(b) is visible in setup()
(c) is re-created (and re-initialised) on every pass of loop()
(d) is stored in EEPROM

5.6.3 Advanced level

Q17 Print all numbers from 1 to 30 that are divisible by 3 but not by 2, on one line separated by spaces.

Q18 What does "300 * 200" give in an int calculation on an Uno R3, and how is it fixed?

(a) 60000 exactly, no fix needed
(b) a compile error; use float only
(c) 0; use byte
(d) it overflows; write 300L * 200 to use long arithmetic

Q19 Write a sketch that counts down from 10 to 1, printing one number per second, then prints "Lift off!" and stops counting.

Q20 What is printed by: for (int i = 0; i < 10; i += 3) Serial.print(i);

(a) 036 **(b)** 0369 **(c)** 3690 **(d)** 0123456789

Q21 Which is true about "#define LED 13"?

(a) the preprocessor replaces LED with 13 everywhere before compiling
(b) it creates a variable stored in SRAM
(c) it must end with a semicolon
(d) it can be changed at run time

Q22 Write a sketch that prints whether each number in the array {4, 7, 10, 13} is even or odd.

Q23 millis() is stored in a 32-bit unsigned long. After about how long does it wrap around to 0?

(a) 32.7 seconds **(b)** 70 minutes **(c)** 49.7 days **(d)** 1 year

Q24 Print the first 12 Fibonacci numbers (0, 1, 1, 2, …) on one line.

Q25 On an ESP32, what is the maximum value of an int?

(a) 2 147 483 647 **(b)** 32 767 **(c)** 255 **(d)** 65 535

5.7 Solutions

Q1 **Answer: (b) once, at power-up or reset**

Once, at start-up.

Q2 **Answer: (a) unsigned long**

millis() returns an **unsigned long**; an int would overflow after 32.7 s.

Q3 **Answer: (c) 2**

17 = 3 × 5 + **2**.

Q4 **Answer: (d) 2 bytes**

On 8-bit AVR boards an int is **2 bytes**.

Q5 **Answer: (d) //**

// starts a single-line comment.

Q6 **Answer: (c) 3**

Integer division drops the fraction: **3**.

Q7 **Answer: (a) &&**

&& is logical AND (& is bitwise AND).

Q8 **Answer: (d) 100**

150 is above the upper limit, so it is clamped to **100**.

Q9 **Answer: Solution program**

A for loop from 1 to 10 prints each line; code that runs once belongs in setup().

```
void setup() {
  Serial.begin(9600);
  for (int i = 1; i <= 10; i++) {
    Serial.print("7 x "); Serial.print(i);
    Serial.print(" = ");  Serial.println(7 * i);
  }
}
void loop() {}
```

SERIAL MONITOR

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Q10 **Answer: (b) 63**

256 × 255 / 1023 = 63.8, and map() uses integer arithmetic, so it gives **63**.

Q11 **Answer: (b) -32768**

A 2-byte int **wraps around** to -32768.

Q12 **Answer: (d) a whole number from 1 to 6**

The upper bound is exclusive: **1 to 6**.

Q13 **Answer: Solution program**

The loop adds each number to sum; 1 + 2 + … + 100 = 5050 (= 100 × 101 / 2). A long is used because sums can grow past the int limit in similar tasks.

```
void setup() {
  Serial.begin(9600);
  int n = 1;
  long sum = 0;
  while (n <= 100) {
    sum += n;
    n++;
  }
  Serial.print("Sum = ");
  Serial.println(sum);
}
void loop() {}
```

SERIAL MONITOR

```
Sum = 5050
```

Q14 **Answer: (a) it assigns 10 to x instead of comparing, so the condition is always true**

Use **==** to compare.

Q15 **Answer: Solution program**

Each case ends with break so execution does not fall through to the next case.

```
void printDay(int d) {
  switch (d) {
    case 1: Serial.println("Monday"); break;
    case 2: Serial.println("Tuesday"); break;
    case 3: Serial.println("Wednesday"); break;
    case 4: Serial.println("Thursday"); break;
    case 5: Serial.println("Friday"); break;
    case 6: Serial.println("Saturday"); break;
    case 7: Serial.println("Sunday"); break;
    default: Serial.println("invalid");
  }
}
void setup() {
  Serial.begin(9600);
  printDay(1); printDay(3); printDay(7);
}
void loop() {}
```

SERIAL MONITOR

```
Monday
Wednesday
Sunday
```

Q16 **Answer: (c) is re-created (and re-initialised) on every pass of loop()**

Local variables are created fresh each time the function runs.

Q17 **Answer: Solution program**

Combine the two conditions with &&: divisible by 3 (i % 3 == 0) and odd (i % 2 != 0).

```
void setup() {
  Serial.begin(9600);
  for (int i = 1; i <= 30; i++) {
    if (i % 3 == 0 && i % 2 != 0) {
      Serial.print(i);
      Serial.print(' ');
    }
  }
  Serial.println();
}
void loop() {}
```

SERIAL MONITOR

```
3 9 15 21 27
```

Q18 **Answer: (d) it overflows; write 300L * 200 to use long arithmetic**

60 000 exceeds 32 767; the **L** suffix forces long arithmetic.

Q19 **Answer: Solution program**

A global counter keeps its value between passes of loop(). Setting n to -1 after the message stops further output.

```
int n = 10;
void setup() {
  Serial.begin(9600);
}
void loop() {
  if (n > 0) {
    Serial.println(n);
    n--;
    delay(1000);
  } else if (n == 0) {
    Serial.println("Lift off!");
    n = -1;          // so the message prints only once
  }
}
```

SERIAL MONITOR

```
10
9
8
7
6
5
4
3
2
1
Lift off!
```

Q20 **Answer: (b) 0369**

i takes 0, 3, 6, 9: prints **0369**.

Q21 **Answer: (a) the preprocessor replaces LED with 13 everywhere before compiling**

#define is **text substitution**; adding a semicolon would be copied too and often causes errors.

Q22 **Answer: Solution program**

The remainder after dividing by 2 is 0 for even numbers.

```
void setup() {
  Serial.begin(9600);
  int nums[] = {4, 7, 10, 13};
  for (int i = 0; i < 4; i++) {
    Serial.print(nums[i]);
    if (nums[i] % 2 == 0) Serial.println(" is even");
    else Serial.println(" is odd");
  }
}
void loop() {}
```

SERIAL MONITOR

```
4 is even
7 is odd
10 is even
13 is odd
```

Q23 **Answer: (c) 49.7 days**

2^{32} ms ≈ 4 294 967 s ≈ **49.7 days**. (micros() wraps after about 70 minutes.)

Q24 **Answer: Solution program**

Each term is the sum of the previous two; keep two variables and shift them along each step.

```
void setup() {
  Serial.begin(9600);
  long a = 0, b = 1;
  for (int i = 0; i < 12; i++) {
    Serial.print(a);
    Serial.print(' ');
    long next = a + b;
    a = b;
    b = next;
  }
  Serial.println();
}
void loop() {}
```

SERIAL MONITOR

```
0 1 1 2 3 5 8 13 21 34 55 89
```

Q25 **Answer: (a) 2 147 483 647**

On 32-bit boards an int is 4 bytes: up to **2 147 483 647**.

setup() runs once, loop() for ever. End statements with ;, group with { }, and remember C++ is case-sensitive. Choose types by range (int is 2 bytes on Uno R3; use unsigned long for millis()). Integer division drops fractions; watch for overflow. Use == to compare, break in every switch case, and const for named pins.

MODULE 06 · BEGINNER

Digital output

Minimum duration **1 day** | Level **Beginner**

Digital output means setting a pin HIGH or LOW to switch something on or off. This module covers pinMode() and digitalWrite(), multiple LEDs, a traffic light, running lights, a binary counter, driving bigger loads with transistors and relays (with mains safety), and the 74HC595 shift register that controls eight outputs from three pins, followed by 25 solved questions and programs, each simulated.

IN THIS MODULE

6.1 Digital output basics
6.2 LED projects
6.3 Driving bigger loads
6.4 Shift registers
6.5 Practice questions
6.6 Solutions

6.1 Digital output basics

6.1.1 pinMode(pin, OUTPUT) and digitalWrite()

Set a pin as an output once in setup() with `pinMode(pin, OUTPUT)`; then `digitalWrite(pin, HIGH)` connects it to 5 V and `digitalWrite(pin, LOW)` to 0 V.

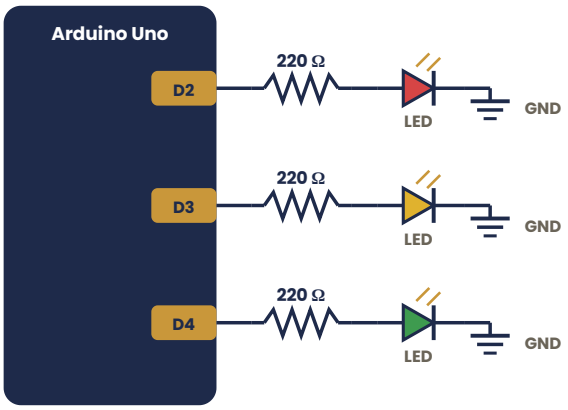
6.1.2 HIGH and LOW

Value	Uno R3 voltage	ESP32 voltage	Meaning
HIGH (1, true)	5 V	3.3 V	on (for an LED wired pin → resistor → LED → GND)
LOW (0, false)	0 V	0 V	off

An LED can be wired to light on HIGH (pin → resistor → LED → GND, "source") or on LOW (5 V → resistor → LED → pin, "sink"). The second way is "active-low": LOW turns it on.

6.2 LED projects

6.2.1 Multiple LEDs



THREE LEDS, EACH WITH ITS OWN RESISTOR

WATCH OUT

Give every LED its own resistor. Sharing one resistor between parallel LEDs makes them light unevenly because their forward voltages differ.

6.2.2 Traffic light

TRAFFICLIGHT.INO

```
const int RED = 2, YELLOW = 3, GREEN = 4;

void setup() {
  pinMode(RED, OUTPUT);
  pinMode(YELLOW, OUTPUT);
  pinMode(GREEN, OUTPUT);
}

void light(int r, int y, int g, unsigned long ms) {
  digitalWrite(RED, r);
  digitalWrite(YELLOW, y);
  digitalWrite(GREEN, g);
  delay(ms);
}

void loop() {
  light(HIGH, LOW, LOW, 5000);   // red
  light(LOW, LOW, HIGH, 4000);   // green
  light(LOW, HIGH, LOW, 1500);   // yellow
}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] pin 2 -> HIGH
[     0 ms] pin 3 -> LOW
[     0 ms] pin 4 -> LOW
[  5000 ms] pin 2 -> LOW
[  5000 ms] pin 3 -> LOW
```

```
[  5000 ms] pin 4 -> HIGH
[  9000 ms] pin 2 -> LOW
[  9000 ms] pin 3 -> HIGH
[  9000 ms] pin 4 -> LOW
```

6.2.3 Running lights (Knight Rider)

KNIGHTRIDER.INO

```
const int pins[] = {2, 3, 4, 5, 6, 7};
const int N = 6;

void setup() {
  for (int i = 0; i < N; i++) pinMode(pins[i], OUTPUT);
}

void loop() {
  for (int i = 0; i < N; i++) {          // left to right
    digitalWrite(pins[i], HIGH); delay(80); digitalWrite(pins[i], LOW);
  }
  for (int i = N - 2; i > 0; i--) {      // back again (skip the ends)
    digitalWrite(pins[i], HIGH); delay(80); digitalWrite(pins[i], LOW);
  }
}
```

Arrays let one loop handle many pins. To add more LEDs, only the array changes.

6.2.4 Binary counter

BINARYCOUNTER.INO

```
const int pins[] = {2, 3, 4, 5};      // pin 2 = bit 0 (least significant)

void setup() {
  Serial.begin(9600);
  for (int i = 0; i < 4; i++) pinMode(pins[i], OUTPUT);
}

void loop() {
  for (int n = 0; n < 16; n++) {
    for (int b = 0; b < 4; b++) digitalWrite(pins[b], bitRead(n, b));
    if (n < 6) { Serial.print(n); Serial.print(" = "); Serial.println(n, BIN); }
    delay(500);
  }
}
```

SERIAL MONITOR

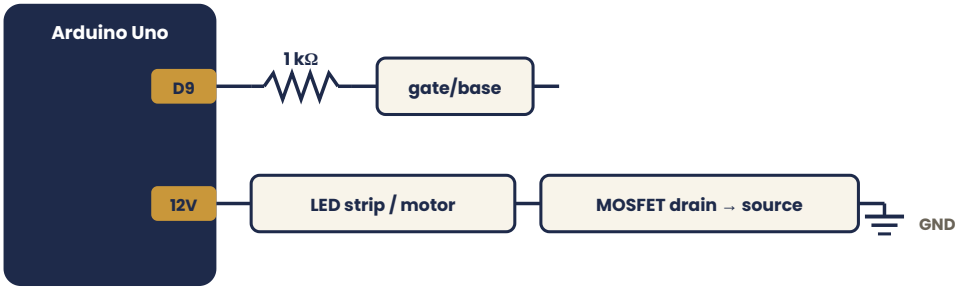
```
0 = 0
1 = 1
2 = 10
3 = 11
4 = 100
5 = 101
```

`bitRead(n, b)` returns bit b of n (0 or 1), which is exactly the HIGH/LOW value for that LED. Four LEDs show 0 to 15.

6.3 Driving bigger loads

6.3.1 Transistors for high-current loads

A pin gives about 20 mA. A small DC motor, a 12 V LED strip or a solenoid needs far more, often from a separate supply. Use an NPN transistor (up to a few hundred mA) or a logic-level N-channel MOSFET (amps): the pin drives the base/gate, the load sits between the supply and the transistor, and the grounds are shared.



LOW-SIDE SWITCHING WITH A MOSFET OR TRANSISTOR (COMMON GROUND)

6.3.2 Relay modules for mains appliances

A relay is an electrically operated switch: a small coil current closes heavy contacts that can switch a lamp or fan on mains. Relay modules include the transistor, the flyback diode and often an optocoupler, so they can be driven straight from a pin. Many modules are **active-low**: LOW energises the relay.

> **WATCH OUT**
> Mains electricity (230 V) can kill. Beginners should switch only low-voltage loads, or use a certified, enclosed smart-plug or relay box. Never touch the screw terminals of a powered relay, and never work on mains wiring while it is plugged in.

6.4 Shift registers

6.4.1 74HC595: many outputs with three pins

A 74HC595 receives 8 bits one at a time on its data pin (DS), clocked in by SHCP, and copies them to its 8 outputs Q0–Q7 when the latch pin (STCP) pulses. Three Arduino pins therefore control 8 outputs, and chips can be chained for 16, 24 …



74HC595 CONNECTIONS

SHIFTREGISTER.INO

```
const int DATA = 11, CLOCK = 12, LATCH = 8;

void setup() {
  pinMode(DATA, OUTPUT); pinMode(CLOCK, OUTPUT); pinMode(LATCH, OUTPUT);
}

void show(byte pattern) {
  digitalWrite(LATCH, LOW);
  shiftOut(DATA, CLOCK, MSBFIRST, pattern);   // send 8 bits
  digitalWrite(LATCH, HIGH);                  // outputs update together
}

void loop() {
  show(0b10101010);
  delay(500);
  show(0b01010101);
  delay(500);
}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] pin 8 -> LOW
[     0 ms] shiftOut 10101010
[     0 ms] pin 8 -> HIGH
[   500 ms] pin 8 -> LOW
[   500 ms] shiftOut 01010101
[   500 ms] pin 8 -> HIGH
```

6.5 Practice questions

6.5.1 Basic level

Q1 Which function makes pin 7 an output?

(a) digitalWrite(7, OUTPUT);
(b) pinMode(7, OUTPUT);
(c) pinMode(OUTPUT, 7);
(d) output(7);

Q2 On an Uno R3, digitalWrite(pin, HIGH) puts how much voltage on the pin?

(a) 3.3 V **(b)** 12 V **(c)** 0 V **(d)** 5 V

Q3 An LED is wired 5 V → resistor → LED → pin 6. Which command turns it ON?

(a) digitalWrite(6, HIGH);
(b) pinMode(6, INPUT);
(c) digitalWrite(6, LOW);
(d) analogRead(6);

Q4 Why does each LED need its own resistor?

(a) to make the code shorter
(b) because LEDs need 12 V
(c) to limit each LED's current; shared resistors make parallel LEDs uneven
(d) resistors are optional

Q5 What does bitRead(6, 1) return?

(a) 0 **(b)** 6 **(c)** 3 **(d)** 1

Q6 How many outputs does one 74HC595 provide?

(a) 4 **(b)** 8 **(c)** 16 **(d)** 3

Q7 Which device lets an Arduino switch a mains lamp?

(a) a relay module
(b) a 220 Ω resistor
(c) a potentiometer
(d) a 74HC595 alone

Q8 How many Arduino pins does a 74HC595 need?

(a) 8 **(b)** 3 **(c)** 1 **(d)** 16

6.5.2 Intermediate level

Q9 Write a sketch that turns on LEDs on pins 2, 3 and 4 one after another (300 ms apart) and then turns them all off together.

Q10 Write a pedestrian-crossing light: car green 5 s, car yellow 2 s, then car red with pedestrian green 4 s. Use pins 2 (car red), 3 (car yellow), 4 (car green), 5 (pedestrian green).

Q11 A relay module is labelled "active-low". What does this mean?

(a) the relay switches on when the control pin is LOW
(b) it works only at low voltages
(c) it switches on when the pin is HIGH
(d) it needs no power

Q12 What does shiftOut(DATA, CLOCK, MSBFIRST, 0b10000001) send first?

(a) the least significant bit (1)
(b) all 8 bits at once
(c) the most significant bit (1)
(d) the clock signal only

Q13 Why is the latch pin of the 74HC595 pulsed after shifting the bits?

(a) to erase the chip
(b) to set the clock speed
(c) to read inputs
(d) so all 8 outputs change at the same moment

Q14 Using four LEDs on pins 2–5, display the binary value of 9 and keep it shown.

Q15 A 12 V LED strip draws 1.5 A. What is the best way to switch it from an Arduino pin?

(a) connect it directly to the pin
(b) use a 220 Ω resistor in series
(c) use a 74HC595 output directly
(d) a logic-level N-channel MOSFET with a common ground

Q16 What is the main safety rule for relays switching mains?

(a) mains is safe below 1 A
(b) use thinner wires for mains
(c) never touch or work on the mains side while it is powered; prefer enclosed, certified modules
(d) mains needs no fuse

6.5.3 Advanced level

Q17 Make an SOS signal on the built-in LED: three short (200 ms), three long (600 ms), three short flashes, with 200 ms gaps and a 2 s pause at the end.

Q18 Two 74HC595 chips are chained. How many outputs do they give, and how many Arduino pins are needed?

(a) 16 outputs, still 3 pins
(b) 16 outputs, 6 pins
(c) 8 outputs, 3 pins
(d) 24 outputs, 4 pins

Q19 Why is a flyback diode needed when a transistor drives a motor or relay coil?

(a) to reverse the motor
(b) to slow the motor
(c) to increase the base current
(d) to protect the transistor from the voltage spike when the coil turns off

Q20 Write a sketch that blinks LED A (pin 8) every 400 ms and LED B (pin 9) every 800 ms using a single counter and delay(400).

Q21 Which pattern does show(0b00001111) produce on Q7…Q0 of a 74HC595 sent MSB first?

(a) Q7–Q4 on, Q3–Q0 off
(b) Q3–Q0 on, Q7–Q4 off
(c) all on
(d) alternate on and off

Q22 Display the decimal digits 0–3 on a 7-segment display (common cathode, segments a–g on pins 2–8) using a lookup table.

Q23 A relay coil needs 70 mA at 5 V. Can it be driven directly from an Uno pin?

(a) Yes, pins supply up to 2 A
(b) No; 70 mA exceeds the pin limit, so use a transistor (or a relay module)
(c) Yes, if the sketch uses HIGH
(d) No, relays need 3.3 V

Q24 In the Knight Rider sketch with 6 LEDs, how many LED flashes make one full left-right-left cycle?

(a) 12 **(b)** 6 **(c)** 11 **(d)** 10

Q25 What is the advantage of storing LED pins in an array?

(a) one loop can control any number of LEDs, and adding an LED changes only the array
(b) it uses less electricity
(c) it makes LEDs brighter
(d) arrays are required by pinMode()

6.6 Solutions

Q1 **Answer: (b) pinMode(7, OUTPUT);**
The pin comes first, then the mode: **pinMode(7, OUTPUT)**.

Q2 **Answer: (d) 5 V**
HIGH is **5 V** on a 5 V board (3.3 V on ESP32).

Q3 **Answer: (c) digitalWrite(6, LOW);**
The pin sinks current: LOW lets current flow, so it is **active-low**.

Q4 **Answer: (c) to limit each LED's current; shared resistors make parallel LEDs uneven**
Each LED needs its own current limit.

Q5 **Answer: (d) 1**
6 is 110 in binary; bit 1 is **1**.

Q6 **Answer: (b) 8**
Q0–Q7: **8** outputs.

Q7 **Answer: (a) a relay module**
A **relay** isolates the mains circuit from the Arduino.

Q8 **Answer: (b) 3**
Data, clock and latch: **3**.

Q9 **Answer: Solution program**

The first loop switches the LEDs on with a delay between them; the second switches them all off without delays, so they go off together.

```
const int pins[] = {2, 3, 4};
void setup() {
  for (int i = 0; i < 3; i++) pinMode(pins[i], OUTPUT);
}
void loop() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(pins[i], HIGH);
    delay(300);
  }
  for (int i = 0; i < 3; i++) digitalWrite(pins[i], LOW);
  delay(300);
}
```

SIMULATED RUN

```
[     0 ms] pin 2 -> HIGH
[   300 ms] pin 3 -> HIGH
[   600 ms] pin 4 -> HIGH
[   900 ms] pin 2 -> LOW
[   900 ms] pin 3 -> LOW
[   900 ms] pin 4 -> LOW
```

Q10 **Answer: Solution program**

Each phase switches on its lights, waits, then switches them off before the next phase. The pedestrian light is green only while the cars have red.

```
const int CR = 2, CY = 3, CG = 4, PG = 5;
void setup() {
  pinMode(CR, OUTPUT); pinMode(CY, OUTPUT); pinMode(CG, OUTPUT); pinMode(PG, OUTPUT);
}
void loop() {
  digitalWrite(CG, HIGH); delay(5000); digitalWrite(CG, LOW);
  digitalWrite(CY, HIGH); delay(2000); digitalWrite(CY, LOW);
  digitalWrite(CR, HIGH); digitalWrite(PG, HIGH); delay(4000);
  digitalWrite(CR, LOW);  digitalWrite(PG, LOW);
}
```

SIMULATED RUN

```
[     0 ms] pin 4 -> HIGH
[  5000 ms] pin 4 -> LOW
[  5000 ms] pin 3 -> HIGH
[  7000 ms] pin 3 -> LOW
[  7000 ms] pin 2 -> HIGH
[  7000 ms] pin 5 -> HIGH
[ 11000 ms] pin 2 -> LOW
[ 11000 ms] pin 5 -> LOW
```

Q11 **Answer: (a) the relay switches on when the control pin is LOW**

Active-low: **LOW energises** the relay.

Q12 **Answer: (c) the most significant bit (1)**

MSBFIRST sends bit 7 first; here it is **1**.

Q13 **Answer: (d) so all 8 outputs change at the same moment**

The latch copies the shifted bits to the outputs **together**, avoiding flicker.

Q14 **Answer: Solution program**

9 = 1001 in binary, so pins 2 and 5 go HIGH and pins 3 and 4 stay LOW.

```
const int pins[] = {2, 3, 4, 5};   // pin 2 = bit 0
void setup() {
  Serial.begin(9600);
  for (int b = 0; b < 4; b++) {
    pinMode(pins[b], OUTPUT);
    digitalWrite(pins[b], bitRead(9, b));
  }
  Serial.println(9, BIN);
}
void loop() {}
```

SIMULATED RUN

```
[     0 ms] pin 2 -> HIGH
[     0 ms] pin 3 -> LOW
[     0 ms] pin 4 -> LOW
[     0 ms] pin 5 -> HIGH
1001
```

Q15 **Answer: (d) a logic-level N-channel MOSFET with a common ground**

Amps need a **MOSFET**; a pin gives only about 20 mA.

Q16 **Answer: (c) never touch or work on the mains side while it is powered; prefer enclosed, certified modules**

Mains is lethal: work only **unpowered** and enclosed.

Q17 **Answer: Solution program**

A helper function flash() avoids repeating the same four lines nine times.

```
void flash(int ms) {
  digitalWrite(LED_BUILTIN, HIGH); delay(ms);
  digitalWrite(LED_BUILTIN, LOW);  delay(200);
}
void setup() { pinMode(LED_BUILTIN, OUTPUT); }
void loop() {
  for (int i = 0; i < 3; i++) flash(200);
  for (int i = 0; i < 3; i++) flash(600);
  for (int i = 0; i < 3; i++) flash(200);
  delay(2000);
}
```

SIMULATED RUN

```
[     0 ms] pin 13 -> HIGH
[   200 ms] pin 13 -> LOW
[   400 ms] pin 13 -> HIGH
[   600 ms] pin 13 -> LOW
[   800 ms] pin 13 -> HIGH
[  1000 ms] pin 13 -> LOW
[  1200 ms] pin 13 -> HIGH
[  1800 ms] pin 13 -> LOW
[  2000 ms] pin 13 -> HIGH
[  2600 ms] pin 13 -> LOW
[  2800 ms] pin 13 -> HIGH
[  3400 ms] pin 13 -> LOW
[  3600 ms] pin 13 -> HIGH
[  3800 ms] pin 13 -> LOW
[  4000 ms] pin 13 -> HIGH
[  4200 ms] pin 13 -> LOW
[  4400 ms] pin 13 -> HIGH
[  4600 ms] pin 13 -> LOW
```

Q18 **Answer: (a) 16 outputs, still 3 pins**

Chaining passes bits from Q7′ to the next chip: **16 outputs on 3 pins**.

Q19 **Answer: (d) to protect the transistor from the voltage spike when the coil turns off**

Coils produce a reverse **spike** when switched off.

Q20 **Answer: Solution program**

With a 400 ms tick, LED A toggles every tick and LED B every second tick. (Module 12 shows a cleaner way with millis().)

```
int tick = 0;
void setup() { pinMode(8, OUTPUT); pinMode(9, OUTPUT); }
void loop() {
  digitalWrite(8, tick % 2 == 0 ? HIGH : LOW);          // changes every tick
  digitalWrite(9, (tick / 2) % 2 == 0 ? HIGH : LOW);    // changes every 2 ticks
  tick++;
  delay(400);
}
```

SIMULATED RUN

```
[     0 ms] pin 8 -> HIGH
[     0 ms] pin 9 -> HIGH
[   400 ms] pin 8 -> LOW
[   400 ms] pin 9 -> HIGH
[   800 ms] pin 8 -> HIGH
[   800 ms] pin 9 -> LOW
[  1200 ms] pin 8 -> LOW
[  1200 ms] pin 9 -> LOW
```

Q21 **Answer: (b) Q3–Q0 on, Q7–Q4 off**

The low four bits are 1: **Q0–Q3 on**.

Q22 **Answer: Solution program**

Each digit is stored as a byte whose bits say which segments are lit; bitRead() turns each bit into a pin level. Leading zeros are not printed by Serial.print(…, BIN).

```
const int seg[7] = {2, 3, 4, 5, 6, 7, 8};          // a b c d e f g
const byte digits[4] = {                            // bit 0 = a … bit 6 = g
  0b0111111,  // 0
  0b0000110,  // 1
  0b1011011,  // 2
  0b1001111   // 3
};
void setup() {
  Serial.begin(9600);
  for (int i = 0; i < 7; i++) pinMode(seg[i], OUTPUT);
}
void loop() {
  for (int d = 0; d < 4; d++) {
    for (int i = 0; i < 7; i++) digitalWrite(seg[i], bitRead(digits[d], i));
    Serial.print("digit "); Serial.print(d); Serial.print(" pattern gfedcba = ");
    Serial.println(digits[d], BIN);
    delay(1000);
  }
}
```

SERIAL MONITOR

```
digit 0 pattern gfedcba = 111111
digit 1 pattern gfedcba = 110
digit 2 pattern gfedcba = 1011011
digit 3 pattern gfedcba = 1001111
```

Q23 **Answer: (b) No; 70 mA exceeds the pin limit, so use a transistor (or a relay module)**

70 mA is well above **20 mA**.

Q24 **Answer: (d) 10**

Forward pass: 6 LEDs; return pass skips both ends: 4 more. Total **10**.

Q25 **Answer: (a) one loop can control any number of LEDs, and adding an LED changes only the array**

Arrays make the code **scalable**.

pinMode(pin, OUTPUT) once, then digitalWrite(pin, HIGH/LOW). One resistor per LED; use arrays and loops for many LEDs; bitRead() maps numbers to LEDs. Drive loads above about 20 mA with transistors, MOSFETs or relay modules (common ground, flyback diode), keep away from live mains, and use a 74HC595 for 8 outputs on 3 pins.

MODULE 07 · BEGINNER

Digital input

Minimum duration **1 day** | Level **Beginner**

Digital input means reading whether a pin is HIGH or LOW. This module covers pinMode() and digitalRead(), floating inputs, pull-up and pull-down resistors and INPUT_PULLUP, reading a button, toggling and counting, why buttons bounce and how to debounce them in software and hardware, state-change (edge) detection, and other switch-type sensors such as tilt, reed and PIR sensors, followed by 25 solved questions and programs simulated with scripted button presses.

IN THIS MODULE

7.1 Digital input basics
7.2 Buttons
7.3 Debouncing
7.4 State change detection
7.5 Other digital input devices
7.6 Practice questions
7.7 Solutions

7.1 Digital input basics

7.1.1 pinMode(pin, INPUT) and digitalRead()

`digitalRead(pin)` returns HIGH if the voltage on the pin is above about 3 V (on a 5 V board) and LOW if it is below about 1.5 V. In between, the result is undefined.

7.1.2 Floating inputs

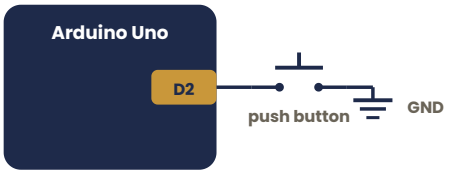
An input pin connected to nothing is "floating": it picks up electrical noise and reads HIGH and LOW at random. A button alone, connecting the pin to 5 V when pressed, leaves the pin floating when released.

7.1.3 Pull-up and pull-down resistors

Wiring	Released	Pressed	Resistor
pull-down: pin → 10 kΩ → GND, button pin → 5 V	LOW	HIGH	holds the pin LOW
pull-up: pin → 10 kΩ → 5 V, button pin → GND	HIGH	LOW	holds the pin HIGH

7.1.4 INPUT_PULLUP

Every Uno pin has an internal pull-up resistor (about 20–50 kΩ). `pinMode(pin, INPUT_PULLUP)` switches it on, so a button can be wired simply from the pin to GND. The logic is inverted: **pressed = LOW**.



BUTTON WITH INPUT_PULLUP: NO EXTERNAL RESISTOR NEEDED

INPUT_PULLUP + button to GND is the simplest reliable wiring. Remember: pressed reads LOW, released reads HIGH.

7.2 Buttons

7.2.1 Reading a push button

BUTTONLED.INO

```
const int BTN = 2, LED = 13;

void setup() {
  pinMode(BTN, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}

void loop() {
  if (digitalRead(BTN) == LOW) digitalWrite(LED, HIGH);   // pressed
  else digitalWrite(LED, LOW);
  delay(10);
}
```

7.2.2 Toggling an LED with a button

To make each press switch the LED on or off (like a light switch), you must act on the **moment** the button goes from released to pressed, not while it is held: see state-change detection in 7.4.

7.2.3 Counting button presses

COUNTPRESSES.INO

```
const int BTN = 2;
int count = 0;
int last = HIGH;

void setup() {
  Serial.begin(9600);
  pinMode(BTN, INPUT_PULLUP);
}

void loop() {
  int now = digitalRead(BTN);
  if (last == HIGH && now == LOW) {      // just pressed
    count++;
    Serial.print("Presses: ");
    Serial.println(count);
  }
  last = now;
  delay(20);                             // simple debounce
}
```

SERIAL MONITOR

```
Presses: 1
Presses: 2
Presses: 3
```

The simulation pressed the button three times (at 200, 900 and 1500 ms). Holding the button does not add extra counts because only the HIGH → LOW change is counted.

7.3 Debouncing

7.3.1 Why buttons bounce

Metal contacts spring apart and together several times in the first few milliseconds of a press. A fast sketch sees each bounce as a separate press.



A REAL BUTTON PRESS: SEVERAL QUICK TRANSITIONS BEFORE IT SETTLES LOW

7.3.2 Software debouncing

DEBOUNCE.INO

```
const int BTN = 2;
const unsigned long DEBOUNCE_MS = 30;
int stable = HIGH, lastRead = HIGH;
unsigned long lastChange = 0;
int presses = 0;

void setup() {
  Serial.begin(9600);
  pinMode(BTN, INPUT_PULLUP);
}

void loop() {
  int r = digitalRead(BTN);
  if (r != lastRead) { lastChange = millis(); lastRead = r; }   // input moved: restart timer
  if (millis() - lastChange > DEBOUNCE_MS && r != stable) {     // unchanged for 30 ms
    stable = r;
    if (stable == LOW) { presses++; Serial.print("Press "); Serial.println(presses); }
  }
}
```

SERIAL MONITOR

```
Press 1
Press 2
```

The scripted input bounced on every press and release, yet only two presses were counted: a change is accepted only after the reading has stayed the same for 30 ms.

7.3.3 Hardware debouncing (overview)

A 0.1 µF capacitor across the button with the pull-up resistor forms an RC filter that smooths the bounces; a Schmitt-trigger buffer (such as 74HC14) then gives a clean edge. Libraries such as Bounce2 wrap the software method.

7.4 State change detection (edge detection)

TOGGLELED.INO

```
const int BTN = 2, LED = 13;
int lastState = HIGH;
bool ledOn = false;

void setup() {
  Serial.begin(9600);
  pinMode(BTN, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}

void loop() {
  int state = digitalRead(BTN);
  if (state != lastState) {              // something changed
    if (state == LOW) {                  // falling edge = press
      ledOn = !ledOn;
      digitalWrite(LED, ledOn);
      Serial.println(ledOn ? "LED ON" : "LED OFF");
    }
    delay(30);                           // ignore bounces
  }
  lastState = state;
}
```

SERIAL MONITOR

```
LED ON
LED OFF
```



ACT ON THE EDGE, NOT THE LEVEL

7.5 Other digital input devices

7.5.1 Tilt switches, reed switches, PIR motion sensors

Device	How it works	Reads
Tilt switch	a ball rolls to close contacts when tilted	like a button; debounce it
Reed switch	contacts close near a magnet (door and window sensors)	like a button
PIR motion sensor (HC-SR501)	detects moving warm bodies; output goes HIGH for a set time	HIGH = motion; no pull-up needed (active output)

Device	How it works	Reads
IR obstacle sensor	reflected infrared light; output usually LOW when an object is near	active-low digital output

7.6 Practice questions

7.6.1 Basic level

Q1 What does digitalRead() return?

(a) a number from 0 to 1023 **(b)** a voltage in volts
(c) HIGH or LOW **(d)** a string

Q2 An input pin connected to nothing is called

(a) grounded **(b)** pulled up **(c)** saturated **(d)** floating

Q3 With pinMode(2, INPUT_PULLUP) and a button from pin 2 to GND, what does a pressed button read?

(a) HIGH **(b)** LOW **(c)** 1023 **(d)** random

Q4 Which wiring needs no external resistor for a button?

(a) INPUT_PULLUP with the button to GND **(b)** INPUT with the button to 5 V
(c) OUTPUT with the button to GND **(d)** INPUT with nothing connected

Q5 A typical external pull-up or pull-down resistor value is

(a) 10 Ω **(b)** 10 MΩ **(c)** 10 kΩ **(d)** 1 Ω

Q6 A PIR sensor output goes HIGH when it detects

(a) motion of a warm body **(b)** light
(c) sound **(d)** magnetic fields

Q7 A reed switch closes when

(a) it is tilted **(b)** a magnet comes near
(c) light falls on it **(d)** it gets hot

Q8 Why do buttons "bounce"?

(a) the code runs too slowly
(b) the resistor is too large
(c) the button is broken
(d) the metal contacts spring open and closed several times before settling

7.6.2 Intermediate level

Q9 Write a sketch that prints "Pressed" each time a button on pin 3 (INPUT_PULLUP) is pressed, counting each press only once.

Q10 In edge detection, a press with INPUT_PULLUP is which kind of edge?

(a) rising (LOW → HIGH) **(b)** no edge
(c) falling (HIGH → LOW) **(d)** both

Q11 What is the main problem with: "if (digitalRead(BTN) == LOW) count++;" inside loop()?

(a) it never counts
(b) it causes a compile error

(c) count increases thousands of times while the button is held
(d) it counts only releases

Q12 Two buttons: pin 2 turns the LED on pin 13 on, pin 3 turns it off. Both use INPUT_PULLUP.

Q13 What does a software debounce typically do?

(a) reads the pin only once per hour
(b) adds a resistor in code
(c) turns off the internal pull-up
(d) accepts a change only after the input has been stable for a short time (e.g. 20–50 ms)

Q14 Which hardware parts form a simple debounce filter?

(a) an LED and a resistor
(b) a resistor and a capacitor (RC)
(c) a relay and a diode
(d) a transistor and a motor

Q15 A tilt sensor on pin 4 (wired like a button with INPUT_PULLUP) reads LOW when tilted. Print "TILTED" or "LEVEL" whenever its state changes.

Q16 An IR obstacle sensor outputs LOW when an object is near. This output is called

(a) active-low (b) active-high (c) analog (d) floating

7.6.3 Advanced level

Q17 Write a debounced press counter that also prints how long each press lasted, in milliseconds.

Q18 Why should a sketch avoid long delay() calls when reading buttons?

(a) delay() damages the button
(b) presses that happen during the delay can be missed
(c) delay() changes the pull-up
(d) delay() makes the LED brighter

Q19 Make a "long-press" detector: a press shorter than 1 s prints "short", longer prints "long".

Q20 A PIR module output is connected to pin 7. Which pinMode is appropriate?

(a) INPUT (the module drives the line itself)
(b) OUTPUT
(c) INPUT_PULLUP is required
(d) no pinMode is allowed

Q21 With a pull-down resistor and a button to 5 V, what does the pin read when the button is released?

(a) HIGH (b) floating (c) LOW (d) 512

Q22 A doorbell: when a button on pin 2 is pressed, sound a buzzer on pin 8 at 1000 Hz for 300 ms, once per press.

Q23 Why does reading a voltage of 2.2 V on a 5 V board digital input give unreliable results?

(a) 2.2 V is always HIGH
(b) 2.2 V is always LOW
(c) digital pins cannot read voltages

(d) it lies in the undefined region between LOW (< 1.5 V) and HIGH (> 3 V)

Q24 Count people passing an IR beam sensor on pin 5 (active-low: LOW while the beam is broken) and print the running total.

Q25 How many states must a toggle sketch remember between passes of loop()?

(a) nothing
(b) only the LED pin number
(c) the time of day
(d) the previous button reading and the LED state

7.7 Solutions

Q1 **Answer: (c) HIGH or LOW**

HIGH or LOW (1 or 0).

Q2 **Answer: (d) floating**

It is **floating** and reads random values.

Q3 **Answer: (b) LOW**

Pressing connects the pin to GND: **LOW**.

Q4 **Answer: (a) INPUT_PULLUP with the button to GND**

The internal pull-up replaces the external resistor.

Q5 **Answer: (c) 10 kΩ**

10 kΩ is the usual choice.

Q6 **Answer: (a) motion of a warm body**

PIR = passive infrared: **moving warm bodies**.

Q7 **Answer: (b) a magnet comes near**

Reed switches respond to a **magnet**.

Q8 **Answer: (d) the metal contacts spring open and closed several times before settling**

Mechanical **contact bounce** lasts a few milliseconds.

Q9 **Answer: Solution program**

Remember the previous reading and act only on the HIGH → LOW change. The simulation pressed the button twice.

```
const int BTN = 3;
int last = HIGH;
void setup() {
  Serial.begin(9600);
  pinMode(BTN, INPUT_PULLUP);
}
void loop() {
  int now = digitalRead(BTN);
  if (last == HIGH && now == LOW) Serial.println("Pressed");
  last = now;
  delay(20);
}
```

SERIAL MONITOR

```
Pressed
Pressed
```

Q10 **Answer: (c) falling (HIGH → LOW)**

Released HIGH → pressed LOW: a **falling edge**.

Q11 **Answer: (c) count increases thousands of times while the button is held**

It counts the **level**, not the edge, so holding the button keeps counting.

Q12 **Answer: Solution program**

Each button sets the LED to a fixed state, so no edge detection is needed. The run shows ON after pin 2 is pressed and OFF after pin 3.

```
void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP);
  pinMode(3, INPUT_PULLUP);
  pinMode(13, OUTPUT);
}
void loop() {
  if (digitalRead(2) == LOW) { digitalWrite(13, HIGH); }
  if (digitalRead(3) == LOW) { digitalWrite(13, LOW); }
  delay(10);
}
```

SIMULATED RUN

```
[   100 ms] pin 13 -> HIGH
[   110 ms] pin 13 -> HIGH
[   120 ms] pin 13 -> HIGH
[   130 ms] pin 13 -> HIGH
[   140 ms] pin 13 -> HIGH
[   500 ms] pin 13 -> LOW
[   510 ms] pin 13 -> LOW
[   520 ms] pin 13 -> LOW
[   530 ms] pin 13 -> LOW
[   540 ms] pin 13 -> LOW
```

Q13 **Answer: (d) accepts a change only after the input has been stable for a short time (e.g. 20–50 ms)**

It waits for the reading to **stay stable**.

Q14 **Answer: (b) a resistor and a capacitor (RC)**

An **RC filter** smooths the bounces.

Q15 **Answer: Solution program**

Starting last at -1 forces the first reading to be printed. Printing only on changes keeps the Serial Monitor readable.

```
const int TILT = 4;
int last = -1;
void setup() {
  Serial.begin(9600);
  pinMode(TILT, INPUT_PULLUP);
}
void loop() {
  int s = digitalRead(TILT);
  if (s != last) {
    Serial.println(s == LOW ? "TILTED" : "LEVEL");
    last = s;
    delay(50);   // ignore the ball rattling
  }
}
```

SERIAL MONITOR

```
LEVEL
TILTED
LEVEL
```

Q16 **Answer: (a) active-low**

LOW means "detected": **active-low**.

Q17 **Answer: Solution program**

Record millis() at the falling edge and subtract it at the rising edge. The scripted presses lasted 300 ms and 800 ms.

```
const int BTN = 2;
int last = HIGH;
unsigned long downAt = 0;
void setup() {
  Serial.begin(9600);
  pinMode(BTN, INPUT_PULLUP);
}
void loop() {
  int now = digitalRead(BTN);
  if (now != last) {
    if (now == LOW) downAt = millis();              // press started
    else {                                          // released
      Serial.print("Press lasted ");
      Serial.print(millis() - downAt);
      Serial.println(" ms");
    }
    last = now;
    delay(20);                                      // debounce
  }
}
```

SERIAL MONITOR

```
Press lasted 300 ms
Press lasted 800 ms
```

Q18 **Answer: (b) presses that happen during the delay can be missed**

During delay() the sketch is not reading: **presses can be missed**.

Q19 **Answer: Solution program**

The decision is made on release, when the press duration is known.

```
const int BTN = 2;
int last = HIGH;
unsigned long t0 = 0;
void setup() {
  Serial.begin(9600);
  pinMode(BTN, INPUT_PULLUP);
}
void loop() {
  int now = digitalRead(BTN);
  if (now != last) {
    if (now == LOW) t0 = millis();
    else Serial.println(millis() - t0 >= 1000 ? "long" : "short");
    last = now;
    delay(20);
  }
}
```

SERIAL MONITOR

```
short
long
```

Q20 **Answer: (a) INPUT (the module drives the line itself)**

The PIR has an **active output**, so plain INPUT works.

Q21 **Answer: (c) LOW**

The pull-down holds the pin at **LOW**.

Q22 **Answer: Solution program**

tone(pin, frequency, duration) plays a note without blocking; edge detection ensures one ring per press.

```
const int BTN = 2, BUZ = 8;
int last = HIGH;
void setup() {
  pinMode(BTN, INPUT_PULLUP);
}
void loop() {
  int now = digitalRead(BTN);
  if (last == HIGH && now == LOW) {
    tone(BUZ, 1000, 300);
    delay(50);
  }
  last = now;
}
```

SIMULATED RUN

```
[   200 ms] tone pin 8 1000 Hz for 300 ms
```

Q23 **Answer: (d) it lies in the undefined region between LOW (< 1.5 V) and HIGH (> 3 V)**

Values between the thresholds are **undefined**.

Q24 **Answer: Solution program**

Each break of the beam is a falling edge. The script broke the beam three times. (The mock reads an undriven INPUT as LOW at start, so the scripted signal begins HIGH.)

```
const int IR = 5;
int last = HIGH;
int people = 0;
void setup() {
  Serial.begin(9600);
  pinMode(IR, INPUT);
}
void loop() {
  int now = digitalRead(IR);
  if (last == HIGH && now == LOW) {
    people++;
    Serial.print("People: ");
    Serial.println(people);
  }
  last = now;
  delay(10);
}
```

SERIAL MONITOR

```
People: 1
People: 2
People: 3
```

Q25 **Answer: (d) the previous button reading and the LED state**

It needs the **last reading** (for edges) and the **LED state** (to flip it).

Never leave inputs floating: use a pull-up/pull-down or INPUT_PULLUP (pressed = LOW). Act on edges, not levels, to count or toggle. Debounce by accepting a change only after it is stable for 20–50 ms. Avoid long delays when reading inputs. PIR and IR modules have active outputs; reed and tilt switches act like buttons.

MODULE 08 · BEGINNER

Analog input and the ADC

Minimum duration **1 day** | Level **Beginner**

Analog inputs measure voltages, not just HIGH or LOW. This module covers analog-to-digital conversion, resolution and the reference voltage, analogRead() and analogReadResolution(), converting readings to volts and scaling with map(), potentiometers, light-dependent resistors with voltage dividers, LM35 and TMP36 temperature sensors, soil-moisture, flex and force sensors, and smoothing noisy readings, followed by 25 solved questions and programs run with scripted sensor values.

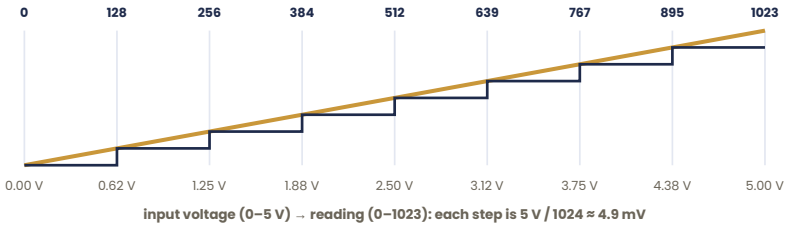
IN THIS MODULE

8.1 Analog-to-digital conversion
8.2 Converting readings
8.3 Analog sensors
8.4 Noise and smoothing
8.5 Practice questions
8.6 Solutions

8.1 Analog-to-digital conversion

8.1.1 ADC resolution

The analog-to-digital converter (ADC) turns a voltage into a number. A 10-bit ADC (Uno R3) has 2^{10} = 1024 levels, 0 to 1023. Newer boards have 12-bit (0–4095) or 14-bit (0–16383) ADCs.



A 10-BIT ADC DIVIDES THE 0–5 V RANGE INTO 1024 STEPS

Board	ADC bits	Range	Step at full scale
Uno R3, Nano, Mega	10	0–1023	5 V / 1024 ≈ 4.88 mV
Uno R4	10 by default, up to 14	0–16383 at 14 bits	5 V / 16384 ≈ 0.31 mV
ESP32	12	0–4095	3.3 V / 4096 ≈ 0.81 mV

8.1.2 Reference voltage

The ADC compares the input with a reference voltage: 5 V by default on the Uno R3. `analogReference(INTERNAL)` selects an internal 1.1 V reference for more resolution on small signals;

EXTERNAL uses the voltage on the AREF pin. Never apply more than the reference (or 5 V) to an analog pin.

8.1.3 analogRead() and analogReadResolution()

`analogRead(A0)` returns the reading. On boards with a better ADC, `analogReadResolution(12)` requests 12-bit values. A conversion takes about 100 µs on the Uno R3.

8.2 Converting readings

8.2.1 Raw values to voltage

Voltage = reading × Vref / 1023 (or / 1024; the difference is less than 0.1 %). With Vref = 5 V: 512 → 2.50 V; 1023 → 5.00 V; 205 → 1.00 V.

READVOLTAGE.INO

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  int raw = analogRead(A0);
  float volts = raw * 5.0 / 1023.0;       // 5.0 forces floating-point maths
  Serial.print("raw = ");  Serial.print(raw);
  Serial.print("  voltage = "); Serial.print(volts, 2); Serial.println(" V");
  delay(500);
}
```

SERIAL MONITOR

```
raw = 0  voltage = 0.00 V
raw = 205  voltage = 1.00 V
raw = 512  voltage = 2.50 V
raw = 1023  voltage = 5.00 V
```

WATCH OUT

raw * 5 / 1023 with integers gives whole volts only (0 to 5). Write 5.0 so the calculation uses floating point.

8.2.2 Scaling with map()

PERCENT.INO

```
void setup() {
  Serial.begin(9600);
}

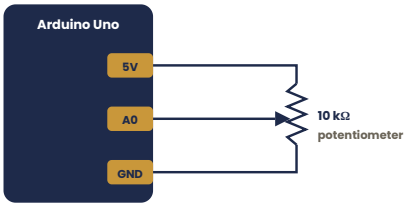
void loop() {
  int raw = analogRead(A0);
  int percent = map(raw, 0, 1023, 0, 100);
  Serial.print(percent); Serial.println(" %");
  delay(500);
}
```

SERIAL MONITOR

```
0 %
29 %
100 %
```

8.3 Analog sensors

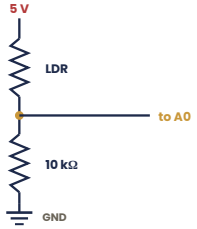
8.3.1 Potentiometers



POTENTIOMETER TO A0: TURNING THE KNOB SWEEPS 0–5 V

8.3.2 Light-dependent resistors (LDR)

An LDR's resistance falls as light increases (from about 1 MΩ in darkness to a few hundred ohms in bright light). With a fixed 10 kΩ resistor it forms a voltage divider; with the LDR on the bottom (to GND), the reading **falls** as the light increases.



LDR DIVIDER: LDR FROM 5 V TO A0, 10 KΩ FROM A0 TO GND (READING RISES WITH LIGHT)

8.3.3 Analog temperature sensors (LM35, TMP36)

Sensor	Output	Formula	Range
LM35	10 mV per °C, 0 V at 0 °C	T = V × 100	0 to 100 °C (basic wiring)
TMP36	10 mV per °C with 500 mV offset	T = (V - 0.5) × 100	-40 to 125 °C

TMP36.INO

```
const int SENSOR = A1;      // TMP36 output

void setup() {
  Serial.begin(9600);
}

void loop() {
  float v = analogRead(SENSOR) * 5.0 / 1023.0;
  float tempC = (v - 0.5) * 100.0;
  Serial.print("Temperature: "); Serial.print(tempC, 1); Serial.println(" C");
  delay(1000);
}
```

SERIAL MONITOR

```
Temperature: 24.8 C
Temperature: 25.3 C
Temperature: 39.9 C
```

8.3.4 Soil moisture sensors

Resistive or capacitive soil probes give a voltage that changes with moisture. Calibrate: note the reading in dry air and in a cup of water, then map() between them. Capacitive probes do not corrode and last much longer.

8.3.5 Flex and force sensors

Flex sensors and force-sensitive resistors (FSRs) change resistance when bent or pressed. Use them, like an LDR, in a divider with a fixed resistor (often 10 kΩ or 47 kΩ).

8.4 Noise and smoothing

8.4.1 Averaging multiple readings

AVERAGE.INO

```
const int N = 8;

int readAverage(int pin) {
  long sum = 0;
  for (int i = 0; i < N; i++) {
    sum += analogRead(pin);
    delay(2);
  }
  return sum / N;
}

void setup() {
  Serial.begin(9600);
  Serial.println(readAverage(A0));
}

void loop() {}
```

SERIAL MONITOR

```
502
```

Eight noisy readings between 485 and 520 average to 502. A moving (rolling) average or an exponential filter (avg = avg + (new - avg) / 8) smooths continuously without waiting for a batch.

8.5 Practice questions

8.5.1 Basic level

Q1 How many different values can a 10-bit ADC produce?

(a) 1023 **(b)** 256 **(c)** 1024 **(d)** 4096

Q2 On an Uno R3 with a 5 V reference, analogRead() returns 512. What is the voltage?

(a) about 2.5 V **(b)** 5 V **(c)** 1 V **(d)** 3.3 V

Q3 What range does analogRead() return on an Uno R3?

(a) 0 to 255 **(b)** 0 to 5 **(c)** -512 to 511 **(d)** 0 to 1023

Q4 Which pins on the Uno R3 are analog inputs?

(a) 0–5 **(b)** A0–A5
(c) 3, 5, 6, 9, 10, 11 **(d)** 13 only

Q5 An LDR's resistance _____ as the light gets brighter.

(a) increases **(b)** decreases **(c)** stays the same **(d)** becomes zero

Q6 An LM35 outputs 0.25 V. What is the temperature?

(a) 2.5 °C **(b)** 250 °C **(c)** 0.25 °C **(d)** 25 °C

Q7 Which function changes a 0–1023 reading into 0–100?

(a) map(raw, 0, 1023, 0, 100) **(b)** constrain(raw, 0, 100)
(c) analogWrite(raw, 100) **(d)** random(0, 100)

Q8 What is the ADC resolution of the ESP32?

(a) 10 bits (0–1023) **(b)** 8 bits (0–255)
(c) 12 bits (0–4095) **(d)** 16 bits

8.5.2 Intermediate level

Q9 Read a potentiometer on A0 and print the value as a percentage (0–100 %) every half second.

Q10 A TMP36 outputs 0.75 V. What temperature is that?

(a) 75 °C **(b)** 7.5 °C **(c)** -25 °C **(d)** 25 °C

Q11 What is the voltage step (resolution) of a 10-bit ADC with a 5 V reference?

(a) about 5 mV × 1023 **(b)** about 4.9 mV
(c) about 1 V **(d)** about 20 mV

Q12 Why does "float v = raw * 5 / 1023;" give only whole numbers?

(a) the calculation is done in integers before being stored in the float
(b) floats cannot hold decimals
(c) analogRead returns a float
(d) 1023 is too large

Q13 Read an LM35 on A2 and print the temperature in °C and °F with one decimal place.

Q14 With the LDR from 5 V to A0 and 10 kΩ from A0 to GND, what happens to the reading as the room gets darker?

(a) it increases **(b)** it stays at 1023
(c) it decreases **(d)** it becomes negative

Q15 Turn on an LED on pin 9 when it gets dark: LDR divider on A0 (reading falls as it gets darker); threshold 300.

Q16 What does analogReference(INTERNAL) do on an Uno R3?

(a) uses the internal 1.1 V reference for finer resolution on small signals
(b) connects A0 to GND
(c) doubles the ADC resolution to 20 bits
(d) sets the reference to 12 V

8.5.3 Advanced level

Q17 Smooth a noisy sensor on A0 by averaging 10 readings, and print the raw first reading and the average.

Q18 A soil sensor reads 820 in dry air and 380 in water. Which map() call gives moisture 0–100 %?

(a) map(raw, 0, 1023, 0, 100) **(b)** map(raw, 820, 380, 0, 100)
(c) map(raw, 380, 820, 0, 100) **(d)** map(raw, 0, 100, 820, 380)

Q19 An ESP32 analog pin reads 2048 (12-bit, 3.3 V reference). What is the voltage?

(a) about 2.5 V **(b)** about 3.3 V **(c)** about 0.8 V **(d)** about 1.65 V

Q20 Use a potentiometer on A0 to set a blink interval between 100 ms and 1000 ms for the built-in LED.

Q21 Why should a 9 V signal never be connected directly to an Uno analog pin?

(a) the reading would simply be 1023 with no harm
(b) analog pins accept up to 12 V
(c) pins accept at most 5 V (the reference); higher voltage can damage the chip
(d) it would give negative readings

Q22 Measure a battery of up to 10 V using a divider of two equal 10 kΩ resistors into A0, and print the battery voltage.

Q23 A sensor reading jumps between 498 and 506. Which technique reduces this noise?

(a) increasing the delay only **(b)** using digitalRead instead
(c) averaging several readings **(d)** removing the ground wire

Q24 Implement an exponential moving average: avg += (new - avg) / 4, and print raw and smoothed values.

Q25 A flex sensor (25 kΩ flat, 100 kΩ bent) is the top resistor of a divider with 47 kΩ to GND from 5 V. What is the voltage when flat?

(a) about 1.60 V **(b)** about 2.50 V **(c)** about 5.00 V **(d)** about 3.26 V

8.6 Solutions

Q1 **Answer: (c) 1024**

2^{10} = **1024** values (0–1023).

Q2 **Answer: (a) about 2.5 V**

512 × 5 / 1023 ≈ **2.50 V**.

Q3 **Answer: (d) 0 to 1023**

A 10-bit range: **0–1023**.

Q4 **Answer: (b) A0–A5**

A0 to A5.

Q5 **Answer: (b) decreases**

More light → **lower resistance**.

Q6 **Answer: (d) 25 °C**

T = V × 100 = **25 °C**.

Q7 **Answer: (a) map(raw, 0, 1023, 0, 100)**

map() rescales one range to another.

Q8 **Answer: (c) 12 bits (0–4095)**

ESP32 has a **12-bit** ADC.

Q9 **Answer: Solution program**

map() converts the 0–1023 reading into 0–100. The script turned the knob to 0, 256, 768 and 1023.

```
void setup() { Serial.begin(9600); }
void loop() {
  int pct = map(analogRead(A0), 0, 1023, 0, 100);
  Serial.print("Knob: "); Serial.print(pct); Serial.println("%");
  delay(500);
}
```

SERIAL MONITOR

```
Knob: 0%
Knob: 25%
Knob: 75%
Knob: 100%
```

Q10 **Answer: (d) 25 °C**

T = (0.75 - 0.5) × 100 = **25 °C**.

Q11 **Answer: (b) about 4.9 mV**

5 / 1024 ≈ **4.88 mV**.

Q12 **Answer: (a) the calculation is done in integers before being stored in the float**

Integer arithmetic happens first; write **5.0** to force floating point.

Q13 **Answer: Solution program**

LM35: 10 mV per °C, so T = V × 100; then F = C × 9/5 + 32. The scripted readings were 51 and 72.

```
void setup() { Serial.begin(9600); }
void loop() {
  float v = analogRead(A2) * 5.0 / 1023.0;
  float c = v * 100.0;
  float f = c * 9.0 / 5.0 + 32.0;
  Serial.print(c, 1); Serial.print(" C = ");
  Serial.print(f, 1); Serial.println(" F");
  delay(1000);
}
```

SERIAL MONITOR

```
24.9 C = 76.9 F
35.2 C = 95.3 F
```

Q14 **Answer: (c) it decreases**

Darker → LDR resistance rises → less voltage reaches A0: the reading **falls**.

Q15 **Answer: Solution program**

Compare the reading with a threshold. In practice choose the threshold by printing readings at dusk.

```
const int LDR = A0, LED = 9;
void setup() {
  Serial.begin(9600);
  pinMode(LED, OUTPUT);
}
void loop() {
  int light = analogRead(LDR);
  bool dark = light < 300;
  digitalWrite(LED, dark ? HIGH : LOW);
  Serial.print(light); Serial.println(dark ? "  dark -> LED on" : "  bright -> LED off");
  delay(500);
}
```

SERIAL MONITOR

```
700  bright -> LED off
250  dark -> LED on
600  bright -> LED off
```

Q16 **Answer: (a) uses the internal 1.1 V reference for finer resolution on small signals**

It selects the **1.1 V** internal reference.

Q17 **Answer: Solution program**

Summing into a long avoids overflow (10 × 1023 fits in an int, but larger counts would not on an Uno R3).

```
void setup() {
  Serial.begin(9600);
  long sum = 0;
  int first = 0;
  for (int i = 0; i < 10; i++) {
    int r = analogRead(A0);
    if (i == 0) first = r;
    sum += r;
  }
  Serial.print("first = "); Serial.print(first);
  Serial.print("  average = "); Serial.println(sum / 10);
}
void loop() {}
```

SERIAL MONITOR

```
first = 540  average = 507
```

Q18 **Answer: (b) map(raw, 820, 380, 0, 100)**

Dry (820) → 0 %, wet (380) → 100 %: **map(raw, 820, 380, 0, 100)**; map() handles reversed ranges.

Q19 **Answer: (d) about 1.65 V**

2048 / 4095 × 3.3 ≈ **1.65 V**.

Q20 **Answer: Solution program**

The knob position, mapped to 100–1000, becomes the delay. Turning it changes the blink speed.

```
void setup() {
  Serial.begin(9600);
  pinMode(LED_BUILTIN, OUTPUT);
}
void loop() {
  int interval = map(analogRead(A0), 0, 1023, 100, 1000);
  Serial.print("interval "); Serial.println(interval);
  digitalWrite(LED_BUILTIN, HIGH); delay(interval);
  digitalWrite(LED_BUILTIN, LOW);  delay(interval);
}
```

SERIAL MONITOR

```
interval 100
interval 550
interval 1000
```

Q21 **Answer: (c) pins accept at most 5 V (the reference); higher voltage can damage the chip**

Use a **voltage divider** to bring it below 5 V.

Q22 **Answer: Solution program**

The divider halves the battery voltage so it stays under 5 V; multiply back by 2. Readings 920 and 742 correspond to about 9.0 V and 7.3 V.

```
void setup() { Serial.begin(9600); }
void loop() {
  float vPin = analogRead(A0) * 5.0 / 1023.0;
  float vBat = vPin * 2.0;            // equal resistors halve the voltage
  Serial.print("Battery: "); Serial.print(vBat, 2); Serial.println(" V");
  delay(1000);
}
```

SERIAL MONITOR

```
Battery: 8.99 V
Battery: 7.25 V
```

Q23 **Answer: (c) averaging several readings**

Averaging (or filtering) smooths random noise.

Q24 **Answer: Solution program**

Each new reading moves the average a quarter of the way towards it, so spikes are softened but the value still follows real changes.

```
float avg = 0;
void setup() {
  Serial.begin(9600);
  avg = analogRead(A0);
}
void loop() {
  int raw = analogRead(A0);
  avg += (raw - avg) / 4.0;
  Serial.print(raw); Serial.print("  ->  "); Serial.println(avg, 1);
  delay(100);
}
```

SERIAL MONITOR

```
500  ->  500.0
600  ->  525.0
500  ->  518.8
500  ->  514.1
500  ->  510.5
```

Q25 **Answer: (d) about 3.26 V**

V = 5 × 47 / (25 + 47) ≈ **3.26 V** (bent: 5 × 47/147 ≈ 1.60 V).

A 10-bit ADC gives 0–1023; voltage = reading × Vref / 1023 (use 5.0 for float maths). map() rescales ranges. Resistive sensors (LDR, flex, FSR, thermistor) need a voltage divider. LM35: T = V × 100; TMP36: T = (V - 0.5) × 100. Average readings to reduce noise, and never exceed the reference voltage on an analog pin.

MODULE 09 · BEGINNER

PWM and analog output

Minimum duration **1 day** | Level **Beginner**

Most Arduino boards cannot output a true in-between voltage; instead they switch a pin on and off very fast. This module covers pulse-width modulation (duty cycle and frequency), the PWM pins on each board and analogWrite(), LED fading, potentiometer brightness control, RGB colour mixing, an introduction to motor speed control, and true analog output with the DAC on the Uno R4, followed by 25 solved questions and programs.

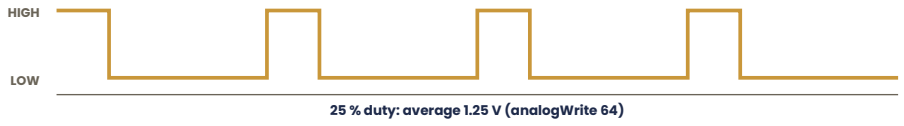
IN THIS MODULE

9.1 Pulse-width modulation
9.2 PWM projects
9.3 True analog output
9.4 Practice questions
9.5 Solutions

9.1 Pulse-width modulation

9.1.1 Duty cycle and frequency

PWM switches a pin between 0 V and 5 V many times a second. The **duty cycle** is the percentage of each period that the pin is HIGH. The average voltage is duty × 5 V, and an LED or motor responds to that average.



25 % DUTY CYCLE



75 % DUTY CYCLE

analogWrite value = duty × 255. Average voltage = value / 255 × 5 V. 0 = always off, 127 ≈ 50 %, 255 = always on.

9.1.2 PWM pins on each board

Board	PWM pins	Default frequency
Uno R3 / Nano	3, 5, 6, 9, 10, 11	490 Hz (5 and 6: 980 Hz)
Mega 2560	2–13 and 44–46	490 Hz (4 and 13: 980 Hz)
Uno R4	3, 5, 6, 9, 10, 11	490 Hz
ESP32	almost any output pin (LEDC)	configurable

9.1.3 analogWrite() (0 to 255)

`analogWrite(pin, value)` starts PWM on a ~ pin; no pinMode() is needed. The value is 0–255 (8-bit). The pin keeps producing PWM until you call analogWrite or digitalWrite again.

> **WATCH OUT**
> analogWrite() on a pin without PWM gives just HIGH (≥128) or LOW (<128). Check the ~ mark.

9.2 PWM projects

9.2.1 LED fading

FADE.INO

```
const int LED = 9;              // a PWM (~) pin

void setup() {}

void loop() {
  for (int b = 0; b <= 255; b += 51) {   // brighter
    analogWrite(LED, b);
    delay(30);
  }
  for (int b = 255; b >= 0; b -= 51) {   // dimmer
    analogWrite(LED, b);
    delay(30);
  }
}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] PWM pin 9 = 0
[    30 ms] PWM pin 9 = 51
[    60 ms] PWM pin 9 = 102
[    90 ms] PWM pin 9 = 153
[   120 ms] PWM pin 9 = 204
[   150 ms] PWM pin 9 = 255
[   180 ms] PWM pin 9 = 255
[   210 ms] PWM pin 9 = 204
[   240 ms] PWM pin 9 = 153
[   270 ms] PWM pin 9 = 102
[   300 ms] PWM pin 9 = 51
[   330 ms] PWM pin 9 = 0
```

Smaller steps (b++ with delay(5)) give a smoother fade; steps of 51 keep the simulated log short.

SHORTCUT TRICK

The eye responds to brightness roughly logarithmically, so a linear fade seems to change mostly at the dark end. For a smoother look, use a gamma table or square the value: analogWrite(LED, b * b / 255).

9.2.2 Brightness control with a potentiometer

DIMMER.INO

```
void setup() {
  Serial.begin(9600);
}

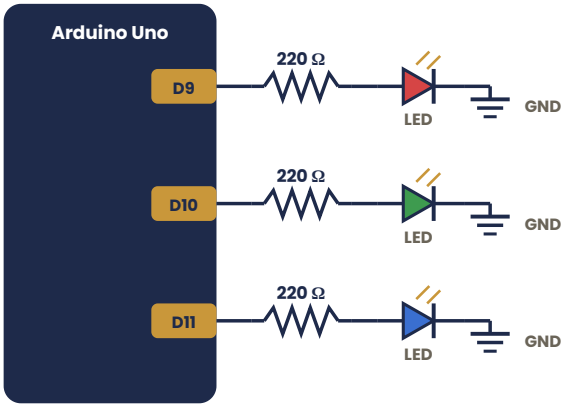
void loop() {
  int raw = analogRead(A0);          // 0–1023
  int level = raw / 4;               // 0–255 (same as map(raw, 0, 1023, 0, 255))
  analogWrite(9, level);
  Serial.print(raw); Serial.print(" -> "); Serial.println(level);
  delay(100);
}
```

SERIAL MONITOR

```
0 -> 0
512 -> 128
1023 -> 255
```

9.2.3 RGB LED colour mixing

An RGB LED contains red, green and blue LEDs. A common-cathode type has one shared GND leg; PWM on each colour (each with its own resistor) mixes any colour. For a **common-anode** LED the shared leg goes to 5 V and the values are inverted (255 - value).



COMMON-CATHODE RGB LED ON PWM PINS 9, 10, 11

RGB.INO

```
const int R = 9, G = 10, B = 11;

void setColor(int r, int g, int b) {
  analogWrite(R, r); analogWrite(G, g); analogWrite(B, b);
}

void setup() {}

void loop() {
  setColor(255, 0, 0);   delay(1000);   // red
  setColor(255, 255, 0); delay(1000);   // yellow = red + green
  setColor(0, 255, 255); delay(1000);   // cyan = green + blue
  setColor(128, 0, 128); delay(1000);   // purple
}
```

Colour	R	G	B
red	255	0	0
green	0	255	0
blue	0	0	255
yellow	255	255	0
cyan	0	255	255
magenta	255	0	255
white	255	255	255
orange	255	100	0

9.2.4 Motor speed control (introduction)

A DC motor driven through a transistor or motor driver from a PWM pin runs faster as the duty cycle rises. Module 16 covers motor drivers, direction and safety in detail.

9.3 True analog output

9.3.1 DAC on Uno R4 and other boards

A digital-to-analog converter (DAC) produces a real, steady voltage. The Uno R4 has a 12-bit DAC on pin A0; the ESP32 has two 8-bit DACs (GPIO 25 and 26); the Arduino Zero and Due also have DACs. Use them for audio and waveform generation, where PWM would need heavy filtering.

UNO R4 DAC (FRAGMENT)

```
analogWriteResolution(12);   // Uno R4: 0–4095
analogWrite(A0, 2048);       // about half of the reference voltage, as a steady voltage
```

On a PWM-only board, an RC low-pass filter (for example 10 kΩ and 10 µF) smooths PWM into an approximately steady voltage.

9.4 Practice questions

9.4.1 Basic level

Q1 What does PWM stand for?

(a) power wave management
(b) pulse-width modulation
(c) pin write mode
(d) pulse wave memory

Q2 What value range does analogWrite() accept on an Uno?

(a) 0 to 255 **(b)** 0 to 1023 **(c)** 0 to 5 **(d)** 0 to 100

Q3 What is the duty cycle when analogWrite(pin, 127) is used?

(a) about 25 % **(b)** about 50 % **(c)** 100 % **(d)** 12.7 %

Q4 Which of these Uno pins supports PWM?

(a) 9 **(b)** 7 **(c)** 8 **(d)** 13

Q5 What average voltage does a 50 % duty cycle give on a 5 V board?

(a) 5 V **(b)** 1.25 V **(c)** 2.5 V **(d)** 0 V

Q6 Does analogWrite() need pinMode(pin, OUTPUT) first?

(a) Yes, always
(b) Only on pin 13
(c) Only for values above 128
(d) No, analogWrite() sets the pin up itself

Q7 Which analogWrite value turns the output fully on?

(a) 1023 **(b)** 100 **(c)** 256 **(d)** 255

Q8 Which colours mix to make yellow on an RGB LED?

(a) red and blue **(b)** green and blue **(c)** red and green **(d)** blue only

9.4.2 Intermediate level

Q9 Fade the LED on pin 6 from off to full brightness in 5 equal steps, 200 ms apart, and then switch it off.

Q10 What analogWrite value gives an average of 1 V on a 5 V board?

(a) 51 **(b)** 20 **(c)** 102 **(d)** 205

Q11 Why does a potentiometer reading need to be divided by 4 before analogWrite()?

(a) to reduce noise
(b) to protect the LED
(c) because of the 4 analog pins
(d) analogRead gives 0–1023 but analogWrite accepts 0–255

Q12 What is the default PWM frequency on pins 5 and 6 of the Uno R3?

(a) about 490 Hz **(b)** about 980 Hz **(c)** 50 Hz **(d)** 16 MHz

Q13 Control the brightness of an LED on pin 9 with a potentiometer on A0, using map().

Q14 For a common-anode RGB LED, which value gives full red?

(a) analogWrite(redPin, 255) **(b)** analogWrite(redPin, 0)
(c) analogWrite(redPin, 127) **(d)** digitalWrite(redPin, HIGH)

Q15 Write a function setColor(r, g, b) for a common-cathode RGB LED on pins 9, 10, 11 and show white, then off.

Q16 What happens if analogWrite(7, 200) is used on an Uno R3 (pin 7 has no PWM)?

(a) the pin outputs PWM anyway
(b) the board resets
(c) the pin goes LOW
(d) the pin simply goes HIGH, because 200 ≥ 128

9.4.3 Advanced level

Q17 Make a "breathing" LED on pin 9: brightness rises and falls smoothly using a gamma-like curve (value² / 255). Print the PWM values for one rise in steps of 64.

Q18 A 12-bit DAC with a 5 V reference is given the value 1024. Approximately what voltage does it output?

(a) 1.25 V **(b)** 1.024 V **(c)** 2.5 V **(d)** 5 V

Q19 Which board has a true DAC output on pin A0?

(a) Uno R3 **(b)** Nano (classic) **(c)** Uno R4 **(d)** Mega 2560

Q20 Cycle an RGB LED through red, green and blue, each for 500 ms, using arrays of colour values.

Q21 Why is PWM effective for dimming an LED even though the LED is only ever fully on or off?

(a) the LED voltage really changes
(b) PWM changes the LED colour
(c) the resistor adjusts automatically
(d) the switching is too fast for the eye to see, so we perceive the average brightness

Q22 Simulate a candle flicker on pin 9 by writing random brightness values between 120 and 255 every 50 ms; print the first 5 values.

Q23 A motor driven by PWM through a transistor runs slowly at analogWrite 60. What happens at 200?

(a) it stops
(b) it runs faster, because the average voltage is higher
(c) it reverses
(d) nothing changes

Q24 An RC filter of 10 kΩ and 10 µF smooths PWM. What is its time constant?

(a) 0.1 s **(b)** 1 s **(c)** 0.01 s **(d)** 10 s

Q25 Use two buttons (pins 2 and 3, INPUT_PULLUP) to step the brightness of an LED on pin 9 up or down by 51, staying within 0–255.

9.5 Solutions

Q1 **Answer: (b) pulse-width modulation**

Pulse-width modulation.

Q2 **Answer: (a) 0 to 255**

An 8-bit value: **0–255**.

Q3 **Answer: (b) about 50 %**

127 / 255 ≈ **50 %**.

Q4 **Answer: (a) 9**

PWM pins are 3, 5, 6, **9**, 10, 11.

Q5 **Answer: (c) 2.5 V**

0.5 × 5 = **2.5 V**.

Q6 **Answer: (d) No, analogWrite() sets the pin up itself**

analogWrite() **configures the pin** automatically.

Q7 **Answer: (d) 255**

255 is 100 % duty.

Q8 **Answer: (c) red and green**

Additive mixing: **red + green = yellow**.

Q9 **Answer: Solution program**

255 / 5 = 51, so the steps are 51, 102, 153, 204, 255.

```
void setup() {}
void loop() {
  for (int i = 1; i <= 5; i++) {
    analogWrite(6, i * 51);
    delay(200);
  }
  analogWrite(6, 0);
  delay(200);
}
```

SIMULATED RUN

```
[     0 ms] PWM pin 6 = 51
[   200 ms] PWM pin 6 = 102
[   400 ms] PWM pin 6 = 153
[   600 ms] PWM pin 6 = 204
```

```
[   800 ms] PWM pin 6 = 255
[  1000 ms] PWM pin 6 = 0
```

Q10 **Answer: (a) 51**

1 / 5 × 255 = **51**.

Q11 **Answer: (d) analogRead gives 0–1023 but analogWrite accepts 0–255**

1024 / 256 = 4.

Q12 **Answer: (b) about 980 Hz**

Pins 5 and 6 run at about **980 Hz**; the others at 490 Hz.

Q13 **Answer: Solution program**

map() converts the reading to 0–255; the run used knob positions 0, 300 and 1023.

```
void setup() { Serial.begin(9600); }
void loop() {
  int level = map(analogRead(A0), 0, 1023, 0, 255);
  analogWrite(9, level);
  Serial.println(level);
  delay(50);
}
```

SERIAL MONITOR

```
0
74
255
```

Q14 **Answer: (b) analogWrite(redPin, 0)**

Common anode is active-low: **0** = fully on.

Q15 **Answer: Solution program**

All three colours at full brightness make white.

```
void setColor(int r, int g, int b) {
  analogWrite(9, r); analogWrite(10, g); analogWrite(11, b);
}
void setup() {
  setColor(255, 255, 255);   // white
  delay(1000);
  setColor(0, 0, 0);         // off
}
void loop() {}
```

SIMULATED RUN

```
[     0 ms] PWM pin 9 = 255
[     0 ms] PWM pin 10 = 255
```

```
[     0 ms] PWM pin 11 = 255
[  1000 ms] PWM pin 9 = 0
[  1000 ms] PWM pin 10 = 0
[  1000 ms] PWM pin 11 = 0
```

Q16 **Answer: (d) the pin simply goes HIGH, because 200 ≥ 128**

Non-PWM pins output **HIGH for values ≥ 128**, LOW otherwise.

Q17 **Answer: Solution program**

Squaring makes the low end change slowly and the high end quickly, matching the eye's response. The cast to long avoids overflow of v × v on an Uno R3 (255 × 255 = 65 025 > 32 767).

```
void setup() {
  Serial.begin(9600);
  for (int b = 0; b <= 256; b += 64) {
    int v = min(b, 255);
    int out = (long)v * v / 255;
    analogWrite(9, out);
    Serial.print(v); Serial.print(" -> "); Serial.println(out);
  }
}
void loop() {}
```

SERIAL MONITOR

```
0 -> 0
64 -> 16
128 -> 64
192 -> 144
255 -> 255
```

Q18 **Answer: (a) 1.25 V**

1024 / 4096 × 5 = **1.25 V**.

Q19 **Answer: (c) Uno R4**

The **Uno R4** has a 12-bit DAC on A0.

Q20 **Answer: Solution program**

A 2-D array stores one row per colour; the inner loop writes the three channels.

```
const int pins[3] = {9, 10, 11};
const byte colors[3][3] = {{255, 0, 0}, {0, 255, 0}, {0, 0, 255}};
void setup() {}
void loop() {
  for (int c = 0; c < 3; c++) {
    for (int k = 0; k < 3; k++) analogWrite(pins[k], colors[c][k]);
    delay(500);
  }
}
```

SIMULATED RUN

```
[     0 ms] PWM pin 9 = 255
[     0 ms] PWM pin 10 = 0
[     0 ms] PWM pin 11 = 0
[   500 ms] PWM pin 9 = 0
[   500 ms] PWM pin 10 = 255
[   500 ms] PWM pin 11 = 0
[  1000 ms] PWM pin 9 = 0
[  1000 ms] PWM pin 10 = 0
[  1000 ms] PWM pin 11 = 255
```

Q21 **Answer: (d) the switching is too fast for the eye to see, so we perceive the average brightness**

At hundreds of hertz the eye **averages** the light.

Q22 **Answer: Solution program**

random(120, 256) gives 120–255. On real hardware, seed with analogRead of an unconnected pin to get a different sequence each time.

```
int shown = 0;
void setup() {
  Serial.begin(9600);
  randomSeed(7);
}
void loop() {
  int v = random(120, 256);
  analogWrite(9, v);
  if (shown < 5) { Serial.println(v); shown++; }
  delay(50);
}
```

SERIAL MONITOR

```
229
247
120
221
176
```

Q23 **Answer: (b) it runs faster, because the average voltage is higher**

Higher duty → higher average voltage → **faster**.

Q24 **Answer: (a) 0.1 s**

$\tau = RC = 10\,000 \times 0.000\,01 =$ **0.1 s**.

Q25 **Answer: Solution program**

Edge detection gives one step per press; constrain() keeps the level in range. The script pressed "up" three times and "down" once.

```
int level = 0;
int last2 = HIGH, last3 = HIGH;
void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP); pinMode(3, INPUT_PULLUP);
}
void loop() {
  int b2 = digitalRead(2), b3 = digitalRead(3);
  if (last2 == HIGH && b2 == LOW) level = constrain(level + 51, 0, 255);
  if (last3 == HIGH && b3 == LOW) level = constrain(level - 51, 0, 255);
  if (b2 != last2 || b3 != last3) { analogWrite(9, level); Serial.println(level); delay(20); }
  last2 = b2; last3 = b3;
}
```

SERIAL MONITOR

```
51
51
102
102
153
153
102
102
```

PWM switches a pin fast; duty cycle = value / 255 and average voltage = duty × Vcc. Use analogWrite(pin, 0–255) only on ~ pins (3, 5, 6, 9, 10, 11 on the Uno R3). Divide analogRead by 4 (or map) before analogWrite. RGB LEDs mix colours with three PWM channels (invert for common anode). True analog output needs a DAC (Uno R4 A0, ESP32 GPIO 25/26).

LEVEL 2

Intermediate level

Use timing without delay, interrupts, PWM and motors, I2C, SPI and UART devices, displays, libraries, data storage, power management and structured multi-file code

10 MODULES · 18 DAYS MINIMUM

Module 10 · Serial communication and debugging	1 day
Module 11 · Functions, arrays, strings and structures	2 days
Module 12 · Timing and non-blocking code	1 day
Module 13 · Sensors	3 days
Module 14 · Displays	2 days
Module 15 · Sound and user-input devices	2 days
Module 16 · Motors and motion control	2 days
Module 17 · Libraries	1 day
Module 18 · Communication protocols in depth	2 days
Module 19 · Data storage and timekeeping	2 days

MODULE 10 · INTERMEDIATE

Serial communication and debugging

Minimum duration **1 day** | Level **Intermediate**

The serial link between the Arduino and the computer is your window into a running sketch. This module covers Serial.begin() and baud rates, print and println, formatted output, reading input with available(), read(), parseInt() and readStringUntil(), controlling the board from the computer, live graphs with the Serial Plotter, and systematic debugging of hardware and software problems, followed by 25 solved questions and programs run with scripted serial input.

IN THIS MODULE

10.1 Serial basics
10.2 Reading serial input
10.3 Serial Plotter
10.4 Debugging techniques
10.5 Practice questions
10.6 Solutions

10.1 Serial basics

10.1.1 Serial.begin() and baud rates

`Serial.begin(9600)` starts the UART at 9600 bits per second. Common rates: 9600 (safe default), 115200 (fast; standard on ESP32). Each character takes about 10 bits (start + 8 data + stop), so 9600 baud carries about 960 characters per second.

Characters per second ≈ baud / 10. At 9600 baud, a 50-character line takes about 52 ms to send; at 115200 baud, about 4 ms.

10.1.2 Serial.print() and Serial.println()

`print()` sends text or a number; `println()` does the same and adds a new line. Numbers can be printed in other bases and floats with a chosen number of decimals.

10.1.3 Printing formatted values

FORMATTING.INO

```
void setup() {
  Serial.begin(9600);
  int n = 200;
  float t = 23.456;
  Serial.println(n);            // 200
  Serial.println(n, HEX);       // C8
  Serial.println(n, BIN);       // 11001000
  Serial.println(t);            // 2 decimals by default
  Serial.println(t, 1);         // 1 decimal
  Serial.print("Temp: "); Serial.print(t, 1); Serial.println(" C");
  char buf[32];
  snprintf(buf, sizeof buf, "x=%d y=%d", 12, -5);   // printf-style into a buffer
  Serial.println(buf);
}
void loop() {}
```

SERIAL MONITOR

```
200
C8
11001000
23.46
23.5
Temp: 23.5 C
x=12 y=-5
```

WATCH OUT

On AVR boards (Uno R3), snprintf() does not support %f for floats. Use dtostrf() or print the float separately.

10.2 Reading serial input

10.2.1 Serial.available() and Serial.read()

`Serial.available()` returns how many bytes are waiting; `Serial.read()` returns the next byte (or -1 if none). Always check available() before reading.

SERIALLED.INO

```
void setup() {
  Serial.begin(9600);
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  if (Serial.available() > 0) {
    char c = Serial.read();
    if (c == '1') { digitalWrite(LED_BUILTIN, HIGH); Serial.println("LED on"); }
    else if (c == '0') { digitalWrite(LED_BUILTIN, LOW); Serial.println("LED off"); }
  }
}
```

SERIAL MONITOR

```
LED on
LED off
LED on
```

The newline characters sent by the Serial Monitor are read too, but ignored because they are neither "1" nor "0".

10.2.2 Serial.parseInt() and Serial.readStringUntil()

COMMANDS.INO

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  if (Serial.available()) {
    String line = Serial.readStringUntil('\n');   // one full command line
    line.trim();
    if (line.startsWith("BRIGHT ")) {
      int v = line.substring(7).toInt();
      v = constrain(v, 0, 255);
      analogWrite(9, v);
      Serial.print("Brightness set to "); Serial.println(v);
    } else {
      Serial.print("Unknown: "); Serial.println(line);
    }
  }
}
```

SERIAL MONITOR

```
Brightness set to 128
Brightness set to 255
Unknown: HELLO
```

10.2.3 Controlling the Arduino from the computer

Choose a simple text protocol: one command per line, a keyword and optional numbers ("LED 1", "SERVO 90"). Set the Serial Monitor line ending to "Newline". The same protocol works from Python (pyserial), a phone app over Bluetooth, or another microcontroller.

10.3 Serial Plotter

10.3.1 Graphing sensor data live

Tools → Serial Plotter draws numbers as live graphs. Print one line per sample; separate several values with spaces or commas, optionally labelled "name:value".

PLOTTER.INO

```
void setup() { Serial.begin(9600); }

void loop() {
  int light = analogRead(A0);
  int knob = analogRead(A1);
  Serial.print("light:"); Serial.print(light);
  Serial.print(",knob:");  Serial.println(knob);
  delay(50);
}
```

SERIAL MONITOR

```
light:300,knob:1000
light:320,knob:900
light:340,knob:800
```

10.4 Debugging techniques

10.4.1 Print debugging

Print the values of key variables and messages such as "entered checkSensor()" to follow what the sketch does. Wrap debug prints in a macro so they can be switched off in one place.

DEBUGMACRO.INO

```
#define DEBUG 1
#if DEBUG
  #define DBG(x) Serial.println(x)
#else
  #define DBG(x)
#endif

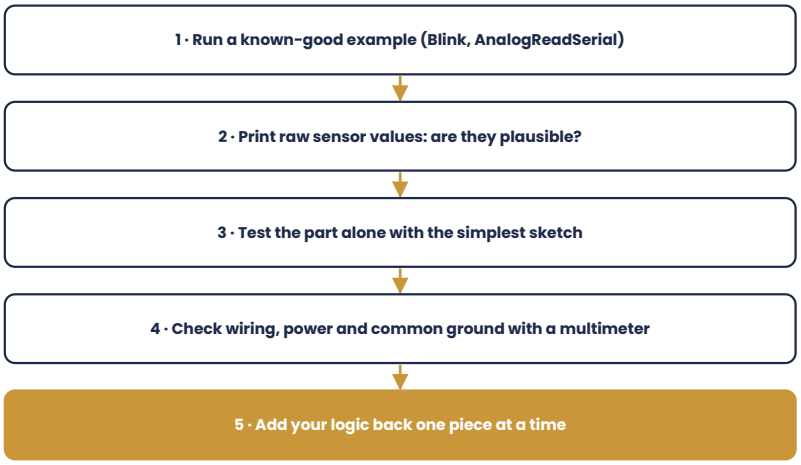
void setup() {
  Serial.begin(9600);
  DBG("setup done");
}

void loop() {
  int v = analogRead(A0);
  DBG(v);
  delay(1000);
}
```

SERIAL MONITOR

```
setup done
412
415
```

10.4.2 Isolating hardware vs software problems



DIVIDE AND CONQUER

10.4.3 Common errors and compiler messages

Message	Meaning and fix
expected ';' before …	missing semicolon on the line above
'x' was not declared in this scope	misspelt name, missing #include, or variable defined in another function
expected '}' at end of input	a brace is not closed
invalid conversion from …	wrong type passed to a function
Sketch too big	program exceeds flash: remove features, use smaller libraries
Low memory available, stability problems may occur	SRAM nearly full: use F("…"), smaller buffers, avoid String

10.5 Practice questions

10.5.1 Basic level

Q1 Which function starts serial communication at 9600 baud?

(a) Serial.start(9600);
(b) Serial.print(9600);
(c) Serial.begin(9600);
(d) Serial.baud(9600);

Q2 What is the difference between Serial.print() and Serial.println()?

(a) println() adds a new line after the output
(b) println() prints faster
(c) print() works only with numbers
(d) there is no difference

Q3 What does Serial.println(255, HEX) print?

(a) 255 **(b)** FF **(c)** 11111111 **(d)** 0xFF

Q4 What does Serial.available() return?

(a) true if the USB cable is connected
(b) the baud rate
(c) the next character
(d) the number of bytes waiting to be read

Q5 What does Serial.read() return when no data is available?

(a) 0
(b) '\n'
(c) -1
(d) it waits for ever

Q6 Which IDE tool draws live graphs of printed numbers?

(a) Library Manager
(b) Boards Manager
(c) Serial Plotter
(d) Board Info

Q7 What does Serial.println(3.14159, 3) print?

(a) 3.14 **(b)** 3.14159 **(c)** 3 **(d)** 3.142

Q8 About how many characters per second can be sent at 9600 baud?

(a) 9600 **(b)** 960 **(c)** 96 **(d)** 1200

10.5.2 Intermediate level

Q9 Read single characters: "r" prints "Red", "g" prints "Green", anything else (except newline) prints "?".

Q10 What does Serial.parseInt() do with the input "abc42xyz"?

(a) skips non-digits and returns 42
(b) returns 0
(c) returns "abc"
(d) causes an error

Q11 Why might "Serial.println(\"Temperature reading complete\")" use F("...") on an Uno R3?

(a) to print faster
(b) to keep the text in flash instead of copying it into the small SRAM
(c) to print in bold
(d) F() means "format"

Q12 Read a number typed in the Serial Monitor and print its square.

Q13 Your sketch prints garbage characters. Serial.begin(115200) is used and the monitor is at 9600. What is the fix?

(a) set the Serial Monitor to 115200 baud
(b) change the USB cable
(c) add delay(1000)
(d) use println instead of print

Q14 What format should a sketch print for the Serial Plotter to show two labelled lines?

(a) temp=23 hum=60 on one line only once
(b) two separate sketches
(c) temp:23,hum:60 on each line
(d) binary data only

Q15 Write a command parser: "ADD a b" prints the sum, "MUL a b" prints the product.

Q16 What does the compiler error "expected ';' before '}' token" usually mean?

(a) a library is missing
(b) the board is not connected
(c) there is an extra semicolon
(d) a semicolon is missing at the end of the previous statement

10.5.3 Advanced level

Q17 A sensor sketch gives readings of 0 all the time. Which step best separates a hardware fault from a software fault?

(a) rewrite the whole program
(b) change the baud rate
(c) add more libraries
(d) run a minimal sketch that only prints the raw reading, and check the wiring with a multimeter

Q18 Print a table of the numbers 0 to 15 in decimal, hexadecimal and binary, separated by tabs.

Q19 How long does it take to send a 100-character line at 9600 baud?

(a) about 10 ms
(b) about 1 s
(c) about 104 ms
(d) about 1 ms

Q20 A "set point" command: typing "SET 30" stores 30 as a temperature limit; "GET" prints the current limit (initially 25).

Q21 Why does printing a lot of debug text inside a fast loop slow the sketch down?

(a) Serial.print waits when the transmit buffer is full, so the loop runs at the speed of the serial link
(b) printing uses EEPROM
(c) it lowers the clock speed
(d) it has no effect

Q22 Echo back each received line in upper case, with its length.

Q23 Which is a valid reason to use 115200 baud instead of 9600?

(a) to save battery
(b) to make text bigger
(c) because 9600 is not supported
(d) to send large amounts of data quickly (e.g. fast sensor logging)

Q24 The message "Low memory available, stability problems may occur" appears after compiling. What should you do first?

(a) buy a bigger USB cable

(b) reduce SRAM use: move string constants into flash with F() and shrink buffers

(c) increase the baud rate

(d) ignore it, it is harmless

Q25 Use a DEBUG flag macro: when DEBUG is 1, print each step of computing the average of three readings; when 0, print only the result.

10.6 Solutions

Q1 **Answer: (c) Serial.begin(9600);**

Serial.begin(9600).

Q2 **Answer: (a) println() adds a new line after the output**

println = print + **new line**.

Q3 **Answer: (b) FF**

255 in hexadecimal is **FF**.

Q4 **Answer: (d) the number of bytes waiting to be read**

It returns the **number of bytes** in the receive buffer.

Q5 **Answer: (c) -1**

It returns **-1**.

Q6 **Answer: (c) Serial Plotter**

The **Serial Plotter**.

Q7 **Answer: (d) 3.142**

Three decimal places, rounded: **3.142**.

Q8 **Answer: (b) 960**

About 10 bits per character: 9600 / 10 = **960**.

Q9 **Answer: Solution program**

Line-ending characters are skipped so that each key press gives exactly one reply. Scripted input: r, g, x.

```
void setup() { Serial.begin(9600); }
void loop() {
  if (Serial.available()) {
    char c = Serial.read();
    if (c == '\n' || c == '\r') return;   // ignore line endings
    if (c == 'r') Serial.println("Red");
    else if (c == 'g') Serial.println("Green");
    else Serial.println("?");
  }
}
```

SERIAL MONITOR

```
Red
Green
?
```

Q10 **Answer: (a) skips non-digits and returns 42**

It skips leading non-numeric characters and reads **42**.

Q11 **Answer: (b) to keep the text in flash instead of copying it into the small SRAM**

F() keeps string constants in **flash**.

Q12 **Answer: Solution program**

parseInt() reads the number; the leftover newline is removed so it is not parsed as another (zero) number. Inputs: 7 and 12.

```
void setup() { Serial.begin(9600); }
void loop() {
  if (Serial.available()) {
    long n = Serial.parseInt();
    while (Serial.available() && (Serial.peek() == '\n' || Serial.peek() == '\r')) Serial.read();
    Serial.print(n); Serial.print(" squared = "); Serial.println(n * n);
  }
}
```

SERIAL MONITOR

```
7 squared = 49
12 squared = 144
```

Q13 **Answer: (a) set the Serial Monitor to 115200 baud**

Both ends must use the **same baud rate**.

Q14 **Answer: (c) temp:23,hum:60 on each line**

One line per sample, values labelled and separated by commas or spaces.

Q15 **Answer: Solution program**

indexOf() finds the spaces, substring() extracts the parts and toInt() converts them. Inputs: ADD 5 7, MUL 6 9, SUB 1 2.

```
void setup() { Serial.begin(9600); }
void loop() {
  if (!Serial.available()) return;
  String cmd = Serial.readStringUntil('\n');
  cmd.trim();
  int s1 = cmd.indexOf(' ');
  int s2 = cmd.indexOf(' ', s1 + 1);
  String op = cmd.substring(0, s1);
  long a = cmd.substring(s1 + 1, s2).toInt();
  long b = cmd.substring(s2 + 1).toInt();
  if (op == "ADD") Serial.println(a + b);
  else if (op == "MUL") Serial.println(a * b);
  else Serial.println("ERR");
}
```

SERIAL MONITOR

```
12
54
ERR
```

Q16 **Answer: (d) a semicolon is missing at the end of the previous statement**

A **missing semicolon**.

Q17 **Answer: (d) run a minimal sketch that only prints the raw reading, and check the wiring with a multimeter**

Simplify to the **smallest test**, then check the hardware.

Q18 **Answer: Solution program**

The base argument (HEX, BIN) changes how integers are printed; \t inserts a tab. The loop steps by 5 to keep the output short.

```
void setup() {
  Serial.begin(9600);
  Serial.println("DEC\tHEX\tBIN");
  for (int i = 0; i < 16; i += 5) {
    Serial.print(i); Serial.print('\t');
    Serial.print(i, HEX); Serial.print('\t');
    Serial.println(i, BIN);
  }
}
void loop() {}
```

SERIAL MONITOR

```
DEC     HEX     BIN
0       0       0
```

```
5       5       101
10      A       1010
15      F       1111
```

Q19 **Answer: (c) about 104 ms**

100 characters × 10 bits / 9600 ≈ **0.104 s**.

Q20 **Answer: Solution program**

A global variable keeps the setting between commands. Inputs: GET, SET 30, GET.

```
int limit = 25;
void setup() { Serial.begin(9600); }
void loop() {
  if (!Serial.available()) return;
  String c = Serial.readStringUntil('\n');
  c.trim();
  if (c.startsWith("SET ")) {
    limit = c.substring(4).toInt();
    Serial.print("Limit = "); Serial.println(limit);
  } else if (c == "GET") {
    Serial.print("Limit is "); Serial.println(limit);
  }
}
```

SERIAL MONITOR

```
Limit is 25
Limit = 30
Limit is 30
```

Q21 **Answer: (a) Serial.print waits when the transmit buffer is full, so the loop runs at the speed of the serial link**

When the 64-byte buffer fills, print() **blocks** until space is free.

Q22 **Answer: Solution program**

toUpperCase() modifies the String in place; length() gives the number of characters. Inputs: hello, Arduino Uno.

```
void setup() { Serial.begin(9600); }
void loop() {
  if (Serial.available()) {
    String s = Serial.readStringUntil('\n');
    s.trim();
    s.toUpperCase();
    Serial.print(s); Serial.print(" ("); Serial.print(s.length()); Serial.println(")");
  }
}
```

SERIAL MONITOR

```
HELLO (5)
ARDUINO UNO (11)
```

Q23 **Answer: (d) to send large amounts of data quickly (e.g. fast sensor logging)**

Higher baud = **faster transfer**.

Q24 **Answer: (b) reduce SRAM use: move string constants into flash with F() and shrink buffers**

It warns that **SRAM** is nearly full.

Q25 **Answer: Solution program**

Changing DEBUG to 0 removes the debug lines at compile time, with no cost in speed or memory.

```
#define DEBUG 1
#if DEBUG
#define DBG(label, v) do { Serial.print(label); Serial.println(v); } while (0)
#else
#define DBG(label, v)
#endif
void setup() {
  Serial.begin(9600);
  long sum = 0;
  for (int i = 0; i < 3; i++) {
    int r = analogRead(A0);
    DBG("reading: ", r);
    sum += r;
  }
  Serial.print("average = "); Serial.println(sum / 3);
}
void loop() {}
```

SERIAL MONITOR

```
reading: 100
reading: 110
reading: 120
average = 110
```

Match baud rates (characters per second ≈ baud / 10). print() vs println(); print(n, HEX/BIN) and print(f, decimals). Check Serial.available() before read(); use readStringUntil('\n') and a simple text protocol for commands. Plot with one line per sample. Debug by printing values, simplifying to the smallest test, and checking hardware separately.

MODULE 11 · INTERMEDIATE

Functions, arrays, strings and structures

Minimum duration **2 days** | Level **Intermediate**

As sketches grow, structure keeps them readable and reliable. This module covers writing functions with parameters and return values, pass by value and by reference, arrays (including arrays of pins and 2-D arrays), C strings and the String class with its memory risks, parsing text commands, struct and enum, and bit manipulation, followed by 25 solved questions and programs.

IN THIS MODULE

11.1 Functions
11.2 Arrays
11.3 Strings
11.4 Structures and enums
11.5 Bit manipulation
11.6 Practice questions
11.7 Solutions

11.1 Functions

11.1.1 Writing functions with parameters and return values

FUNCTIONS.INO

```
float celsiusToF(float c) {          // return type, name, parameters
  return c * 9.0 / 5.0 + 32.0;
}

void blink(int pin, int times, int ms) {   // void: returns nothing
  for (int i = 0; i < times; i++) {
    digitalWrite(pin, HIGH); delay(ms);
    digitalWrite(pin, LOW);  delay(ms);
  }
}

void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);
  Serial.println(celsiusToF(25));    // 77.00
  Serial.println(celsiusToF(-40));   // -40.00
  blink(13, 2, 100);
}
void loop() {}
```

SERIAL MONITOR

```
77.00
-40.00
```

11.1.2 Pass by value and by reference

REFERENCES.INO

```
void byValue(int x)      { x = x * 2; }       // works on a copy
void byReference(int &x) { x = x * 2; }       // works on the original

void minMax(const int a[], int n, int &lo, int &hi) {   // "return" two results
  lo = hi = a[0];
  for (int i = 1; i < n; i++) { if (a[i] < lo) lo = a[i]; if (a[i] > hi) hi = a[i]; }
}

void setup() {
  Serial.begin(9600);
  int v = 5;
  byValue(v);     Serial.println(v);   // still 5
  byReference(v); Serial.println(v);   // now 10
  int data[] = {12, 7, 30, 18};
  int lo, hi;
  minMax(data, 4, lo, hi);
  Serial.print("min "); Serial.print(lo); Serial.print(", max "); Serial.println(hi);
}
void loop() {}
```

SERIAL MONITOR

```
5
10
min 7, max 30
```

11.2 Arrays

11.2.1 Declaring and using arrays

`int readings[5];` holds five ints, indexed 0 to 4. `int a[] = {3, 1, 4};` lets the compiler count the elements. The number of elements is `sizeof(a) / sizeof(a[0])`.

> **WATCH OUT**
> C++ does not check array bounds. Writing readings[5] in a 5-element array silently overwrites other variables, causing bizarre bugs.

11.2.2 Arrays of pins

PINS IN AN ARRAY

```
const int leds[] = {2, 3, 4, 5, 6};
const int N = sizeof(leds) / sizeof(leds[0]);
void setup() {
  for (int i = 0; i < N; i++) pinMode(leds[i], OUTPUT);
}
void loop() {}
```

11.2.3 Two-dimensional arrays

ARRAY2D.INO

```
const byte pattern[3][4] = {      // 3 frames × 4 LEDs
  {1, 0, 0, 1},
  {0, 1, 1, 0},
  {1, 1, 1, 1}
};

void setup() {
  Serial.begin(9600);
  for (int f = 0; f < 3; f++) {
    for (int i = 0; i < 4; i++) Serial.print(pattern[f][i] ? '*' : '.');
    Serial.println();
  }
}
void loop() {}
```

SERIAL MONITOR

```
*..*
.**.
****
```

11.3 Strings

11.3.1 C strings (char arrays)

A C string is an array of char ending with the null character `'\0'`. `char name[10] = "Uno";` uses 4 bytes ("U", "n", "o", "\0"). Functions: strlen(), strcpy(), strcat(), strcmp(), strncpy(), atoi().

11.3.2 The String class and its memory risks

The String class is convenient (+ to join, indexOf(), substring(), toInt()), but each change may allocate new memory on the heap. On a 2 KB Uno R3, repeated String building can fragment memory until allocations fail. Prefer fixed char buffers in long-running sketches, or call reserve() once.

	C string (char[])	String class
Memory	fixed, known at compile time	dynamic (heap)
Ease of use	needs library functions	operators and methods
Risk	buffer overflow if too small	heap fragmentation on small boards
Best for	Uno R3, long-running code	quick prototypes, ESP32

11.3.3 Parsing text commands

STRTOK.INO

```
void setup() {
  Serial.begin(9600);
  char line[] = "LED,9,128";
  char *cmd = strtok(line, ",");          // split at commas
  int pin = atoi(strtok(NULL, ","));
  int val = atoi(strtok(NULL, ","));
  Serial.print("cmd=");  Serial.print(cmd);
  Serial.print(" pin="); Serial.print(pin);
  Serial.print(" val="); Serial.println(val);
}
void loop() {}
```

SERIAL MONITOR

```
cmd=LED pin=9 val=128
```

11.4 Structures and enums

11.4.1 struct for grouping data

STRUCT.INO

```
struct Reading {
  unsigned long time;
  float temp;
  int light;
};

void printReading(const Reading &r) {
  Serial.print(r.time); Serial.print(" ms: ");
  Serial.print(r.temp, 1); Serial.print(" C, light ");
  Serial.println(r.light);
}

void setup() {
  Serial.begin(9600);
  Reading log[2] = {{1000, 23.5, 640}, {2000, 23.7, 610}};
  for (int i = 0; i < 2; i++) printReading(log[i]);
}
void loop() {}
```

SERIAL MONITOR

```
1000 ms: 23.5 C, light 640
2000 ms: 23.7 C, light 610
```

11.4.2 enum for states

ENUM.INO

```
enum Mode { OFF, SLOW, FAST };
Mode mode = OFF;

const char *name(Mode m) {
  switch (m) { case OFF: return "OFF"; case SLOW: return "SLOW"; default: return "FAST"; }
}

void setup() {
  Serial.begin(9600);
  for (int i = 0; i < 4; i++) {
    Serial.println(name(mode));
    mode = (Mode)((mode + 1) % 3);       // cycle OFF -> SLOW -> FAST -> OFF
  }
}
void loop() {}
```

SERIAL MONITOR

```
OFF
SLOW
FAST
OFF
```

11.5 Bit manipulation

11.5.1 bitRead(), bitWrite(), bitSet(), bitwise operators

Operator	Meaning	Example (a = 0b1100, b = 0b1010)
&	AND: 1 where both are 1	a & b = 0b1000
|	OR: 1 where either is 1	a | b = 0b1110
^	XOR: 1 where they differ	a ^ b = 0b0110
~	NOT: flip every bit	~a flips all bits
<<	shift left (× 2 per place)	1 << 3 = 8
>>	shift right (÷ 2 per place)	0b1100 >> 2 = 0b11

BITS.INO

```
void setup() {
  Serial.begin(9600);
  byte flags = 0;
  bitSet(flags, 0);            // flags = 00000001
  bitSet(flags, 3);            // flags = 00001001
  Serial.println(flags, BIN);
  Serial.println(bitRead(flags, 3));
  bitClear(flags, 0);
  Serial.println(flags, BIN);
  byte mask = 0b11110000;
  Serial.println(0b10110110 & mask, BIN);   // keep the high 4 bits
  Serial.println(1 << 5);
}
void loop() {}
```

SERIAL MONITOR

```
1001
1
1000
10110000
32
```

11.6 Practice questions

11.6.1 Basic level

Q1 What does the keyword void mean as a function return type?

(a) the function is empty
(b) the function returns no value
(c) the function returns 0
(d) the function cannot have parameters

Q2 int a[] = {4, 8, 15, 16}; What is a[2]?

(a) 8 **(b)** 16 **(c)** 4 **(d)** 15

Q3 How many bytes does the C string "Uno" occupy?

(a) 4 **(b)** 3 **(c)** 6 **(d)** 8

Q4 What is 0b1100 & 0b1010?

(a) 0b1110 **(b)** 0b0110 **(c)** 0b1000 **(d)** 0b0010

Q5 What is 1 << 4?

(a) 4 **(b)** 5 **(c)** 8 **(d)** 16

Q6 Which keyword groups several variables of different types into one type?

(a) enum **(b)** struct **(c)** array **(d)** define

Q7 In int x[5], what is the valid index range?

(a) 0 to 4 **(b)** 1 to 5 **(c)** 0 to 5 **(d)** 1 to 4

Q8 What does an enum provide?

(a) dynamic memory
(b) floating-point maths
(c) named integer constants, useful for states
(d) a list of pins automatically

11.6.2 Intermediate level

Q9 Write a function average(const int a[], int n) that returns the average as a float, and test it on {10, 20, 25}.

Q10 After "void f(int x) { x = 10; }" is called as f(v) with v = 3, what is v?

(a) 3 **(b)** 10 **(c)** 0 **(d)** 13

Q11 After "void f(int &x) { x = 10; }" is called as f(v) with v = 3, what is v?

(a) 3 **(b)** 10 **(c)** 0 **(d)** 13

Q12 Write a function swap(int &a, int &b) and use it to sort two numbers into ascending order.

Q13 Why can heavy use of String on an Uno R3 make a sketch crash after hours of running?

(a) Strings are slower to print
(b) Strings use EEPROM
(c) Strings cannot hold numbers

(d) repeated allocations fragment the small heap until memory runs out

Q14 Find the largest value in an array of 6 readings and the index where it occurs.

Q15 What does 0b0110 ^ 0b0011 equal?

(a) 0b0111 (b) 0b0010 (c) 0b0101 (d) 0b1001

Q16 Parse the C string "SERVO:90" into a name and a number using strchr() and atoi().

11.6.3 Advanced level

Q17 What is sizeof(a) / sizeof(a[0]) used for?

(a) to measure free memory
(b) to find the largest element
(c) to count the elements of an array
(d) to copy an array

Q18 Use a struct Led {int pin; int onMs; int offMs;} array to describe two LEDs and print their settings.

Q19 Which expression tests whether bit 2 of x is set?

(a) x | 2
(b) x << 2
(c) x == 2
(d) (x & (1 << 2)) != 0

Q20 Write a function that counts how many bits are 1 in a byte, and test it on 0b10110110.

Q21 What is the risk of: char buf[5]; strcpy(buf, "Arduino");

(a) none
(b) buffer overflow: 8 bytes are written into a 5-byte array
(c) it prints in capitals
(d) strcpy adds spaces

Q22 Implement a traffic-light state machine with an enum {RED, GREEN, YELLOW}: print the state and its duration for one full cycle.

Q23 Which of these turns OFF bit 3 of a variable x?

(a) x &= ~(1 << 3);
(b) x |= (1 << 3);
(c) x ^= 3;
(d) x = x >> 3;

Q24 Store 5 temperature readings in a circular buffer of size 3 and print the buffer contents after each insertion.

Q25 Why are fixed char buffers often preferred to String on the Uno R3 for long-running code?

(a) they print faster
(b) their memory is fixed at compile time, so the heap cannot fragment
(c) they use EEPROM
(d) they support Unicode emojis

11.7 Solutions

Q1 **Answer: (b) the function returns no value**

void = returns nothing.

Q2 **Answer: (d) 15**

Indices start at 0: a[2] = **15**.

Q3 **Answer: (a) 4**

Three letters plus the terminating '\0': **4**.

Q4 **Answer: (c) 0b1000**

AND keeps bits set in both: **1000**.

Q5 **Answer: (d) 16**

Shifting left by 4 multiplies by 2^4 = **16**.

Q6 **Answer: (b) struct**

struct groups related data.

Q7 **Answer: (a) 0 to 4**

Five elements: indices **0–4**.

Q8 **Answer: (c) named integer constants, useful for states**

An **enum** names states such as OFF, SLOW, FAST.

Q9 **Answer: Solution program**

Cast to float before dividing, otherwise integer division would give 18 instead of 18.33.

```
float average(const int a[], int n) {
  long sum = 0;
  for (int i = 0; i < n; i++) sum += a[i];
  return (float)sum / n;
}
void setup() {
  Serial.begin(9600);
  int d[] = {10, 20, 25};
  Serial.println(average(d, 3));
}
void loop() {}
```

SERIAL MONITOR

```
18.33
```

Q10 **Answer: (a) 3**

Pass by value changes only a **copy**.

Q11 **Answer: (b) 10**

Pass by reference changes the **original**.

Q12 **Answer: Solution program**

References let the function change the caller's variables. (The name swap2 avoids a clash with std::swap.)

```
void swap2(int &a, int &b) { int t = a; a = b; b = t; }
void setup() {
  Serial.begin(9600);
  int x = 42, y = 17;
  if (x > y) swap2(x, y);
  Serial.print(x); Serial.print(" "); Serial.println(y);
}
void loop() {}
```

SERIAL MONITOR

```
17 42
```

Q13 **Answer: (d) repeated allocations fragment the small heap until memory runs out**

Heap fragmentation in 2 KB of SRAM.

Q14 **Answer: Solution program**

Track the index of the best value so far; sizeof gives the element count.

```
void setup() {
  Serial.begin(9600);
  int r[] = {512, 640, 590, 701, 655, 700};
  int n = sizeof(r) / sizeof(r[0]);
  int best = 0;
  for (int i = 1; i < n; i++) if (r[i] > r[best]) best = i;
  Serial.print("max "); Serial.print(r[best]);
  Serial.print(" at index "); Serial.println(best);
}
void loop() {}
```

SERIAL MONITOR

```
max 701 at index 3
```

Q15 **Answer: (c) 0b0101**

XOR sets bits that differ: **0101**.

Q16 **Answer: Solution program**

Replacing the colon with '\0' ends the first string there; the text after it is converted with atoi().

```
void setup() {
  Serial.begin(9600);
  char msg[] = "SERVO:90";
  char *colon = strchr(msg, ':');
  *colon = '\0';                 // split the string in two
  int angle = atoi(colon + 1);
  Serial.print("name="); Serial.print(msg);
  Serial.print(" angle="); Serial.println(angle);
}
void loop() {}
```

SERIAL MONITOR

```
name=SERVO angle=90
```

Q17 **Answer: (c) to count the elements of an array**

Total bytes ÷ bytes per element = **element count**.

Q18 **Answer: Solution program**

A range-based for loop with a reference (auto &l) visits each struct without copying it.

```
struct Led { int pin; int onMs; int offMs; };
Led leds[] = {{9, 200, 800}, {10, 500, 500}};
void setup() {
  Serial.begin(9600);
  for (auto &l : leds) {
    Serial.print("pin "); Serial.print(l.pin);
    Serial.print(": on "); Serial.print(l.onMs);
    Serial.print(" ms, off "); Serial.print(l.offMs); Serial.println(" ms");
  }
}
void loop() {}
```

SERIAL MONITOR

```
pin 9: on 200 ms, off 800 ms
pin 10: on 500 ms, off 500 ms
```

Q19 **Answer: (d) (x & (1 << 2)) != 0**

Mask bit 2 with AND: **(x & (1 << 2))**.

Q20 **Answer: Solution program**

Test the lowest bit with & 1, then shift right until the byte is zero. 10110110 has five 1-bits.

```
int countOnes(byte b) {
  int n = 0;
  while (b) { n += b & 1; b >>= 1; }
  return n;
}
void setup() {
  Serial.begin(9600);
  Serial.println(countOnes(0b10110110));
}
void loop() {}
```

SERIAL MONITOR

```
5
```

Q21 **Answer: (b) buffer overflow: 8 bytes are written into a 5-byte array**

"Arduino" needs 8 bytes including '\0': a **buffer overflow**.

Q22 **Answer: Solution program**

Enum values are integers 0, 1, 2, so they index the name and duration arrays.

```
enum Light { RED, GREEN, YELLOW };
const char *names[] = {"RED", "GREEN", "YELLOW"};
const int durations[] = {5000, 4000, 1500};
void setup() {
  Serial.begin(9600);
  Light s = RED;
  for (int i = 0; i < 3; i++) {
    Serial.print(names[s]); Serial.print(" for "); Serial.print(durations[s]); Serial.println("
ms");
    s = (Light)((s + 1) % 3);
  }
}
void loop() {}
```

SERIAL MONITOR

```
RED for 5000 ms
GREEN for 4000 ms
YELLOW for 1500 ms
```

Q23 **Answer: (a) x &= ~(1 << 3);**

AND with an inverted mask **clears** the bit.

Q24 **Answer: Solution program**

The head index wraps round with % SIZE, overwriting the oldest value: a fixed-memory way to keep the last N readings.

```
const int SIZE = 3;
float buf[SIZE];
int head = 0, count = 0;
void add(float v) {
  buf[head] = v;
  head = (head + 1) % SIZE;
  if (count < SIZE) count++;
}
void setup() {
  Serial.begin(9600);
  float temps[] = {21.0, 21.5, 22.0, 22.5, 23.0};
  for (float t : temps) {
    add(t);
    for (int i = 0; i < count; i++) { Serial.print(buf[i], 1); Serial.print(' '); }
    Serial.println();
  }
}
void loop() {}
```

SERIAL MONITOR

```
21.0
21.0 21.5
21.0 21.5 22.0
22.5 21.5 22.0
22.5 23.0 22.0
```

Q25 **Answer: (b) their memory is fixed at compile time, so the heap cannot fragment**

Fixed buffers avoid **dynamic allocation**.

Break sketches into functions with clear parameters and return values; use references (&) to change the caller's variables. Arrays are 0-indexed and unchecked: count with sizeof. Prefer char buffers to String on small boards. Group data with struct, name states with enum, and use &, |, ^, ~, << and >> (or bitRead/bitSet) for bits.

MODULE 12 · INTERMEDIATE

Timing and non-blocking code

Minimum duration **1 day** | Level **Intermediate**

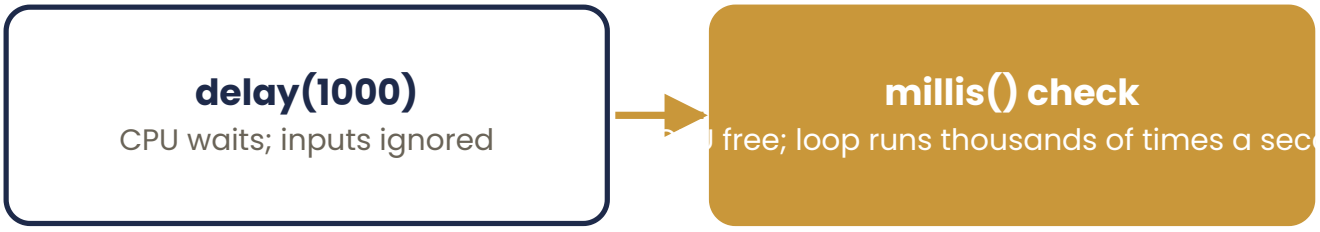
delay() is simple but freezes the whole sketch. This module explains the problems with delay(), millis()-based timing ("blink without delay"), running several timed tasks together, handling millis() overflow correctly, micros() for short intervals, cooperative task scheduling and an introduction to state machines, followed by 25 solved questions and programs, each run in simulated time.

IN THIS MODULE

12.1	Problems with delay()	12.2	millis()-based timing
12.3	micros() for short intervals	12.4	Simple task scheduling
12.5	State machines (introduction)	12.6	Practice questions
12.7	Solutions		

12.1 Problems with delay()

While delay(1000) runs, the sketch does nothing else: it cannot read buttons, update a display or answer serial commands. A button pressed and released during the delay is simply missed, and two LEDs cannot blink at independent rates.



BLOCKING VS NON-BLOCKING

12.2 millis()-based timing

12.2.1 "Blink without delay"

BLINKWITHOUTDELAY.INO

```
const int LED = 13;
const unsigned long INTERVAL = 500;
unsigned long previous = 0;
int state = LOW;

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  unsigned long now = millis();
  if (now - previous >= INTERVAL) {    // time for the next change?
    previous = now;
    state = !state;
    digitalWrite(LED, state);
  }
  // other code here keeps running all the time
}
```

SIMULATED RUN (pin changes and Serial output)

```
[   500 ms] pin 13 -> HIGH
[  1000 ms] pin 13 -> LOW
[  1500 ms] pin 13 -> HIGH
[  2000 ms] pin 13 -> LOW
```

The pattern: if (millis() - previous >= interval) { previous = millis(); do the task; }. Store times in unsigned long.

12.2.2 Running several timed tasks together

TWOTASKS.INO

```
unsigned long tLed = 0, tPrint = 0;
int led = LOW;

void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);
}

void loop() {
  unsigned long now = millis();
  if (now - tLed >= 300) {          // task 1: blink every 300 ms
    tLed = now;
    led = !led;
    digitalWrite(13, led);
  }
  if (now - tPrint >= 1000) {       // task 2: report every 1000 ms
    tPrint = now;
    Serial.print("t = "); Serial.print(now); Serial.println(" ms");
  }
}
```

SERIAL MONITOR

```
t = 1000 ms
t = 2000 ms
t = 3000 ms
```

12.2.3 Handling millis() overflow correctly

millis() returns to 0 after about 49.7 days. With unsigned arithmetic, `now - previous` still gives the right elapsed time across the wrap, so the subtraction pattern keeps working.

OVERFLOW.INO

```
void setup() {
  Serial.begin(9600);
  uint32_t previous = 4294967000UL;        // 296 ms before the wrap (32-bit, like millis())
  uint32_t now = 200UL;                    // 200 ms after the wrap
  uint32_t elapsed = now - previous;
  Serial.println(elapsed);                 // correct elapsed time: 496
  uint32_t due = previous + 400;           // overflows to 104
  Serial.println(now >= due ? "due" : "not due");   // happens to work here...
  uint32_t due2 = previous + 100;          // 4294967100: no overflow
  Serial.println(now >= due2 ? "due" : "not due (WRONG: 496 ms have passed)");
}
void loop() {}
```

SERIAL MONITOR

```
496
due
not due (WRONG: 496 ms have passed)
```

WATCH OUT
Never write if (millis() >= previous + interval): previous + interval can overflow and the comparison then fails. Always subtract.

12.3 micros() for short intervals

`micros()` counts microseconds (resolution 4 µs on a 16 MHz Uno R3) and wraps after about 70 minutes. Use it to measure short pulses or to run tasks faster than 1 kHz, with the same subtraction pattern.

12.4 Simple task scheduling

12.4.1 Cooperative multitasking patterns

SCHEDULER.INO

```
struct Task {
  unsigned long interval, last;
  void (*run)();
};

void blinkTask()  { static int s = 0; s = !s; digitalWrite(13, s); }
void readTask()   { Serial.print("sensor "); Serial.println(analogRead(A0)); }

Task tasks[] = {{250, 0, blinkTask}, {1000, 0, readTask}};

void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);
}

void loop() {
  unsigned long now = millis();
  for (auto &t : tasks) {
    if (now - t.last >= t.interval) { t.last = now; t.run(); }
  }
}
```

SERIAL MONITOR

```
sensor 400
sensor 410
```

Each task must finish quickly and never call delay(); the scheduler gives every task its turn. This is **cooperative** multitasking. Module 24 introduces FreeRTOS for pre-emptive multitasking.

12.5 State machines (introduction)

A state machine is in exactly one state at a time and changes state on events (a timer expiring, a button press). It replaces long chains of delays with a clear, non-blocking structure.



TRAFFIC-LIGHT STATES: EACH TIMES OUT INTO THE NEXT

STATEMACHINE.INO

```
enum State { RED, GREEN, YELLOW };
State state = RED;
unsigned long entered = 0;
const unsigned long DUR[] = {5000, 4000, 1500};
const char *NAME[] = {"RED", "GREEN", "YELLOW"};

void setup() {
  Serial.begin(9600);
  Serial.println("RED");
}

void loop() {
  if (millis() - entered >= DUR[state]) {
    state = (State)((state + 1) % 3);
    entered = millis();
    Serial.print(millis()); Serial.print(" ms: "); Serial.println(NAME[state]);
  }
  // the loop is free here to read a pedestrian button
}
```

SERIAL MONITOR

```
RED
5000 ms: GREEN
9000 ms: YELLOW
10500 ms: RED
```

12.6 Practice questions

12.6.1 Basic level

Q1 What is the main drawback of delay()?

(a) the sketch cannot do anything else while waiting
(b) it uses too much flash
(c) it is inaccurate by seconds
(d) it works only on pin 13

Q2 Which data type should hold a time from millis()?

(a) int **(b)** float **(c)** unsigned long **(d)** byte

Q3 What is the correct non-blocking test that 500 ms have passed?

(a) if (millis() == 500)
(b) if (millis() >= previous + 500)
(c) if (delay(500))
(d) if (millis() - previous >= 500)

Q4 After about how many days does millis() wrap around to 0?

(a) 7 **(b)** 30 **(c)** 365 **(d)** 49.7

Q5 What is the resolution of micros() on a 16 MHz Uno R3?

(a) 1 µs **(b)** 1 ms **(c)** 4 µs **(d)** 16 µs

Q6 In cooperative multitasking, each task must

(a) finish quickly and never block with delay()
(b) run for at least one second
(c) use interrupts only
(d) have its own Arduino

Q7 A state machine is always in

(a) all states at once
(b) no state until a button is pressed
(c) two states
(d) exactly one state at a time

Q8 Why does "previous = now;" appear inside the if-block of blink without delay?

(a) to reset the board
(b) to start timing the next interval from this moment
(c) to stop the LED
(d) to read the button

12.6.2 Intermediate level

Q9 Blink LED A (pin 8) every 200 ms and LED B (pin 9) every 700 ms, independently, without delay().

Q10 previous = 4294967200 and now = 100 (after a millis() wrap). What does now - previous give with unsigned long arithmetic?

(a) a huge number **(b)** 196 **(c)** -4294967100 **(d)** 0

Q11 Why is "if (millis() >= previous + interval)" unsafe?

(a) it is slower
(b) millis() cannot be compared
(c) interval must be an int
(d) previous + interval can overflow near the wrap, making the test wrong

Q12 Print "tick" every 250 ms and "TOCK" every second, using millis().

Q13 Which function would you use to measure a 150 µs pulse?

(a) micros() (b) millis() (c) delay() (d) analogRead()

Q14 Turn an LED on pin 13 on when a button (pin 2, INPUT_PULLUP) is pressed and switch it off automatically 3 seconds later, without delay().

Q15 What does the "static" keyword do to a local variable inside a function?

(a) makes it global
(b) stores it in EEPROM
(c) keeps its value between calls
(d) makes it read-only

Q16 Implement a countdown timer that prints the remaining seconds from 5 to 0 once per second, without delay().

12.6.3 Advanced level

Q17 A loop takes 2 ms per pass and contains a 500 ms blink-without-delay task. Roughly how precise is the blink timing?

(a) within about 500 ms
(b) exactly 0 ms error
(c) within about 1 s
(d) within about 2 ms

Q18 Write a non-blocking "debounced button + blinking LED" sketch: the LED on pin 13 blinks every 400 ms, and each button press on pin 2 prints a press count, debounced with millis().

Q19 In the task-scheduler pattern, what happens if one task calls delay(2000)?

(a) only that task waits
(b) every other task is delayed by 2 seconds
(c) the scheduler skips it
(d) nothing

Q20 A state machine for a washing cycle: FILL (2 s) → WASH (5 s) → DRAIN (2 s) → DONE. Print each state change with its time.

Q21 Why is a state machine better than a long chain of delay() calls for a vending machine?

(a) it can react to coins and buttons at any moment while timing other steps
(b) it uses less flash in every case
(c) it makes the motors faster
(d) delay() cannot be used on vending machines

Q22 Measure how many times loop() runs in one second and print the count (loop rate).

Q23 A 1 kHz square wave must be generated without blocking. What half-period should the micros() timer use?

(a) 1000 µs
(b) 1 ms per edge pair

(c) 500 µs (d) 50 µs

Q24 Fade an LED on pin 9 up from 0 to 255 in steps of 51 every 100 ms without delay(), then stop.

Q25 What is the elapsed time if previous = 1000 and now = 1350 (both from millis())?

(a) 350 ms (b) 2350 ms (c) 1350 ms (d) -350 ms

12.7 Solutions

Q1 **Answer: (a) the sketch cannot do anything else while waiting**

delay() **blocks** the whole sketch.

Q2 **Answer: (c) unsigned long**

unsigned long matches millis().

Q3 **Answer: (d) if (millis() - previous >= 500)**

Subtract and compare: **millis() - previous >= 500**.

Q4 **Answer: (d) 49.7**

2^{32} ms ≈ **49.7 days**.

Q5 **Answer: (c) 4 µs**

micros() advances in steps of **4 µs**.

Q6 **Answer: (a) finish quickly and never block with delay()**

Tasks **share** the loop and must return quickly.

Q7 **Answer: (d) exactly one state at a time**

One state at a time, changing on events.

Q8 **Answer: (b) to start timing the next interval from this moment**

It records when the task last ran.

Q9 **Answer: Solution program**

Each LED has its own timer variable, so their rates are completely independent.

```
unsigned long ta = 0, tb = 0;
int a = LOW, b = LOW;
void setup() { pinMode(8, OUTPUT); pinMode(9, OUTPUT); }
void loop() {
  unsigned long now = millis();
  if (now - ta >= 200) { ta = now; a = !a; digitalWrite(8, a); }
  if (now - tb >= 700) { tb = now; b = !b; digitalWrite(9, b); }
}
```

SIMULATED RUN

```
[   200 ms] pin 8 -> HIGH
[   400 ms] pin 8 -> LOW
[   600 ms] pin 8 -> HIGH
[   700 ms] pin 9 -> HIGH
[   800 ms] pin 8 -> LOW
[  1000 ms] pin 8 -> HIGH
[  1200 ms] pin 8 -> LOW
[  1400 ms] pin 8 -> HIGH
[  1400 ms] pin 9 -> LOW
```

Q10 **Answer: (b) 196**

Unsigned subtraction wraps too: 2^{32} - 4294967200 + 100 = **196** ms.

Q11 **Answer: (d) previous + interval can overflow near the wrap, making the test wrong**

The sum **overflows**; subtraction does not suffer from this.

Q12 **Answer: Solution program**

Two independent timers. Both start at 0, so the first tick appears at 250 ms and the first TOCK at 1000 ms.

```
unsigned long t1 = 0, t2 = 0;
void setup() { Serial.begin(9600); }
void loop() {
  unsigned long now = millis();
  if (now - t1 >= 250) { t1 = now; Serial.print("tick "); }
  if (now - t2 >= 1000) { t2 = now; Serial.println("TOCK"); }
}
```

SERIAL MONITOR

```
tick tick tick tick TOCK
tick tick tick tick TOCK
```

Q13 **Answer: (a) micros()**

Short intervals need **micros()**.

Q14 **Answer: Solution program**

A timestamp replaces the delay, so the sketch keeps checking the button. This is how stair lights and auto-off lamps work.

```
unsigned long onAt = 0;
bool lit = false;
void setup() {
  pinMode(2, INPUT_PULLUP);
  pinMode(13, OUTPUT);
}
void loop() {
  if (digitalRead(2) == LOW && !lit) {
    lit = true; onAt = millis(); digitalWrite(13, HIGH);
  }
  if (lit && millis() - onAt >= 3000) {
    lit = false; digitalWrite(13, LOW);
  }
}
```

SIMULATED RUN

```
[   500 ms] pin 13 -> HIGH
[  3500 ms] pin 13 -> LOW
```

Q15 **Answer: (c) keeps its value between calls**

A static local is created once and **keeps its value**.

Q16 **Answer: Solution program**

The countdown runs in the background; other code could run in loop() at the same time.

```
unsigned long last = 0;
int remaining = 5;
void setup() { Serial.begin(9600); Serial.println(remaining); }
void loop() {
  if (remaining > 0 && millis() - last >= 1000) {
    last = millis();
    remaining--;
    Serial.println(remaining);
    if (remaining == 0) Serial.println("Done");
  }
}
```

SERIAL MONITOR

```
5
4
3
2
1
0
Done
```

Q17 **Answer: (d) within about 2 ms**

The check happens once per pass, so error ≤ one loop time (**about 2 ms**).

Q18 **Answer: Solution program**

Both jobs run together because neither blocks. The scripted button bounced but was counted twice.

```
unsigned long tBlink = 0, tChange = 0;
int led = LOW, stable = HIGH, lastRead = HIGH, presses = 0;
void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT); pinMode(2, INPUT_PULLUP);
}
void loop() {
  unsigned long now = millis();
  if (now - tBlink >= 400) { tBlink = now; led = !led; digitalWrite(13, led); }
  int r = digitalRead(2);
  if (r != lastRead) { tChange = now; lastRead = r; }
  if (now - tChange > 30 && r != stable) {
    stable = r;
    if (stable == LOW) { presses++; Serial.print("press "); Serial.println(presses); }
  }
}
```

SERIAL MONITOR

```
press 1
press 2
```

Q19 **Answer: (b) every other task is delayed by 2 seconds**

Cooperative scheduling **breaks** if any task blocks.

Q20 **Answer: Solution program**

Each state has a duration; when it expires the machine moves on. DONE has no exit, so the machine stops there.

```
enum S { FILL, WASH, DRAIN, DONE };
const char *N[] = {"FILL", "WASH", "DRAIN", "DONE"};
const unsigned long D[] = {2000, 5000, 2000, 0};
S s = FILL;
unsigned long since = 0;
void setup() { Serial.begin(9600); Serial.println("0 ms: FILL"); }
void loop() {
  if (s != DONE && millis() - since >= D[s]) {
    s = (S)(s + 1);
    since = millis();
    Serial.print(since); Serial.print(" ms: "); Serial.println(N[s]);
  }
}
```

SERIAL MONITOR

```
0 ms: FILL
2000 ms: WASH
7000 ms: DRAIN
9000 ms: DONE
```

Q21 **Answer: (a) it can react to coins and buttons at any moment while timing other steps**

The loop stays **responsive** to events.

Q22 **Answer: Solution program**

Counting passes per second shows how responsive the sketch is. In this simulation each empty pass costs 0.1 ms, giving about 10 000 passes per second; a real Uno R3 runs an empty loop far faster.

```
unsigned long start = 0, count = 0;
void setup() { Serial.begin(9600); }
void loop() {
  count++;
  if (millis() - start >= 1000) {
    Serial.print("loops per second: "); Serial.println(count);
    count = 0;
    start = millis();
  }
}
```

SERIAL MONITOR

```
loops per second: 10001
loops per second: 10000
```

Q23 **Answer: (c) 500 µs**

Period 1 ms → toggle every **500 µs**.

Q24 **Answer: Solution program**

Each step happens when 100 ms have passed; between steps the loop is free.

```
unsigned long last = 0;
int level = 0;
void setup() { analogWrite(9, 0); }
void loop() {
  if (level < 255 && millis() - last >= 100) {
    last = millis();
    level += 51;
    analogWrite(9, level);
  }
}
```

SIMULATED RUN

```
[     0 ms] PWM pin 9 = 0
[   100 ms] PWM pin 9 = 51
[   200 ms] PWM pin 9 = 102
[   300 ms] PWM pin 9 = 153
[   400 ms] PWM pin 9 = 204
[   500 ms] PWM pin 9 = 255
```

Q25 **Answer: (a) 350 ms**

now - previous = **350 ms**.

Replace delay() with "if (millis() - previous >= interval) { previous = millis(); … }" and unsigned long times; subtraction survives the 49.7-day wrap. Give each task its own timer, keep tasks short, use micros() for short intervals, and organise behaviour as state machines that change state on events.

MODULE 13 · INTERMEDIATE

Sensors

Minimum duration **3 days** | Level **Intermediate**

Sensors connect your sketch to the physical world. This module covers distance sensors (HC-SR04 ultrasonic, IR, VL53L0X time-of-flight), environment sensors (DHT11/DHT22, BMP280/BME280, DS18B20, MQ gas, rain and water level), motion and position sensors (MPU6050, PIR, Hall effect), other sensors (sound, colour, pulse, HX711 load cells, RFID and fingerprint readers), and calibration and accuracy, followed by 25 solved questions and programs run with simulated sensor data.

IN THIS MODULE

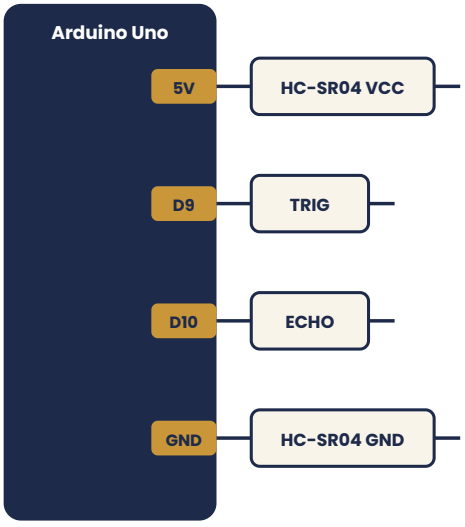
13.1 Distance sensors
13.2 Environment sensors
13.3 Motion and position sensors
13.4 Other sensors
13.5 Calibration and accuracy
13.6 Practice questions
13.7 Solutions

13.1 Distance sensors

13.1.1 Ultrasonic HC-SR04

The HC-SR04 sends a 40 kHz ultrasonic burst when its TRIG pin gets a 10 µs pulse, then holds ECHO HIGH for as long as the sound takes to travel to the object and back. Sound travels about 343 m/s (0.0343 cm/µs) at 20 °C.

distance (cm) = echo time (µs) × 0.0343 / 2 ≈ echo time / 58. Range about 2 cm to 4 m.



HC-SR04 WIRING (5 V MODULE)

ULTRASONIC.INO

```
const int TRIG = 9, ECHO = 10;

long readCm() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);    // 10 µs trigger pulse
  digitalWrite(TRIG, LOW);
  unsigned long us = pulseIn(ECHO, HIGH, 30000);      // timeout 30 ms (about 5 m)
  if (us == 0) return -1;                             // no echo
  return us / 58;
}

void setup() {
  Serial.begin(9600);
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
}

void loop() {
  long cm = readCm();
  if (cm < 0) Serial.println("out of range");
  else { Serial.print(cm); Serial.println(" cm"); }
  delay(100);
}
```

SERIAL MONITOR

```
10 cm
20 cm
50 cm
out of range
```

WATCH OUT

On a 3.3 V board (ESP32), the HC-SR04 ECHO output is 5 V: use a voltage divider or the 3.3 V HC-SR04P version.

13.1.2 IR distance and obstacle sensors

IR obstacle modules give a digital LOW when something reflects their infrared beam (adjust range with the potentiometer). Sharp GP2Y0A21 sensors give an analog voltage that falls non-linearly with distance (10–80 cm); use a lookup table or the datasheet curve.

13.1.3 Time-of-flight VL53L0X

The VL53L0X measures the flight time of a laser pulse and reports millimetres over I2C (up to about 2 m), with a narrow beam that ultrasonic sensors cannot match. Libraries: Pololu VL53L0X or Adafruit_VL53L0X.

13.2 Environment sensors

13.2.1 DHT11 and DHT22 temperature and humidity

	DHT11	DHT22 (AM2302)
Temperature	0–50 °C, ±2 °C	-40–80 °C, ±0.5 °C
Humidity	20–80 %, ±5 %	0–100 %, ±2–5 %
Read rate	once per second	once every 2 seconds
Cost	lowest	higher

DHT22.INO

```
#include <DHT.h>
DHT dht(2, DHT22);          // data pin 2, sensor type

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  if (isnan(h) || isnan(t)) { Serial.println("read failed"); return; }
  Serial.print("T = "); Serial.print(t, 1); Serial.print(" C, RH = ");
  Serial.print(h, 0); Serial.println(" %");
  delay(2000);              // DHT22 needs 2 s between readings
}
```

SERIAL MONITOR

```
T = 25.0 C, RH = 50 %
T = 25.0 C, RH = 50 %
```

13.2.2 BMP280 and BME280 pressure and temperature

The BMP280 measures air pressure and temperature; the BME280 adds humidity. Both use I2C (address 0x76 or 0x77) or SPI. Pressure falls by about 12 Pa per metre near sea level, so the sensor can estimate altitude.

13.2.3 DS18B20 one-wire temperature sensor

The DS18B20 is a digital, waterproof-probe temperature sensor (-55 to 125 °C, ±0.5 °C) on the One-Wire bus: one data pin with a 4.7 kΩ pull-up, and several sensors can share it, each with a unique 64-bit address.

DS18B20.INO

```
#include <OneWire.h>
#include <DallasTemperature.h>
OneWire bus(4);
DallasTemperature sensors(&bus);

void setup() {
  Serial.begin(9600);
  sensors.begin();
}

void loop() {
  sensors.requestTemperatures();
  Serial.print("Water: "); Serial.print(sensors.getTempCByIndex(0), 2); Serial.println(" C");
  delay(1000);
}
```

SERIAL MONITOR

```
Water: 24.50 C
```

13.2.4 MQ gas sensors

MQ-2 (smoke, LPG), MQ-3 (alcohol), MQ-135 (air quality) have a heater and give an analog voltage that rises with gas concentration. They need a warm-up of a minute or more (24–48 h burn-in for accuracy), draw about 150 mA, and give relative rather than precise readings unless calibrated.

13.2.5 Rain and water-level sensors

Rain and water-level boards have exposed traces: the more water, the lower their resistance and the higher (or lower, depending on the module) the analog output. Power them only when reading (from a digital pin) to slow corrosion.

13.3 Motion and position sensors

13.3.1 MPU6050 accelerometer and gyroscope

The MPU6050 combines a 3-axis accelerometer (measures acceleration including gravity, so it shows tilt) and a 3-axis gyroscope (measures rotation rate) over I2C at address 0x68. Tilt angle from the accelerometer: angle = atan2(ay, az) × 180 / π.

TILTANGLE.INO

```
void setup() {
  Serial.begin(9600);
  // example accelerometer readings (in g) for three orientations
  float samples[3][3] = {{0, 0, 1.0}, {0, 0.5, 0.866}, {0, 1.0, 0}};
  for (int i = 0; i < 3; i++) {
    float ay = samples[i][1], az = samples[i][2];
    float angle = atan2(ay, az) * 180.0 / PI;
    Serial.print("tilt = "); Serial.print(angle, 1); Serial.println(" deg");
  }
}
void loop() {}
```

SERIAL MONITOR

```
tilt = 0.0 deg
tilt = 30.0 deg
tilt = 90.0 deg
```

Accelerometer angles are noisy when the sensor vibrates; gyroscope angles drift over time. A **complementary filter** combines them: angle = 0.98 × (angle + gyroRate × dt) + 0.02 × accelAngle.

13.3.2 PIR motion sensors

HC-SR501: digital HIGH on motion; two potentiometers set sensitivity and hold time (about 3 s to 5 min); allow about a minute to settle after power-up.

13.3.3 Hall-effect sensors

Hall sensors detect magnetic fields: digital types (A3144) switch when a magnet is near (use for RPM counting on a wheel); analog types (49E) give a voltage proportional to field strength.

13.4 Other sensors

Sensor	Output	Notes
Sound (KY-038, MAX4466)	analog level / digital threshold	detect claps or noise level, not speech
Colour (TCS3200, TCS34725)	frequency or I2C RGB values	calibrate with white and black references
Pulse (PulseSensor, MAX30102)	analog wave / I2C	heart rate and SpO_2 (MAX30102); not medical grade
Load cell + HX711	24-bit digital (2 pins)	weighing scales; needs calibration factor and tare
RFID RC522	SPI	reads 13.56 MHz cards and tags (UID)
Fingerprint (R307, AS608)	UART	enrol and match fingerprints on the module

SCALEMATHS.INO

```
// HX711 weighing logic with simulated raw readings
long raw[] = {84210, 84210, 126310, 168420};   // first two: empty scale
const float CAL = 42.1;                        // raw counts per gram (from calibration)

void setup() {
  Serial.begin(9600);
  long tare = (raw[0] + raw[1]) / 2;           // zero point
  for (int i = 2; i < 4; i++) {
    float grams = (raw[i] - tare) / CAL;
    Serial.print(grams, 0); Serial.println(" g");
  }
}
void loop() {}
```

SERIAL MONITOR

```
1000 g
2000 g
```

13.5 Calibration and accuracy

Sensors have offset (reads 1 °C high everywhere), gain error (reads 5 % high), noise and drift. Calibrate with known references: measure two points (for example ice water at 0 °C and boiling water at 100 °C) and fit value = (raw - raw_1) × ($true_2$ - $true_1$) / (raw_2 - raw_1) + $true_1$. Check accuracy in the datasheet; resolution (smallest step) is not the same as accuracy.

Two-point calibration: true = (raw - r1) × (t2 - t1) / (r2 - r1) + t1. Then average readings to reduce noise.

13.6 Practice questions

13.6.1 Basic level

Q1 Sound travels at about 343 m/s. How far does it travel in 1 µs?

(a) about 3.43 cm **(b)** about 0.0343 cm
(c) about 343 cm **(d)** about 0.343 mm per ms

Q2 An HC-SR04 echo pulse lasts 1160 µs. What is the distance?

(a) about 40 cm **(b)** about 10 cm **(c)** about 116 cm **(d)** about 20 cm

Q3 How long must the HC-SR04 TRIG pulse be?

(a) 10 ms **(b)** 1 s **(c)** 10 µs **(d)** 58 µs

Q4 Which sensor measures both temperature and humidity?

(a) DS18B20 **(b)** HC-SR04 **(c)** DHT22 **(d)** MPU6050

Q5 Which protocol does the DS18B20 use?

(a) I2C **(b)** SPI **(c)** analog voltage **(d)** One-Wire

Q6 What does a PIR sensor detect?

(a) distance **(b)** movement of warm bodies
(c) humidity **(d)** colour

Q7 Which sensor is used with an HX711 amplifier?

(a) load cell **(b)** LDR **(c)** PIR **(d)** DHT11

Q8 What is the default I2C address of the MPU6050?

(a) 0x27 **(b)** 0x68 **(c)** 0x76 **(d)** 0x3C

13.6.2 Intermediate level

Q9 Read an HC-SR04 and print "STOP" when an obstacle is closer than 15 cm, otherwise print the distance.

Q10 Why must a DHT22 not be read more often than every 2 seconds?

(a) the sensor needs that time between measurements; faster reads return stale or failed data
(b) it would overheat
(c) the library forbids it
(d) humidity changes too fast

Q11 What value pull-up resistor does the DS18B20 data line need?

(a) 220 Ω **(b)** 1 MΩ **(c)** 4.7 kΩ **(d)** none

Q12 Read a DHT22 on pin 2 and print a warning when the heat is high (temperature above 30 °C and humidity above 70 %).

Q13 An accelerometer at rest reads ay = 0.5 g, az = 0.866 g. What is the tilt angle?

(a) 60° **(b)** 45° **(c)** 0° **(d)** 30°

Q14 Why do MQ gas sensors need a warm-up period?

(a) to charge a battery
(b) to download data
(c) the gas must warm up
(d) their internal heater must reach operating temperature before readings are stable

Q15 Calibrate a thermistor reading: raw 900 at 0 °C and raw 300 at 100 °C. Convert raws 600 and 450 to °C using two-point calibration.

Q16 What is the difference between resolution and accuracy?

(a) they mean the same
(b) accuracy is the number of decimal places
(c) resolution is the smallest change shown; accuracy is how close the reading is to the true value
(d) resolution is the sensor price

13.6.3 Advanced level

Q17 Which sensor would best detect a magnet on a spinning wheel to count RPM?

(a) digital Hall-effect sensor **(b)** DHT11
(c) LDR **(d)** MQ-2

Q18 Count wheel revolutions with a Hall sensor on pin 2 (LOW when the magnet passes) and print the RPM after 2 seconds.

Q19 A load cell reads raw 84 210 empty and 126 310 with a 1000 g weight. What is the calibration factor (counts per gram)?

(a) 84.2 **(b)** 126.3 **(c)** 4.21 **(d)** 42.1

Q20 The HC-SR04 ECHO pin outputs 5 V. What must you do on an ESP32?

(a) nothing; ESP32 is 5 V tolerant
(b) reduce it to 3.3 V with a divider (or use a 3.3 V sensor)
(c) use a bigger resistor on TRIG
(d) power the sensor from VIN

Q21 Implement a complementary filter: angle = 0.98 × (angle + gyro × dt) + 0.02 × accelAngle, with gyro = 10 °/s, accelAngle = 5°, dt = 0.1 s, for 5 steps starting at 0°.

Q22 The speed of sound rises by about 0.6 m/s per °C. At 35 °C, how does an HC-SR04 calibrated for 20 °C behave?

(a) it reads exactly right
(b) it reads slightly short (about 2.5 % less than the true distance)
(c) it reads twice the distance
(d) it stops working

Q23 Read three DS18B20 temperatures over time and print the minimum, maximum and average.

Q24 Why are rain-sensor boards often powered from a digital pin only while reading?

(a) to make readings faster
(b) to increase sensitivity

(c) they need 12 V

(d) to reduce electrolytic corrosion of the exposed traces

Q25 Which reader would you use for contactless 13.56 MHz access cards?

(a) RC522 RFID module

(b) HC-SR04

(c) DHT22

(d) MQ-135

13.7 Solutions

Q1 **Answer: (b) about 0.0343 cm**

343 m/s = 34 300 cm/s = **0.0343 cm/µs**.

Q2 **Answer: (d) about 20 cm**

1160 / 58 = **20 cm**.

Q3 **Answer: (c) 10 µs**

A **10 µs** HIGH pulse starts a measurement.

Q4 **Answer: (c) DHT22**

The **DHT22** (and DHT11).

Q5 **Answer: (d) One-Wire**

The DS18B20 is a **One-Wire** device.

Q6 **Answer: (b) movement of warm bodies**

Moving warm bodies (infrared).

Q7 **Answer: (a) load cell**

The HX711 reads a **load cell**.

Q8 **Answer: (b) 0x68**

MPU6050: **0x68** (0x69 if AD0 is high).

Q9 **Answer: Solution program**

Simulated echoes of 2320, 812 and 580 µs give 40 cm, 14 cm (STOP) and 10 cm (STOP).

```
const int TRIG = 9, ECHO = 10;
void setup() { Serial.begin(9600); pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT); }
void loop() {
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  long cm = pulseIn(ECHO, HIGH, 30000) / 58;
  if (cm > 0 && cm < 15) Serial.println("STOP");
  else { Serial.print(cm); Serial.println(" cm"); }
  delay(100);
}
```

SERIAL MONITOR

```
40 cm
STOP
STOP
```

Q10 **Answer: (a) the sensor needs that time between measurements; faster reads return stale or failed data**

The DHT22 sampling period is **2 s**.

Q11 **Answer: (c) 4.7 kΩ**

A **4.7 kΩ** pull-up on the data line.

Q12 **Answer: Solution program**

Both conditions must hold, so they are combined with &&. Simulated readings: 28 °C/65 %, 32 °C/75 %.

```
#include <DHT.h>
DHT dht(2, DHT22);
void setup() { Serial.begin(9600); dht.begin(); }
void loop() {
  float t = dht.readTemperature(), h = dht.readHumidity();
  Serial.print(t, 1); Serial.print(" C, "); Serial.print(h, 0); Serial.print(" %");
  Serial.println((t > 30 && h > 70) ? "  HOT AND HUMID!" : "");
  delay(2000);
}
```

SERIAL MONITOR

```
28.0 C, 65 %
32.0 C, 75 %  HOT AND HUMID!
```

Q13 **Answer: (d) 30°**

atan2(0.5, 0.866) = **30°**.

Q14 **Answer: (d) their internal heater must reach operating temperature before readings are stable**

The **heater** needs time.

Q15 **Answer: Solution program**

A straight line through the two reference points: 600 lies halfway, giving 50 °C; 450 gives 75 °C. (Real thermistors are non-linear; this two-point fit is accurate only near the reference points.)

```
float calibrate(int raw) {
  const float r1 = 900, t1 = 0, r2 = 300, t2 = 100;
  return (raw - r1) * (t2 - t1) / (r2 - r1) + t1;
}
void setup() {
  Serial.begin(9600);
  Serial.println(calibrate(600), 1);
  Serial.println(calibrate(450), 1);
}
void loop() {}
```

SERIAL MONITOR

```
50.0
75.0
```

Q16 **Answer: (c) resolution is the smallest change shown; accuracy is how close the reading is to the true value**

A sensor can show 0.01 °C steps yet be wrong by 1 °C.

Q17 **Answer: (a) digital Hall-effect sensor**

A **Hall sensor** switches each time the magnet passes.

Q18 **Answer: Solution program**

The script simulates 5 magnet passes in 2 seconds: 5 × 30 = 150 RPM. (In a real sketch, use a non-blocking timer and an interrupt, Module 20.)

```
int last = HIGH; unsigned long pulses = 0; bool done = false;
void setup() { Serial.begin(9600); pinMode(2, INPUT_PULLUP); }
void loop() {
  int s = digitalRead(2);
  if (last == HIGH && s == LOW) pulses++;
  last = s;
  if (!done && millis() >= 2000) {
    Serial.print("RPM = "); Serial.println(pulses * 30);   // pulses in 2 s × 30 = per minute
    done = true;                                           // report once (demo)
  }
}
```

SERIAL MONITOR

```
RPM = 150
```

Q19 **Answer: (d) 42.1**

(126 310 - 84 210) / 1000 = **42.1**.

Q20 **Answer: (b) reduce it to 3.3 V with a divider (or use a 3.3 V sensor)**

ESP32 inputs are **not 5 V tolerant**.

Q21 **Answer: Solution program**

The gyroscope term follows fast changes; the small accelerometer term slowly corrects drift.

```
void setup() {
  Serial.begin(9600);
  float angle = 0, gyro = 10.0, accelAngle = 5.0, dt = 0.1;
  for (int i = 1; i <= 5; i++) {
    angle = 0.98 * (angle + gyro * dt) + 0.02 * accelAngle;
    Serial.print("step "); Serial.print(i); Serial.print(": "); Serial.println(angle, 2);
  }
}
void loop() {}
```

SERIAL MONITOR

```
step 1: 1.08
step 2: 2.14
step 3: 3.18
step 4: 4.19
step 5: 5.19
```

Q22 **Answer: (b) it reads slightly short (about 2.5 % less than the true distance)**

Sound is faster (≈ 352 m/s), the echo returns sooner, so distance is **under-read by about 2.5 %**.

Q23 **Answer: Solution program**

Track running min, max and sum; the simulated readings were 24.50, 25.00 and 25.75 °C.

```
#include <OneWire.h>
#include <DallasTemperature.h>
OneWire bus(4); DallasTemperature s(&bus);
void setup() {
  Serial.begin(9600); s.begin();
  float mn = 1000, mx = -1000, sum = 0;
  for (int i = 0; i < 3; i++) {
    s.requestTemperatures();
    float t = s.getTempCByIndex(0);
    mn = min(mn, t); mx = max(mx, t); sum += t;
  }
  Serial.print("min "); Serial.print(mn, 2);
  Serial.print("  max "); Serial.print(mx, 2);
  Serial.print("  avg "); Serial.println(sum / 3, 2);
}
void loop() {}
```

SERIAL MONITOR

```
min 24.50  max 25.75  avg 25.08
```

Q24 **Answer: (d) to reduce electrolytic corrosion of the exposed traces**

Constant current through water **corrodes** the traces.

Q25 **Answer: (a) RC522 RFID module**

The **RC522** reads 13.56 MHz MIFARE cards over SPI.

HC-SR04: cm = echo µs / 58 (10 µs trigger, timeout for no echo). DHT22: read every 2 s, check isnan(). DS18B20: One-Wire with 4.7 kΩ pull-up. MPU6050: tilt from atan2, gyro for rate, combine with a complementary filter. Match logic levels, warm up gas sensors, and calibrate with two known points.

MODULE 14 · INTERMEDIATE

Displays

Minimum duration **2 days** | Level **Intermediate**

Displays let a project show information without a computer. This module covers single and multi-digit 7-segment displays and TM1637 modules, 16×2 character LCDs in parallel mode and with I2C backpacks, custom characters, SSD1306 OLED displays with text, shapes and bitmaps, MAX7219 LED matrices, WS2812B addressable LED strips, and an overview of TFT colour displays and touchscreens, followed by 25 solved questions and programs.

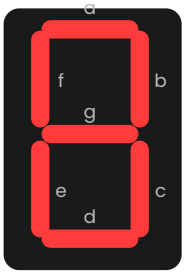
IN THIS MODULE

14.1 7-segment displays
14.2 Character LCDs
14.3 OLED displays
14.4 LED matrices and addressable LEDs
14.5 TFT colour displays and touchscreens (overview)
14.6 Practice questions
14.7 Solutions

14.1 7-segment displays

14.1.1 Single and multi-digit displays

A 7-segment display has seven LED bars (a–g) plus a decimal point. In a **common-cathode** display all cathodes join to GND and a HIGH segment pin lights a segment; in a **common-anode** display the shared pin goes to 5 V and LOW lights a segment.



segments a–g

SEGMENT NAMES

Digit	Segments lit	Byte (gfedcba)
0	a b c d e f	0b0111111
1	b c	0b0000110
2	a b d e g	0b1011011
3	a b c d g	0b1001111

Digit	Segments lit	Byte (gfedcba)
4	b c f g	0b1100110
5	a c d f g	0b1101101
6	a c d e f g	0b1111101
7	a b c	0b0000111
8	all	0b1111111
9	a b c d f g	0b1101111

Multi-digit displays share the segment pins and light one digit at a time very quickly (**multiplexing**); persistence of vision makes all digits appear lit. This needs many pins and constant refreshing, so driver modules are usually preferred.

14.1.2 TM1637 modules

TM1637.INO

```
#include <TM1637Display.h>
TM1637Display disp(2, 3);            // CLK, DIO

void setup() {
  disp.setBrightness(5);
  disp.showNumberDec(42);            // right-aligned
  delay(1000);
  disp.showNumberDecEx(1230, 0b01000000, true);   // 12:30 with the colon
}
void loop() {}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] TM1637 shows [  42]
[  1000 ms] TM1637 shows [12:30]
```

14.2 Character LCDs

14.2.1 16×2 LCD in parallel mode

HD44780-compatible LCDs show 2 rows of 16 characters. In 4-bit parallel mode they need 6 Arduino pins (RS, E, D4–D7) plus power, a contrast potentiometer on V0 and a backlight resistor.

PARALLEL LCD

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);   // RS, E, D4, D5, D6, D7
void setup() {
  lcd.begin(16, 2);
  lcd.print("Hello, world!");
}
void loop() {}
```

14.2.2 I2C LCD backpacks

A PCF8574 backpack converts the LCD to I2C: only SDA (A4) and SCL (A5) plus power. Common addresses are 0x27 and 0x3F (find yours with the I2C scanner in Module 18).

I2C_LCD.INO

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);     // address, columns, rows

void setup() {
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0); lcd.print("Temp: 24.5 C");
  lcd.setCursor(0, 1); lcd.print("Hum : 61 %");
}
void loop() {}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] LCD (0,0) "Temp: 24.5 C"
[     0 ms] LCD (0,1) "Hum : 61 %"
```



WHAT THE 16×2 LCD SHOWS

WATCH OUT

If the LCD lights but shows nothing (or only blocks), adjust the contrast potentiometer on the backpack, then check the I2C address.

14.2.3 Custom characters

Each LCD can hold 8 custom 5×8-pixel characters. Define them as 8 bytes (one per row, 5 low bits used) and store with `lcd.createChar(n, bytes)`; print with `lcd.write(n)`.

A DEGREE SYMBOL

```
byte degree[8] = {0b00110, 0b01001, 0b01001, 0b00110, 0, 0, 0, 0};   // the ° symbol
```

14.3 OLED displays

14.3.1 SSD1306 OLED with Adafruit GFX or U8g2

SSD1306 OLEDs (128×64 or 128×32 pixels, I2C address 0x3C) give sharp graphics with no backlight. Adafruit_SSD1306 + Adafruit_GFX draw into a 1 KB buffer in RAM (half the Uno R3's SRAM!) and send it with display(); U8g2 can draw page by page with less RAM.

14.3.2 Text, shapes and bitmaps

OLED.INO

```
#include <Wire.h>
#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 oled(128, 64, &Wire, -1);

void setup() {
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  oled.clearDisplay();
  oled.setTextSize(2);
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(0, 0);
  oled.println("23.5 C");
  oled.drawRect(0, 40, 128, 20, SSD1306_WHITE);   // frame
  oled.fillRect(2, 42, 70, 16, SSD1306_WHITE);    // bar graph
  oled.display();                                  // nothing appears until display()
}
void loop() {}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] OLED cleared
[     0 ms] OLED text (0,0): 23.5 C
[     0 ms] OLED display() refresh
```

With Adafruit_SSD1306, every drawing command only changes the buffer in RAM: call display() to show it. Call clearDisplay() before redrawing a frame.

14.4 LED matrices and addressable LEDs

14.4.1 MAX7219 matrices

The MAX7219 drives an 8×8 LED matrix (64 LEDs) or 8 seven-segment digits over SPI-like 3-wire interface (DIN, CLK, CS), handling multiplexing and current limiting itself. Modules chain for scrolling text (libraries: LedControl, MD_Parola).

14.4.2 WS2812B (NeoPixel) strips

Each WS2812B LED contains a tiny controller; the whole strip is driven from one data pin with 24-bit colour per LED. Each LED can draw up to 60 mA at full white, so power long strips from a separate 5 V supply, add a 300–500 Ω resistor in the data line and a 1000 µF capacitor across the supply.

NEOPIXEL.INO

```
#include <Adafruit_NeoPixel.h>
Adafruit_NeoPixel strip(5, 6, NEO_GRB + NEO_KHZ800);   // 5 LEDs on pin 6

void setup() {
  strip.begin();
  for (int i = 0; i < 5; i++) strip.setPixelColor(i, strip.Color(255, 0, 0));   // red
  strip.setPixelColor(2, strip.Color(0, 0, 255));                               // middle blue
  strip.show();                                                                 // send the data
}
void loop() {}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] NeoPixels: FF0000 FF0000 0000FF FF0000 FF0000
```

NeoPixel current: up to 60 mA per LED at full white. 30 LEDs → up to 1.8 A: never power long strips from the Arduino 5V pin.

14.5 TFT colour displays and touchscreens (overview)

TFT displays (ILI9341, ST7735, ST7789) show colour graphics over SPI; many include resistive or capacitive touch and an SD slot. Their frame buffers are large, so they suit faster boards (ESP32, Uno R4, Mega) better than the Uno R3. Libraries: Adafruit_GFX with device drivers, TFT_eSPI (ESP32), LVGL for full GUIs.

14.6 Practice questions

14.6.1 Basic level

Q1 How many segments (excluding the decimal point) does a 7-segment display have?

(a) 8 **(b)** 6 **(c)** 7 **(d)** 16

Q2 In a common-cathode 7-segment display, how is a segment lit?

(a) by setting its pin LOW **(b)** by connecting it to 12 V
(c) by PWM only **(d)** by setting its pin HIGH

Q3 Which segments light to show the digit 1?

(a) a and d **(b)** b and c **(c)** e and f **(d)** g only

Q4 A 16×2 LCD shows how many characters in total?

(a) 32 **(b)** 16 **(c)** 18 **(d)** 64

Q5 Which two pins does an I2C LCD use on an Uno R3?

(a) D0 and D1 **(b)** D10 and D11
(c) A4 (SDA) and A5 (SCL) **(d)** A0 and A1

Q6 What are common I2C addresses of LCD backpacks?

(a) 0x27 and 0x3F **(b)** 0x68 and 0x69 **(c)** 0x3C only **(d)** 0x76 and 0x77

Q7 What is the usual I2C address of an SSD1306 OLED?

(a) 0x27 **(b)** 0x3C **(c)** 0x68 **(d)** 0x50

Q8 How many data pins does a WS2812B strip need?

(a) 3 **(b)** 8 **(c)** 24 **(d)** 1

14.6.2 Intermediate level

Q9 Show "Count: n" on line 1 of an I2C LCD (0x27, 16×2), counting from 0 to 2 once per second.

Q10 Your LCD lights up but shows no text. What should you adjust first?

(a) the baud rate **(b)** the USB cable
(c) the contrast potentiometer **(d)** the sketch delay

Q11 With Adafruit_SSD1306, text is drawn but nothing appears on the screen. What is missing?

(a) a call to begin(9600) **(b)** a delay
(c) a call to display() **(d)** pinMode()

Q12 Why can a 128×64 SSD1306 with Adafruit's library be a problem on an Uno R3?

(a) it needs 12 V
(b) it uses SPI only
(c) it needs 20 pins
(d) its 1 KB frame buffer uses half of the 2 KB SRAM

Q13 Light the digit 7 on a common-cathode 7-segment display with segments a–g on pins 2–8.

Q14 A 60-LED WS2812B strip at full white can draw up to

(a) 60 mA **(b)** 3.6 A **(c)** 360 mA **(d)** 36 A

Q15 Use a TM1637 to show a 4-digit count-up timer in seconds (show 0, 1, 2).

Q16 What is multiplexing in a 4-digit 7-segment display?

(a) lighting one digit at a time very quickly so all appear lit
(b) using four Arduinos
(c) showing hexadecimal
(d) dimming with PWM

14.6.3 Advanced level

Q17 Make the first 3 NeoPixels of a 5-LED strip green and the rest off.

Q18 Which byte (bits 0–4 used) makes the top row of a custom LCD character fully lit?

(a) 0b00000 **(b)** 0b11111 **(c)** 0b10001 **(d)** 0b11111111

Q19 Which component should be placed in series with the NeoPixel data line?

(a) a 300–500 Ω resistor **(b)** a 10 kΩ pull-up only
(c) a diode **(d)** a 100 µF capacitor

Q20 Display a temperature with one decimal and a degree-like "C" on an I2C LCD line 2, e.g. "T=23.4 C".

Q21 Which display is most suitable for showing a small graph on a battery-powered gadget?

(a) 16×2 character LCD **(b)** single 7-segment digit
(c) SSD1306 OLED **(d)** MAX7219 matrix

Q22 Draw a horizontal bar graph on an OLED whose length is proportional to a potentiometer on A0 (full width 128 pixels).

Q23 Why are TFT colour displays better suited to an ESP32 than to an Uno R3?

(a) they need Wi-Fi
(b) they work only at 3.3 V
(c) Uno R3 has no SPI
(d) they need much more memory and speed than the Uno R3 has

Q24 Scroll a "moving dot" along a 5-LED NeoPixel strip, one position every 200 ms (show 3 steps).

Q25 The MAX7219 can drive how many LEDs in an 8×8 matrix?

(a) 8 **(b)** 16 **(c)** 128 **(d)** 64

14.7 Solutions

Q1 **Answer: (c) 7**

Segments a–g: **7**.

Q2 **Answer: (d) by setting its pin HIGH**

Cathodes go to GND; **HIGH** lights the segment.

Q3 **Answer: (b) b and c**

The right-hand vertical bars: **b and c**.

Q4 **Answer: (a) 32**

2 rows × 16 = **32**.

Q5 **Answer: (c) A4 (SDA) and A5 (SCL)**

I2C on the Uno is **A4/A5** (also the SDA/SCL header pins).

Q6 **Answer: (a) 0x27 and 0x3F**

PCF8574 backpacks: **0x27 or 0x3F**.

Q7 **Answer: (b) 0x3C**

SSD1306: **0x3C** (sometimes 0x3D).

Q8 **Answer: (d) 1**

One data line drives the whole strip.

Q9 **Answer: Solution program**

setCursor(0, 0) returns to the start of line 1 each time; the number overwrites the previous one.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
int n = 0;
void setup() { lcd.init(); lcd.backlight(); }
void loop() {
  lcd.setCursor(0, 0);
  lcd.print("Count: ");
  lcd.print(n);
  n++;
  delay(1000);
}
```

SIMULATED RUN

```
[     0 ms] LCD (0,0) "Count: "
[     0 ms] LCD (7,0) "0"
[  1000 ms] LCD (0,0) "Count: "
[  1000 ms] LCD (7,0) "1"
[  2000 ms] LCD (0,0) "Count: "
[  2000 ms] LCD (7,0) "2"
```

Q10 **Answer: (c) the contrast potentiometer**

Wrong **contrast** is the commonest cause.

Q11 **Answer: (c) a call to display()**

Drawing goes to the buffer; **display()** sends it.

Q12 **Answer: (d) its 1 KB frame buffer uses half of the 2 KB SRAM**

128 × 64 / 8 = **1024 bytes** of RAM.

Q13 **Answer: Solution program**

Bits 0–2 (a, b, c) are 1, so only pins 2, 3 and 4 go HIGH.

```
const int seg[7] = {2, 3, 4, 5, 6, 7, 8};      // a b c d e f g
const byte SEVEN = 0b0000111;                  // a, b, c
void setup() {
  for (int i = 0; i < 7; i++) {
    pinMode(seg[i], OUTPUT);
    digitalWrite(seg[i], bitRead(SEVEN, i));
  }
}
void loop() {}
```

SIMULATED RUN

```
[     0 ms] pin 2 -> HIGH
[     0 ms] pin 3 -> HIGH
[     0 ms] pin 4 -> HIGH
[     0 ms] pin 5 -> LOW
[     0 ms] pin 6 -> LOW
[     0 ms] pin 7 -> LOW
[     0 ms] pin 8 -> LOW
```

Q14 **Answer: (b) 3.6 A**

60 × 60 mA = **3.6 A**.

Q15 **Answer: Solution program**

showNumberDec(n, true) pads with zeros (0001). The timer is non-blocking.

```
#include <TM1637Display.h>
TM1637Display d(2, 3);
unsigned long last = 0; int sec = 0;
void setup() { d.setBrightness(4); d.showNumberDec(0, true); }
void loop() {
  if (millis() - last >= 1000) {
    last = millis(); sec++;
    d.showNumberDec(sec, true);     // true = show leading zeros
  }
}
```

SIMULATED RUN

```
[     0 ms] TM1637 shows [0000]
[  1000 ms] TM1637 shows [0001]
[  2000 ms] TM1637 shows [0002]
```

Q16 **Answer: (a) lighting one digit at a time very quickly so all appear lit**

Fast switching + **persistence of vision**.

Q17 **Answer: Solution program**

clear() sets all pixels to off in the buffer; show() sends the colours. 00FF00 is green.

```
#include <Adafruit_NeoPixel.h>
Adafruit_NeoPixel strip(5, 6, NEO_GRB + NEO_KHZ800);
void setup() {
  strip.begin();
  strip.clear();
  for (int i = 0; i < 3; i++) strip.setPixelColor(i, strip.Color(0, 255, 0));
  strip.show();
}
void loop() {}
```

SIMULATED RUN

```
[     0 ms] NeoPixels: 00FF00 00FF00 00FF00 000000 000000
```

Q18 **Answer: (b) 0b11111**

Five pixels wide: **0b11111**.

Q19 **Answer: (a) a 300–500 Ω resistor**

A **300–500 Ω** resistor reduces ringing on the data line.

Q20 **Answer: Solution program**

print(float, 1) rounds to one decimal place: 23.4.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
void setup() {
  lcd.init(); lcd.backlight();
  float t = 23.44;
  lcd.setCursor(0, 1);
  lcd.print("T=");
  lcd.print(t, 1);
  lcd.print(" C");
}
void loop() {}
```

SIMULATED RUN

```
[     0 ms] LCD (0,1) "T="
[     0 ms] LCD (2,1) "23.4"
[     0 ms] LCD (6,1) " C"
```

Q21 **Answer: (c) SSD1306 OLED**

OLEDs draw pixels and need no backlight, so they are **graphic and low power**.

Q22 **Answer: Solution program**

map() converts the reading to a pixel width. Each frame clears, draws and displays.

```
#include <Wire.h>
#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 oled(128, 64, &Wire, -1);
void setup() {
  Serial.begin(9600);
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}
void loop() {
  int w = map(analogRead(A0), 0, 1023, 0, 128);
  oled.clearDisplay();
  oled.fillRect(0, 24, w, 16, SSD1306_WHITE);
  oled.display();
  Serial.print("bar width "); Serial.println(w);
  delay(100);
}
```

SERIAL MONITOR

```
bar width 0
bar width 64
bar width 128
```

Q23 **Answer: (d) they need much more memory and speed than the Uno R3 has**

Colour frame buffers need **RAM and speed**.

Q24 **Answer: Solution program**

Each frame clears the strip, lights one pixel and advances the position, wrapping at the end.

```
#include <Adafruit_NeoPixel.h>
Adafruit_NeoPixel strip(5, 6, NEO_GRB + NEO_KHZ800);
int pos = 0;
void setup() { strip.begin(); }
void loop() {
  strip.clear();
  strip.setPixelColor(pos, strip.Color(255, 255, 255));
  strip.show();
  pos = (pos + 1) % 5;
  delay(200);
}
```

SIMULATED RUN

```
[     0 ms] NeoPixels: FFFFFF 000000 000000 000000 000000
[   200 ms] NeoPixels: 000000 FFFFFF 000000 000000 000000
[   400 ms] NeoPixels: 000000 000000 FFFFFF 000000 000000
```

Q25 **Answer: (d) 64**

8 × 8 = **64**.

7-segment: common cathode lights on HIGH; use TM1637 or MAX7219 to save pins. LCD 16×2: I2C backpack (0x27/0x3F) on SDA/SCL; adjust contrast. OLED SSD1306 (0x3C): draw into the buffer, then display(); mind the 1 KB RAM. NeoPixels: one data pin, up to 60 mA per LED, separate supply, resistor and capacitor.

MODULE 15 · INTERMEDIATE

Sound and user-input devices

Minimum duration **2 days** | Level **Intermediate**

Sound and controls make projects interactive. This module covers active and passive buzzers, tone() and playing melodies, DFPlayer Mini MP3 modules, matrix keypads, rotary encoders, joysticks, IR remote receivers, capacitive touch sensors, and building simple menus on an LCD, followed by 25 solved questions and programs with simulated key presses and encoder turns.

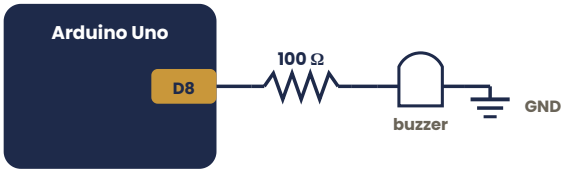
IN THIS MODULE

15.1 Sound output
15.2 Input devices
15.3 Simple user interfaces
15.4 Practice questions
15.5 Solutions

15.1 Sound output

15.1.1 Active and passive buzzers

	Active buzzer	Passive buzzer
Inside	its own oscillator	just a speaker element
How to drive	digitalWrite HIGH = beep	tone(pin, frequency) square wave
Pitch	fixed	any frequency: melodies
How to tell	beeps on plain DC; often sealed underneath	only clicks on DC; open circuit board underneath



A PASSIVE BUZZER ON PIN 8 (SERIES RESISTOR LIMITS CURRENT)

15.1.2 tone() and playing melodies

`tone(pin, frequency)` produces a square wave until `noTone(pin)`; `tone(pin, frequency, duration)` stops automatically. Only one tone at a time on an Uno, and tone() uses Timer2, which disables PWM on pins 3 and 11 while it runs.

Note	C4	D4	E4	F4	G4	A4	B4	C5
Hz	262	294	330	349	392	440	494	523

MELODY.INO

```
const int BUZ = 8;
int melody[]   = {262, 294, 330, 262, 0, 392};      // 0 = rest
int duration[] = {250, 250, 250, 250, 250, 500};

void setup() {
  for (int i = 0; i < 6; i++) {
    if (melody[i]) tone(BUZ, melody[i], duration[i]);
    delay(duration[i] * 1.3);                        // note plus a short gap
  }
  noTone(BUZ);
}
void loop() {}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] tone pin 8 262 Hz for 250 ms
[   325 ms] tone pin 8 294 Hz for 250 ms
[   650 ms] tone pin 8 330 Hz for 250 ms
[   975 ms] tone pin 8 262 Hz for 250 ms
[  1625 ms] tone pin 8 392 Hz for 500 ms
[  2275 ms] noTone pin 8
```

Frequency doubles every octave: A4 = 440 Hz, A5 = 880 Hz. Each semitone multiplies the frequency by about 1.0595 (the twelfth root of 2).

15.1.3 MP3 player modules (DFPlayer Mini)

The DFPlayer Mini plays MP3 files from a micro-SD card through a small speaker (up to 3 W) and is controlled over serial at 9600 baud (use SoftwareSerial on the Uno, with a 1 kΩ resistor in its RX line). Files are named 0001.mp3, 0002.mp3…; the DFRobotDFPlayerMini library provides play(n), volume(0–30), next() and more.

15.2 Input devices

15.2.1 Matrix keypads

A 4×4 keypad has 16 buttons arranged in 4 rows and 4 columns: 8 wires instead of 16. The Keypad library scans by driving each row and reading the columns.

KEYPADLOCK.INO

```
#include <Keypad.h>
const byte ROWS = 4, COLS = 4;
char keys[ROWS][COLS] = {{'1','2','3','A'}, {'4','5','6','B'}, {'7','8','9','C'},
{'*','0','#','D'}};
byte rowPins[ROWS] = {9, 8, 7, 6};
byte colPins[COLS] = {5, 4, 3, 2};
Keypad pad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

String code = "";

void setup() { Serial.begin(9600); }

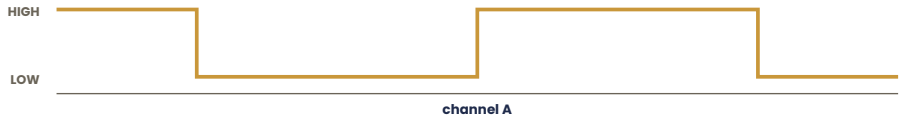
void loop() {
  char k = pad.getKey();
  if (k == NO_KEY) return;
  if (k == '#') {                                  // enter
    Serial.println(code == "1234" ? "Access granted" : "Wrong code");
    code = "";
  } else if (k == '*') code = "";                  // clear
  else code += k;
}
```

SERIAL MONITOR

```
Wrong code
Access granted
```

15.2.2 Rotary encoders

A rotary encoder outputs two square waves, A and B, a quarter cycle apart. When A falls, B tells the direction: B HIGH → one way, B LOW → the other. Encoders turn endlessly (unlike a potentiometer) and often include a push button.



ENCODER CHANNEL A



ENCODER CHANNEL B

ENCODER.INO

```
const int A = 2, B = 3;
int lastA = HIGH;
long position = 0;

void setup() {
  Serial.begin(9600);
  pinMode(A, INPUT_PULLUP);
  pinMode(B, INPUT_PULLUP);
}

void loop() {
  int a = digitalRead(A);
  if (lastA == HIGH && a == LOW) {                    // falling edge on A
    if (digitalRead(B) == HIGH) position++; else position--;
    Serial.print("position "); Serial.println(position);
  }
  lastA = a;
}
```

SERIAL MONITOR

```
position 1
position 2
position 1
```

In the simulation the encoder clicked twice one way (B HIGH) and once the other way (B LOW). For fast turning, read the encoder with interrupts (Module 20).

15.2.3 Joysticks

A thumb joystick is two potentiometers (X on one analog pin, Y on another) plus a push button. At rest each axis reads about 512; use a dead zone (for example ±50) so small drift is ignored.

15.2.4 IR remote control receivers

IR receivers (TSOP1838, VS1838B) demodulate the 38 kHz infrared bursts from TV remotes and output a digital pulse train. The IRremote library decodes it into a code for each button (NEC protocol is the most common), which the sketch maps to actions.

15.2.5 Capacitive touch sensors

TTP223 touch modules give a digital HIGH when a finger touches the pad (through thin plastic too). ESP32 boards have built-in touch pins read with touchRead().

15.3 Simple user interfaces

15.3.1 Menus on an LCD controlled by buttons or an encoder

MENU.INO

```
const char *items[] = {"Set time", "Set alarm", "Brightness", "Exit"};
const int N = 4;
int sel = 0;

void showMenu() {
  Serial.print("> "); Serial.println(items[sel]);       // on an LCD: line 1
  Serial.print("  "); Serial.println(items[(sel + 1) % N]);   // line 2
}

void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP);                            // "next" button
  showMenu();
}

int last = HIGH;
void loop() {
  int b = digitalRead(2);
  if (last == HIGH && b == LOW) { sel = (sel + 1) % N; showMenu(); delay(20); }
  last = b;
}
```

SERIAL MONITOR

```
> Set time
  Set alarm
> Set alarm
  Brightness
> Brightness
  Exit
```

A menu is a state (the selected item) plus actions: "next" moves the selection, "select" enters the item. With an encoder, turning moves the selection and pressing selects.

15.4 Practice questions

15.4.1 Basic level

Q1 Which buzzer can play different notes using tone()?

(a) active buzzer
(b) piezo disc with oscillator only
(c) passive buzzer
(d) relay

Q2 What does tone(8, 440, 500) do?

(a) plays 440 Hz on pin 8 for 500 ms
(b) plays 500 Hz for 440 ms
(c) sets pin 8 to 440 V
(d) plays forever

Q3 Which function stops a tone?

(a) tone(pin, 0, 0) only
(b) stopTone()
(c) digitalWrite(pin, LOW) always
(d) noTone(pin)

Q4 What is the frequency of A4 (concert pitch)?

(a) 262 Hz (b) 440 Hz (c) 880 Hz (d) 523 Hz

Q5 How many wires does a 4×4 matrix keypad need?

(a) 16 (b) 8 (c) 4 (d) 17

Q6 A joystick at rest reads about what on each axis (10-bit ADC)?

(a) 0 (b) 1023 (c) 255 (d) 512

Q7 What carrier frequency do most IR remote controls use?

(a) 38 kHz (b) 2.4 GHz (c) 50 Hz (d) 433 MHz

Q8 A rotary encoder differs from a potentiometer because it

(a) gives a voltage 0–5 V
(b) needs 12 V
(c) turns endlessly and reports steps and direction
(d) has no pins

15.4.2 Intermediate level

Q9 Play a rising scale C4 to C5 (262, 294, 330, 349, 392, 440, 494, 523 Hz), 200 ms per note, on a passive buzzer on pin 8.

Q10 What is the frequency one octave above 330 Hz (E4)?

(a) 440 Hz (b) 495 Hz (c) 165 Hz (d) 660 Hz

Q11 Why can tone() interfere with analogWrite() on pins 3 and 11 of the Uno?

(a) tone() uses too much current
(b) tone() uses Timer2, which also generates PWM on those pins
(c) pins 3 and 11 are analog only
(d) they share the USB

Q12 Read a 4×4 keypad and print each key pressed; when "D" is pressed print "Delete".

Q13 An encoder's A channel falls while B is LOW. With the convention used in this module, what happened?

(a) one step in the negative direction **(b)** one step in the positive direction
(c) the button was pressed **(d)** nothing

Q14 What is a joystick "dead zone"?

(a) a broken area of the joystick
(b) the maximum deflection
(c) a range around the centre treated as zero to ignore small drift
(d) a zone with no power

Q15 Convert joystick X and Y readings into "LEFT", "RIGHT", "UP", "DOWN" or "CENTRE" with a dead zone of ±100.

Q16 How is a DFPlayer Mini usually controlled from an Uno?

(a) serial commands at 9600 baud (often via SoftwareSerial)
(b) I2C at 0x27
(c) PWM only
(d) SPI with an SD card on the Arduino

15.4.3 Advanced level

Q17 Build a code lock: the correct code is 2580. Print "OPEN" for the right code entered with "#", "ALARM" after three wrong codes.

Q18 Each semitone multiplies frequency by about

(a) 1.5 **(b)** 1.0595 **(c)** 2 **(d)** 1.1

Q19 Use a rotary encoder (A on pin 2, B on pin 3) to change a volume value from 0 to 10, clamped at the limits; print each change.

Q20 A TTP223 touch module outputs what when touched?

(a) an analog voltage proportional to pressure
(b) a PWM signal
(c) a serial message
(d) a digital HIGH (default mode)

Q21 Which library is commonly used to decode IR remote buttons?

(a) Servo **(b)** Wire **(c)** IRremote **(d)** FastLED

Q22 Make an alarm siren: sweep a passive buzzer on pin 8 from 500 Hz to 1000 Hz in steps of 250 Hz, then back down, 100 ms per step.

Q23 In a 4-item menu with "next" wrapping round, the selection is item 3 (0-based). After "next" it becomes

(a) 4 **(b)** 3 **(c)** 0 **(d)** 1

Q24 Why does a DFPlayer Mini need a 1 kΩ resistor in its RX line when used with a 5 V Arduino?

(a) to make it louder

(b) to set the volume

(c) to slow the baud rate

(d) its serial input is 3.3 V and the resistor limits current from the 5 V TX

Q25 Print a two-line menu view for items {"Start", "Settings", "Info"} after pressing "next" (pin 2) twice.

15.5 Solutions

Q1 **Answer: (c) passive buzzer**

A **passive** buzzer follows the frequency you send.

Q2 **Answer: (a) plays 440 Hz on pin 8 for 500 ms**

pin, frequency, duration.

Q3 **Answer: (d) noTone(pin)**

noTone(pin).

Q4 **Answer: (b) 440 Hz**

A4 = **440 Hz**.

Q5 **Answer: (b) 8**

4 rows + 4 columns = **8**.

Q6 **Answer: (d) 512**

The centre of 0–1023 is about **512**.

Q7 **Answer: (a) 38 kHz**

IR remotes modulate at about **38 kHz**.

Q8 **Answer: (c) turns endlessly and reports steps and direction**

Encoders are **incremental** and have no end stops.

Q9 **Answer: Solution program**

A range-based for loop plays each frequency; the 20 ms extra delay separates the notes.

```
int scale[] = {262, 294, 330, 349, 392, 440, 494, 523};
void setup() {
  for (int f : scale) {
    tone(8, f, 200);
    delay(220);
  }
}
void loop() {}
```

SIMULATED RUN

```
[     0 ms] tone pin 8 262 Hz for 200 ms
[   220 ms] tone pin 8 294 Hz for 200 ms
[   440 ms] tone pin 8 330 Hz for 200 ms
[   660 ms] tone pin 8 349 Hz for 200 ms
[   880 ms] tone pin 8 392 Hz for 200 ms
[  1100 ms] tone pin 8 440 Hz for 200 ms
[  1320 ms] tone pin 8 494 Hz for 200 ms
[  1540 ms] tone pin 8 523 Hz for 200 ms
```

Q10 **Answer: (d) 660 Hz**

An octave **doubles** the frequency.

Q11 **Answer: (b) tone() uses Timer2, which also generates PWM on those pins**

Both use **Timer2**.

Q12 **Answer: Solution program**

getKey() returns NO_KEY when nothing is pressed. Simulated keys: 7, A, D.

```
#include <Keypad.h>
char keys[4][4] = {{'1','2','3','A'},{'4','5','6','B'},{'7','8','9','C'},{'*','0','#','D'}};
byte rp[4] = {9, 8, 7, 6}, cp[4] = {5, 4, 3, 2};
Keypad kp = Keypad(makeKeymap(keys), rp, cp, 4, 4);
void setup() { Serial.begin(9600); }
void loop() {
  char k = kp.getKey();
  if (k == NO_KEY) return;
  if (k == 'D') Serial.println("Delete");
  else { Serial.print("Key: "); Serial.println(k); }
}
```

SERIAL MONITOR

```
Key: 7
Key: A
Delete
```

Q13 **Answer: (a) one step in the negative direction**

B LOW at A's falling edge → **position--**.

Q14 **Answer: (c) a range around the centre treated as zero to ignore small drift**

It ignores readings near **512 ± a margin**.

Q15 **Answer: Solution program**

Centre the readings by subtracting 512, ignore small values, and choose the axis with the larger deflection.

```
void setup() { Serial.begin(9600); }
void loop() {
  int x = analogRead(A0) - 512, y = analogRead(A1) - 512;
  if (abs(x) < 100 && abs(y) < 100) Serial.println("CENTRE");
  else if (abs(x) > abs(y)) Serial.println(x > 0 ? "RIGHT" : "LEFT");
  else Serial.println(y > 0 ? "UP" : "DOWN");
  delay(100);
}
```

SERIAL MONITOR

```
CENTRE
RIGHT
DOWN
LEFT
```

Q16 **Answer: (a) serial commands at 9600 baud (often via SoftwareSerial)**

The DFPlayer takes **UART** commands.

Q17 **Answer: Solution program**

A counter tracks failures and resets after success or an alarm. Simulated entries: 1111#, 2222#, 3333#, 2580#.

```
#include <Keypad.h>
char keys[4][4] = {{'1','2','3','A'},{'4','5','6','B'},{'7','8','9','C'},{'*','0','#','D'}};
byte rp[4] = {9, 8, 7, 6}, cp[4] = {5, 4, 3, 2};
Keypad kp = Keypad(makeKeymap(keys), rp, cp, 4, 4);
String entry = ""; int wrong = 0;
void setup() { Serial.begin(9600); }
void loop() {
  char k = kp.getKey();
  if (k == NO_KEY) return;
  if (k != '#') { entry += k; return; }
  if (entry == "2580") { Serial.println("OPEN"); wrong = 0; }
  else if (++wrong >= 3) { Serial.println("ALARM"); wrong = 0; }
  else { Serial.print("wrong ("); Serial.print(wrong); Serial.println(")"); }
  entry = "";
}
```

SERIAL MONITOR

```
wrong (1)
wrong (2)
```

```
ALARM
OPEN
```

Q18 **Answer: (b) 1.0595**

The twelfth root of 2 ≈ **1.0595**.

Q19 **Answer: Solution program**

constrain() keeps the value in range however far the knob turns. Simulated: three clicks up, one down.

```
int lastA = HIGH, vol = 5;
void setup() { Serial.begin(9600); pinMode(2, INPUT_PULLUP); pinMode(3, INPUT_PULLUP); }
void loop() {
  int a = digitalRead(2);
  if (lastA == HIGH && a == LOW) {
    vol += digitalRead(3) == HIGH ? 1 : -1;
    vol = constrain(vol, 0, 10);
    Serial.print("volume "); Serial.println(vol);
  }
  lastA = a;
}
```

SERIAL MONITOR

```
volume 6
volume 7
volume 8
volume 7
```

Q20 **Answer: (d) a digital HIGH (default mode)**

It acts like a **digital button**.

Q21 **Answer: (c) IRremote**

IRremote decodes NEC and other protocols.

Q22 **Answer: Solution program**

Two loops sweep up and down. Larger step counts with smaller steps sound smoother.

```
void setup() {
  for (int f = 500; f <= 1000; f += 250) { tone(8, f); delay(100); }
  for (int f = 750; f >= 500; f -= 250) { tone(8, f); delay(100); }
  noTone(8);
}
void loop() {}
```

SIMULATED RUN

```
[     0 ms] tone pin 8 500 Hz
[   100 ms] tone pin 8 750 Hz
[   200 ms] tone pin 8 1000 Hz
[   300 ms] tone pin 8 750 Hz
```

```
[   400 ms] tone pin 8 500 Hz
[   500 ms] noTone pin 8
```

Q23 **Answer: (c) 0**

(3 + 1) % 4 = **0**.

Q24 **Answer: (d) its serial input is 3.3 V and the resistor limits current from the 5 V TX**

It protects the **3.3 V** input (and reduces noise).

Q25 **Answer: Solution program**

The highlighted item is marked with ">"; the next item fills the second line, wrapping round at the end.

```
const char *it[] = {"Start", "Settings", "Info"};
int sel = 0, last = HIGH;
void show() { Serial.print("> "); Serial.println(it[sel]); Serial.print("  ");
Serial.println(it[(sel + 1) % 3]); Serial.println("--"); }
void setup() { Serial.begin(9600); pinMode(2, INPUT_PULLUP); show(); }
void loop() {
  int b = digitalRead(2);
  if (last == HIGH && b == LOW) { sel = (sel + 1) % 3; show(); delay(20); }
  last = b;
}
```

SERIAL MONITOR

```
> Start
  Settings
--
> Settings
  Info
--
> Info
  Start
--
```

Active buzzers beep on HIGH; passive buzzers need tone(pin, Hz[, ms]) and noTone(). Octaves double the frequency (A4 = 440 Hz). Keypads scan rows and columns (Keypad library). Encoders: on A's falling edge, B gives the direction. Joysticks are two pots with a dead zone. IR remotes use 38 kHz (IRremote). Menus are a selected-item state changed by buttons or an encoder.

MODULE 16 · INTERMEDIATE

Motors and motion control

Minimum duration **2 days** | Level **Intermediate**

Motors turn electrical signals into movement. This module covers DC motors with transistors and H-bridge drivers (L298N, L293D, TB6612FNG) for speed and direction, servo motors with the Servo library and the PCA9685 driver, stepper motors (28BYJ-48 with ULN2003, NEMA 17 with A4988/DRV8825, AccelStepper), powering motors safely with separate supplies, common ground, flyback diodes and decoupling, and three motor projects, followed by 25 solved questions and programs.

IN THIS MODULE

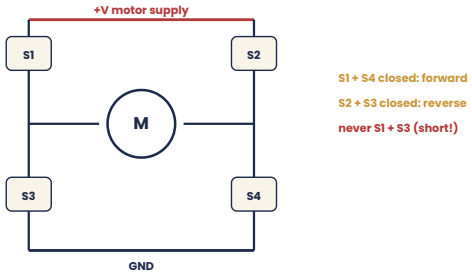
16.1 DC motors
16.2 Servo motors
16.3 Stepper motors
16.4 Power for motors
16.5 Motor projects
16.6 Practice questions
16.7 Solutions

16.1 DC motors

16.1.1 Transistor control

A transistor or MOSFET (Module 6) switches a DC motor on and off and, with PWM, controls its speed. It cannot reverse the motor. Always fit a flyback diode across the motor.

16.1.2 H-bridge drivers



AN H-BRIDGE: FOUR SWITCHES LET CURRENT FLOW THROUGH THE MOTOR EITHER WAY

Driver	Motors	Current per channel	Notes
L298N module	2 DC or 1 stepper	up to 2 A	bipolar transistors: drops about 2 V, gets hot; has a 5 V regulator
L293D chip	2 DC	0.6 A	built-in flyback diodes; good for small motors
TB6612FNG	2 DC	1.2 A (3.2 A peak)	MOSFET: efficient, little heat; 2.7–5.5 V logic

16.1.3 Speed and direction control

Each channel has two direction inputs (IN1, IN2) and an enable (ENA) that takes PWM for speed.

IN1	IN2	Motor
HIGH	LOW	forward
LOW	HIGH	reverse
LOW	LOW	coast (free stop)
HIGH	HIGH	brake

DCMOTOR.INO

```
const int ENA = 9, IN1 = 7, IN2 = 8;

void motor(int speed) {              // -255 … 255
  digitalWrite(IN1, speed > 0);
  digitalWrite(IN2, speed < 0);
  analogWrite(ENA, abs(speed));
}

void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  motor(200);  delay(2000);          // forward, fast
  motor(0);    delay(500);           // stop
  motor(-120); delay(2000);          // reverse, slower
  motor(0);
}
void loop() {}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] pin 7 -> HIGH
[     0 ms] pin 8 -> LOW
[     0 ms] PWM pin 9 = 200
[  2000 ms] pin 7 -> LOW
[  2000 ms] pin 8 -> LOW
[  2000 ms] PWM pin 9 = 0
[  2500 ms] pin 7 -> LOW
[  2500 ms] pin 8 -> HIGH
[  2500 ms] PWM pin 9 = 120
[  4500 ms] pin 7 -> LOW
[  4500 ms] pin 8 -> LOW
[  4500 ms] PWM pin 9 = 0
```

16.2 Servo motors

16.2.1 The Servo library

A hobby servo contains a motor, gears, a potentiometer and a controller. It moves to an angle (0–180°) set by a pulse 1–2 ms long (about 0.5–2.5 ms for full range), repeated every 20 ms (50 Hz). Wires: brown/black GND, red 5 V, orange/yellow signal.



SERVO CONTROL SIGNAL

SERVO.INO

```
#include <Servo.h>
Servo arm;

void setup() {
  arm.attach(9);
  arm.write(0);   delay(500);
  arm.write(90);  delay(500);
  arm.write(180); delay(500);
}
void loop() {}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] servo attached to pin 9
[     0 ms] servo pin 9 -> 0 deg
[   500 ms] servo pin 9 -> 90 deg
[  1000 ms] servo pin 9 -> 180 deg
```

WATCH OUT

A servo can draw 500 mA–1 A when starting or stalled. Power more than one small servo (or any large servo) from a separate 5–6 V supply, with grounds connected. On an Uno, the Servo library disables PWM on pins 9 and 10.

16.2.2 Controlling several servos

The Servo library drives up to 12 servos on an Uno, on any digital pins. Move them in small steps with short delays (or millis()) for smooth, coordinated motion.

16.2.3 PCA9685 servo driver

The PCA9685 board generates 16 independent PWM channels over I2C (address 0x40), so a robot arm or hexapod can drive 16 servos with just two pins; boards can be chained for more. Library: Adafruit_PWMServoDriver.

16.3 Stepper motors

16.3.1 28BYJ-48 with ULN2003

The cheap 28BYJ-48 is a geared unipolar stepper: about 2048 steps per output revolution (full-step mode, 4096 half-step). The ULN2003 board contains Darlington drivers. It is slow (about 10–15 rpm) but precise and holds its position.

STEPPER.INO

```
#include <Stepper.h>
const int STEPS_PER_REV = 2048;
Stepper motor(STEPS_PER_REV, 8, 10, 9, 11);   // IN1, IN3, IN2, IN4 order for the 28BYJ-48

void setup() {
  motor.setSpeed(10);              // rpm
  motor.step(STEPS_PER_REV / 4);   // quarter turn = 90 degrees
  motor.step(-STEPS_PER_REV / 4);  // back again
}
void loop() {}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] stepper speed 10 rpm
[     0 ms] stepper moves 512 steps
[     0 ms] stepper moves -512 steps
```

16.3.2 NEMA 17 with A4988 or DRV8825

NEMA 17 bipolar steppers (200 steps/rev, 1.8° per step) power 3D printers and CNC machines. A4988 and DRV8825 drivers need just STEP and DIR pins; each pulse on STEP moves one (micro)step. Microstepping (up to 1/16 on A4988, 1/32 on DRV8825) gives smoother motion. Set the current limit with the driver's potentiometer and add a 100 µF capacitor across the motor supply.

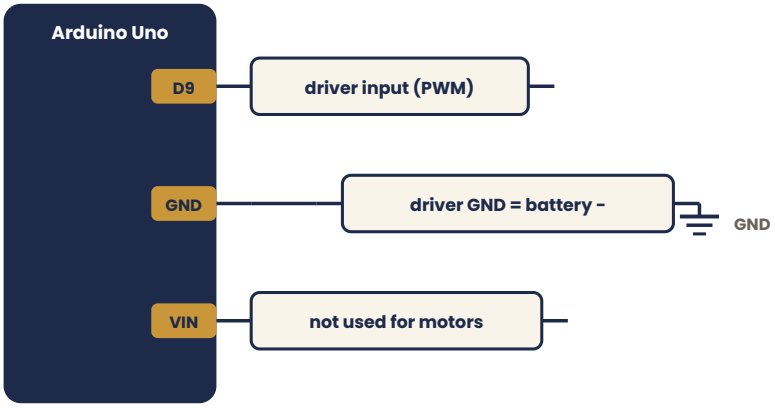
Steps per revolution = 360° / step angle × microsteps. NEMA 17 at 1/16 microstepping: 200 × 16 = 3200 steps per revolution.

16.3.3 AccelStepper library

AccelStepper adds acceleration and deceleration and non-blocking movement: call moveTo(target) once, then run() as often as possible in loop(); the motor ramps up, cruises and slows down while the rest of the sketch keeps working.

16.4 Power for motors

16.4.1 Separate power supplies and common ground



ARDUINO AND MOTOR SUPPLY SHARE GROUND; THE MOTOR SUPPLY FEEDS THE DRIVER DIRECTLY

Motors cause current surges and electrical noise that can reset the Arduino. Feed motors from their own battery or adapter through the driver, never from the Arduino 5V pin, and connect the grounds so the control signals have a common reference.

16.4.2 Flyback diodes and decoupling

Fit a flyback diode across DC motors and coils (many driver boards include them), and a 0.1 µF ceramic capacitor across the motor terminals to reduce electrical noise. A large electrolytic capacitor (100–1000 µF) across the motor supply smooths current surges.

16.5 Motor projects

16.5.1 Two-wheel robot car

ROBOTCAR.INO

```
// TB6612FNG or L298N, two motors
const int ENA = 5, IN1 = 7, IN2 = 8, ENB = 6, IN3 = 9, IN4 = 10;

void drive(int left, int right) {                    // -255 … 255 each
  digitalWrite(IN1, left > 0);  digitalWrite(IN2, left < 0);  analogWrite(ENA, abs(left));
  digitalWrite(IN3, right > 0); digitalWrite(IN4, right < 0); analogWrite(ENB, abs(right));
}

void setup() {
  for (int p : {IN1, IN2, IN3, IN4}) pinMode(p, OUTPUT);
  Serial.begin(9600);
  Serial.println("forward");    drive(200, 200);  delay(1000);
  Serial.println("turn right"); drive(200, -200); delay(400);   // spin on the spot
  Serial.println("stop");       drive(0, 0);
}
void loop() {}
```

SERIAL MONITOR

```
forward
turn right
stop
```

Differential drive: equal speeds go straight, unequal speeds curve, opposite speeds spin on the spot. Add an HC-SR04 (Module 13) to stop or turn when an obstacle is near.

16.5.2 Pan-tilt camera mount

Two servos (pan: left–right, tilt: up–down) on a bracket, controlled by a joystick (Module 15): map each axis to 0–180° and move the servos in small steps for smooth motion.

16.5.3 Smart door lock

A servo or solenoid moves a bolt; a keypad, RFID reader or Bluetooth command (Module 21) authorises opening. Add a reed switch to confirm the door is closed and a timeout that relocks automatically.

16.6 Practice questions

16.6.1 Basic level

Q1 Which device lets a DC motor run in both directions?

(a) a single transistor
(b) an H-bridge driver
(c) a potentiometer
(d) a 74HC595

Q2 With an L298N, IN1 = HIGH and IN2 = LOW gives

(a) forward rotation
(b) reverse rotation
(c) brake
(d) coast

Q3 Which pin of a motor driver takes PWM to set speed?

(a) IN1 **(b)** ENA (enable) **(c)** GND **(d)** VCC

Q4 What angle range does a standard hobby servo cover?

(a) 0–180°
(b) 0–360° continuous
(c) 0–90°
(d) 0–45°

Q5 How often is the servo control pulse repeated?

(a) every 1 ms
(b) every second
(c) every 20 ms (50 Hz)
(d) once only

Q6 Which servo wire is usually the signal?

(a) brown
(b) black
(c) red
(d) orange or yellow

Q7 How many steps per revolution does a 1.8° stepper have (full steps)?

(a) 180 **(b)** 360 **(c)** 100 **(d)** 200

Q8 Why should motors not be powered from the Arduino 5V pin?

(a) motors need AC
(b) the 5V pin is input only
(c) their current surges and noise can exceed the supply and reset the board
(d) motors run backwards

16.6.2 Intermediate level

Q9 Sweep a servo on pin 9 from 0° to 180° in 45° steps, 300 ms apart.

Q10 An H-bridge closes switches S1 and S3 on the same side at once. What happens?

(a) a short circuit from supply to ground ("shoot-through")
(b) the motor runs faster
(c) the motor brakes gently
(d) nothing

Q11 Why does the L298N waste more power than the TB6612FNG?

(a) it has no PWM

(b) it runs at 12 V only
(c) it has more channels
(d) its bipolar transistors drop about 2 V, which becomes heat

Q12 Write a function motor(speed) for one L298N channel (ENA 9, IN1 7, IN2 8) taking -255…255, and run it forward at 150 then reverse at 150.

Q13 How many steps make one full output revolution of a 28BYJ-48 in full-step mode?

(a) 200 **(b)** about 2048 **(c)** 360 **(d)** 64

Q14 A NEMA 17 driven by an A4988 at 1/16 microstepping needs how many STEP pulses per revolution?

(a) 200 **(b)** 3200 **(c)** 1600 **(d)** 6400

Q15 Move a 28BYJ-48 (2048 steps/rev) exactly 180° clockwise, then 90° back.

Q16 What does AccelStepper's run() need to be effective?

(a) to be called once in setup()
(b) a delay(1000) after it
(c) interrupts disabled
(d) to be called as often as possible in loop()

16.6.3 Advanced level

Q17 Drive a two-wheel robot: forward 1 s, gentle left curve (left 100, right 200) 1 s, stop. Print each action.

Q18 A servo jitters and the Arduino resets when the servo moves. The most likely cause is

(a) the servo draws too much current from the Arduino's 5 V supply
(b) the sketch uses Servo.h
(c) the angle is above 90°
(d) the signal wire is orange

Q19 Which IC can drive 16 servos over I2C?

(a) ULN2003 **(b)** L298N **(c)** PCA9685 **(d)** HX711

Q20 Control a servo with a potentiometer on A0, mapping 0–1023 to 0–180°.

Q21 Why is a capacitor of about 100 µF placed across a stepper driver's motor supply?

(a) to change the step angle
(b) to set the microstepping
(c) to slow the motor
(d) to absorb voltage spikes and smooth current surges that could destroy the driver

Q22 Ramp a DC motor (ENA 9, IN1 7, IN2 8) from stop to full speed in 5 steps of 51, 200 ms apart, forward.

Q23 A servo signal pulse is 1.0 ms. Approximately what angle results (1 ms = 0°, 2 ms = 180°)?

(a) 90° **(b)** 0° **(c)** 180° **(d)** 45°

Q24 Which driver has built-in flyback diodes and suits small 600 mA motors?

(a) L293D **(b)** A4988

(c) PCA9685 (d) ULN2003 for servos

Q25 Smoothly move a servo from 30° to 150° one degree at a time with 5 ms per degree, and print the time taken.

16.7 Solutions

Q1 **Answer: (b) an H-bridge driver**

An **H-bridge** reverses the current.

Q2 **Answer: (a) forward rotation**

HIGH/LOW = **forward** (by convention).

Q3 **Answer: (b) ENA (enable)**

The **enable** pin takes PWM.

Q4 **Answer: (a) 0–180°**

Standard servos: **0–180°**.

Q5 **Answer: (c) every 20 ms (50 Hz)**

Servo frames repeat at **50 Hz**.

Q6 **Answer: (d) orange or yellow**

Signal is **orange/yellow**; red is power; brown/black ground.

Q7 **Answer: (d) 200**

360 / 1.8 = **200**.

Q8 **Answer: (c) their current surges and noise can exceed the supply and reset the board**

Use a **separate supply** with common ground.

Q9 **Answer: Solution program**

write() sets the target angle; the delay gives the servo time to move.

```
#include <Servo.h>
Servo s;
void setup() {
  s.attach(9);
  for (int a = 0; a <= 180; a += 45) {
    s.write(a);
    delay(300);
  }
}
void loop() {}
```

SIMULATED RUN

```
[     0 ms] servo attached to pin 9
[     0 ms] servo pin 9 -> 0 deg
[   300 ms] servo pin 9 -> 45 deg
[   600 ms] servo pin 9 -> 90 deg
[   900 ms] servo pin 9 -> 135 deg
[  1200 ms] servo pin 9 -> 180 deg
```

Q10 **Answer: (a) a short circuit from supply to ground ("shoot-through")**

That shorts the supply: **shoot-through**.

Q11 **Answer: (d) its bipolar transistors drop about 2 V, which becomes heat**

The L298N's **voltage drop** is about 2 V.

Q12 **Answer: Solution program**

The sign of speed picks the direction; its absolute value sets the PWM duty.

```
const int ENA = 9, IN1 = 7, IN2 = 8;
void motor(int sp) {
  digitalWrite(IN1, sp > 0 ? HIGH : LOW);
  digitalWrite(IN2, sp < 0 ? HIGH : LOW);
  analogWrite(ENA, abs(sp));
}
void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  motor(150); delay(1000);
  motor(-150); delay(1000);
  motor(0);
}
void loop() {}
```

SIMULATED RUN

```
[     0 ms] pin 7 -> HIGH
[     0 ms] pin 8 -> LOW
[     0 ms] PWM pin 9 = 150
[  1000 ms] pin 7 -> LOW
[  1000 ms] pin 8 -> HIGH
[  1000 ms] PWM pin 9 = 150
[  2000 ms] pin 7 -> LOW
```

```
[  2000 ms] pin 8 -> LOW
[  2000 ms] PWM pin 9 = 0
```

Q13 **Answer: (b) about 2048**

With its gearbox: about **2048** (4096 half-steps).

Q14 **Answer: (b) 3200**

200 × 16 = **3200**.

Q15 **Answer: Solution program**

Steps = angle / 360 × steps per revolution.

```
#include <Stepper.h>
Stepper m(2048, 8, 10, 9, 11);
void setup() {
  m.setSpeed(12);
  m.step(1024);     // 180 degrees = half of 2048
  m.step(-512);     // 90 degrees back
}
void loop() {}
```

SIMULATED RUN

```
[     0 ms] stepper speed 12 rpm
[     0 ms] stepper moves 1024 steps
[     0 ms] stepper moves -512 steps
```

Q16 **Answer: (d) to be called as often as possible in loop()**

run() moves at most one step per call, so call it **very frequently**.

Q17 **Answer: Solution program**

A slower left wheel makes the robot curve to the left.

```
const int ENA = 5, IN1 = 7, IN2 = 8, ENB = 6, IN3 = 9, IN4 = 10;
void drive(int l, int r) {
  digitalWrite(IN1, l > 0); digitalWrite(IN2, l < 0); analogWrite(ENA, abs(l));
  digitalWrite(IN3, r > 0); digitalWrite(IN4, r < 0); analogWrite(ENB, abs(r));
}
void setup() {
  Serial.begin(9600);
  Serial.println("forward"); drive(200, 200); delay(1000);
  Serial.println("curve left"); drive(100, 200); delay(1000);
  Serial.println("stop"); drive(0, 0);
}
void loop() {}
```

SERIAL MONITOR

```
forward
curve left
```

```
stop
```

Q18 **Answer: (a) the servo draws too much current from the Arduino's 5 V supply**

Starting current of **0.5–1 A** sags the supply.

Q19 **Answer: (c) PCA9685**

The **PCA9685**, address 0x40.

Q20 **Answer: Solution program**

This is the classic "Knob" example; the 15 ms delay lets the servo follow.

```
#include <Servo.h>
Servo s;
void setup() { s.attach(9); }
void loop() {
  s.write(map(analogRead(A0), 0, 1023, 0, 180));
  delay(15);
}
```

SIMULATED RUN

```
[     0 ms] servo attached to pin 9
[     0 ms] servo pin 9 -> 0 deg
[    15 ms] servo pin 9 -> 90 deg
[    30 ms] servo pin 9 -> 180 deg
```

Q21 **Answer: (d) to absorb voltage spikes and smooth current surges that could destroy the driver**

It protects the driver from **spikes**.

Q22 **Answer: Solution program**

Ramping avoids a sudden current surge and mechanical shock.

```
void setup() {
  pinMode(7, OUTPUT); pinMode(8, OUTPUT);
  digitalWrite(7, HIGH); digitalWrite(8, LOW);
  for (int s = 51; s <= 255; s += 51) { analogWrite(9, s); delay(200); }
}
void loop() {}
```

SIMULATED RUN

```
[     0 ms] pin 7 -> HIGH
[     0 ms] pin 8 -> LOW
[     0 ms] PWM pin 9 = 51
[   200 ms] PWM pin 9 = 102
[   400 ms] PWM pin 9 = 153
[   600 ms] PWM pin 9 = 204
[   800 ms] PWM pin 9 = 255
```

Q23 **Answer: (b) 0°**

Using the 1–2 ms convention: **0°**. (Many servos need 0.5–2.5 ms for the full range.)

Q24 **Answer: (a) L293D**

The **L293D** includes clamp diodes.

Q25 **Answer: Solution program**

121 positions × 5 ms = 605 ms. Small steps make the movement smooth instead of a sudden jump.

```
#include <Servo.h>
Servo s;
void setup() {
  Serial.begin(9600);
  s.attach(9);
  unsigned long t0 = millis();
  for (int a = 30; a <= 150; a++) { s.write(a); delay(5); }
  Serial.print("took "); Serial.print(millis() - t0); Serial.println(" ms");
}
void loop() {}
```

SERIAL MONITOR

```
took 605 ms
```

DC motors: transistor for on/off and speed; H-bridge (L298N, L293D, TB6612FNG) for direction; IN1/IN2 set direction, PWM on EN sets speed. Servos: 50 Hz pulses, Servo.write(0–180), separate power. Steppers: steps = angle/360 × steps per rev; 28BYJ-48 ≈ 2048, NEMA 17 = 200 × microsteps. Separate motor supply, common ground, flyback diodes and capacitors.

MODULE 17 · INTERMEDIATE

Libraries

Minimum duration **1 day** | Level **Intermediate**

Libraries package tested code for sensors, displays, motors and protocols. This module covers installing libraries from the Library Manager, ZIP files and GitHub, reading their examples and documentation, choosing well-maintained libraries, how header and source files and #include work, writing your own library as a class with keywords.txt, examples and library.properties, and solving library conflicts and version problems, followed by 25 solved questions and programs.

IN THIS MODULE

17.1 Using libraries
17.2 How libraries work
17.3 Writing your own library
17.4 Library conflicts and version issues
17.5 Practice questions
17.6 Solutions

17.1 Using libraries

17.1.1 Installing from the Library Manager

Tools → Manage Libraries (or the books icon in IDE 2): search, choose a version, click Install. The IDE also offers to install dependencies (for example Adafruit_SSD1306 needs Adafruit_GFX and Adafruit_BusIO).

17.1.2 Installing from ZIP files and GitHub

Sketch → Include Library → Add .ZIP Library installs a downloaded ZIP (for example from a GitHub "Code → Download ZIP" button). Libraries live in the **libraries** folder of your sketchbook (Documents/Arduino/libraries); each in its own folder.

17.1.3 Reading library examples and documentation

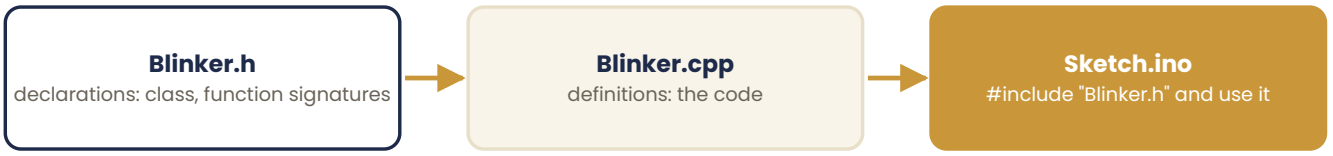
File → Examples → (library name) opens working example sketches: start from one, confirm your hardware works, then adapt it. The README and header file (.h) list the available functions.

17.1.4 Choosing well-maintained libraries

Check	Good sign
Recent activity	updates or answered issues in the last year or two
Popularity	many users, stars, mentions in tutorials
Documentation	clear README and examples
Board support	states that it works on your board (AVR, ESP32, RP2040…)
Author	a known maker or the chip manufacturer (Adafruit, SparkFun, Arduino)

17.2 How libraries work

17.2.1 Header (.h) and source (.cpp) files



A LIBRARY IS A HEADER PLUS A SOURCE FILE

17.2.2 #include

`#include <Servo.h>` searches the installed libraries; `#include "MyFile.h"` searches the sketch folder first. The preprocessor pastes the header in place; the linker then joins the compiled .cpp code. An **include guard** (#pragma once or #ifndef … #define … #endif) stops a header being pasted twice.

17.3 Writing your own library

17.3.1 Creating a class for a component

BLINKER.H

```
// Blinker.h
#pragma once
#include <Arduino.h>

class Blinker {
 public:
  Blinker(int pin, unsigned long interval);   // constructor
  void begin();
  void update();                              // call often from loop()
 private:
  int _pin;
  unsigned long _interval, _last = 0;
  bool _state = false;
};
```

BLINKER LIBRARY IN USE

```
// Blinker.cpp (shown together with the sketch so it compiles as one file here)
class Blinker {
 public:
  Blinker(int pin, unsigned long interval) : _pin(pin), _interval(interval) {}
  void begin() { pinMode(_pin, OUTPUT); }
  void update() {
    if (millis() - _last >= _interval) {
      _last = millis();
      _state = !_state;
      digitalWrite(_pin, _state);
    }
  }
 private:
  int _pin;
  unsigned long _interval, _last = 0;
  bool _state = false;
};

// Sketch: two independent blinkers in three lines of loop() code
Blinker red(8, 300), green(9, 700);

void setup() { red.begin(); green.begin(); }
void loop()  { red.update(); green.update(); }
```

SIMULATED RUN (pin changes and Serial output)

```
[   300 ms] pin 8 -> HIGH
[   600 ms] pin 8 -> LOW
[   700 ms] pin 9 -> HIGH
[   900 ms] pin 8 -> HIGH
[  1200 ms] pin 8 -> LOW
[  1400 ms] pin 9 -> LOW
```

Each object keeps its own pin, interval and timer: the class hides the millis() bookkeeping. Private members start with an underscore by convention.

17.3.2 keywords.txt, examples and library.properties

File	Purpose
library.properties	name, version, author, description, architectures (required for the Library Manager)
src/Blinker.h, src/Blinker.cpp	the code
examples/Basic/Basic.ino	appears under File → Examples
keywords.txt	tells the IDE which words to colour (KEYWORD1 classes, KEYWORD2 methods)
README.md, LICENSE	documentation and terms of use

17.4 Library conflicts and version issues

Symptom	Cause and fix
"Multiple libraries were found for X.h"	two copies installed: the IDE lists which one it used; delete the unwanted folder
Example no longer compiles after an update	API changed: read the changelog, or install the older version in the Library Manager
"X.h: No such file or directory"	library not installed, or misspelt include (names are case-sensitive on Linux/macOS)
Works on Uno, fails on ESP32	library is AVR-only (uses AVR registers); find a compatible fork
Two libraries use the same timer	for example Servo and tone() / IRremote: change pins or choose another library

WATCH OUT

Keep a note of library names and versions used by each project (for example in a comment at the top of the sketch), so you can rebuild it later.

17.5 Practice questions

17.5.1 Basic level

Q1 Where do you install most libraries in Arduino IDE 2?

(a) the Boards Manager
(b) the Serial Monitor
(c) the Library Manager
(d) File → Preferences only

Q2 Which file of a library contains the class declaration?

(a) the header file (.h)
(b) the .cpp file only
(c) keywords.txt
(d) library.properties

Q3 Which include form searches the sketch folder first?

(a) #include
(b) #include "MyLib.h"
(c) #import MyLib
(d) using MyLib;

Q4 Where are user-installed libraries stored?

(a) inside each .ino file
(b) on the Arduino board
(c) in EEPROM
(d) in the libraries folder inside the sketchbook (Documents/Arduino/libraries)

Q5 How do you install a library downloaded from GitHub as a ZIP?

(a) Upload it to the board
(b) Paste it into the Serial Monitor
(c) Sketch → Include Library → Add .ZIP Library
(d) Rename it .ino

Q6 What is the purpose of an include guard (#pragma once)?

(a) to speed up uploads
(b) to protect against viruses
(c) to prevent a header from being included twice
(d) to make variables private

Q7 Where can you find working example sketches for an installed library?

(a) Tools → Port
(b) Help → About
(c) Sketch → Upload
(d) File → Examples → library name

Q8 Which file makes the IDE colour your library's class and function names?

(a) library.properties
(b) keywords.txt
(c) README.md
(d) LICENSE

17.5.2 Intermediate level

Q9 Write a class Counter with methods increment(), reset() and value(), and test it.

Q10 "Multiple libraries were found for Servo.h" appears. What does it mean?

(a) two copies of a library providing Servo.h are installed
(b) the sketch has two servos
(c) Servo.h is missing
(d) the board has two USB ports

Q11 A library compiles for Uno but not for ESP32. The most likely reason is

(a) ESP32 cannot use libraries
(b) the library uses AVR-specific hardware registers
(c) the USB cable is faulty
(d) the sketch is too short

Q12 Turn the button-debounce logic into a class Button(pin) with a method pressed() that returns true once per press.

Q13 Which file is required for a library to appear in the Library Manager index?

(a) library.properties
(b) keywords.txt
(c) README.md
(d) a .ino file in the root

Q14 Which is the best sign of a well-maintained library?

(a) it has the longest name
(b) it was never updated since 2012
(c) recent updates and answered issues, with clear examples
(d) it has no documentation

Q15 Create a class Led with on(), off() and toggle(), and use two objects on pins 8 and 9.

Q16 After a library update, an old example fails with "no matching function". What is the quickest fix?

(a) reinstall the IDE
(b) change the baud rate
(c) delete the sketch
(d) install the previous version from the Library Manager (or update the code to the new API)

17.5.3 Advanced level

Q17 Why should a library class store the pin number in a member variable?

(a) to save flash
(b) pins cannot be passed to functions
(c) it is required by Serial
(d) so each object remembers its own pin and several objects can coexist

Q18 Write a class Average that accumulates samples with add(x) and returns mean() as a float; test with 3, 5 and 10.

Q19 Servo and IRremote stop working together on an Uno. A likely cause is

(a) both need 12 V
(b) they use the same Serial port
(c) both libraries use the same hardware timer
(d) Uno cannot run two libraries

Q20 What does the "architectures" field in library.properties specify?

(a) which board families the library supports (e.g. avr, esp32, *)

(b) the CPU speed

(c) the author's address

(d) the library size

Q21 Implement a class Timer with start(ms) and done() that returns true once the time has elapsed; use it to print "ding" after 1500 ms.

Q22 Why do names of library headers need exact capitalisation?

(a) Arduino IDE ignores case

(b) only for Wi-Fi libraries

(c) capitals make code faster

(d) file systems on Linux and macOS are case-sensitive

Q23 Add a static member to a class Sensor that counts how many Sensor objects exist; create three and print the count.

Q24 What should you record so a project can be rebuilt reliably a year later?

(a) only the sketch name

(b) the names and versions of all libraries and the board package used

(c) the room temperature

(d) the USB port number

Q25 A library's .cpp file includes "Arduino.h". Why?

(a) to make the library smaller

(b) to access Arduino types and functions such as pinMode() and millis()

(c) it is optional decoration

(d) to connect to the internet

17.6 Solutions

Q1 **Answer: (c) the Library Manager**

Library Manager.

Q2 **Answer: (a) the header file (.h)**

Declarations go in the **.h**.

Q3 **Answer: (b) #include "MyLib.h"**

Quotes search the **sketch folder** first.

Q4 **Answer: (d) in the libraries folder inside the sketchbook (Documents/Arduino/libraries)**

The sketchbook **libraries** folder.

Q5 **Answer: (c) Sketch → Include Library → Add .ZIP Library**

Add .ZIP Library.

Q6 **Answer: (c) to prevent a header from being included twice**

It stops **duplicate definitions**.

Q7 **Answer: (d) File → Examples → library name**

Examples appear under **File → Examples**.

Q8 **Answer: (b) keywords.txt**

keywords.txt.

Q9 **Answer: Solution program**

Private data (_n) can only be changed through the public methods, which keeps the object consistent.

```
class Counter {
 public:
  void increment() { _n++; }
  void reset() { _n = 0; }
  long value() const { return _n; }
 private:
  long _n = 0;
};
void setup() {
  Serial.begin(9600);
  Counter c;
  for (int i = 0; i < 3; i++) c.increment();
  Serial.println(c.value());
  c.reset();
  Serial.println(c.value());
}
void loop() {}
```

SERIAL MONITOR

```
3
0
```

Q10 **Answer: (a) two copies of a library providing Servo.h are installed**

Remove the **duplicate** folder if the wrong one is used.

Q11 **Answer: (b) the library uses AVR-specific hardware registers**

It is **AVR-only**.

Q12 **Answer: Solution program**

The class keeps its own last state and timer, so several buttons need only several objects. Simulated: two presses.

```
class Button {
 public:
  Button(int pin) : _pin(pin) {}
  void begin() { pinMode(_pin, INPUT_PULLUP); }
  bool pressed() {
    int r = digitalRead(_pin);
    bool edge = (_last == HIGH && r == LOW && millis() - _t > 30);
    if (r != _last) _t = millis();
    _last = r;
    return edge;
  }
 private:
  int _pin, _last = HIGH;
  unsigned long _t = 0;
};
Button b(2);
int count = 0;
void setup() { Serial.begin(9600); b.begin(); }
void loop() { if (b.pressed()) { count++; Serial.print("press "); Serial.println(count); } }
```

SERIAL MONITOR

```
press 1
press 2
```

Q13 **Answer: (a) library.properties**

library.properties describes the library.

Q14 **Answer: (c) recent updates and answered issues, with clear examples**

Activity and documentation.

Q15 **Answer: Solution program**

toggle() reads the stored state, so the object always knows whether it is on.

```
class Led {
 public:
  Led(int pin) : _p(pin) {}
  void begin() { pinMode(_p, OUTPUT); off(); }
  void on() { _s = true; digitalWrite(_p, HIGH); }
  void off() { _s = false; digitalWrite(_p, LOW); }
  void toggle() { _s ? off() : on(); }
 private:
  int _p; bool _s = false;
};
Led a(8), b(9);
void setup() { a.begin(); b.begin(); a.on(); b.toggle(); b.toggle(); }
void loop() {}
```

SIMULATED RUN

```
[     0 ms] pin 8 -> LOW
[     0 ms] pin 9 -> LOW
[     0 ms] pin 8 -> HIGH
```

```
[     0 ms] pin 9 -> HIGH
[     0 ms] pin 9 -> LOW
```

Q16 **Answer: (d) install the previous version from the Library Manager (or update the code to the new API)**

The **API changed**; roll back or adapt.

Q17 **Answer: (d) so each object remembers its own pin and several objects can coexist**

Each object has its **own state**.

Q18 **Answer: Solution program**

Checking _n before dividing avoids division by zero when no samples have been added. (3 + 5 + 10) / 3 = 6.

```
class Average {
 public:
  void add(float x) { _sum += x; _n++; }
  float mean() const { return _n ? _sum / _n : 0; }
 private:
  float _sum = 0; int _n = 0;
};
void setup() {
  Serial.begin(9600);
  Average a;
  a.add(3); a.add(5); a.add(10);
  Serial.println(a.mean());
}
void loop() {}
```

SERIAL MONITOR

```
6.00
```

Q19 **Answer: (c) both libraries use the same hardware timer**

Hardware **timer conflicts** are common.

Q20 **Answer: (a) which board families the library supports (e.g. avr, esp32, *)**

It lists **supported architectures**.

Q21 **Answer: Solution program**

done() returns true only once, then stops the timer: a reusable non-blocking timer object.

```
class Timer {
 public:
  void start(unsigned long ms) { _t0 = millis(); _ms = ms; _run = true; }
  bool done() { if (_run && millis() - _t0 >= _ms) { _run = false; return true; } return false; }
 private:
  unsigned long _t0 = 0, _ms = 0; bool _run = false;
};
Timer t;
void setup() { Serial.begin(9600); t.start(1500); }
void loop() { if (t.done()) { Serial.print("ding at "); Serial.println(millis()); } }
```

SERIAL MONITOR

```
ding at 1500
```

Q22 **Answer: (d) file systems on Linux and macOS are case-sensitive**

"servo.h" ≠ "Servo.h" on **case-sensitive** systems.

Q23 **Answer: Solution program**

A static member belongs to the class, not to any one object, so all constructors increment the same counter.

```
class Sensor {
 public:
  Sensor(int pin) : _pin(pin) { count++; }
  static int count;
 private:
  int _pin;
};
int Sensor::count = 0;
Sensor s1(A0), s2(A1), s3(A2);
void setup() {
  Serial.begin(9600);
  Serial.print("sensors: "); Serial.println(Sensor::count);
}
void loop() {}
```

SERIAL MONITOR

```
sensors: 3
```

Q24 **Answer: (b) the names and versions of all libraries and the board package used**

Record **library and core versions**.

Q25 **Answer: (b) to access Arduino types and functions such as pinMode() and millis()**

.ino files include it automatically; **.cpp files must include it** themselves.

Install libraries with the Library Manager (or Add .ZIP), start from their examples, and prefer active, documented ones that support your board. A library = header (.h, declarations, include guard) + source (.cpp, include Arduino.h). Wrap components in classes with private state. Resolve duplicate copies, API changes and timer conflicts, and record versions.

MODULE 18 · INTERMEDIATE

Communication protocols in depth

Minimum duration **2 days** | Level **Intermediate**

Sensors, displays and other boards talk to the Arduino through a handful of protocols. This module covers UART (TX, RX, baud rate, hardware and software serial, linking two Arduinos, GPS NMEA sentences), I2C (SDA and SCL, addresses and the scanner, the Wire library, several devices, Arduino-to-Arduino), SPI (MOSI, MISO, SCK, CS, the SPI library, SD cards, RFID and TFTs), the One-Wire bus, and how to choose a protocol, followed by 25 solved questions and programs.

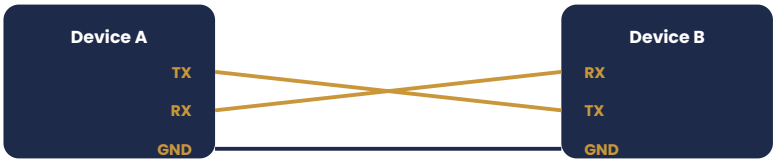
IN THIS MODULE

18.1 UART
18.2 I2C
18.3 SPI
18.4 One-Wire protocol
18.5 Choosing a protocol
18.6 Practice questions
18.7 Solutions

18.1 UART

18.1.1 TX, RX and baud rate

UART (serial) sends bytes one bit at a time over a TX (transmit) wire, receiving on RX. There is no clock wire, so both ends must agree on the baud rate. Each byte is framed by a start bit and a stop bit ("8N1": 8 data bits, no parity, 1 stop bit).



UART WIRING: TX CROSSES TO RX, AND GROUNDS ARE JOINED

18.1.2 Hardware vs software serial

The Uno R3 has one hardware UART (pins 0/1, shared with USB). **SoftwareSerial** emulates another on almost any pins (reliable up to about 38 400 baud, one port listening at a time). The Mega has four hardware UARTs (Serial1–Serial3 extra); ESP32 has three.

18.1.3 Communicating between two Arduinos

RECEIVER.INO

```
#include <SoftwareSerial.h>
SoftwareSerial link(10, 11);          // RX, TX  (cross-wired to the other board)

void setup() {
  Serial.begin(9600);
  link.begin(9600);
}

void loop() {
  if (link.available()) {
    String msg = link.readStringUntil('\n');
    Serial.print("received: "); Serial.println(msg);
    if (msg.startsWith("T=")) {
      float t = msg.substring(2).toFloat();
      link.print("ACK "); link.println(t, 1);    // reply to the sender
    }
  }
}
```

SIMULATED RUN (pin changes and Serial output)

```
received: T=23.5
[     0 ms] SoftwareSerial sends: ACK 23.5
received: T=24.0
[     0 ms] SoftwareSerial sends: ACK 24.0
```

WATCH OUT

Connect a 5 V Arduino's TX to a 3.3 V board's RX only through a voltage divider; the 3.3 V TX into a 5 V RX usually works directly.

18.1.4 GPS modules (NEO-6M) with NMEA sentences

A GPS module sends text lines ("NMEA sentences") at 9600 baud once per second. The $GPGGA sentence carries time, latitude, longitude, fix quality, satellites and altitude. Coordinates are in degrees and minutes: 1259.2840,N means 12° 59.2840′ = 12.98807°. The TinyGPS++ library parses them for you.

PARSEGGA.INO

```
void setup() {
  Serial.begin(9600);
  String s = "$GPGGA,064036.00,1259.2840,N,07735.6460,E,1,08,0.9,920.0,M,,,,*47";
  int f[15], n = 0;
  for (unsigned i = 0; i < s.length() && n < 15; i++) if (s[i] == ',') f[n++] = i;
  String lat = s.substring(f[1] + 1, f[2]);
  String lon = s.substring(f[3] + 1, f[4]);
  float latDeg = lat.substring(0, 2).toInt() + lat.substring(2).toFloat() / 60.0;
  float lonDeg = lon.substring(0, 3).toInt() + lon.substring(3).toFloat() / 60.0;
  Serial.print("lat ");  Serial.println(latDeg, 5);
  Serial.print("lon ");  Serial.println(lonDeg, 5);
  Serial.print("sats "); Serial.println(s.substring(f[6] + 1, f[7]).toInt());
}
void loop() {}
```

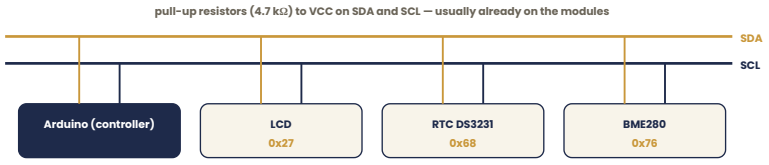
SERIAL MONITOR

```
lat 12.98807
lon 77.59410
sats 8
```

18.2 I2C

18.2.1 SDA and SCL lines

I2C (inter-integrated circuit) uses two shared wires: SDA (data) and SCL (clock), plus GND, with pull-up resistors (usually on the modules). A controller (the Arduino) addresses up to about 100 devices on one bus at 100 kHz (standard) or 400 kHz (fast mode). Uno: SDA = A4, SCL = A5; ESP32: GPIO 21 and 22 by default.



SEVERAL DEVICES SHARE ONE I2C BUS, EACH WITH ITS OWN ADDRESS

18.2.2 Addresses and the I2C scanner

I2CSCANNER.INO

```
#include <Wire.h>

void setup() {
  Serial.begin(9600);
  Wire.begin();
  Serial.println("Scanning I2C bus...");
  int found = 0;
  for (byte addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {            // 0 = a device acknowledged
      Serial.print("device at 0x");
      if (addr < 16) Serial.print('0');
      Serial.println(addr, HEX);
      found++;
    }
  }
  Serial.print(found); Serial.println(" device(s) found");
}
void loop() {}
```

SERIAL MONITOR

```
Scanning I2C bus...
device at 0x27
device at 0x68
device at 0x76
3 device(s) found
```

Device	Typical address
LCD backpack (PCF8574)	0x27 or 0x3F
SSD1306 OLED	0x3C
DS3231 / DS1307 RTC	0x68 (AT24C32 EEPROM on the module at 0x57)
MPU6050	0x68 or 0x69
BMP280 / BME280	0x76 or 0x77
PCA9685	0x40

18.2.3 The Wire library

WRITING A REGISTER ADDRESS, THEN READING BYTES

```
#include <Wire.h>
const byte DEV = 0x68;
void setup() {
  Wire.begin();
  Wire.beginTransmission(DEV);   // start talking to the device
  Wire.write(0x00);              // register to read from
  Wire.endTransmission();
  Wire.requestFrom(DEV, 3);      // ask for 3 bytes
  while (Wire.available()) {
    byte b = Wire.read();
  }
}
void loop() {}
```

18.2.4 Several devices on one bus

Devices share SDA and SCL as long as their addresses differ. Two identical modules need an address-select pin (often A0/AD0) set differently, or an I2C multiplexer (TCA9548A). Keep wires short (under about 1 m) and avoid too many pull-ups in parallel.

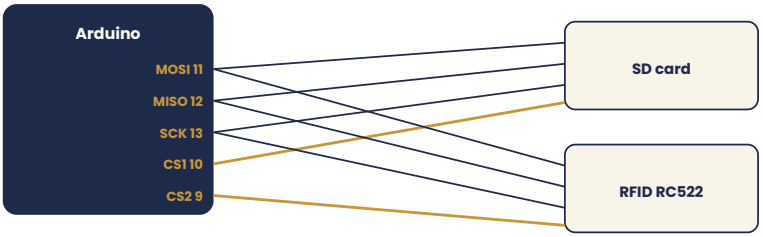
18.2.5 Arduino-to-Arduino I2C

One board acts as controller; another calls Wire.begin(8) to become a peripheral at address 8 and registers onReceive() and onRequest() handlers that run when the controller writes or reads.

18.3 SPI

18.3.1 MOSI, MISO, SCK, CS

SPI uses four wires: MOSI (controller out, peripheral in), MISO (controller in), SCK (clock) and one CS (chip select, active LOW) per device. It is full-duplex and fast (several MHz). Uno: MOSI 11, MISO 12, SCK 13, CS usually 10 (also on the ICSP header).



SPI: SHARED MOSI, MISO, SCK; A SEPARATE CS LINE FOR EACH DEVICE

18.3.2 The SPI library

ONE SPI TRANSFER

```
#include <SPI.h>
const int CS = 10;
void setup() {
  pinMode(CS, OUTPUT);
  digitalWrite(CS, HIGH);                    // deselect
  SPI.begin();
  SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0));
  digitalWrite(CS, LOW);                     // select the device
  byte reply = SPI.transfer(0x9F);           // send a byte, receive one at the same time
  digitalWrite(CS, HIGH);
  SPI.endTransaction();
}
void loop() {}
```

18.3.3 SPI devices: SD cards, RFID, TFT displays

SD card modules (Module 19), RC522 RFID readers (MFRC522 library) and TFT displays all use SPI. Each needs its own CS pin; libraries handle the transfers. SD cards are 3.3 V devices: use a module with a level shifter on a 5 V board.

18.4 One-Wire protocol

One-Wire (Dallas/Maxim) uses a single data wire plus ground, with a 4.7 kΩ pull-up. Every device has a unique 64-bit ROM address, so many DS18B20 sensors can share one pin. It is slow (about 16 kbit/s) but needs only one pin over long cables.

18.5 Choosing a protocol

	UART	I2C	SPI	One-Wire
Wires	TX, RX (+GND)	SDA, SCL	MOSI, MISO, SCK + CS each	1 data
Devices	2 (point to point)	many (by address)	many (one CS each)	many (by ROM ID)
Speed	up to ~1 Mbit/s	100–400 kHz	1–20+ MHz	~16 kbit/s

	UART	I2C	SPI	One-Wire
Typical use	GPS, Bluetooth, PC link	sensors, RTC, small displays	SD, TFT, RFID, fast ADCs	temperature sensors

Few pins and many slow sensors → I2C. Speed (SD cards, displays) → SPI. A text stream between two devices → UART. Many temperature probes on one long cable → One-Wire.

18.6 Practice questions

18.6.1 Basic level

Q1 How many wires carry data in I2C?

(a) 1 **(b)** 4
(c) 3 **(d)** 2 (SDA and SCL)

Q2 Which Uno pins are SDA and SCL?

(a) A4 and A5 **(b)** 0 and 1 **(c)** 11 and 13 **(d)** A0 and A1

Q3 In UART wiring between two boards, TX connects to

(a) the other board's TX **(b)** GND
(c) the other board's RX **(d)** 5 V

Q4 What does CS stand for in SPI?

(a) clock signal **(b)** control status **(c)** current sense **(d)** chip select

Q5 Which SPI pin carries data from the controller to the peripheral?

(a) MISO **(b)** MOSI **(c)** SCK **(d)** CS

Q6 What is the usual I2C address of a DS3231 RTC?

(a) 0x68 **(b)** 0x27 **(c)** 0x3C **(d)** 0x40

Q7 Which protocol identifies devices by a unique 64-bit ROM code?

(a) UART **(b)** SPI **(c)** One-Wire **(d)** PWM

Q8 Which Wire function result means a device acknowledged its address?

(a) endTransmission() returned 0 **(b)** available() returned 0
(c) read() returned -1 **(d)** begin() returned true

18.6.2 Intermediate level

Q9 Write an I2C scanner that prints the address of every device found in hexadecimal.

Q10 Two identical I2C sensors have the same fixed address. How can both be used?

(a) connect them to the same address anyway
(b) change one with its address-select pin, or use an I2C multiplexer such as TCA9548A
(c) use two different baud rates
(d) it is impossible

Q11 Why does UART need both ends set to the same baud rate?

(a) to save power
(b) because of the pull-ups
(c) only for Bluetooth
(d) there is no clock wire, so timing must be agreed in advance

Q12 Convert the NMEA latitude "2830.1234,N" to decimal degrees.

Q13 How many hardware UARTs does a Mega 2560 have?

(a) 1 **(b)** 2 **(c)** 4 **(d)** 8

Q14 Which protocol would you choose to drive an SD card at high speed?

(a) I2C **(b)** One-Wire **(c)** SPI **(d)** UART at 9600

Q15 Using SoftwareSerial on pins 10 (RX) and 11 (TX), forward every line received from the other board to the Serial Monitor with a line number.

Q16 Why are pull-up resistors needed on SDA and SCL?

(a) to limit LED current
(b) to set the address
(c) they are not needed
(d) devices only pull the lines LOW; resistors pull them back HIGH

18.6.3 Advanced level

Q17 What is the standard and fast-mode I2C clock speed?

(a) 9600 and 115200 Hz **(b)** 100 kHz and 400 kHz
(c) 1 MHz and 10 MHz **(d)** 50 Hz and 500 Hz

Q18 Extract the UTC time from "$GPGGA,064036.00,..." and print it as HH:MM:SS.

Q19 A 5 V Arduino TX feeds a 3.3 V ESP32 RX directly. What is the risk?

(a) the 5 V signal can damage the 3.3 V input; use a divider or level shifter
(b) nothing; it always works
(c) the baud rate doubles
(d) the ESP32 sends faster

Q20 SPI devices share MOSI, MISO and SCK. How does the controller choose which one to talk to?

(a) by sending its address first **(b)** it pulls that device's CS line LOW
(c) by changing the baud rate **(d)** by toggling MISO

Q21 Make an Arduino reply to "PING" lines received on SoftwareSerial with "PONG" (log the reply).

Q22 Why is SoftwareSerial unreliable at 115 200 baud on a 16 MHz Uno?

(a) it times bits in software, and interrupts and CPU speed limit its accuracy
(b) 115 200 is not a real baud rate
(c) it needs I2C
(d) the USB blocks it

Q23 An I2C bus works on the bench but fails with 3 m cables. The best explanation is

(a) I2C needs 12 V
(b) the address changed
(c) long wires add capacitance, so the open-drain lines rise too slowly
(d) the baud rate is wrong

Q24 Compute the checksum of an NMEA sentence: XOR of all characters between "$" and "*". Test with "$GPGLL,4916.45,N,12311.12,W,225444,A*" and print it in hex.

Q25 Which protocol suits 10 DS18B20 probes spread along a 20 m greenhouse cable on one pin?

(a) SPI **(b)** I2C **(c)** PWM **(d)** One-Wire

18.7 Solutions

Q1 **Answer: (d) 2 (SDA and SCL)**

SDA and SCL.

Q2 **Answer: (a) A4 and A5**

A4 = SDA, A5 = SCL.

Q3 **Answer: (c) the other board's RX**

TX → **RX** (crossed).

Q4 **Answer: (d) chip select**

Chip select, active LOW.

Q5 **Answer: (b) MOSI**

MOSI: controller out, peripheral in.

Q6 **Answer: (a) 0x68**

0x68.

Q7 **Answer: (c) One-Wire**

One-Wire.

Q8 **Answer: (a) endTransmission() returned 0**

0 = success (ACK).

Q9 **Answer: Solution program**

Each address is tried; a device that acknowledges returns 0. The simulated bus had an OLED (0x3C) and an LCD (0x27).

```
#include <Wire.h>
void setup() {
  Serial.begin(9600);
  Wire.begin();
  for (byte a = 1; a < 127; a++) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0) { Serial.print("0x"); Serial.println(a, HEX); }
  }
}
void loop() {}
```

SERIAL MONITOR

```
0x27
0x3C
```

Q10 **Answer: (b) change one with its address-select pin, or use an I2C multiplexer such as TCA9548A**

Addresses must differ, or a **multiplexer** switches buses.

Q11 **Answer: (d) there is no clock wire, so timing must be agreed in advance**

UART is **asynchronous**.

Q12 **Answer: Solution program**

Degrees are the first two digits (28); the rest is minutes (30.1234) ÷ 60 = 0.50206. Total 28.50206°. (Longitude has three degree digits.)

```
void setup() {
  Serial.begin(9600);
  String lat = "2830.1234";
  float deg = lat.substring(0, 2).toInt() + lat.substring(2).toFloat() / 60.0;
  Serial.println(deg, 5);
}
void loop() {}
```

SERIAL MONITOR

```
28.50206
```

Q13 **Answer: (c) 4**

Serial and Serial1–Serial3: **4**.

Q14 **Answer: (c) SPI**

SD cards use **SPI** (fast).

Q15 **Answer: Solution program**

readStringUntil() collects one full message at a time. Simulated messages: hello, distance=42.

```
#include <SoftwareSerial.h>
SoftwareSerial link(10, 11);
int n = 0;
void setup() { Serial.begin(9600); link.begin(9600); }
void loop() {
  if (link.available()) {
    String s = link.readStringUntil('\n');
    n++;
    Serial.print(n); Serial.print(": "); Serial.println(s);
  }
}
```

SERIAL MONITOR

```
1: hello
2: distance=42
```

Q16 **Answer: (d) devices only pull the lines LOW; resistors pull them back HIGH**

I2C lines are **open-drain**.

Q17 **Answer: (b) 100 kHz and 400 kHz**

100 kHz standard, **400 kHz** fast.

Q18 **Answer: Solution program**

The time field follows the first comma as hhmmss.ss. India Standard Time would be UTC + 5:30.

```
void setup() {
  Serial.begin(9600);
  String s = "$GPGGA,064036.00,1259.2840,N,07735.6460,E,1,08,0.9,920.0,M,,,,*47";
  int c = s.indexOf(',');
  String t = s.substring(c + 1, c + 7);         // "064036"
  Serial.print(t.substring(0, 2)); Serial.print(':');
  Serial.print(t.substring(2, 4)); Serial.print(':');
  Serial.println(t.substring(4, 6));
}
void loop() {}
```

SERIAL MONITOR

```
06:40:36
```

Q19 **Answer: (a) the 5 V signal can damage the 3.3 V input; use a divider or level shifter**

ESP32 pins are **not 5 V tolerant**.

Q20 **Answer: (b) it pulls that device's CS line LOW**

Each device has its own **CS**.

Q21 **Answer: Solution program**

A simple request–reply protocol; the simulated run shows the module sending PONG twice.

```
#include <SoftwareSerial.h>
SoftwareSerial link(10, 11);
void setup() { link.begin(9600); }
void loop() {
  if (link.available()) {
    String s = link.readStringUntil('\n');
    s.trim();
    if (s == "PING") link.println("PONG");
  }
}
```

SIMULATED RUN

```
[     0 ms] SoftwareSerial sends: PONG
[     0 ms] SoftwareSerial sends: PONG
```

Q22 **Answer: (a) it times bits in software, and interrupts and CPU speed limit its accuracy**

Software bit-timing is fragile; stay at or below **38 400**.

Q23 **Answer: (c) long wires add capacitance, so the open-drain lines rise too slowly**

I2C is meant for **short, on-board** distances.

Q24 **Answer: Solution program**

Every character between $ and * is XORed together; the receiver compares this with the two hex digits after *, rejecting corrupted sentences.

```
void setup() {
  Serial.begin(9600);
  const char *s = "$GPGLL,4916.45,N,12311.12,W,225444,A*";
  byte cs = 0;
  for (const char *p = s + 1; *p && *p != '*'; p++) cs ^= *p;
  Serial.print("checksum 0x"); Serial.println(cs, HEX);
}
void loop() {}
```

SERIAL MONITOR

```
checksum 0x31
```

Q25 **Answer: (d) One-Wire**

One-Wire: one pin, long cable, unique IDs.

UART: TX→RX crossed, common ground, same baud; SoftwareSerial for extra ports at low speed. I2C: SDA/SCL with pull-ups, unique addresses (scan to find them), 100/400 kHz, short wires. SPI: MOSI, MISO, SCK shared, one CS (active LOW) per device, fast. One-Wire: one pin, 4.7 kΩ pull-up, 64-bit IDs. Match logic levels.

MODULE 19 · INTERMEDIATE

Data storage and timekeeping

Minimum duration **2 days** | Level **Intermediate**

Many projects must remember settings, record data and know the time. This module covers EEPROM (reading and writing bytes, put() and get() for larger data, write endurance), SD cards (modules and the SD library, writing CSV log files, reading files), real-time clocks (DS3231 and DS1307, setting and reading the date and time, time-stamped logging) and a complete data-logger project, followed by 25 solved questions and programs.

IN THIS MODULE

19.1 EEPROM
19.2 SD cards
19.3 Real-time clocks
19.4 Data logger project
19.5 Practice questions
19.6 Solutions

19.1 EEPROM

19.1.1 Reading and writing bytes

EEPROM keeps its contents with the power off: 1 KB on the Uno R3 (4 KB on the Mega). Each address holds one byte (0–255); a new chip reads 255 (0xFF) everywhere.

EEPROMBYTE.INO

```
#include <EEPROM.h>

void setup() {
  Serial.begin(9600);
  Serial.print("address 0 before: "); Serial.println(EEPROM.read(0));   // 255 when new
  EEPROM.update(0, 42);       // writes only if the value is different
  Serial.print("address 0 after: ");  Serial.println(EEPROM.read(0));
}
void loop() {}
```

SERIAL MONITOR

```
address 0 before: 255
address 0 after: 42
```

19.1.2 EEPROM.put() and EEPROM.get() for larger data

EEPROMSTRUCT.INO

```
#include <EEPROM.h>

struct Settings {
  int threshold;
  float calibration;
  byte mode;
};

void setup() {
  Serial.begin(9600);
  Settings s = {300, 1.025, 2};
  EEPROM.put(0, s);                       // writes sizeof(Settings) bytes
  Settings back;
  EEPROM.get(0, back);
  Serial.print("threshold "); Serial.println(back.threshold);
  Serial.print("calibration "); Serial.println(back.calibration, 3);
  Serial.print("mode "); Serial.println(back.mode);
  Serial.print("bytes used: "); Serial.println(sizeof(Settings));
}
void loop() {}
```

SERIAL MONITOR

```
threshold 300
calibration 1.025
mode 2
bytes used: 12
```

On an Uno R3 this struct takes 7 bytes (int 2 + float 4 + byte 1); the simulation ran on a 64-bit host, where padding and a 4-byte int make it larger. Always use sizeof() rather than counting by hand, and store a "version" or magic number so the sketch can tell whether the EEPROM holds valid settings.

19.1.3 Write endurance limits

Each EEPROM cell survives about 100 000 writes (reads are unlimited). Writing one cell every second would wear it out in about 28 hours: use update(), write only when values change, and spread writes over several addresses.

19.2 SD cards

19.2.1 SD card modules and the SD library

An SD module connects over SPI (MOSI 11, MISO 12, SCK 13, CS often 4 or 10 on an Uno) and should include a 3.3 V regulator and level shifter. Format the card FAT32 (cards up to 32 GB are safest). The SD library uses 8.3 file names (up to 8 characters, a dot, 3 characters).

19.2.2 Writing CSV log files

SDLOGGER.INO

```
#include <SPI.h>
#include <SD.h>
const int CS = 4;

void setup() {
  Serial.begin(9600);
  if (!SD.begin(CS)) { Serial.println("SD card failed"); return; }
  File f = SD.open("LOG.CSV", FILE_WRITE);       // appends if the file exists
  if (f) {
    f.println("time_ms,light");
    for (int i = 0; i < 3; i++) {
      f.print(millis()); f.print(','); f.println(analogRead(A0));
      delay(1000);
    }
    f.close();                                   // flush data to the card
    Serial.println("logged 3 rows");
  }
  f = SD.open("LOG.CSV");                       // read it back
  while (f.available()) Serial.write(f.read());
  f.close();
}
void loop() {}
```

SERIAL MONITOR

```
logged 3 rows
time_ms,light
0,610
1000,598
2000,605
```

WATCH OUT

Data is written to the card in 512-byte blocks: until close() or flush() runs, recent rows may be only in RAM. If power is cut before then, they are lost. Flush after each important write.

19.2.3 Reading files

`SD.open(name)` opens for reading; `available()` and `read()` return bytes; `readStringUntil('\n')` returns one line, which you can split at commas. `SD.exists()` and `SD.remove()` check and delete files.

19.3 Real-time clocks

19.3.1 DS3231 and DS1307 RTC modules

	DS1307	DS3231
Accuracy	crystal-based, drifts minutes per month	temperature-compensated, about ±2 ppm (≈1 min/year)
Supply	5 V	2.3–5.5 V

	DS1307	DS3231
Extras	56 bytes RAM	temperature sensor, alarms, square-wave output
Interface	I2C 0x68	I2C 0x68

A coin cell (CR2032) keeps the clock running when the main power is off. millis() cannot replace an RTC: it resets on every restart and drifts.

19.3.2 Setting and reading date and time

RTCCLOCK.INO

```
#include <Wire.h>
#include <RTClib.h>
RTC_DS3231 rtc;

void print2(int v) { if (v < 10) Serial.print('0'); Serial.print(v); }

void setup() {
  Serial.begin(9600);
  rtc.begin();
  if (rtc.lostPower()) rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));   // set from compile time
}

void loop() {
  DateTime now = rtc.now();
  Serial.print(now.year()); Serial.print('-'); print2(now.month()); Serial.print('-');
print2(now.day());
  Serial.print(' '); print2(now.hour()); Serial.print(':'); print2(now.minute()); Serial.print(':');
print2(now.second());
  Serial.println();
  delay(1000);
}
```

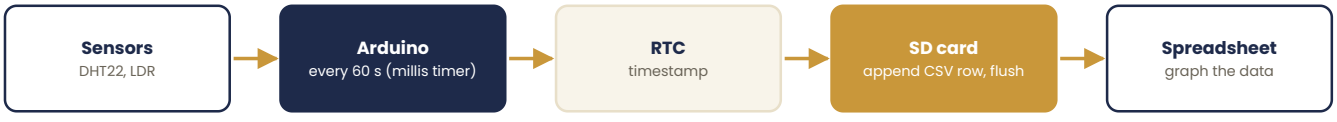
SERIAL MONITOR

```
2026-11-02 10:30:00
2026-11-02 10:30:01
2026-11-02 10:30:02
```

19.3.3 Time-stamped data logging

Combine an RTC with an SD card: each CSV row starts with the date and time, so the data can be graphed against real time in a spreadsheet.

19.4 Data logger project



A TIME-STAMPED ENVIRONMENTAL LOGGER

ENVLOGGER.INO

```
#include <SD.h>
#include <RTClib.h>
#include <DHT.h>
RTC_DS3231 rtc;
DHT dht(2, DHT22);
unsigned long last = 0;
const unsigned long PERIOD = 60000UL;     // one row per minute

void logRow() {
  DateTime t = rtc.now();
  File f = SD.open("ENV.CSV", FILE_WRITE);
  if (!f) return;
  f.print(t.hour()); f.print(':'); f.print(t.minute()); f.print(',');
  f.print(dht.readTemperature(), 1); f.print(',');
  f.print(dht.readHumidity(), 0); f.print(',');
  f.println(analogRead(A0));
  f.close();
}

void setup() {
  Serial.begin(9600);
  SD.begin(4); rtc.begin(); dht.begin();
}

bool first = true;
int rows = 0;

void loop() {
  if (first || millis() - last >= PERIOD) {
    first = false;
    last = millis();
    logRow();
    if (++rows == 3) {                    // demo: show the file after three rows
      File f = SD.open("ENV.CSV");
      while (f.available()) Serial.write(f.read());
      f.close();
    }
  }
}
```

SERIAL MONITOR

```
10:30,24.1,58,620
10:31,24.3,57,615
10:32,24.6,57,600
```

19.5 Practice questions

19.5.1 Basic level

Q1 How many bytes of EEPROM does the Uno R3 have?

(a) 512 **(b)** 2048 **(c)** 32 768 **(d)** 1024

Q2 What value does an unused EEPROM cell usually read?

(a) 0 **(b)** 1023 **(c)** 255 **(d)** -1

Q3 About how many write cycles can each EEPROM cell survive?

(a) 100 000 **(b)** 1 000 **(c)** 10 million **(d)** unlimited

Q4 What does EEPROM.update() do differently from EEPROM.write()?

(a) it writes twice
(b) it erases the whole EEPROM
(c) it reads only
(d) it writes only if the new value differs, saving wear

Q5 Which interface do SD card modules use with the Arduino?

(a) I2C **(b)** SPI **(c)** One-Wire **(d)** analog

Q6 Which file system should an SD card use for the Arduino SD library?

(a) NTFS **(b)** FAT32 (or FAT16)
(c) ext4 **(d)** APFS

Q7 Which RTC is temperature-compensated and more accurate?

(a) DS1307 **(b)** ATmega328P **(c)** HC-SR04 **(d)** DS3231

Q8 What keeps an RTC running when the main power is off?

(a) a coin cell such as CR2032 **(b)** the USB cable
(c) EEPROM **(d)** the SD card

19.5.2 Intermediate level

Q9 Store a counter in EEPROM address 0 that increases by one every time the board starts (simulate one start).

Q10 Why should you call close() or flush() after writing to an SD file?

(a) to format the card
(b) to reduce SPI speed
(c) to make sure buffered data is actually written to the card
(d) it is optional and has no effect

Q11 A sketch writes a value to the same EEPROM cell every 10 seconds. Roughly how long until the cell wears out?

(a) about 10 years **(b)** about 1 hour
(c) never **(d)** about 11.6 days

Q12 Save a float calibration factor 1.0375 to EEPROM with put() and read it back with get().

Q13 Which file name is valid for the classic SD library (8.3 format)?

(a) temperature_log_2026.csv **(b)** DATA01.CSV
(c) my data.csv **(d)** LOGFILE.CSVX

Q14 Why is millis() not a substitute for an RTC in a data logger?

(a) it restarts at 0 on every reset and drifts, so it does not give the real date and time
(b) it is too fast
(c) it uses the SD card
(d) it only counts seconds

Q15 Write three CSV rows "n,square" (n = 1…3) to SQ.CSV and print the file back.

Q16 What I2C address do DS1307 and DS3231 use?

(a) 0x27 **(b)** 0x3C **(c)** 0x68 **(d)** 0x76

19.5.3 Advanced level

Q17 Using RTClib, print the time as HH:MM every 30 seconds for one minute (simulated).

Q18 A struct of int (2 B), float (4 B) and byte (1 B) is saved with EEPROM.put on an Uno R3. How many bytes does it occupy?

(a) 7 **(b)** 3 **(c)** 12 **(d)** 4

Q19 Read a CSV file line by line and print the second field of each line (file contains "10:00,24.5", "10:01,24.7").

Q20 Why do many sketches store a "magic number" alongside settings in EEPROM?

(a) to encrypt the data
(b) to detect whether valid settings were ever saved (e.g. on a new board)
(c) to make EEPROM faster
(d) to reduce wear

Q21 The DS3231 accuracy of ±2 ppm corresponds to about how much drift per year?

(a) about 1 second **(b)** about 1 hour **(c)** about 1 day **(d)** about 1 minute

Q22 Implement wear levelling: write a value to one of 4 EEPROM cells in turn (round robin), keeping the index in RAM, for 6 writes, and print where each went.

Q23 Your logger loses the last few rows whenever the battery is unplugged. The best fix is

(a) use a bigger SD card **(b)** raise the baud rate
(c) flush or close the file after each row **(d)** log less often only

Q24 Log a minimum/maximum temperature pair to EEPROM only when a new extreme occurs, for readings 22.5, 25.0, 21.0, 24.0.

Q25 Which SD card size is the safest choice for the Arduino SD library?

(a) 1 TB exFAT **(b)** 128 GB NTFS
(c) up to 32 GB, formatted FAT32 **(d)** any card, any format

19.6 Solutions

Q1 **Answer: (d) 1024**

1 KB.

Q2 **Answer: (c) 255**

Erased EEPROM reads **0xFF = 255**.

Q3 **Answer: (a) 100 000**

About **100 000** writes.

Q4 **Answer: (d) it writes only if the new value differs, saving wear**

update() **skips identical writes**.

Q5 **Answer: (b) SPI**

SPI.

Q6 **Answer: (b) FAT32 (or FAT16)**

FAT file systems.

Q7 **Answer: (d) DS3231**

The **DS3231**.

Q8 **Answer: (a) a coin cell such as CR2032**

A **backup coin cell**.

Q9 **Answer: Solution program**

Treating 255 as "never written" handles a new chip. On a real board each reset increases the count; a byte wraps after 254 boots, so use put()/get() with an int for more.

```
#include <EEPROM.h>
void setup() {
  Serial.begin(9600);
  byte boots = EEPROM.read(0);
  if (boots == 255) boots = 0;          // first use: fresh EEPROM
  boots++;
  EEPROM.update(0, boots);
  Serial.print("Boot number "); Serial.println(boots);
}
void loop() {}
```

SERIAL MONITOR

```
Boot number 1
```

Q10 **Answer: (c) to make sure buffered data is actually written to the card**

Data waits in a **512-byte buffer** until flushed.

Q11 **Answer: (d) about 11.6 days**

100 000 × 10 s = 1 000 000 s ≈ **11.6 days**.

Q12 **Answer: Solution program**

put() and get() handle any type, writing sizeof(float) = 4 bytes starting at address 10.

```
#include <EEPROM.h>
void setup() {
  Serial.begin(9600);
  float cal = 1.0375;
  EEPROM.put(10, cal);
  float back = 0;
  EEPROM.get(10, back);
  Serial.println(back, 4);
}
void loop() {}
```

SERIAL MONITOR

```
1.0375
```

Q13 **Answer: (b) DATA01.CSV**

Up to 8 characters, a dot, up to 3: **DATA01.CSV**.

Q14 **Answer: (a) it restarts at 0 on every reset and drifts, so it does not give the real date and time**

millis() is **time since start-up**, not calendar time.

Q15 **Answer: Solution program**

Open, write with print()/println(), close; then reopen for reading.

```
#include <SD.h>
void setup() {
  Serial.begin(9600);
  SD.begin(4);
  File f = SD.open("SQ.CSV", FILE_WRITE);
  for (int n = 1; n <= 3; n++) { f.print(n); f.print(','); f.println(n * n); }
  f.close();
  f = SD.open("SQ.CSV");
  while (f.available()) Serial.write(f.read());
  f.close();
}
void loop() {}
```

SERIAL MONITOR

```
1,1
2,4
3,9
```

Q16 **Answer: (c) 0x68**

0x68.

Q17 **Answer: Solution program**

Leading zeros are added by hand so 9:5 prints as 09:05. The simulated clock starts at 10:30.

```
#include <RTClib.h>
RTC_DS3231 rtc;
void setup() { Serial.begin(9600); rtc.begin(); }
void loop() {
  DateTime t = rtc.now();
  if (t.hour() < 10) Serial.print('0'); Serial.print(t.hour()); Serial.print(':');
  if (t.minute() < 10) Serial.print('0'); Serial.println(t.minute());
  delay(30000);
}
```

SERIAL MONITOR

```
10:30
10:30
10:31
```

Q18 **Answer: (a) 7**

2 + 4 + 1 = **7** (AVR does not pad structs).

Q19 **Answer: Solution program**

readStringUntil() returns one line; the text after the first comma is the second field.

```
#include <SD.h>
void setup() {
  Serial.begin(9600);
  SD.begin(4);
  File w = SD.open("T.CSV", FILE_WRITE);
  w.println("10:00,24.5"); w.println("10:01,24.7"); w.close();
  File f = SD.open("T.CSV");
  while (f.available()) {
    String line = f.readStringUntil('\n');
    Serial.println(line.substring(line.indexOf(',') + 1));
  }
  f.close();
}
void loop() {}
```

SERIAL MONITOR

```
24.5
24.7
```

Q20 **Answer: (b) to detect whether valid settings were ever saved (e.g. on a new board)**

If the magic number is missing, load **defaults**.

Q21 **Answer: (d) about 1 minute**

$2 \times 10^{-6} \times 31.5$ million s ≈ **63 s**.

Q22 **Answer: Solution program**

Spreading writes over 4 cells makes the EEPROM last 4 times longer. A real design also stores a sequence number to find the newest value after a restart.

```
#include <EEPROM.h>
int idx = 0;
void save(byte v) {
  EEPROM.update(idx, v);
  Serial.print("value "); Serial.print(v); Serial.print(" -> address "); Serial.println(idx);
  idx = (idx + 1) % 4;
}
void setup() {
  Serial.begin(9600);
  for (byte v = 10; v < 16; v++) save(v);
}
void loop() {}
```

SERIAL MONITOR

```
value 10 -> address 0
value 11 -> address 1
value 12 -> address 2
value 13 -> address 3
value 14 -> address 0
value 15 -> address 1
```

Q23 **Answer: (c) flush or close the file after each row**

Unflushed data is lost: **flush()** after writing.

Q24 **Answer: Solution program**

Writing only on change saves EEPROM wear: here only 4 writes for 4 readings, and none at all once the extremes are known.

```
#include <EEPROM.h>
float mn = 1000, mx = -1000;
void setup() {
  Serial.begin(9600);
  float r[] = {22.5, 25.0, 21.0, 24.0};
  for (float t : r) {
    if (t < mn) { mn = t; EEPROM.put(0, mn); Serial.print("new min "); Serial.println(mn, 1); }
    if (t > mx) { mx = t; EEPROM.put(4, mx); Serial.print("new max "); Serial.println(mx, 1); }
  }
}
void loop() {}
```

SERIAL MONITOR

```
new min 22.5
new max 22.5
new max 25.0
new min 21.0
```

Q25 **Answer: (c) up to 32 GB, formatted FAT32**

SDHC cards up to **32 GB, FAT32**.

EEPROM (1 KB on Uno R3) survives power-off but only about 100 000 writes per cell: use update(), put()/get() and write only on change. SD cards: SPI, FAT32, 8.3 names; close() or flush() after writing. RTCs (DS3231 best, I2C 0x68, coin cell) give real time for time-stamped CSV logging.

LEVEL 3

Advanced level

Build IoT projects with ESP32 and Wi-Fi, use Bluetooth and wireless links, program at register level, use RTOS, design custom PCBs, debug and test firmware and complete a capstone product

8 MODULES · 18 DAYS MINIMUM

Module 20 · Interrupts and timers	2 days
Module 21 · Wireless communication	2 days
Module 22 · IoT with Wi-Fi boards	3 days
Module 23 · Power management and battery projects	1 day
Module 24 · Advanced Arduino programming	2 days
Module 25 · Beyond the Arduino IDE	2 days
Module 26 · From prototype to product: PCB design and enclosures	2 days
Module 27 · Projects and careers	4 days

MODULE 20 · ADVANCED

Interrupts and timers

Minimum duration **2 days** | Level **Advanced**

Interrupts let the microcontroller react to events immediately, and hardware timers keep precise time without the CPU's attention. This module covers attachInterrupt(), interrupt pins and modes, writing short ISRs, volatile variables and atomic access, pin-change interrupts, hardware timers and timer interrupts, their effect on PWM and millis(), reading encoders, measuring frequency and RPM, precise periodic sampling, and the watchdog timer, followed by 25 solved questions and programs.

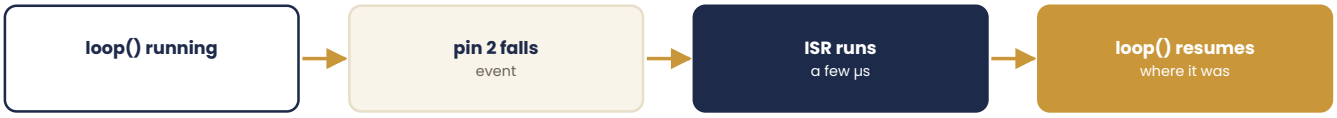
IN THIS MODULE

20.1 External interrupts
20.2 Pin-change interrupts (overview)
20.3 Hardware timers
20.4 Applications
20.5 Watchdog timer
20.6 Practice questions
20.7 Solutions

20.1 External interrupts

20.1.1 attachInterrupt() and interrupt pins

An interrupt pauses the main program the instant an event occurs, runs a short function (the interrupt service routine, ISR) and then resumes exactly where it left off. No polling, no missed events.



WHAT HAPPENS ON AN INTERRUPT

Board	External interrupt pins
Uno R3, Nano	2 and 3 (INT0, INT1)
Mega 2560	2, 3, 18, 19, 20, 21
Uno R4, Leonardo	several; check the pinout
ESP32	any GPIO

Always write attachInterrupt(digitalPinToInterrupt(pin), isr, mode): it maps the pin to the right interrupt number on every board.

20.1.2 RISING, FALLING, CHANGE modes

Mode	Triggers when
RISING	the pin goes LOW → HIGH
FALLING	the pin goes HIGH → LOW (a press with INPUT_PULLUP)
CHANGE	either edge
LOW	continuously while the pin is LOW (rarely useful)

20.1.3 Writing short interrupt service routines

INTERRUPTCOUNTER.INO

```
const int BTN = 2;
volatile unsigned long presses = 0;    // shared with the ISR

void onPress() {                        // ISR: short, no delay(), no Serial
  presses++;
}

void setup() {
  Serial.begin(9600);
  pinMode(BTN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(BTN), onPress, FALLING);
}

void loop() {
  static unsigned long shown = 0;
  noInterrupts();                       // copy the shared value safely
  unsigned long p = presses;
  interrupts();
  if (p != shown) { shown = p; Serial.print("presses: "); Serial.println(p); }
  delay(400);                           // even during long delays, no press is missed
}
```

SERIAL MONITOR

```
presses: 2
presses: 3
```

Two presses happened during the same 400 ms delay, and both were counted. Real buttons bounce, so an ISR on a raw button may count several times per press; debounce with a time check inside the ISR (ignore edges within 30 ms of the last) or use a hardware RC filter.

Do in an ISR	Avoid in an ISR
update a counter or flag	delay() (it relies on interrupts)
read millis() or micros() (frozen during the ISR)	Serial.print (may block)
keep it to a few lines	long calculations, floating point
set a flag for loop() to act on	I2C, SPI, SD library calls

20.1.4 volatile variables and atomic access

volatile tells the compiler that a variable can change at any moment, so it must be re-read from memory every time. On an 8-bit AVR, reading a 2- or 4-byte variable takes several instructions; if the ISR changes it halfway, loop() can read a corrupted value. Copy multi-byte shared variables with interrupts briefly disabled, as above.

20.2 Pin-change interrupts (overview)

Every Uno pin can raise a **pin-change interrupt**, but pins are grouped into three ports (PCINT0–2), one ISR per group, triggered on any change; the ISR must work out which pin changed. Libraries such as EnableInterrupt or PinChangeInterrupt hide the details.

20.3 Hardware timers

20.3.1 Timer basics

A timer is a hardware counter clocked from the 16 MHz system clock through a prescaler (÷1, 8, 64, 256, 1024). In CTC mode it counts up to a compare value and raises an interrupt each time it matches.

> Interrupt frequency = 16 MHz / (prescaler × (compare + 1)). For 1 Hz with prescaler 256: compare = 16 000 000 / 256 - 1 = 62 499 (fits the 16-bit Timer1).

Timer (Uno R3)	Size	Used by
Timer0	8-bit	millis(), micros(), delay(); PWM on pins 5 and 6
Timer1	16-bit	Servo library; PWM on pins 9 and 10
Timer2	8-bit	tone(); PWM on pins 3 and 11

20.3.2 Timer interrupts with libraries

TIMER1INTERRUPT.INO

```
#include <avr/io.h>
volatile bool tick = false;

ISR(TIMER1_COMPA_vect) {          // runs once per second
  tick = true;
}

void setup() {
  Serial.begin(9600);
  noInterrupts();
  TCCR1A = 0;
  TCCR1B = _BV(WGM12) | _BV(CS12);   // CTC mode, prescaler 256
  OCR1A = 62499;                      // 16 MHz / 256 / 62 500 = 1 Hz
  TIMSK1 = _BV(OCIE1A);               // enable compare-match interrupt
  interrupts();
}

void loop() {
  if (tick) { tick = false; Serial.println("one second"); }
}
```

Libraries such as TimerOne (Timer1.initialize(1000000); Timer1.attachInterrupt(f);) wrap the register settings. On ESP32 use hardware timer functions (timerBegin, timerAttachInterrupt) or esp_timer.

20.3.3 Effects on PWM and millis()

WATCH OUT

Re-configuring Timer0 breaks millis() and delay(); changing Timer1 stops PWM on pins 9 and 10 (and conflicts with Servo); changing Timer2 affects tone() and PWM on 3 and 11. Choose the timer that your other libraries do not use.

20.4 Applications

20.4.1 Reading rotary and wheel encoders

ENCODERISR.INO

```
const int A = 2, B = 4;
volatile long ticks = 0;

void onA() {                          // A fell: B gives the direction
  if (digitalRead(B) == HIGH) ticks++; else ticks--;
}

void setup() {
  Serial.begin(9600);
  pinMode(A, INPUT_PULLUP); pinMode(B, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(A), onA, FALLING);
}

void loop() {
  noInterrupts(); long t = ticks; interrupts();
  Serial.print("ticks: "); Serial.println(t);
  delay(500);
}
```

SERIAL MONITOR

```
ticks: 0
ticks: 3
ticks: 2
```

20.4.2 Measuring frequency and RPM

Count pulses in an ISR for a fixed window (say 1 s): frequency = count / window. Or time the gap between consecutive pulses with micros(): frequency = 1 000 000 / period in µs. RPM = pulses per second × 60 / pulses per revolution.

20.4.3 Precise periodic sampling

A timer interrupt that sets a flag (or reads the ADC) at an exact rate, for example 1 kHz, gives evenly spaced samples for signal processing, independent of how long loop() takes.

20.5 Watchdog timer

20.5.1 Automatic recovery from freezes

The watchdog is an independent timer that resets the microcontroller if the sketch fails to "pat" it within a timeout. Remote or unattended devices use it so that a hang (stuck I2C bus, endless loop) recovers by itself.

WATCHDOG.INO

```
#include <avr/wdt.h>

void setup() {
  Serial.begin(9600);
  Serial.println("started");
  wdt_enable(WDTO_2S);            // reset if not patted within 2 s
}

void loop() {
  // ... normal work, each pass well under 2 s ...
  wdt_reset();                    // "pat the dog"
  delay(500);
}
```

SIMULATED RUN (pin changes and Serial output)

```
started
[     0 ms] watchdog enabled, timeout code 7
```

20.6 Practice questions

20.6.1 Basic level

Q1 Which Uno R3 pins support attachInterrupt() external interrupts?

(a) 0 and 1 **(b)** 9 and 10 **(c)** A0 and A1 **(d)** 2 and 3

Q2 What is an ISR?

(a) internal serial register
(b) interrupt service routine: the function run when the interrupt occurs
(c) an input sensor reading
(d) a library

Q3 Which mode triggers on a HIGH → LOW transition?

(a) FALLING **(b)** RISING **(c)** CHANGE only **(d)** HIGH

Q4 Why must a variable shared with an ISR be declared volatile?

(a) to save memory
(b) so the compiler always re-reads it from memory
(c) to make it constant
(d) so it is stored in EEPROM

Q5 Which should NOT be used inside an ISR?

(a) delay() **(b)** incrementing a counter
(c) setting a flag **(d)** reading a digital pin

Q6 Which timer drives millis() on the Uno R3?

(a) Timer1 **(b)** Timer2 **(c)** Timer0 **(d)** the watchdog

Q7 What does the watchdog timer do if the sketch stops resetting it?

(a) turns off the LED **(b)** erases flash
(c) sends an email **(d)** resets the microcontroller

Q8 What is the recommended way to get the interrupt number for a pin?

(a) pin - 2 **(b)** analogRead(pin)
(c) interrupt(pin) **(d)** digitalPinToInterrupt(pin)

20.6.2 Intermediate level

Q9 Count FALLING edges on pin 3 with an interrupt and print the count every second.

Q10 With a 16 MHz clock and prescaler 64, what compare value gives a 1 kHz timer interrupt?

(a) 250 **(b)** 1000 **(c)** 249 **(d)** 15999

Q11 Changing Timer1 settings on an Uno R3 affects

(a) PWM on pins 9 and 10 and the Servo library
(b) millis()
(c) tone() only

(d) nothing

Q12 Measure the frequency of a signal on pin 2: count rising edges in an ISR over 1 second.

Q13 Why should Serial.print() be avoided in an ISR?

(a) it prints in reverse
(b) it changes the baud rate
(c) it is not allowed in C++
(d) it can block waiting for the transmit buffer, freezing the system

Q14 A 4-byte volatile counter is read in loop() without disabling interrupts. What can go wrong on an 8-bit AVR?

(a) nothing; reads are atomic
(b) the ISR may update it mid-read, giving a corrupted value
(c) the board resets
(d) the counter becomes negative always

Q15 Debounce an interrupt-driven button: in the ISR ignore edges that come within 50 ms of the last accepted one.

Q16 Which interrupt type is available on every Uno pin but shared by groups of pins?

(a) external interrupt INT0 **(b)** pin-change interrupt (PCINT)
(c) watchdog interrupt **(d)** timer overflow

20.6.3 Advanced level

Q17 Prescaler 1024 and a 16-bit Timer1: what is the longest interrupt period possible?

(a) about 1 ms **(b)** about 65 s **(c)** about 0.26 s **(d)** about 4.19 s

Q18 Compute and print OCR1A values for 1 Hz, 10 Hz and 100 Hz with prescaler 256.

Q19 Wheel encoder: 20 pulses per revolution, 400 pulses counted in 2 s. What is the RPM?

(a) 600 **(b)** 200 **(c)** 400 **(d)** 1200

Q20 Inside an ISR, does millis() advance?

(a) Yes, normally
(b) It runs twice as fast
(c) No; it is frozen while the ISR runs (so keep ISRs short)
(d) It resets to 0

Q21 Measure the period of pulses on pin 2 with micros() in an ISR and compute the frequency from the last period.

Q22 Why does a watchdog help an unattended weather station?

(a) it measures temperature
(b) it saves battery
(c) it uploads data
(d) it automatically resets the board if the code hangs (e.g. a stuck sensor bus)

Q23 Which is the best pattern for heavy work triggered by an interrupt?

(a) do all the work in the ISR
(b) set a volatile flag in the ISR; do the work in loop() when the flag is set
(c) call delay() in the ISR
(d) disable interrupts permanently

Q24 Use the watchdog with a 1 s timeout; print "alive" in loop() and pat it each pass (simulate 3 passes of 300 ms).

Q25 What frequency does Timer1 in CTC mode produce with prescaler 8 and OCR1A = 1999?

(a) 1000 Hz (b) 2000 Hz (c) 500 Hz (d) 8000 Hz

20.7 Solutions

Q1 **Answer: (d) 2 and 3**

INT0 and INT1 are on pins **2 and 3**.

Q2 **Answer: (b) interrupt service routine: the function run when the interrupt occurs**

Interrupt **service routine**.

Q3 **Answer: (a) FALLING**

FALLING.

Q4 **Answer: (b) so the compiler always re-reads it from memory**

It can change **outside the normal flow**.

Q5 **Answer: (a) delay()**

delay() relies on interrupts and blocks.

Q6 **Answer: (c) Timer0**

Timer0.

Q7 **Answer: (d) resets the microcontroller**

It **resets** the board.

Q8 **Answer: (d) digitalPinToInterrupt(pin)**

Portable across boards.

Q9 **Answer: Solution program**

The ISR only increments; loop() copies the value with interrupts off. Simulated: 3 edges in the first second, 1 in the second.

```
volatile unsigned int edges = 0;
void isr() { edges++; }
void setup() {
  Serial.begin(9600);
  pinMode(3, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(3), isr, FALLING);
}
void loop() {
  delay(1000);
  noInterrupts(); unsigned int e = edges; interrupts();
  Serial.print("edges: "); Serial.println(e);
}
```

SERIAL MONITOR

```
edges: 3
edges: 4
```

Q10 **Answer: (c) 249**

16 000 000 / (64 × 1000) − 1 = **249**.

Q11 **Answer: (a) PWM on pins 9 and 10 and the Servo library**

Timer1 drives PWM on **9 and 10** and Servo.

Q12 **Answer: Solution program**

Edges counted in a 1-second window equal the frequency. Simulated: a 5 Hz square wave.

```
volatile unsigned long n = 0;
void rise() { n++; }
void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT);
  attachInterrupt(digitalPinToInterrupt(2), rise, RISING);
  delay(1000);
  noInterrupts(); unsigned long c = n; interrupts();
  Serial.print("frequency = "); Serial.print(c); Serial.println(" Hz");
}
void loop() {}
```

SERIAL MONITOR

```
frequency = 5 Hz
```

Q13 **Answer: (d) it can block waiting for the transmit buffer, freezing the system**

Printing may **block**, and Serial itself uses interrupts.

Q14 **Answer: (b) the ISR may update it mid-read, giving a corrupted value**

Multi-byte reads are **not atomic** on 8-bit AVRs.

Q15 **Answer: Solution program**

The bouncing edges (2 ms apart) are rejected; only one press per real push is counted. Simulated: two bouncy presses.

```
volatile unsigned long count = 0, lastT = 0;
void press() {
  unsigned long now = millis();
  if (now - lastT > 50) { count++; lastT = now; }
}
void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(2), press, FALLING);
}
void loop() {
  delay(1000);
  noInterrupts(); unsigned long c = count; interrupts();
  Serial.print("presses: "); Serial.println(c);
}
```

SERIAL MONITOR

```
presses: 2
```

Q16 **Answer: (b) pin-change interrupt (PCINT)**

Pin-change interrupts cover all pins in three groups.

Q17 **Answer: (d) about 4.19 s**

65 536 × 1024 / 16 MHz ≈ **4.19 s**.

Q18 **Answer: Solution program**

compare = 16 MHz / (prescaler × f) − 1. All values fit in Timer1's 16 bits (≤ 65 535).

```
void setup() {
  Serial.begin(9600);
  int f[] = {1, 10, 100};
  for (int i = 0; i < 3; i++) {
    long ocr = 16000000L / (256L * f[i]) - 1;
    Serial.print(f[i]); Serial.print(" Hz -> OCR1A = "); Serial.println(ocr);
  }
}
void loop() {}
```

SERIAL MONITOR

```
1 Hz -> OCR1A = 62499
10 Hz -> OCR1A = 6249
100 Hz -> OCR1A = 624
```

Q19 **Answer: (a) 600**

400 / 2 = 200 pulses/s ÷ 20 = 10 rev/s × 60 = **600 RPM**.

Q20 **Answer: (c) No; it is frozen while the ISR runs (so keep ISRs short)**

Timer0's interrupt cannot run during another ISR.

Q21 **Answer: Solution program**

Timing between edges gives the period; f = 1 000 000 / period µs. Simulated rising edges every 250 ms → 4 Hz.

```
volatile unsigned long lastUs = 0, period = 0;
void edge() { unsigned long t = micros(); period = t - lastUs; lastUs = t; }
void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT);
  attachInterrupt(digitalPinToInterrupt(2), edge, RISING);
  delay(1000);
  noInterrupts(); unsigned long p = period; interrupts();
  Serial.print("period "); Serial.print(p); Serial.print(" us -> ");
  Serial.print(1000000.0 / p, 1); Serial.println(" Hz");
}
void loop() {}
```

SERIAL MONITOR

```
period 250000 us -> 4.0 Hz
```

Q22 **Answer: (d) it automatically resets the board if the code hangs (e.g. a stuck sensor bus)**

Self-recovery from **freezes**.

Q23 **Answer: (b) set a volatile flag in the ISR; do the work in loop() when the flag is set**

Keep ISRs tiny; **defer** work to loop().

Q24 **Answer: Solution program**

Each pass takes 300 ms, well within the 1 s timeout. If a pass ever took more than a second, the watchdog would reset the board.

```
#include <avr/wdt.h>
void setup() { Serial.begin(9600); wdt_enable(WDTO_1S); }
void loop() {
  Serial.println("alive");
  wdt_reset();
  delay(300);
}
```

SERIAL MONITOR

```
alive
alive
```

```
alive
```

Q25 **Answer: (a) 1000 Hz**

16 000 000 / (8 × 2000) = **1000 Hz**.

attachInterrupt(digitalPinToInterrupt(pin), isr, RISING/FALLING/CHANGE) on pins 2 and 3 (Uno). ISRs: short, no delay() or Serial; shared variables volatile and copied with interrupts off. Timer frequency = 16 MHz / (prescaler × (compare + 1)); Timer0 = millis, Timer1 = Servo/PWM 9–10, Timer2 = tone/PWM 3–11. The watchdog resets a hung sketch.

MODULE 21 · ADVANCED

Wireless communication

Minimum duration **2 days** | Level **Advanced**

Wireless links free projects from cables. This module covers Bluetooth Classic with HC-05 and HC-06 modules (AT commands, pairing, smartphone control), Bluetooth Low Energy with ArduinoBLE, radio links with nRF24L01 transceivers, 433 MHz modules and LoRa, infrared communication, and how to choose a wireless technology by range, power, data rate and cost, followed by 25 solved questions and programs.

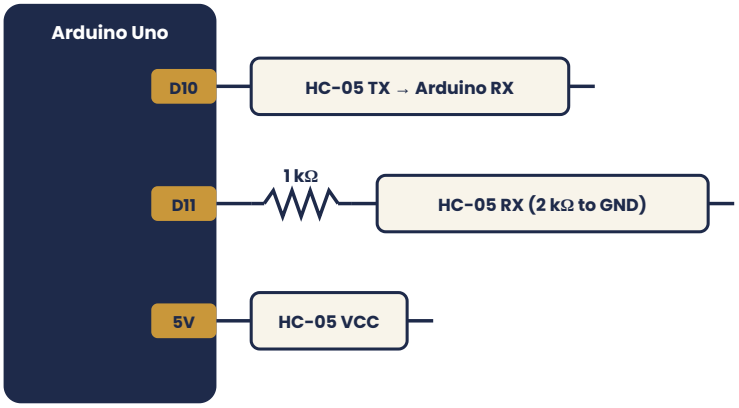
IN THIS MODULE

21.1 Bluetooth
21.2 Radio
21.3 Infrared communication
21.4 Choosing a wireless technology
21.5 Practice questions
21.6 Solutions

21.1 Bluetooth

21.1.1 HC-05 and HC-06 Bluetooth Classic modules

HC-05 and HC-06 modules act as a wireless serial cable (Serial Port Profile): whatever the phone sends arrives on the module's TX pin. HC-06 is slave-only; HC-05 can also be a master and connect to another module. Default baud 9600, PIN 1234 or 0000, range about 10 m. Their RX pin is 3.3 V: feed it from a 5 V TX through a divider (1 kΩ / 2 kΩ).



HC-05 ON SOFTWARESERIAL PINS 10 AND 11, WITH A DIVIDER ON ITS RX

21.1.2 AT commands and pairing

AT command	Purpose
AT	test: replies OK
AT+NAME=Robot1	change the Bluetooth name

AT command	Purpose
AT+PSWD=4321 (HC-05) / AT+PIN4321 (HC-06)	change the pairing PIN
AT+UART=38400,0,0	change the baud rate (HC-05)
AT+ROLE=1	make an HC-05 a master

The HC-05 enters AT mode when its KEY/EN pin is held HIGH at power-up (AT-mode baud 38 400). The HC-06 accepts AT commands whenever it is not connected.

21.1.3 Controlling projects from a smartphone app

BLUETOOTHCONTROL.INO

```
#include <SoftwareSerial.h>
SoftwareSerial bt(10, 11);               // RX, TX to the HC-05

void setup() {
  Serial.begin(9600);
  bt.begin(9600);
  pinMode(13, OUTPUT);
}

void loop() {
  if (bt.available()) {
    char c = bt.read();
    switch (c) {
      case 'F': Serial.println("forward"); break;
      case 'B': Serial.println("back");    break;
      case 'L': digitalWrite(13, HIGH); bt.println("LED ON");  break;
      case 'l': digitalWrite(13, LOW);  bt.println("LED OFF"); break;
    }
  }
}
```

SIMULATED RUN (pin changes and Serial output)

```
forward
[     0 ms] pin 13 -> HIGH
[     0 ms] SoftwareSerial sends: LED ON
[     0 ms] pin 13 -> LOW
[     0 ms] SoftwareSerial sends: LED OFF
back
```

Many free "Bluetooth controller" apps send single characters for each button, which makes the sketch simple. The ESP32 has Bluetooth built in: `#include <BluetoothSerial.h>`, `SerialBT.begin("MyESP32")`, then use it like Serial.

21.1.4 Bluetooth Low Energy with ArduinoBLE

BLE (Nano 33 BLE, Nano ESP32, Uno R4 WiFi, ESP32) uses very little power and is supported by every phone. Instead of a serial stream, a device offers **services** containing **characteristics** (small values identified by UUIDs) that a phone can read, write or subscribe to (notifications). Apps such as nRF Connect or LightBlue explore them.

ARDUINOBLE PERIPHERAL SKETCH

```
#include <ArduinoBLE.h>
BLEService envService("181A");                                  // Environmental Sensing
BLEFloatCharacteristic tempChar("2A6E", BLERead | BLENotify);   // Temperature
void setup() {
  BLE.begin();
  BLE.setLocalName("EnvNode");
  envService.addCharacteristic(tempChar);
  BLE.addService(envService);
  BLE.advertise();
}
void loop() {
  BLE.poll();
  tempChar.writeValue(24.5);
  delay(1000);
}
```

21.2 Radio

21.2.1 nRF24L01 transceivers

The nRF24L01+ is a cheap 2.4 GHz transceiver on SPI (CE and CSN pins plus MOSI, MISO, SCK), up to 2 Mbit/s, with 125 channels and 32-byte packets; range about 100 m (over 1 km with the PA+LNA antenna version). It must be powered from 3.3 V with a 10 µF capacitor across its supply pins. Library: RF24 (radio.openWritingPipe, radio.write, radio.available, radio.read).

WATCH OUT
Most nRF24L01 problems are power problems: the Uno's 3.3 V pin is weak. Add a 10–100 µF capacitor right at the module, or use an adapter board with its own regulator.

21.2.2 433 MHz RF modules

Very cheap 433 MHz transmitter/receiver pairs (FS1000A and XY-MK-5V) send simple one-way, slow data (a few kbit/s) using the RadioHead (RH_ASK) library. Range improves greatly with a 17 cm wire antenna (quarter wavelength). Also used to copy remote-control sockets (rc-switch library).

21.2.3 LoRa for long-range communication

LoRa (SX1276/RFM95 modules, 433/868/915 MHz depending on region; 865–867 MHz in India) trades speed for range: kilometres in open country with tiny power, at a few hundred bits to a few kbit/s. LoRaWAN networks (The Things Network) connect LoRa sensors to the internet through gateways. Use the legal frequency band for your country.

Range ≈ inversely related to data rate: LoRa (km, slow) > nRF24 with antenna (hundreds of m) > Bluetooth Classic/BLE (10–30 m) > IR (line of sight, a few m).

21.3 Infrared communication

Infrared links (TV-remote style) need line of sight and work over a few metres. An IR LED driven at 38 kHz with the IRremote library can send NEC codes, so an Arduino can act as a universal remote; a TSOP receiver decodes them on the other side (Module 15).

21.4 Choosing a wireless technology

21.4.1 Range, power, data rate and cost

Technology	Range	Data rate	Power	Typical use
IR	≤ 10 m, line of sight	very low	very low	remote controls
Bluetooth Classic (HC-05)	≈ 10 m	≈ 1–2 Mbit/s	medium	phone control, serial bridge
BLE	10–50 m	≈ 1 Mbit/s	very low	wearables, phone sensors
nRF24L01	100 m – 1 km	up to 2 Mbit/s	low	robot remotes, sensor networks
433 MHz ASK	20–100 m	≈ 2 kbit/s	low	simple one-way signals
LoRa	2–15 km	0.3–20 kbit/s	very low	farm and city sensors
Wi-Fi (Module 22)	30–50 m	many Mbit/s	high	internet, web dashboards

21.5 Practice questions

21.5.1 Basic level

Q1 What is the default baud rate of an HC-05/HC-06 in data mode?

(a) 115200 **(b)** 38400 **(c)** 9600 **(d)** 4800

Q2 What is the typical range of an HC-05 module?

(a) about 10 m **(b)** about 1 km **(c)** about 1 m **(d)** about 100 km

Q3 Which module can work as both Bluetooth master and slave?

(a) HC-06 **(b)** HC-05 **(c)** nRF24L01 **(d)** FS1000A

Q4 Which command tests communication with an HC-05 in AT mode?

(a) TEST **(b)** PING **(c)** HELLO **(d)** AT

Q5 What frequency band does the nRF24L01 use?

(a) 433 MHz **(b)** 868 MHz **(c)** 2.4 GHz **(d)** 38 kHz

Q6 Which technology offers the longest range at very low power?

(a) Bluetooth Classic **(b)** IR
(c) LoRa **(d)** nRF24L01 without antenna

Q7 What supply voltage does the nRF24L01 need?

(a) 5 V **(b)** 12 V **(c)** 1.5 V **(d)** 3.3 V

Q8 In BLE, values that a phone can read or subscribe to are called

(a) sketches **(b)** characteristics
(c) pipes **(d)** channels

21.5.2 Intermediate level

Q9 Receive Bluetooth commands over SoftwareSerial: "1" turns pin 13 on, "0" off, and reply with the new state.

Q10 Why should a 5 V Arduino TX be connected to the HC-05 RX through a voltage divider?

(a) the HC-05 RX pin is a 3.3 V input **(b)** to increase range
(c) to set the PIN **(d)** the HC-05 needs 12 V

Q11 How is an HC-05 put into AT command mode?

(a) send "AT" while connected to a phone
(b) hold its KEY/EN pin HIGH while powering it up
(c) press reset three times
(d) connect TX to GND

Q12 On an ESP32, create a Bluetooth serial device called "LBF-ESP32" that echoes received text back in upper case.

Q13 What is the maximum payload size of one nRF24L01 packet?

(a) 32 bytes (b) 1 byte (c) 1024 bytes (d) 64 kB

Q14 What length of wire makes a good quarter-wave antenna for 433 MHz?

(a) about 1 m (b) about 2 cm (c) about 17 cm (d) about 7 mm

Q15 Parse joystick commands from a phone app of the form "X:120,Y:-40" received over Bluetooth and print the two numbers.

Q16 Which frequency band should a LoRa project in India use?

(a) 915 MHz (b) 433 MHz only (c) 2.4 GHz (d) 865–867 MHz

21.5.3 Advanced level

Q17 Why do many nRF24L01 projects fail to communicate?

(a) the library is too new

(b) SPI is too slow

(c) they need line of sight like IR

(d) insufficient or noisy 3.3 V power; fixed with a capacitor or regulator

Q18 Encode a 3-value sensor packet (temperature ×10, humidity, battery %) into a comma-separated string for a radio link, and decode it on the "receiver" side.

Q19 Which wireless option best suits a soil sensor 3 km from the farmhouse, reporting every 15 minutes on batteries?

(a) Bluetooth Classic (b) IR

(c) LoRa (d) Wi-Fi

Q20 What makes BLE better than Bluetooth Classic for a coin-cell wearable?

(a) much lower power consumption (b) higher audio quality

(c) longer range always (d) no need for a phone

Q21 Implement a simple packet with a checksum: send "DATA:42*" plus the XOR of the payload characters, then verify it on reception.

Q22 A TV-remote style IR link fails when someone walks between the devices. Why?

(a) IR uses 2.4 GHz (b) the baud rate is wrong

(c) IR has a 1 km range (d) IR needs line of sight

Q23 An app shows "Connected" to an HC-05 but the Arduino receives nothing. Which is the most likely wiring fault?

(a) the PIN is 1234

(b) HC-05 TX and RX are connected to the wrong Arduino pins (not crossed)

(c) the phone is too close

(d) the LED is red

Q24 Which library is commonly used with nRF24L01 modules?

(a) Servo (b) RF24 (c) DHT (d) IRremote

Q25 Which characteristic property lets a phone receive automatic updates when a BLE value changes?

(a) Write only
(b) Broadcast power
(c) Pairing PIN
(d) Notify

21.6 Solutions

Q1 **Answer: (c) 9600**

9600 (AT mode on HC-05: 38 400).

Q2 **Answer: (a) about 10 m**

Class 2 Bluetooth: **≈ 10 m**.

Q3 **Answer: (b) HC-05**

The **HC-05**.

Q4 **Answer: (d) AT**

Send **AT**; it replies OK.

Q5 **Answer: (c) 2.4 GHz**

2.4 GHz.

Q6 **Answer: (c) LoRa**

LoRa: kilometres.

Q7 **Answer: (d) 3.3 V**

3.3 V (its data pins tolerate 5 V).

Q8 **Answer: (b) characteristics**

Services contain **characteristics**.

Q9 **Answer: Solution program**

The HC-05 delivers each phone keypress as a character; the reply goes back to the phone app.
Simulated input: 1, 0.

```
#include <SoftwareSerial.h>
SoftwareSerial bt(10, 11);
void setup() { bt.begin(9600); pinMode(13, OUTPUT); }
void loop() {
  if (!bt.available()) return;
  char c = bt.read();
  if (c == '1') { digitalWrite(13, HIGH); bt.println("ON"); }
  else if (c == '0') { digitalWrite(13, LOW); bt.println("OFF"); }
}
```

SIMULATED RUN

```
[     0 ms] pin 13 -> HIGH
[     0 ms] SoftwareSerial sends: ON
[     0 ms] pin 13 -> LOW
[     0 ms] SoftwareSerial sends: OFF
```

Q10 **Answer: (a) the HC-05 RX pin is a 3.3 V input**

Protect the **3.3 V** input.

Q11 **Answer: (b) hold its KEY/EN pin HIGH while powering it up**

KEY/EN HIGH at **power-up**.

Q12 **Answer: Solution program**

The ESP32's built-in Bluetooth replaces an external module. Simulated message: hello.

```
#include <BluetoothSerial.h>
BluetoothSerial SerialBT;
void setup() { SerialBT.begin("LBF-ESP32"); }
void loop() {
  if (SerialBT.available()) {
    String s = SerialBT.readStringUntil('\n');
    s.toUpperCase();
    SerialBT.println(s);
  }
}
```

SIMULATED RUN

```
[     0 ms] Bluetooth device name: LBF-ESP32
[     0 ms] Bluetooth sends: HELLO
```

Q13 **Answer: (a) 32 bytes**

32 bytes per packet.

Q14 **Answer: (c) about 17 cm**

$\lambda = c / f \approx 69$ cm; $\lambda/4 \approx$ **17 cm**.

Q15 **Answer: Solution program**

Split at the comma and skip the "X:" and "Y:" labels. Simulated: X:120,Y:-40 and X:0,Y:255.

```
#include <SoftwareSerial.h>
SoftwareSerial bt(10, 11);
void setup() { Serial.begin(9600); bt.begin(9600); }
void loop() {
  if (!bt.available()) return;
  String s = bt.readStringUntil('\n');
  int comma = s.indexOf(',');
  int x = s.substring(2, comma).toInt();
  int y = s.substring(comma + 3).toInt();
  Serial.print("x="); Serial.print(x); Serial.print(" y="); Serial.println(y);
}
```

SERIAL MONITOR

```
x=120 y=-40
x=0 y=255
```

Q16 **Answer: (d) 865–867 MHz**

India's licence-free LoRa band is **865–867 MHz**.

Q17 **Answer: (d) insufficient or noisy 3.3 V power; fixed with a capacitor or regulator**

Power is the usual culprit.

Q18 **Answer: Solution program**

Sending the temperature ×10 as an integer avoids floats in the packet (snprintf has no %f on AVR) and keeps it within 32 bytes.

```
void setup() {
  Serial.begin(9600);
  int t10 = 245, hum = 61, bat = 87;                // 24.5 C
  char pkt[32];
  snprintf(pkt, sizeof pkt, "%d,%d,%d", t10, hum, bat);
  Serial.print("sent: "); Serial.println(pkt);
  int a, b, c;
  sscanf(pkt, "%d,%d,%d", &a, &b, &c);              // receiver side
  Serial.print("temp "); Serial.print(a / 10.0, 1);
  Serial.print(" C, hum "); Serial.print(b); Serial.print(" %, battery "); Serial.print(c);
Serial.println(" %");
}
void loop() {}
```

SERIAL MONITOR

```
sent: 245,61,87
temp 24.5 C, hum 61 %, battery 87 %
```

Q19 **Answer: (c) LoRa**

Long range, low data, low power: **LoRa**.

Q20 **Answer: (a) much lower power consumption**

BLE is designed for **very low power**.

Q21 **Answer: Solution program**

Radio links can corrupt bytes; a checksum lets the receiver reject bad packets. (RF24 and LoRa hardware add CRCs too.)

```
byte xorOf(const String &s) { byte c = 0; for (unsigned i = 0; i < s.length(); i++) c ^= s[i];
return c; }
void setup() {
  Serial.begin(9600);
  String payload = "DATA:42";
  String pkt = payload + "*" + String(xorOf(payload), HEX);
  Serial.print("packet "); Serial.println(pkt);
  int star = pkt.indexOf('*');
  String body = pkt.substring(0, star);
  byte got = strtol(pkt.substring(star + 1).c_str(), NULL, 16);
  Serial.println(got == xorOf(body) ? "checksum OK" : "corrupted");
}
void loop() {}
```

SERIAL MONITOR

```
packet DATA:42*2C
checksum OK
```

Q22 **Answer: (d) IR needs line of sight**

Infrared is **blocked** by obstacles.

Q23 **Answer: (b) HC-05 TX and RX are connected to the wrong Arduino pins (not crossed)**

TX must go to the Arduino's **RX** pin.

Q24 **Answer: (b) RF24**

RF24.

Q25 **Answer: (d) Notify**

BLE **notifications** push changes to the phone.

HC-05/HC-06: wireless serial at 9600 baud, about 10 m, 3.3 V RX (use a divider), AT commands to configure. BLE: services and characteristics, very low power. nRF24L01: 2.4 GHz, 3.3 V with a capacitor, 32-byte packets. 433 MHz: cheap, slow, one-way. LoRa: kilometres, slow, legal band (865–867 MHz in India). IR: line of sight. Choose by range, power, data rate and cost.

MODULE 22 · ADVANCED

IoT with Wi-Fi boards

Minimum duration **3 days** | Level **Advanced**

Wi-Fi boards put your projects on the internet. This module covers the ESP32 and ESP8266 with the Arduino IDE, the Uno R4 WiFi and Nano 33 IoT, connecting to a network and access-point mode, hosting web pages that control outputs and show sensor data, calling web APIs with HTTP GET and POST and parsing JSON, MQTT publish and subscribe, the Arduino Cloud, Blynk, ThingSpeak and Firebase platforms, over-the-air updates and IoT security basics, followed by 25 solved questions and programs simulated with scripted network traffic.

IN THIS MODULE

22.1 Wi-Fi boards
22.2 Wi-Fi basics
22.3 Web servers
22.4 Calling web APIs
22.5 MQTT
22.6 IoT platforms
22.7 Over-the-air (OTA) updates
22.8 IoT security basics
22.9 Practice questions
22.10 Solutions

22.1 Wi-Fi boards

22.1.1 ESP32 and ESP8266 with the Arduino IDE

Add the board package (Boards Manager: "esp32 by Espressif" or "esp8266"), select your board (for example "ESP32 Dev Module") and upload as usual; some boards need the BOOT button held while uploading starts. The ESP32 offers a dual-core 240 MHz CPU, Wi-Fi, Bluetooth, 12-bit ADCs, touch pins and 3.3 V logic.

> **WATCH OUT**
> ESP32 pins are 3.3 V and not 5 V tolerant. ADC2 pins cannot be used while Wi-Fi is on (use ADC1: GPIO 32–39). GPIO 6–11 are connected to the flash chip: never use them.

22.1.2 Uno R4 WiFi and Nano 33 IoT

The Uno R4 WiFi (with an ESP32-S3 co-processor) and Nano 33 IoT use the WiFiS3 / WiFiNINA libraries, whose functions closely match the ESP32 WiFi library used here.

22.2 Wi-Fi basics

22.2.1 Connecting to a network

WIFICONNECT.INO

```
#include <WiFi.h>
const char *SSID = "HomeNet";
const char *PASS = "secret123";

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);                  // station: join an existing network
  WiFi.begin(SSID, PASS);
  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print('.');
  }
  Serial.println();
  Serial.print("Connected, IP address: ");
  Serial.println(WiFi.localIP().toString());
  Serial.print("Signal strength (RSSI): "); Serial.print(WiFi.RSSI()); Serial.println(" dBm");
}
void loop() {}
```

SERIAL MONITOR

```
Connecting...
Connected, IP address: 192.168.1.42
Signal strength (RSSI): -58 dBm
```

22.2.2 Access-point mode

In access-point (AP) mode the ESP32 creates its own network (WiFi.softAP("Setup-Node", "password")); phones connect to it directly and open http://192.168.4.1. AP mode is ideal for first-time setup (entering home Wi-Fi details) or projects with no router.

22.3 Web servers

22.3.1 Hosting a web page on the board

22.3.2 Controlling outputs from a browser

WEBLED.INO

```
#include <WiFi.h>
#include <WebServer.h>
WebServer server(80);
const int LED = 2;
bool ledOn = false;

void handleRoot() {
  String html = "<h1>ESP32</h1><p>LED is ";
  html += ledOn ? "ON" : "OFF";
  html += "</p><a href='/on'>ON</a> <a href='/off'>OFF</a>";
  server.send(200, "text/html", html);
}
void handleOn()  { ledOn = true;  digitalWrite(LED, HIGH); handleRoot(); }
void handleOff() { ledOn = false; digitalWrite(LED, LOW);  handleRoot(); }

void setup() {
  pinMode(LED, OUTPUT);
  WiFi.begin("HomeNet", "secret123");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  server.on("/", handleRoot);
  server.on("/on", handleOn);
  server.on("/off", handleOff);
  server.begin();
}

void loop() {
  server.handleClient();            // must be called often
}
```

SERIAL MONITOR

```
--> browser requests GET /
<-- 200 text/html: <h1>ESP32</h1><p>LED is OFF</p><a href='/on'>ON</a> <a href='/off'>OFF</a>
--> browser requests GET /on
<-- 200 text/html: <h1>ESP32</h1><p>LED is ON</p><a href='/on'>ON</a> <a href='/off'>OFF</a>
--> browser requests GET /off
<-- 200 text/html: <h1>ESP32</h1><p>LED is OFF</p><a href='/on'>ON</a> <a href='/off'>OFF</a>
```

The simulation shows three browser requests and the HTML the board sent back. A real page would add buttons and CSS; keep it small, because it is stored as a string in the board's memory.

22.3.3 Showing sensor data on a web page

SENSORJSON.INO

```
#include <WiFi.h>
#include <WebServer.h>
WebServer server(80);

void handleData() {                    // a JSON endpoint for a dashboard
  float t = 24.0 + analogRead(34) / 4095.0 * 2;   // demo: sensor on GPIO 34 (ADC1)
  String json = "{\"temp\":" + String(t, 1) + ",\"uptime\":" + String(millis() / 1000) + "}";
  server.send(200, "application/json", json);
}

void setup() {
  WiFi.begin("HomeNet", "secret123");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  server.on("/data", handleData);
  server.begin();
}

void loop() { server.handleClient(); }
```

SERIAL MONITOR

```
--> browser requests GET /data
<-- 200 application/json: {"temp":25.0,"uptime":1}
--> browser requests GET /data
<-- 200 application/json: {"temp":25.0,"uptime":1}
```

A web page can fetch /data every few seconds with JavaScript (fetch()) and update a gauge, so the page itself never needs reloading.

22.4 Calling web APIs

22.4.1 HTTP GET and POST

GET requests data (a weather report); **POST** sends data (a sensor reading to a cloud database). HTTPClient handles the request; check the status code (200 = OK, 404 = not found, 401 = unauthorised).

22.4.2 Parsing JSON with ArduinoJson

WEATHERAPI.INO

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

void setup() {
  Serial.begin(115200);
  WiFi.begin("HomeNet", "secret123");
  while (WiFi.status() != WL_CONNECTED) delay(500);

  HTTPClient http;
  http.begin("http://api.example.com/weather?city=Pune");
  int code = http.GET();
  if (code == 200) {
    JsonDocument doc;
    DeserializationError err = deserializeJson(doc, http.getString());
    if (!err) {
      float temp = doc["main"]["temp"];
      int hum = doc["main"]["humidity"];
      const char *sky = doc["weather"][0]["description"];
      Serial.print("Pune: "); Serial.print(temp, 1); Serial.print(" C, ");
      Serial.print(hum); Serial.print(" %, "); Serial.println(sky);
    }
  } else {
    Serial.print("HTTP error "); Serial.println(code);
  }
  http.end();
}
void loop() {}
```

SERIAL MONITOR

```
Pune: 27.4 C, 58 %, scattered clouds
```

Real weather APIs need a free API key in the URL and usually HTTPS (WiFiClientSecure). Nested JSON is read by chaining keys and array indices, exactly like the structure of the data.

22.5 MQTT

22.5.1 Brokers, topics, publish and subscribe



PUBLISHERS AND SUBSCRIBERS NEVER TALK DIRECTLY: THE BROKER RELAYS MESSAGES BY TOPIC

MQTT is a lightweight publish/subscribe protocol. Devices **publish** messages to topics ("home/garden/temp") on a **broker**; any client **subscribed** to the topic receives them. Wildcards: + matches one level, # matches all lower levels. QoS levels 0, 1, 2 trade overhead for delivery guarantees.

MQTTNODE.INO

```
#include <WiFi.h>
#include <PubSubClient.h>
WiFiClient net;
PubSubClient mqtt(net);
unsigned long last = 0;

void onMessage(char *topic, byte *payload, unsigned int len) {
  String msg;
  for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];
  Serial.print("command on "); Serial.print(topic); Serial.print(": "); Serial.println(msg);
  digitalWrite(2, msg == "ON" ? HIGH : LOW);
}

void setup() {
  Serial.begin(115200);
  pinMode(2, OUTPUT);
  WiFi.begin("HomeNet", "secret123");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  mqtt.setServer("broker.hivemq.com", 1883);
  mqtt.setCallback(onMessage);
  mqtt.connect("lbf-node-01");
  mqtt.subscribe("lbf/demo/led");
}

void loop() {
  mqtt.loop();                                  // keep the connection alive, deliver messages
  if (millis() - last >= 5000) {
    last = millis();
    char buf[8];
    snprintf(buf, sizeof buf, "%d", 240 + (int)(millis() / 1000 % 10));   // demo reading ×10
    mqtt.publish("lbf/demo/temp", buf);
  }
}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] WiFi connecting to HomeNet
[  1500 ms] MQTT broker broker.hivemq.com:1883
[  1500 ms] MQTT connected as lbf-node-01
[  1500 ms] MQTT subscribe lbf/demo/led
command on lbf/demo/led: ON
[  1500 ms] pin 2 -> HIGH
[  5000 ms] MQTT publish lbf/demo/temp = 245
[ 10000 ms] MQTT publish lbf/demo/temp = 240
```

22.6 IoT platforms

Platform	What it offers	Typical use
Arduino Cloud	dashboards, variables synced with the sketch, OTA, phone app	quick dashboards for Arduino and ESP32 boards

Platform	What it offers	Typical use
Blynk	drag-and-drop phone and web dashboards, virtual pins	phone control of home projects
ThingSpeak	stores channels of numbers, graphs, MATLAB analysis; free tier ≈ 15 s between updates	data logging and charts
Firebase	real-time database and authentication (Google)	apps that need live shared data

WORKED EXAMPLE

ThingSpeak update by HTTP GET

http://api.thingspeak.com/update?api_key=YOUR_WRITE_KEY&field1=24.5&field2=61 stores two values in a channel, which ThingSpeak graphs automatically.

22.7 Over-the-air (OTA) updates

OTA lets you upload new firmware over Wi-Fi instead of USB: the ArduinoOTA library makes the board appear as a network port in the IDE; ElegantOTA adds a browser upload page; Arduino Cloud offers OTA for supported boards. The ESP32 keeps two firmware slots, so a failed update can fall back. Protect OTA with a password.

22.8 IoT security basics

Risk	Good practice
Passwords in source code shared on GitHub	keep secrets in a separate secrets.h that is not committed
Unencrypted traffic	use HTTPS and MQTT over TLS (port 8883) where possible
Open public brokers	use your own broker or authenticated accounts; unique, unguessable topics are not security
Unprotected web pages and OTA	add authentication and an OTA password
Old firmware with known bugs	keep libraries and board packages updated; plan for OTA updates
Default credentials on devices	change them; use per-device credentials

22.9 Practice questions

22.9.1 Basic level

Q1 Which board package do you install to program an ESP32 in the Arduino IDE?

(a) "esp32 by Espressif" in the Boards Manager
(b) "Arduino AVR Boards"
(c) the Servo library
(d) nothing; it is built in

Q2 What logic voltage does the ESP32 use?

(a) 5 V **(b)** 12 V **(c)** 3.3 V **(d)** 1.8 V

Q3 Which WiFi mode makes the ESP32 join an existing router?

(a) access point (WIFI_AP) **(b)** monitor
(c) Bluetooth **(d)** station (WIFI_STA)

Q4 What is the usual IP address of an ESP32 in access-point mode?

(a) 127.0.0.1 **(b)** 192.168.4.1 **(c)** 8.8.8.8 **(d)** 10.0.0.255

Q5 Which HTTP status code means success?

(a) 200 **(b)** 404 **(c)** 500 **(d)** 301

Q6 In MQTT, the server that relays messages is called the

(a) router **(b)** gateway pin **(c)** broker **(d)** sketch

Q7 What does the MQTT wildcard # match?

(a) all remaining topic levels **(b)** exactly one level
(c) only numbers **(d)** nothing

Q8 Which HTTP method is normally used to send a sensor reading to a server?

(a) GET only **(b)** POST **(c)** DELETE **(d)** HEAD

22.9.2 Intermediate level

Q9 Connect an ESP32 to Wi-Fi and print the IP address and MAC address.

Q10 Which ESP32 ADC pins remain usable while Wi-Fi is active?

(a) ADC2 pins **(b)** GPIO 6–11
(c) none **(d)** ADC1 pins (GPIO 32–39)

Q11 Why must server.handleClient() be called frequently in loop()?

(a) it connects to Wi-Fi
(b) it keeps the LED on
(c) it checks for and answers incoming browser requests
(d) it updates the clock

Q12 Add a web route /toggle that flips an LED on GPIO 2 and replies with "LED ON" or "LED OFF" as plain text.

Q13 What does deserializeJson() do?

(a) sends JSON to a server
(b) compresses data
(c) parses a JSON text into a document whose fields can be read by name
(d) encrypts a password

Q14 Parse the JSON {"device":"node7","readings":[21.5,22.0,22.8]} and print the device name and the average reading.

Q15 A request returns HTTP 401. What does it mean?

(a) success
(b) server crashed
(c) page moved permanently
(d) unauthorised: the API key or credentials are missing or wrong

Q16 Which port does unencrypted MQTT normally use?

(a) 80 **(b)** 1883 **(c)** 8883 **(d)** 22

22.9.3 Advanced level

Q17 Build a JSON message {"id":"node7","temp":24.5,"ok":true} with ArduinoJson and print it.

Q18 Why is a free public MQTT broker unsuitable for controlling a real door lock?

(a) anyone can subscribe or publish to topics there; there is no authentication or privacy
(b) it is too fast
(c) it only supports numbers
(d) it needs Bluetooth

Q19 Publish a temperature to MQTT topic "lbf/room/temp" every 10 seconds (simulate 20 s) using PubSubClient.

Q20 What is OTA?

(a) a sensor type **(b)** over-the-air firmware update via Wi-Fi
(c) an MQTT topic **(d)** a JSON format

Q21 ThingSpeak's free tier accepts updates roughly how often per channel?

(a) every 15 seconds **(b)** every 1 ms
(c) once a day **(d)** unlimited

Q22 Create a JSON endpoint /status that reports uptime in seconds and free "slots" (fixed 3), and simulate one request.

Q23 Where should Wi-Fi passwords and API keys be kept in a project shared on GitHub?

(a) directly in the main sketch
(b) in the README
(c) in a separate secrets file excluded from the repository
(d) in the commit message

Q24 Which ESP32 GPIOs must never be used because they connect to the flash chip?

(a) 32 to 39 (b) 2 and 4 (c) 21 and 22 (d) 6 to 11

Q25 Handle a query parameter: /set?level=128 sets a PWM level (0–255) and replies with the stored value.

22.10 Solutions

Q1 **Answer: (a) "esp32 by Espressif" in the Boards Manager**

Install **esp32 by Espressif**.

Q2 **Answer: (c) 3.3 V**

ESP32: **3.3 V**.

Q3 **Answer: (d) station (WIFI_STA)**

Station mode.

Q4 **Answer: (b) 192.168.4.1**

softAP default: **192.168.4.1**.

Q5 **Answer: (a) 200**

200 OK.

Q6 **Answer: (c) broker**

The **broker**.

Q7 **Answer: (a) all remaining topic levels**

= **all lower levels**; + = one level.

Q8 **Answer: (b) POST**

POST sends data (GET with query parameters is also common for simple APIs).

Q9 **Answer: Solution program**

Wait for WL_CONNECTED before using the network.

```
#include <WiFi.h>
void setup() {
  Serial.begin(115200);
  WiFi.begin("HomeNet", "secret123");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.print("IP: "); Serial.println(WiFi.localIP().toString());
  Serial.print("MAC: "); Serial.println(WiFi.macAddress());
}
void loop() {}
```

SERIAL MONITOR

```
IP: 192.168.1.42
MAC: 24:6F:28:AA:BB:CC
```

Q10 **Answer: (d) ADC1 pins (GPIO 32–39)**

Wi-Fi uses ADC2, so read analog sensors on **ADC1**.

Q11 **Answer: (c) it checks for and answers incoming browser requests**

Without it, requests are **never answered**.

Q12 **Answer: Solution program**

Each request to /toggle flips the state. Simulated: two requests.

```
#include <WiFi.h>
#include <WebServer.h>
WebServer server(80);
bool on = false;
void handleToggle() {
  on = !on;
  digitalWrite(2, on);
  server.send(200, "text/plain", on ? "LED ON" : "LED OFF");
}
void setup() {
  pinMode(2, OUTPUT);
  WiFi.begin("HomeNet", "secret123");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  server.on("/toggle", handleToggle);
  server.begin();
}
void loop() { server.handleClient(); }
```

SERIAL MONITOR

```
--> browser requests GET /toggle
<-- 200 text/plain: LED ON
--> browser requests GET /toggle
<-- 200 text/plain: LED OFF
```

Q13 **Answer: (c) parses a JSON text into a document whose fields can be read by name**

It **parses** JSON.

Q14 **Answer: Solution program**

Arrays are indexed like C arrays; size() gives the element count.

```
#include <ArduinoJson.h>
void setup() {
  Serial.begin(115200);
  JsonDocument doc;
  deserializeJson(doc, "{\"device\":\"node7\",\"readings\":[21.5,22.0,22.8]}");
  const char *name = doc["device"];
  float sum = 0;
  int n = doc["readings"].size();
  for (int i = 0; i < n; i++) sum += (float)doc["readings"][i];
  Serial.print(name); Serial.print(" average: "); Serial.println(sum / n, 2);
}
void loop() {}
```

SERIAL MONITOR

```
node7 average: 22.10
```

Q15 **Answer: (d) unauthorised: the API key or credentials are missing or wrong**

401 Unauthorized.

Q16 **Answer: (b) 1883**

1883 (8883 for MQTT over TLS).

Q17 **Answer: Solution program**

serializeJson() writes compact JSON, ready for an HTTP POST or MQTT publish.

```
#include <ArduinoJson.h>
void setup() {
  Serial.begin(115200);
  JsonDocument doc;
  doc["id"] = "node7";
  doc["temp"] = 24.5;
  doc["ok"] = true;
  serializeJson(doc, Serial);
  Serial.println();
}
void loop() {}
```

SERIAL MONITOR

```
{"id":"node7","temp":24.5,"ok":true}
```

Q18 **Answer: (a) anyone can subscribe or publish to topics there; there is no authentication or privacy**

Public brokers are **open to everyone**.

Q19 **Answer: Solution program**

mqtt.loop() keeps the connection alive; a millis() timer spaces the messages.

```
#include <WiFi.h>
#include <PubSubClient.h>
WiFiClient net; PubSubClient mqtt(net);
unsigned long last = 0;
void setup() {
  WiFi.begin("HomeNet", "secret123");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  mqtt.setServer("broker.example.com", 1883);
  mqtt.connect("room-sensor");
}
void loop() {
  mqtt.loop();
  if (millis() - last >= 10000) {
    last = millis();
    mqtt.publish("lbf/room/temp", "24.5");
  }
}
```

SIMULATED RUN

```
[     0 ms] WiFi connecting to HomeNet
[  1500 ms] MQTT broker broker.example.com:1883
[  1500 ms] MQTT connected as room-sensor
[ 10000 ms] MQTT publish lbf/room/temp = 24.5
[ 20000 ms] MQTT publish lbf/room/temp = 24.5
```

Q20 **Answer: (b) over-the-air firmware update via Wi-Fi**

Over-the-air updates.

Q21 **Answer: (a) every 15 seconds**

About **15 s** minimum interval.

Q22 **Answer: Solution program**

Building JSON by string concatenation works for small objects; ArduinoJson is safer for larger ones.

```
#include <WiFi.h>
#include <WebServer.h>
WebServer server(80);
void handleStatus() {
  String j = "{\"uptime\":" + String(millis() / 1000) + ",\"slots\":3}";
  server.send(200, "application/json", j);
}
void setup() {
  WiFi.begin("HomeNet", "secret123");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  server.on("/status", handleStatus);
  server.begin();
}
void loop() { server.handleClient(); }
```

SERIAL MONITOR

```
--> browser requests GET /status
<-- 200 application/json: {"uptime":1,"slots":3}
```

Q23 **Answer: (c) in a separate secrets file excluded from the repository**

Keep secrets **out of version control**.

Q24 **Answer: (d) 6 to 11**

GPIO **6–11** are flash pins.

Q25 **Answer: Solution program**

arg() returns the parameter as a String; constrain() protects against out-of-range input. Simulated: level=128, then level=999.

```
#include <WiFi.h>
#include <WebServer.h>
WebServer server(80);
int level = 0;
void handleSet() {
  if (server.hasArg("level")) level = constrain(server.arg("level").toInt(), 0, 255);
  analogWrite(2, level);
  server.send(200, "text/plain", "level=" + String(level));
}
void setup() {
  WiFi.begin("HomeNet", "secret123");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  server.on("/set", handleSet);
  server.begin();
}
void loop() { server.handleClient(); }
```

SERIAL MONITOR

```
--> browser requests GET /set?level=128
<-- 200 text/plain: level=128
```

```
--> browser requests GET /set?level=999
<-- 200 text/plain: level=255
```

ESP32: 3.3 V, ADC1 with Wi-Fi, avoid GPIO 6–11. WiFi.begin() and wait for WL_CONNECTED; softAP for setup at 192.168.4.1. WebServer: server.on(path, handler), send(code, type, body), handleClient() in loop. HTTPClient + ArduinoJson for APIs (check status codes). MQTT: broker, topics, publish/subscribe, mqtt.loop(). Use platforms for dashboards, OTA with a password, secrets out of code, TLS where possible.

MODULE 23 · ADVANCED

Power management and battery projects

Minimum duration **1 day** | Level **Advanced**

A battery-powered project is only useful if it runs for weeks or months. This module covers measuring current consumption and estimating battery life, AA, 9 V, Li-ion and LiPo batteries, TP4056 charging and protection, linear regulators versus buck and boost converters, sleep modes and waking from interrupts and timers, ESP32 deep sleep, reducing power wasted by LEDs and regulators, and solar-powered projects, followed by 25 solved questions and programs with every calculation checked.

IN THIS MODULE

23.1 Power requirements
23.2 Batteries
23.3 Voltage regulation
23.4 Low-power techniques
23.5 Solar-powered projects
23.6 Practice questions
23.7 Solutions

23.1 Power requirements

23.1.1 Measuring current consumption

Put a multimeter set to mA (or µA) **in series** with the supply, or use a USB power meter or a current-sensor module (INA219). Measure each state of the project: active, transmitting, sleeping. An Uno R3 board draws about 45–50 mA even doing nothing, mostly in its USB chip, regulator and power LED; a bare ATmega328P can sleep at under 1 µA.

Device / state	Typical current
Arduino Uno R3 board, idle	≈ 45–50 mA
Arduino Pro Mini 3.3 V, power LED and regulator removed, sleeping	≈ 5 µA
ESP32 with Wi-Fi transmitting	≈ 160–260 mA peaks
ESP32 modem sleep / light sleep / deep sleep	≈ 20 mA / 0.8 mA / 10 µA
Red LED at full brightness	≈ 10–20 mA
HC-SR04 / DHT22 measuring	≈ 15 mA / 1.5 mA

23.1.2 Estimating battery life

Battery life (hours) ≈ capacity (mAh) / average current (mA). Average current = Σ(current × time in that state) / total time. Use about 70–80 % of the rated capacity for a realistic estimate.

WORKED EXAMPLE

Duty-cycled sensor node

Awake 2 s at 50 mA, then asleep 58 s at 0.1 mA, repeating each minute. Average = (50 × 2 + 0.1 × 58) / 60 = 1.76 mA. A 2000 mAh battery at 80 % → 1600 / 1.76 ≈ 909 h ≈ **38 days**. Without sleep (50 mA continuously): 32 h.

BATTERYLIFE.INO

```
// battery-life estimator for a duty-cycled device
void setup() {
  Serial.begin(9600);
  float activeMa = 50, activeS = 2, sleepMa = 0.1, sleepS = 58;
  float capacity = 2000, usable = 0.8;
  float avg = (activeMa * activeS + sleepMa * sleepS) / (activeS + sleepS);
  float hours = capacity * usable / avg;
  Serial.print("average current: "); Serial.print(avg, 2); Serial.println(" mA");
  Serial.print("battery life: "); Serial.print(hours, 0); Serial.print(" h = ");
  Serial.print(hours / 24, 1); Serial.println(" days");
}
void loop() {}
```

SERIAL MONITOR

```
average current: 1.76 mA
battery life: 907 h = 37.8 days
```

23.2 Batteries

23.2.1 AA, 9 V, Li-ion and LiPo batteries

Type	Voltage	Typical capacity	Notes
AA alkaline	1.5 V (×3 or ×4)	2000–2800 mAh	cheap, safe; capacity falls at high current
AA NiMH rechargeable	1.2 V	1900–2500 mAh	4 cells ≈ 4.8 V
9 V PP3	9 V	≈ 500 mAh	poor choice: small capacity, regulator wastes energy
Li-ion 18650	3.7 V nominal (4.2 V full, 3.0 V empty)	2000–3500 mAh	high energy; needs protection and a proper charger
LiPo pouch	3.7 V nominal	100–5000 mAh	thin and light; puncture or overcharge can cause fire
CR2032 coin	3 V	≈ 220 mAh	only for µA-level loads (RTC backup, BLE beacons)

23.2.2 Charging modules (TP4056) and battery protection

A TP4056 module charges a single Li-ion/LiPo cell from 5 V USB with the correct constant-current then constant-voltage (4.2 V) profile; the version with a DW01 protection chip also cuts off at over-discharge

(≈ 2.5 V), over-charge and short circuit. Set its charge current (default 1 A) to suit the cell with the PROG resistor.

> **WATCH OUT**
> Never charge lithium cells without a proper charger and protection circuit, never puncture or crush them, and do not leave them charging unattended in hot places.

23.3 Voltage regulation

23.3.1 Linear regulators vs buck and boost converters

	Linear regulator (7805, AMS1117, LDO)	Buck converter	Boost converter
Function	drops voltage by burning the excess as heat	steps voltage down efficiently	steps voltage up
Efficiency	Vout / Vin (5/9 = 56 %)	85–95 %	80–90 %
Noise	very low	some switching noise	some switching noise
Use	small currents, small voltage drops	batteries above the needed voltage	one Li-ion cell to 5 V

A linear regulator wastes P = (Vin - Vout) × I. From 12 V to 5 V at 200 mA: 7 × 0.2 = 1.4 W of heat. A buck converter wastes only about 0.1–0.2 W.

23.4 Low-power techniques

23.4.1 Sleep modes

The ATmega328P has six sleep modes; **power-down** stops almost everything (current below 1 µA for the chip alone) and can be woken only by external interrupts, pin changes or the watchdog. The LowPower library (LowPower.powerDown(SLEEP_8S, ADC_OFF, BOD_OFF)) makes this easy.

23.4.2 Waking from interrupts and timers

SLEEPWAKE.INO

```
#include <avr/sleep.h>
const int WAKE = 2;
volatile bool woke = false;
void wakeUp() { woke = true; }

void setup() {
  Serial.begin(9600);
  pinMode(WAKE, INPUT_PULLUP);
}

void loop() {
  Serial.println("going to sleep");
  Serial.flush();                                  // finish sending before sleeping
  attachInterrupt(digitalPinToInterrupt(WAKE), wakeUp, LOW);
  set_sleep_mode(SLEEP_MODE_PWR_DOWN);
  sleep_enable();
  sleep_cpu();                                     // stops here until pin 2 goes LOW
  sleep_disable();
  detachInterrupt(digitalPinToInterrupt(WAKE));
  Serial.println("woken by button: take a reading, then sleep again");
  delay(100);
}
```

SIMULATED RUN (pin changes and Serial output)

```
going to sleep
[     0 ms] CPU sleeping (power-down)
woken by button: take a reading, then sleep again
```

The watchdog can also wake the chip every 1–8 s without any external signal: sleep for 8 s repeatedly and take a reading every n-th wake-up.

23.4.3 Deep sleep on ESP32

DEEPSLEEP.INO

```
#include <esp_sleep.h>
RTC_DATA_ATTR int bootCount = 0;          // survives deep sleep (kept in RTC memory)

void setup() {
  Serial.begin(115200);
  bootCount++;
  Serial.print("wake-up number "); Serial.println(bootCount);
  // read sensors and send data here (Wi-Fi, MQTT …)
  esp_sleep_enable_timer_wakeup(10ULL * 60 * 1000000);   // 10 minutes in µs
  esp_deep_sleep_start();                                 // the sketch restarts from setup()
}
void loop() {}                                            // never reached
```

SIMULATED RUN (pin changes and Serial output)

```
wake-up number 1
[     0 ms] deep-sleep wake-up timer 600 s
```

```
[     0 ms] entering deep sleep
(deep sleep: the sketch restarts from setup() on wake-up)
```

In deep sleep the ESP32 draws about 10 µA (the chip; many development boards draw more through their USB chip and regulator). It restarts from setup() on wake-up; variables marked RTC_DATA_ATTR survive.

23.4.4 Reducing power from LEDs and regulators

Change	Typical saving
Remove or cut the power LED	2–5 mA
Replace the AMS1117 regulator with a low-quiescent LDO (MCP1700, HT7333)	5 mA → a few µA
Run at 3.3 V and 8 MHz (Pro Mini 3.3 V)	roughly half the active current
Turn sensors on only when reading (power them from a pin or a MOSFET)	the sensor's whole idle current
Disable the ADC and brown-out detector while sleeping	≈ 100 µA and ≈ 20 µA

23.5 Solar-powered projects

A small solar panel (5–6 V, 1–3 W) charges a Li-ion cell through a solar-capable charger (TP4056 works with a stable panel; CN3791 or dedicated MPPT boards do better). Size the panel and battery for the worst season: daily energy used × number of cloudy days the battery must cover.

WORKED EXAMPLE

Panel sizing

Node average 2 mA at 3.7 V ≈ 0.18 Wh per day. A 1 W panel giving 3 effective sun-hours ≈ 3 Wh per day minus about 30 % losses ≈ 2 Wh: more than ten times the need. A 2000 mAh cell (≈ 7.4 Wh) covers more than a month without sun.

23.6 Practice questions

23.6.1 Basic level

Q1 How is a multimeter connected to measure a circuit's current?

(a) in parallel with the supply **(b)** across the LED
(c) to the 3.3 V pin only **(d)** in series with the supply

Q2 A device draws 20 mA from a 2000 mAh battery. About how long will it last (ideal)?

(a) 40 hours **(b)** 1000 hours **(c)** 100 hours **(d)** 10 hours

Q3 What is the nominal voltage of a Li-ion cell?

(a) 3.7 V **(b)** 1.5 V **(c)** 9 V **(d)** 12 V

Q4 Which module charges a single Li-ion cell from USB?

(a) L298N **(b)** HC-05 **(c)** PCA9685 **(d)** TP4056

Q5 Which converter steps a voltage down efficiently?

(a) boost converter **(b)** buck converter
(c) linear regulator always **(d)** H-bridge

Q6 Which ATmega328P sleep mode saves the most power?

(a) idle **(b)** power-down
(c) active **(d)** ADC noise reduction

Q7 Why is a 9 V PP3 battery a poor choice for an Arduino project?

(a) it is too heavy
(b) it gives AC
(c) it is 3.3 V
(d) small capacity (≈ 500 mAh), and the regulator wastes much of its energy

Q8 About how much current does an idle Uno R3 board draw?

(a) 45–50 mA **(b)** 1 µA **(c)** 1 A **(d)** 500 mA

23.6.2 Intermediate level

Q9 Compute the battery life of a node that is awake 1 s at 80 mA and asleep 299 s at 0.05 mA, with a 3000 mAh battery used to 80 %.

Q10 A linear regulator drops 9 V to 5 V at 100 mA. How much power becomes heat?

(a) 0.9 W **(b)** 0.05 W **(c)** 0.4 W **(d)** 4 W

Q11 What is the efficiency of a linear regulator from 9 V to 5 V (ignoring quiescent current)?

(a) about 95 % **(b)** about 100 % **(c)** about 10 % **(d)** about 56 %

Q12 A sensor is powered from pin 7 only while reading: switch it on, wait 50 ms to settle, read A0, switch it off. Show the simulated sequence.

Q13 What wakes an ATmega328P from power-down sleep?

(a) millis()
(b) an external or pin-change interrupt, or the watchdog
(c) Serial.print()
(d) analogWrite()

Q14 Why call Serial.flush() before sleeping?

(a) so the last message finishes sending before the clock stops
(b) to erase EEPROM
(c) to raise the baud rate
(d) to reset the board

Q15 On an ESP32, keep a counter in RTC memory across deep sleeps and wake every 30 s (show one cycle).

Q16 What current does an ESP32 chip draw in deep sleep?

(a) about 100 mA (b) about 20 mA (c) about 10 µA (d) about 1 A

23.6.3 Advanced level

Q17 A boost converter powers a 5 V, 100 mA load from a 3.7 V cell at 85 % efficiency. What current does the cell supply?

(a) about 159 mA (b) 100 mA (c) 85 mA (d) 135 mA

Q18 Print a table of battery life (days) for average currents of 0.5, 2 and 10 mA with a 2500 mAh battery at 80 %.

Q19 Which change saves the most power on a sleeping Pro Mini?

(a) changing the baud rate
(b) removing the power LED and the regulator's quiescent current
(c) using shorter wires
(d) adding comments to the code

Q20 What voltage indicates a Li-ion cell is empty and should stop discharging?

(a) 4.2 V
(b) 1.5 V
(c) 0 V
(d) about 3.0 V (protection cuts at ≈ 2.5 V)

Q21 Measure a single Li-ion cell (up to 4.2 V) on A0 of a 5 V board and convert it to an approximate state of charge (3.0 V = 0 %, 4.2 V = 100 %).

Q22 A 5 V, 1 W panel gets 4 effective sun-hours. About how much energy can it deliver per day after 30 % losses?

(a) about 4 Wh (b) about 1 Wh (c) about 2.8 Wh (d) about 20 Wh

Q23 What is the main safety risk with LiPo batteries?

(a) they leak acid always
(b) they need 12 V chargers
(c) fire if punctured, over-charged or short-circuited

(d) none

Q24 Simulate a watchdog-style schedule: "sleep" in 8-second units and take a reading every 8th wake-up (about once a minute); show which wake-ups read for 17 wake-ups.

Q25 Why are LoRa or BLE often preferred to Wi-Fi in battery sensor nodes?

(a) Wi-Fi cannot send numbers

(b) LoRa is faster

(c) BLE needs no antenna

(d) Wi-Fi transmission draws much more current (hundreds of mA) and takes longer to connect

23.7 Solutions

Q1 **Answer: (d) in series with the supply**

Current flows **through** the meter.

Q2 **Answer: (c) 100 hours**

2000 / 20 = **100 h**.

Q3 **Answer: (a) 3.7 V**

3.7 V nominal (4.2 V full).

Q4 **Answer: (d) TP4056**

The **TP4056**.

Q5 **Answer: (b) buck converter**

A **buck** converter.

Q6 **Answer: (b) power-down**

Power-down.

Q7 **Answer: (d) small capacity (≈ 500 mAh), and the regulator wastes much of its energy**

Low capacity and regulator losses.

Q8 **Answer: (a) 45–50 mA**

USB chip, regulator and LED: **≈ 45–50 mA**.

Q9 **Answer: Solution program**

Weighted average current, then usable capacity ÷ average. Sleep dominates the time but the 1 s awake dominates the energy.

```
void setup() {
  Serial.begin(9600);
  float avg = (80.0 * 1 + 0.05 * 299) / 300;
  float days = 3000 * 0.8 / avg / 24;
  Serial.print("average "); Serial.print(avg, 3); Serial.println(" mA");
  Serial.print("about "); Serial.print(days, 0); Serial.println(" days");
}
void loop() {}
```

SERIAL MONITOR

```
average 0.317 mA
about 316 days
```

Q10 **Answer: (c) 0.4 W**

(9 − 5) × 0.1 = **0.4 W**.

Q11 **Answer: (d) about 56 %**

Vout / Vin = 5 / 9 ≈ **56 %**.

Q12 **Answer: Solution program**

The sensor draws current for 50 ms out of every 5 s: about 1 % of the time. (Pins can supply only about 20 mA; larger sensors need a MOSFET switch.)

```
void setup() {
  Serial.begin(9600);
  pinMode(7, OUTPUT);
}
void loop() {
  digitalWrite(7, HIGH);        // power the sensor
  delay(50);                    // let it settle
  int v = analogRead(A0);
  digitalWrite(7, LOW);         // power off again
  Serial.print("reading "); Serial.println(v);
  delay(5000);
}
```

SIMULATED RUN

```
[     0 ms] pin 7 -> HIGH
[    50 ms] pin 7 -> LOW
reading 512
[  5050 ms] pin 7 -> HIGH
[  5100 ms] pin 7 -> LOW
reading 515
```

Q13 **Answer: (b) an external or pin-change interrupt, or the watchdog**

Only interrupts and the **watchdog** run in power-down.

Q14 **Answer: (a) so the last message finishes sending before the clock stops**

Otherwise the text is **cut off**.

Q15 **Answer: Solution program**

Ordinary variables are lost in deep sleep; RTC_DATA_ATTR keeps them. The time is given in microseconds.

```
#include <esp_sleep.h>
RTC_DATA_ATTR int wakes = 0;
void setup() {
  Serial.begin(115200);
  wakes++;
  Serial.print("wake "); Serial.println(wakes);
  esp_sleep_enable_timer_wakeup(30ULL * 1000000);
  esp_deep_sleep_start();
}
void loop() {}
```

SIMULATED RUN

```
wake 1
[     0 ms] deep-sleep wake-up timer 30 s
[     0 ms] entering deep sleep
(deep sleep: the sketch restarts from setup() on wake-up)
```

Q16 **Answer: (c) about 10 µA**

≈ 10 µA (development boards often more).

Q17 **Answer: (a) about 159 mA**

Pin = 0.5 W / 0.85 = 0.588 W; I = 0.588 / 3.7 ≈ **159 mA**.

Q18 **Answer: Solution program**

Life is inversely proportional to average current: cutting current by 20× extends life by 20×.

```
void setup() {
  Serial.begin(9600);
  float currents[] = {0.5, 2, 10};
  for (float i : currents) {
    Serial.print(i, 1); Serial.print(" mA -> ");
    Serial.print(2500 * 0.8 / i / 24, 1); Serial.println(" days");
  }
}
void loop() {}
```

SERIAL MONITOR

```
0.5 mA -> 166.7 days
2.0 mA -> 41.7 days
10.0 mA -> 8.3 days
```

Q19 **Answer: (b) removing the power LED and the regulator's quiescent current**

LED and regulator draw **milliamps** while the chip sleeps at microamps.

Q20 **Answer: (d) about 3.0 V (protection cuts at ≈ 2.5 V)**

Below **≈ 3.0 V** capacity is exhausted; deeper discharge damages the cell.

Q21 **Answer: Solution program**

A linear mapping is only approximate (the real discharge curve is flat in the middle). The reading 778 corresponds to about 3.80 V.

```
void setup() {
  Serial.begin(9600);
  float v = analogRead(A0) * 5.0 / 1023.0;
  float pct = constrain((v - 3.0) / 1.2 * 100.0, 0.0, 100.0);
  Serial.print(v, 2); Serial.print(" V -> "); Serial.print(pct, 0); Serial.println(" %");
}
void loop() {}
```

SERIAL MONITOR

```
3.80 V -> 67 %
```

Q22 **Answer: (c) about 2.8 Wh**

1 W × 4 h × 0.7 = **2.8 Wh**.

Q23 **Answer: (c) fire if punctured, over-charged or short-circuited**

LiPos can **catch fire**.

Q24 **Answer: Solution program**

The watchdog wakes at most every 8 s; counting wake-ups gives longer intervals while the chip spends almost all its time in power-down.

```
void setup() {
  Serial.begin(9600);
  int wakes = 0;
  for (int w = 1; w <= 17; w++) {
    wakes++;
    if (wakes % 8 == 0) { Serial.print("wake "); Serial.print(w); Serial.print(" at ");
Serial.print(w * 8); Serial.println(" s: reading"); }
  }
}
void loop() {}
```

SERIAL MONITOR

```
wake 8 at 64 s: reading
wake 16 at 128 s: reading
```

Q25 **Answer: (d) Wi-Fi transmission draws much more current (hundreds of mA) and takes longer to connect**

Wi-Fi's **high power** drains batteries.

Battery life ≈ usable mAh / average mA, with average = Σ(current × time) / total time. Li-ion 3.7 V (4.2 full, 3.0 empty) needs a protected charger (TP4056 + DW01). Linear regulators waste (Vin - Vout) × I; use buck/boost converters for efficiency. Sleep (power-down, ESP32 deep sleep), wake by interrupts or timers, power sensors only when needed, remove LEDs and wasteful regulators.

MODULE 24 · ADVANCED

Advanced Arduino programming

Minimum duration **2 days** | Level **Advanced**

Larger projects need disciplined code. This module covers object-oriented classes for components, designing behaviour as state machines with enum and switch, memory optimisation (flash versus SRAM, F() and PROGMEM, avoiding String fragmentation), speed optimisation (direct port manipulation, avoiding floating point), FreeRTOS multitasking on the ESP32, and code quality with naming, modular files and Git, followed by 25 solved questions and programs.

IN THIS MODULE

24.1 Object-oriented programming
24.2 State machines
24.3 Memory optimisation
24.4 Speed optimisation
24.5 Multitasking
24.6 Code quality
24.7 Practice questions
24.8 Solutions

24.1 Object-oriented programming

24.1.1 Classes for components (LED, Button, Motor)

A class bundles a component's data (pin, state, timers) with the functions that use it. Each physical component becomes one object, so adding a second button or motor is one line.

24.1.2 Constructors and member functions

MOTORCLASS.INO

```
class Motor {
 public:
  Motor(int en, int in1, int in2) : _en(en), _in1(in1), _in2(in2) {}   // constructor
  void begin() { pinMode(_in1, OUTPUT); pinMode(_in2, OUTPUT); stop(); }
  void drive(int speed) {                                              // -255 … 255
    _speed = constrain(speed, -255, 255);
    digitalWrite(_in1, _speed > 0);
    digitalWrite(_in2, _speed < 0);
    analogWrite(_en, abs(_speed));
  }
  void stop() { drive(0); }
  int speed() const { return _speed; }
 private:
  int _en, _in1, _in2, _speed = 0;
};

Motor left(5, 7, 8), right(6, 9, 10);

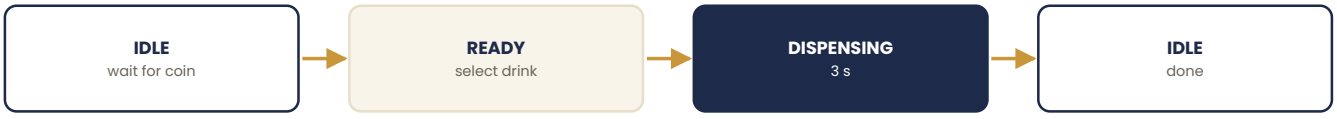
void setup() {
  Serial.begin(9600);
  left.begin(); right.begin();
  left.drive(200); right.drive(-300);          // -300 is clamped to -255
  Serial.print("left "); Serial.print(left.speed());
  Serial.print(", right "); Serial.println(right.speed());
}
void loop() {}
```

SERIAL MONITOR

```
left 200, right -255
```

24.2 State machines

24.2.1 Designing behaviour with states and events



A VENDING MACHINE: STATES AND THE EVENTS THAT CHANGE THEM

State	Event	Action	Next state
IDLE	coin inserted	show "select"	READY
READY	button pressed	start pump	DISPENSING
READY	10 s timeout	refund coin	IDLE
DISPENSING	3 s elapsed	stop pump	IDLE

24.2.2 Implementing with enum and switch

VENDING.INO

```
enum State { IDLE, READY, DISPENSING };
State state = IDLE;
unsigned long since = 0;
const int COIN = 2, BUTTON = 3, PUMP = 13;

void enter(State s, const char *msg) { state = s; since = millis(); Serial.print(millis());
Serial.print(" ms: "); Serial.println(msg); }

void setup() {
  Serial.begin(9600);
  pinMode(COIN, INPUT_PULLUP); pinMode(BUTTON, INPUT_PULLUP); pinMode(PUMP, OUTPUT);
}

void loop() {
  switch (state) {
    case IDLE:
      if (digitalRead(COIN) == LOW) enter(READY, "coin -> select a drink");
      break;
    case READY:
      if (digitalRead(BUTTON) == LOW) { digitalWrite(PUMP, HIGH); enter(DISPENSING, "dispensing"); }
      else if (millis() - since > 10000) enter(IDLE, "timeout -> refund");
      break;
    case DISPENSING:
      if (millis() - since >= 3000) { digitalWrite(PUMP, LOW); enter(IDLE, "done"); }
      break;
  }
}
```

SERIAL MONITOR

```
500 ms: coin -> select a drink
2000 ms: dispensing
5000 ms: done
6000 ms: coin -> select a drink
16001 ms: timeout -> refund
```

The simulation inserted a coin, pressed the button, waited for dispensing to finish, then inserted a second coin and walked away, triggering the 10-second timeout.

24.3 Memory optimisation

24.3.1 Flash vs SRAM usage

After compiling, the IDE reports "Sketch uses X bytes (Y %) of program storage space" (flash) and "Global variables use Z bytes" (SRAM). The rest of SRAM is shared by the stack (local variables, function calls) and the heap (String, new). Keep at least 20–25 % of SRAM free on an Uno R3.

24.3.2 F() macro and PROGMEM

KEEPING CONSTANTS IN FLASH

```
#include <avr/pgmspace.h>
const char msg[] PROGMEM = "Long constant text kept in flash";
const int table[] PROGMEM = {0, 87, 174, 259, 342};   // a lookup table in flash
void setup() {
  Serial.begin(9600);
  Serial.println(F("This literal stays in flash"));   // F() for print literals
  int v = pgm_read_word(&table[2]);                     // read a word from flash
}
void loop() {}
```

On AVR boards, string literals and const arrays are copied into SRAM at start-up unless marked F() or PROGMEM. On ESP32 and other 32-bit boards const data stays in flash automatically, and F() is harmless.

24.3.3 Avoiding String fragmentation

Prefer fixed char buffers with snprintf(), call reserve() if you must use String, pass Strings by const reference (const String &s) rather than by value, and avoid building Strings in fast loops.

24.4 Speed optimisation

24.4.1 Direct port manipulation

digitalWrite() takes about 4 µs on an Uno R3 because it looks up the pin and checks PWM. Writing the port register directly takes about 0.06 µs (one instruction) and can change up to 8 pins at the same instant.

Uno pins	Port	Registers
0–7	D	DDRD (direction), PORTD (output), PIND (input)
8–13	B	DDRB, PORTB, PINB (pin 13 = bit 5)
A0–A5	C	DDRC, PORTC, PINC

PORTMANIPULATION.INO

```
#include <avr/io.h>
void setup() {
  Serial.begin(9600);
  DDRB |= _BV(PB5);          // pin 13 as output (same as pinMode(13, OUTPUT))
  PORTB |= _BV(PB5);         // pin 13 HIGH
  Serial.println(PORTB, BIN);
  PORTB &= ~_BV(PB5);        // pin 13 LOW
  Serial.println(PORTB, BIN);
  PORTB ^= _BV(PB5);         // toggle
  Serial.println(PORTB, BIN);
}
void loop() {}
```

SERIAL MONITOR

```
100000
0
100000
```

WATCH OUT

Port code is fast but not portable: register names and pin mappings differ between boards. Use it only where speed really matters, and comment which pins the bits are.

24.4.2 Avoiding floating-point where possible

The ATmega328P has no floating-point hardware: a float multiply takes tens of times longer than an integer one. Use scaled integers (store 24.5 °C as 245 tenths), integer maths in tight loops, and convert to float only for display.

INTEGERMATHS.INO

```
void setup() {
  Serial.begin(9600);
  int raw = 153;                              // TMP36 reading
  long mv = (long)raw * 5000 / 1023;          // millivolts with integer maths
  Serial.print(mv); Serial.println(" mV");
  int tenths = (mv - 500);                    // TMP36: (mV - 500) / 10 = degrees, so this is
tenths
  Serial.print(tenths / 10); Serial.print('.'); Serial.print(abs(tenths % 10)); Serial.println("
C");
}
void loop() {}
```

SERIAL MONITOR

```
747 mV
24.7 C
```

24.5 Multitasking

24.5.1 FreeRTOS tasks on ESP32

The ESP32 core runs FreeRTOS, a real-time operating system that switches between tasks automatically (pre-emptive multitasking). Each task is a function with its own infinite loop and stack, can be pinned to core 0 or 1, and uses vTaskDelay() to sleep without blocking others. Queues and mutexes pass data safely between tasks.

FREERTOS TASKS (ESP32; ON THE REAL BOARD NO #INCLUDE IS NEEDED)

```
#include "freertos.h"
void blinkTask(void *param) {
  pinMode(2, OUTPUT);
  for (;;) {                                  // each task loops for ever
    digitalWrite(2, !digitalRead(2));
    vTaskDelay(500 / portTICK_PERIOD_MS);     // other tasks run meanwhile
  }
}
void sensorTask(void *param) {
  for (;;) {
    int v = analogRead(34);
    vTaskDelay(pdMS_TO_TICKS(2000));
  }
}
void setup() {
  xTaskCreatePinnedToCore(blinkTask, "blink", 2048, NULL, 1, NULL, 1);
  xTaskCreatePinnedToCore(sensorTask, "sensor", 4096, NULL, 1, NULL, 0);
}
void loop() {}                                // can be empty
```

	Cooperative (millis)	FreeRTOS
Switching	each task must return quickly	the scheduler pre-empts tasks
Blocking calls	break everything	allowed inside a task
Shared data	plain variables	queues, mutexes, semaphores
Boards	any	ESP32 (built in), others with a library

24.6 Code quality

24.6.1 Comments, naming, modular files and version control with Git

Practice	Example
Descriptive names	pumpPin, readMoisture(), MOISTURE_THRESHOLD instead of p, f(), 512
Named constants	const int PUMP_PIN = 7;
Small functions	one job each: readSensors(), updateDisplay(), checkAlarms()
Comments explain why	// wait for the sensor to settle (datasheet: 30 ms)
Modular files	extra tabs in the IDE: sensors.h/.cpp, display.h/.cpp, config.h
Version control	git init; git add .; git commit -m "add pump timeout"; push to GitHub

Readable code is easier to debug and extend: name things clearly, use constants, write small functions, split files by component, and commit to Git after each working change.

24.7 Practice questions

24.7.1 Basic level

Q1 What is a constructor?

(a) a function that deletes objects
(b) a special member function that runs when an object is created
(c) a type of loop
(d) a library manager

Q2 Which keyword hides members so that only the class itself can access them?

(a) private **(b)** public **(c)** static **(d)** volatile

Q3 Which register sets the direction of pins 8–13 on an Uno?

(a) PORTD **(b)** DDRB **(c)** PINC **(d)** TCCR1A

Q4 Pin 13 on the Uno corresponds to which bit of PORTB?

(a) bit 5 **(b)** bit 0 **(c)** bit 13 **(d)** bit 7

Q5 What does the F() macro do on an AVR board?

(a) formats a float
(b) flips a bit
(c) keeps a string literal in flash instead of copying it to SRAM
(d) sets a frequency

Q6 Which construct best represents the states of a machine?

(a) float **(b)** String
(c) array of floats **(d)** enum

Q7 Which operating system does the ESP32 Arduino core run underneath?

(a) Linux **(b)** Windows **(c)** Android **(d)** FreeRTOS

Q8 What is Git used for?

(a) uploading to the board
(b) measuring current
(c) tracking versions of code and collaborating
(d) drawing schematics

24.7.2 Intermediate level

Q9 Write a class Led with blink(times, ms) and use it on pin 9.

Q10 How long does digitalWrite() take on an Uno R3 compared with direct port writing?

(a) about 4 µs versus about 0.06 µs **(b)** both about 1 ms
(c) port writing is slower **(d)** both exactly 1 µs

Q11 What does "PORTB ^= _BV(PB5);" do?

(a) sets pin 13 HIGH **(b)** sets pin 13 LOW

(c) reads pin 13
(d) toggles pin 13

Q12 Implement a door state machine (CLOSED, OPENING, OPEN, CLOSING) where OPENING and CLOSING each take 2 s and OPEN lasts 3 s; print transitions.

Q13 Why is a float multiplication slow on an Uno R3?

(a) floats are stored in EEPROM
(b) the ATmega328P has no floating-point hardware, so it is done in software
(c) floats need Wi-Fi
(d) it is not slow

Q14 A sketch reports "Global variables use 1850 bytes (90%) of dynamic memory". What is the risk?

(a) flash will wear out
(b) too little SRAM left for the stack and heap; random crashes
(c) the sketch will not upload
(d) nothing

Q15 Store temperature as integer tenths: convert 245 tenths to "24.5 C" and -37 tenths to "-3.7 C" without floats.

Q16 Which is a valid reason to use PROGMEM?

(a) to make variables change faster
(b) to share data with an ISR
(c) to store Wi-Fi passwords securely
(d) to keep a large constant lookup table out of the Uno's 2 KB SRAM

24.7.3 Advanced level

Q17 What does vTaskDelay() do in a FreeRTOS task?

(a) pauses that task only, letting other tasks run
(b) stops all tasks
(c) resets the ESP32
(d) deletes the task

Q18 Use direct port access to set pins 8, 9 and 10 HIGH at the same instant (PORTB bits 0–2) and print PORTB in binary.

Q19 Why should a String be passed to a function as const String & rather than String?

(a) to make it uppercase
(b) because String cannot be passed
(c) to avoid copying it (saving time and heap memory)
(d) to store it in EEPROM

Q20 In the vending machine, which event moves READY back to IDLE?

(a) inserting a coin
(b) the pump finishing
(c) power-up
(d) a 10-second timeout

Q21 Write a simple FreeRTOS-style comparison: compute how many 500 ms blinks and 2000 ms sensor reads happen in 10 s, as two independent tasks would.

Q22 Which is the best name for a constant holding the soil-moisture threshold?

(a) x
(b) MOISTURE_THRESHOLD
(c) val2
(d) temp

Q23 Which tool lets you return to the last working version of your sketch after a change breaks it?

(a) Git
(b) the Serial Plotter
(c) the Library Manager
(d) a multimeter

Q24 Write a Button class with a pressed() method based on edge detection and use two buttons (pins 2 and 3) to count presses separately.

Q25 What is the main disadvantage of direct port manipulation?

(a) it is slow
(b) it uses more SRAM
(c) it is not portable between boards and is harder to read
(d) it needs Wi-Fi

24.8 Solutions

Q1 **Answer: (b) a special member function that runs when an object is created**

It **initialises** a new object.

Q2 **Answer: (a) private**

private.

Q3 **Answer: (b) DDRB**

DDRB (data direction register B).

Q4 **Answer: (a) bit 5**

Pins 8–13 = PB0–PB5, so 13 is **PB5**.

Q5 **Answer: (c) keeps a string literal in flash instead of copying it to SRAM**

Flash strings.

Q6 **Answer: (d) enum**

An **enum** names the states.

Q7 **Answer: (d) FreeRTOS**

FreeRTOS.

Q8 **Answer: (c) tracking versions of code and collaborating**

Version control.

Q9 **Answer: Solution program**

The pin is stored once in the object, so callers only say what to do.

```
class Led {
 public:
  Led(int p) : _p(p) {}
  void begin() { pinMode(_p, OUTPUT); }
  void blink(int times, int ms) {
    for (int i = 0; i < times; i++) { digitalWrite(_p, HIGH); delay(ms); digitalWrite(_p, LOW);
delay(ms); }
  }
 private:
  int _p;
};
Led led(9);
void setup() { led.begin(); led.blink(2, 250); }
void loop() {}
```

SIMULATED RUN

```
[     0 ms] pin 9 -> HIGH
[   250 ms] pin 9 -> LOW
[   500 ms] pin 9 -> HIGH
[   750 ms] pin 9 -> LOW
```

Q10 **Answer: (a) about 4 µs versus about 0.06 µs**

Port access is roughly **60× faster**.

Q11 **Answer: (d) toggles pin 13**

XOR flips the bit: **toggle**.

Q12 **Answer: Solution program**

Each case checks only its own exit condition; the switch keeps the logic readable.

```
enum Door { CLOSED, OPENING, OPEN, CLOSING };
const char *N[] = {"CLOSED", "OPENING", "OPEN", "CLOSING"};
Door d = OPENING;
unsigned long t = 0;
void go(Door n) { d = n; t = millis(); Serial.print(millis()); Serial.print(" ms ");
Serial.println(N[n]); }
void setup() { Serial.begin(9600); Serial.println("0 ms OPENING"); }
void loop() {
  switch (d) {
    case OPENING: if (millis() - t >= 2000) go(OPEN); break;
    case OPEN:    if (millis() - t >= 3000) go(CLOSING); break;
    case CLOSING: if (millis() - t >= 2000) go(CLOSED); break;
    case CLOSED:  break;
  }
}
```

SERIAL MONITOR

```
0 ms OPENING
2000 ms OPEN
5000 ms CLOSING
7000 ms CLOSED
```

Q13 **Answer: (b) the ATmega328P has no floating-point hardware, so it is done in software**

Software floating point takes **many cycles**.

Q14 **Answer: (b) too little SRAM left for the stack and heap; random crashes**

Only ~200 bytes remain: **instability**.

Q15 **Answer: Solution program**

Handle the sign first, then split into whole and tenth parts with / and %.

```
void printTenths(int t) {
  if (t < 0) { Serial.print('-'); t = -t; }
  Serial.print(t / 10); Serial.print('.'); Serial.print(t % 10); Serial.println(" C");
}
void setup() {
  Serial.begin(9600);
  printTenths(245);
  printTenths(-37);
}
void loop() {}
```

SERIAL MONITOR

```
24.5 C
-3.7 C
```

Q16 **Answer: (d) to keep a large constant lookup table out of the Uno's 2 KB SRAM**

Large constant tables belong in **flash**.

Q17 **Answer: (a) pauses that task only, letting other tasks run**

It **blocks only the calling task**.

Q18 **Answer: Solution program**

Changing several bits in one write switches the pins simultaneously, which digitalWrite() calls cannot do.

```
#include <avr/io.h>
void setup() {
  Serial.begin(9600);
  DDRB |= 0b00000111;     // pins 8, 9, 10 as outputs
  PORTB |= 0b00000111;    // all three HIGH in one instruction
  Serial.println(PORTB, BIN);
}
void loop() {}
```

SERIAL MONITOR

```
111
```

Q19 **Answer: (c) to avoid copying it (saving time and heap memory)**

Pass by **const reference** avoids copies.

Q20 **Answer: (d) a 10-second timeout**

The **timeout** refunds and returns to IDLE.

Q21 **Answer: Solution program**

With pre-emptive tasks, each runs at its own period regardless of the other; the counts are simply the duration divided by each period.

```
void setup() {
  Serial.begin(9600);
  int blinks = 10000 / 500, reads = 10000 / 2000;
  Serial.print("blink task runs "); Serial.print(blinks); Serial.println(" times");
  Serial.print("sensor task runs "); Serial.print(reads); Serial.println(" times");
}
void loop() {}
```

SERIAL MONITOR

```
blink task runs 20 times
sensor task runs 5 times
```

Q22 **Answer: (b) MOISTURE_THRESHOLD**

Descriptive, uppercase constant names.

Q23 **Answer: (a) Git**

Git keeps every committed version.

Q24 **Answer: Solution program**

Two objects, two independent states. Simulated: A twice, B once.

```
class Button {
 public:
  Button(int p) : _p(p) {}
  void begin() { pinMode(_p, INPUT_PULLUP); }
  bool pressed() { int r = digitalRead(_p); bool e = (_l == HIGH && r == LOW); _l = r; return e; }
 private:
  int _p, _l = HIGH;
};
Button a(2), b(3);
int na = 0, nb = 0;
void setup() { Serial.begin(9600); a.begin(); b.begin(); }
void loop() {
  if (a.pressed()) { na++; Serial.print("A "); Serial.println(na); }
  if (b.pressed()) { nb++; Serial.print("B "); Serial.println(nb); }
  delay(10);
}
```

SERIAL MONITOR

```
A 1
A 2
B 1
```

Q25 **Answer: (c) it is not portable between boards and is harder to read**

Registers are **board-specific**.

Model components as classes (constructor, private state, public methods) and behaviour as state machines (enum + switch, transitions on events and timeouts). Save SRAM with F(), PROGMEM, fixed buffers and const references. Speed up hot spots with port registers and integer maths. Use FreeRTOS tasks on ESP32 for true multitasking. Write clean, modular code and commit it to Git.

MODULE 25 · ADVANCED

Beyond the Arduino IDE

Minimum duration **2 days** | Level **Advanced**

Professional embedded developers use richer tools and many chip families. This module covers PlatformIO in VS Code (projects, environments, platformio.ini and library management), hardware debugging with SWD/JTAG and breakpoints on the Uno R4 and ESP32, the ESP32 in depth, the Raspberry Pi Pico (RP2040 and RP2350), STM32 boards, MicroPython and CircuitPython, and an introduction to bare-metal programming with registers and datasheets, followed by 25 solved questions and programs.

IN THIS MODULE

25.1 PlatformIO
25.2 Debugging
25.3 Other microcontroller families
25.4 MicroPython and CircuitPython (overview)
25.5 Introduction to bare-metal programming (overview)
25.6 Practice questions
25.7 Solutions

25.1 PlatformIO

25.1.1 Installing in VS Code

PlatformIO is an extension for Visual Studio Code that supports over 1000 boards. Install VS Code, open the Extensions panel, search "PlatformIO IDE" and install; its home screen creates projects for any board. You get a professional editor: IntelliSense, refactoring, Git integration, and a unit-testing framework.

25.1.2 Projects, environments and platformio.ini

PLATFORMIO.INI WITH TWO ENVIRONMENTS

```
; platformio.ini
[env:uno]
platform = atmelavr
board = uno
framework = arduino
monitor_speed = 9600
lib_deps =
    adafruit/DHT sensor library@^1.4.6
    marcoschwartz/LiquidCrystal_I2C@^1.1.4

[env:esp32]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
build_flags = -D USE_WIFI
```

Code lives in **src/main.cpp** (a .cpp file, so add `#include <Arduino.h>` and declare functions before use). Each [env:...] section builds the same code for a different board; build_flags let code adapt with #ifdef USE_WIFI.

25.1.3 Library management

Libraries listed in lib_deps (with version ranges such as @^1.4.6) are downloaded automatically for each project, so every computer builds with exactly the same versions: no more "works on my machine" problems.

	Arduino IDE 2	PlatformIO
Editor	simple, beginner-friendly	VS Code: advanced editing, Git, extensions
Libraries	installed globally	per project, pinned versions
Multiple boards	switch manually	several environments in one project
Testing / CI	limited	unit tests, command line, CI builds
Best for	learning, quick sketches	larger and team projects

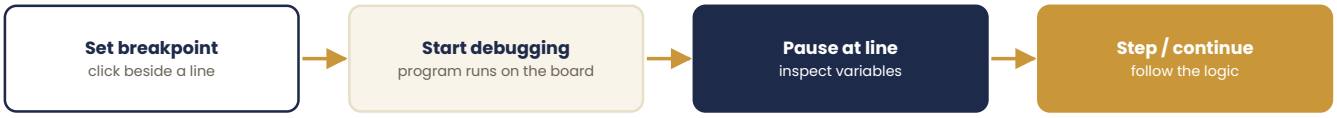
25.2 Debugging

25.2.1 Hardware debuggers (SWD/JTAG) and breakpoints

A hardware debugger connects to the chip's debug port (SWD on Arm chips, JTAG on ESP32) and lets you pause the program at a **breakpoint**, step line by line, inspect and change variables, and view registers and memory, all while the real hardware runs. Probes: J-Link, ST-Link, CMSIS-DAP, ESP-Prog.

25.2.2 Debugging on Uno R4 and ESP32

Arduino IDE 2 has a built-in debugger for supported boards (Uno R4 via its debug pins and a CMSIS-DAP probe, Nano ESP32 and ESP32-S3 via their built-in USB-JTAG). PlatformIO supports debugging on most Arm and ESP32 boards. The classic Uno R3 (AVR) has no easy hardware debugging, so Serial printing remains the main tool there.



A HARDWARE-DEBUGGING SESSION

25.3 Other microcontroller families

25.3.1 ESP32 in depth

Feature	ESP32 (classic)
CPU	dual-core Xtensa LX6, up to 240 MHz (core 0 often runs Wi-Fi)

Feature	ESP32 (classic)
Memory	520 KB SRAM, 4 MB+ external flash
Wireless	Wi-Fi 802.11 b/g/n, Bluetooth Classic and BLE
Peripherals	18 ADC channels (12-bit), 2 DACs, 10 touch pins, hall sensor, I2S, CAN, LEDC PWM on most pins
Variants	ESP32-S3 (AI/USB), ESP32-C3/C6 (RISC-V, low cost; C6 adds Wi-Fi 6 and Thread)

25.3.2 Raspberry Pi Pico (RP2040 and RP2350)

The Pico uses Raspberry Pi's own chips: RP2040 (dual Arm Cortex-M0+ at 133 MHz, 264 KB SRAM) and RP2350 in the Pico 2 (dual Cortex-M33 or RISC-V at 150 MHz, 520 KB SRAM). Unique PIO (programmable I/O) blocks implement custom protocols in hardware. Program it with the Arduino IDE (Earle Philhower core), the C/C++ SDK or MicroPython. The Pico W adds Wi-Fi and Bluetooth.

25.3.3 STM32 boards

STMicroelectronics' STM32 family (Arm Cortex-M0 to M7) is widely used in industry. The cheap "Blue Pill" (STM32F103) and "Black Pill" (STM32F401/F411) and official Nucleo boards can be programmed from the Arduino IDE (STM32duino), PlatformIO or STM32CubeIDE, with full hardware debugging via ST-Link.

25.4 MicroPython and CircuitPython (overview)

MicroPython and Adafruit's CircuitPython run Python directly on microcontrollers (ESP32, Pico, many others). You edit a file such as main.py and it runs immediately, with an interactive prompt (REPL) for experiments. They are slower and use more memory than C++, but are excellent for learning and quick prototypes.

MAIN.PY (MICROPYTHON)

```
# MicroPython on a Raspberry Pi Pico: blink the on-board LED
from machine import Pin
import time
led = Pin("LED", Pin.OUT)
while True:
    led.toggle()
    time.sleep(0.5)
```

25.5 Introduction to bare-metal programming (overview)

25.5.1 Registers and datasheets

Every peripheral is controlled by registers: memory locations whose bits switch features on and off. The datasheet (the ATmega328P datasheet is about 300 pages) describes every register bit. Arduino functions such as pinMode() are wrappers that set those bits. Bare-metal code writes the registers directly: maximum speed and control, at the cost of portability.

TIMERREGISTERS.INO

```
#include <avr/io.h>
// Configure Timer1 for 2 Hz in CTC mode: show the register values a datasheet calculation gives
void setup() {
  Serial.begin(9600);
  long prescaler = 256, f = 2;
  long ocr = 16000000L / (prescaler * f) - 1;
  TCCR1A = 0;
  TCCR1B = _BV(WGM12) | _BV(CS12);     // CTC mode (WGM12), clock / 256 (CS12)
  OCR1A = ocr;
  Serial.print("TCCR1B = 0b"); Serial.println(TCCR1B, BIN);
  Serial.print("OCR1A  = ");   Serial.println(OCR1A);
}
void loop() {}
```

SERIAL MONITOR

```
TCCR1B = 0b1100
OCR1A  = 31249
```

25.6 Practice questions

25.6.1 Basic level

Q1 PlatformIO runs as an extension of which editor?

(a) Visual Studio Code **(b)** Notepad
(c) Microsoft Word **(d)** Arduino IDE 1.8

Q2 Which file configures a PlatformIO project?

(a) library.properties **(b)** platformio.ini
(c) Makefile only **(d)** sketch.ino

Q3 In PlatformIO, where is the main code placed?

(a) main.ino in the root **(b)** lib/main.h
(c) platformio.ini **(d)** src/main.cpp

Q4 What does a breakpoint do?

(a) breaks the board
(b) stops Wi-Fi
(c) pauses the program at a chosen line so you can inspect it
(d) deletes a variable

Q5 Which debug interface is used by Arm Cortex-M chips?

(a) I2C **(b)** UART **(c)** SWD **(d)** PWM

Q6 Which chip is on the original Raspberry Pi Pico?

(a) ATmega328P **(b)** ESP32 **(c)** STM32F103 **(d)** RP2040

Q7 MicroPython lets you program microcontrollers in

(a) Java **(b)** Python **(c)** C# **(d)** Scratch only

Q8 What is a register?

(a) a memory location whose bits control a hardware feature
(b) a type of resistor
(c) a library
(d) a sensor

25.6.2 Intermediate level

Q9 Why must a PlatformIO main.cpp include and declare functions before use?

(a) PlatformIO uses Python
(b) it is a normal C++ file; the Arduino IDE adds these automatically only for .ino files
(c) Arduino.h adds Wi-Fi
(d) it is optional

Q10 What advantage does lib_deps in platformio.ini give?

(a) each project gets exactly the listed library versions automatically

(b) libraries load faster on the board
(c) it removes the need for #include
(d) it compresses the firmware

Q11 Which board can be debugged with hardware breakpoints using its built-in USB-JTAG?

(a) Uno R3
(b) Nano (ATmega328P)
(c) Nano ESP32 / ESP32-S3
(d) Mega 2560 without a probe

Q12 Which feature is unique to the RP2040/RP2350?

(a) Wi-Fi on every board
(b) 5 V logic
(c) a built-in OLED
(d) PIO (programmable I/O) state machines

Q13 Which ESP32 variants use RISC-V cores?

(a) ESP32 classic
(b) ESP8266
(c) ESP32-S3
(d) ESP32-C3 and ESP32-C6

Q14 Compute OCR1A for a 5 Hz Timer1 interrupt with prescaler 64 and check that it fits in 16 bits.

Q15 What is the main trade-off of MicroPython versus C++?

(a) faster and smaller
(b) only runs on PCs
(c) easier and interactive, but slower and uses more memory
(d) cannot use GPIO

Q16 What is the "Blue Pill"?

(a) a cheap STM32F103 development board
(b) an ESP32 module
(c) a LiPo battery
(d) a debugging cable

25.6.3 Advanced level

Q17 Set bits to configure a register: start with 0, set bits 3 and 2 (WGM12 = 3, CS12 = 2) and print the value in binary and decimal.

Q18 How much SRAM does the classic ESP32 have?

(a) 2 KB
(b) 32 KB
(c) 8 MB
(d) 520 KB

Q19 Two [env:...] sections exist in platformio.ini. What does this allow?

(a) running two sketches at once on one board
(b) building the same project for two different boards
(c) using two USB cables
(d) nothing

Q20 Which tool is ST's official IDE for STM32?

(a) Thonny
(b) STM32CubeIDE
(c) Tinkercad
(d) Fritzing

Q21 Show what pinMode(13, OUTPUT) and digitalWrite(13, HIGH) do at register level on an Uno.

Q22 Why is Serial printing still the main debugging method on the Uno R3?

(a) the Uno has no USB

(b) printing is faster than breakpoints
(c) breakpoints damage AVR chips
(d) the AVR chip has no simple hardware debugging support in the IDE

Q23 Which microcontroller family is most common in industrial products among those listed?

(a) STM32 (b) ATtiny85 (c) Arduino Uno (d) BBC micro:bit

Q24 What file does MicroPython run automatically at start-up?

(a) setup.ino (b) boot.cpp (c) main.py (d) index.html

Q25 Using a PlatformIO-style build flag, compile different code for Wi-Fi and non-Wi-Fi builds (here USE_WIFI is defined).

25.7 Solutions

Q1 **Answer: (a) Visual Studio Code**

VS Code.

Q2 **Answer: (b) platformio.ini**

platformio.ini.

Q3 **Answer: (d) src/main.cpp**

src/main.cpp.

Q4 **Answer: (c) pauses the program at a chosen line so you can inspect it**

It **pauses execution**.

Q5 **Answer: (c) SWD**

SWD (serial wire debug).

Q6 **Answer: (d) RP2040**

RP2040.

Q7 **Answer: (b) Python**

Python.

Q8 **Answer: (a) a memory location whose bits control a hardware feature**

Registers **control peripherals**.

Q9 **Answer: (b) it is a normal C++ file; the Arduino IDE adds these automatically only for .ino files**

.ino preprocessing is **not applied** to .cpp files.

Q10 **Answer: (a) each project gets exactly the listed library versions automatically**

Reproducible builds.

Q11 **Answer: (c) Nano ESP32 / ESP32-S3**

ESP32-S3 has **built-in USB-JTAG**.

Q12 **Answer: (d) PIO (programmable I/O) state machines**

PIO blocks implement custom protocols.

Q13 **Answer: (d) ESP32-C3 and ESP32-C6**

The **C3 and C6**.

Q14 **Answer: Solution program**

16 000 000 / 320 - 1 = 49 999, which is below 65 535.

```
void setup() {
  Serial.begin(9600);
  long ocr = 16000000L / (64L * 5) - 1;
  Serial.print("OCR1A = "); Serial.println(ocr);
  Serial.println(ocr <= 65535 ? "fits in 16 bits" : "too big: use a larger prescaler");
}
void loop() {}
```

SERIAL MONITOR

```
OCR1A = 49999
fits in 16 bits
```

Q15 **Answer: (c) easier and interactive, but slower and uses more memory**

Convenience vs **speed and memory**.

Q16 **Answer: (a) a cheap STM32F103 development board**

A low-cost **STM32** board.

Q17 **Answer: Solution program**

Bits 3 and 2 give 0b1100 = 12. Datasheets list each register bit by name and position.

```
void setup() {
  Serial.begin(9600);
  byte reg = 0;
  reg |= (1 << 3) | (1 << 2);
  Serial.print("0b"); Serial.print(reg, BIN); Serial.print(" = "); Serial.println(reg);
}
void loop() {}
```

SERIAL MONITOR

```
0b1100 = 12
```

Q18 **Answer: (d) 520 KB**

520 KB.

Q19 **Answer: (b) building the same project for two different boards**

Multiple **environments**, one per target.

Q20 **Answer: (b) STM32CubeIDE**

STM32CubeIDE.

Q21 **Answer: Solution program**

Pin 13 is bit 5 of port B, so both functions just set bit 5 in the relevant register.

```
#include <avr/io.h>
void setup() {
  Serial.begin(9600);
  DDRB |= (1 << 5);        // pinMode(13, OUTPUT): set bit 5 of the direction register
  PORTB |= (1 << 5);       // digitalWrite(13, HIGH): set bit 5 of the output register
  Serial.print("DDRB = 0b"); Serial.println(DDRB, BIN);
  Serial.print("PORTB = 0b"); Serial.println(PORTB, BIN);
}
void loop() {}
```

SERIAL MONITOR

```
DDRB = 0b100000
PORTB = 0b100000
```

Q22 **Answer: (d) the AVR chip has no simple hardware debugging support in the IDE**

No easy **hardware debug** on AVR.

Q23 **Answer: (a) STM32**

The **STM32** family is an industry standard.

Q24 **Answer: (c) main.py**

main.py (after boot.py).

Q25 **Answer: Solution program**

In PlatformIO the define would come from build_flags = -D USE_WIFI in one environment, so the same source serves several products.

```
#define USE_WIFI 1
void setup() {
  Serial.begin(9600);
#ifdef USE_WIFI
  Serial.println("build with Wi-Fi support");
#else
  Serial.println("build without Wi-Fi");
#endif
}
void loop() {}
```

SERIAL MONITOR

```
build with Wi-Fi support
```

PlatformIO (VS Code): src/main.cpp, platformio.ini environments, lib_deps with pinned versions. Hardware debuggers (SWD/JTAG) give breakpoints and variable inspection on Uno R4, ESP32-S3 and Arm boards. ESP32, RP2040/RP2350 (PIO) and STM32 extend your options; MicroPython/CircuitPython trade speed for convenience. Bare-metal code writes the registers described in the datasheet.

MODULE 26 · ADVANCED

From prototype to product: PCB design and enclosures

Minimum duration **2 days** | Level **Advanced**

A breadboard proves an idea; a product needs a reliable board and a case. This module covers perfboard and stripboard soldering, designing PCBs in KiCad or EasyEDA (schematic capture, footprints, layout, traces, ground planes, design rules, Gerber files and ordering), building a standalone ATmega328P circuit and burning its bootloader, 3D-printed and laser-cut enclosures with proper connectors, mounting and labels, and reliability testing, followed by 25 solved questions.

IN THIS MODULE

26.1 Moving off the breadboard
26.2 PCB design
26.3 Using a bare microcontroller
26.4 Enclosures
26.5 Reliability and testing
26.6 Practice questions
26.7 Solutions

26.1 Moving off the breadboard

26.1.1 Perfboard and stripboard soldering

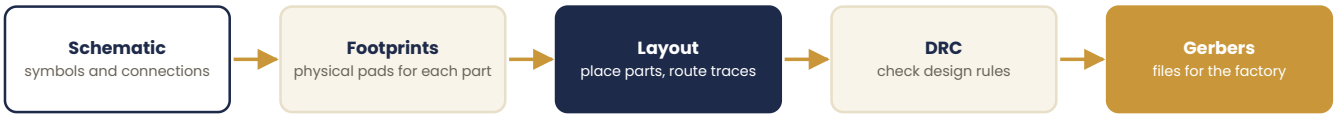
Breadboard contacts loosen and wires fall out. For a permanent prototype, solder components onto **perfboard** (isolated pads, connected with wire) or **stripboard** (Veroboard: copper strips you cut where needed). Use headers so the Arduino Nano and modules can be unplugged.

Soldering step	Tip
Heat	iron at about 330–350 °C; touch pad and lead together for 1–2 s
Solder	feed solder to the joint, not the iron; use rosin-core solder
Good joint	shiny, concave cone ("volcano"), wets both pad and lead
Bad joint	dull ball sitting on the pad (cold joint) or bridges between pads
Safety	ventilation for fumes, safety glasses, wash hands (especially with leaded solder)

26.2 PCB design

26.2.1 KiCad or EasyEDA

KiCad is free, open-source and professional; **EasyEDA** runs in a browser and links directly to JLCPCB ordering and the LCSC parts catalogue. Both follow the same workflow.



THE PCB DESIGN WORKFLOW

26.2.2 Schematic capture

Draw the circuit with symbols from the libraries, connect them with wires or net labels (two wires with the same label are connected), add power symbols, and run the Electrical Rules Check (ERC) to find unconnected pins and outputs driving outputs.

26.2.3 PCB layout, footprints, traces and ground planes

Each symbol is assigned a footprint (the copper pads matching the physical part: through-hole or surface-mount). On the board, place components logically (connectors at edges, decoupling capacitors close to IC power pins), then route traces. A **ground plane** (a copper pour connected to GND) lowers noise and simplifies routing.

Guideline	Typical value
Signal trace width	0.25 mm (10 mil)
Power trace (1 A, 1 oz copper, outer layer)	about 0.5–0.75 mm
Clearance between copper	≥ 0.2 mm
Via drill / pad	0.3 mm / 0.6 mm
Decoupling capacitor	0.1 µF within a few mm of each IC power pin

26.2.4 Design rules and Gerber files

The Design Rules Check (DRC) compares the layout with the manufacturer's capabilities (minimum trace width, clearance, drill size). Gerber files describe each layer (copper, solder mask, silkscreen, outline); an Excellon file lists the drill holes. Zip them for the factory.

26.2.5 Ordering PCBs from manufacturers

Upload the Gerber ZIP to a manufacturer (JLCPCB, PCBWay, and Indian services such as Lion Circuits). A typical order is 5 two-layer boards, 1.6 mm FR-4, 1 oz copper, HASL finish, at a low cost, delivered in one to two weeks. Many also offer assembly (PCBA) of surface-mount parts from a bill of materials (BOM) and placement (CPL) file.

26.3 Using a bare microcontroller

26.3.1 Standalone ATmega328P circuit

A product does not need a whole Uno: the ATmega328P chip needs only a 5 V supply, a 16 MHz crystal with two 22 pF capacitors, a 10 kΩ pull-up on RESET, and 0.1 µF decoupling capacitors on VCC and AVCC. With the internal 8 MHz oscillator even the crystal can be omitted.

Part	Connection
16 MHz crystal + 2 × 22 pF	pins 9 and 10 (XTAL1, XTAL2), capacitors to GND
10 kΩ resistor	RESET (pin 1) to VCC
0.1 µF capacitors	VCC (7) and AVCC (20) to GND, close to the chip
Power	VCC (7), AVCC (20) to 5 V; GND (8, 22)
Programming header	TX, RX, DTR (via 0.1 µF to RESET) for a USB-serial adapter, or ICSP (MOSI, MISO, SCK, RESET)

26.3.2 Burning the bootloader

A blank chip has no bootloader, so it cannot be programmed over serial. Use an Uno running the "ArduinoISP" example (or a USBasp programmer) wired to the chip's ICSP pins, select the board and "Arduino as ISP", then Tools → Burn Bootloader. Afterwards sketches upload through a USB-serial adapter as usual, or can be uploaded directly with "Upload Using Programmer" (no bootloader needed).

26.4 Enclosures

26.4.1 3D-printed and laser-cut enclosures

Design a case in CAD (Fusion 360, Onshape, FreeCAD, Tinkercad) from the PCB dimensions (export a STEP model from KiCad). 3D-print in PLA (indoor) or PETG/ASA (heat and outdoor UV); laser-cut acrylic or plywood for box shapes. Leave clearance for connectors, ventilation slots for regulators, and screw bosses for mounting.

26.4.2 Connectors, mounting and labelling

Use panel-mount connectors (DC barrel jack, USB-C breakout, screw terminals, JST connectors for batteries) instead of loose wires; strain-relieve cables; mount the PCB on standoffs; label connectors, switches and polarity on the case. For outdoor use choose an IP65 box with cable glands.

26.5 Reliability and testing

26.5.1 Testing checklists and long-term operation

Test	What to check
Power-on	correct voltages at test points; no excessive current draw
Function	every input, output, sensor and screen against the specification
Edge cases	sensor disconnected, Wi-Fi lost, power dip, wrong user input
Endurance (burn-in)	run for days; watch for crashes, memory leaks, overheating
Environment	temperature, humidity, vibration, sunlight
Recovery	watchdog resets cleanly; settings survive power loss

A product is reliable when it recovers from every failure you can imagine: use a watchdog, validate every input, reconnect networks automatically, protect power inputs (fuse, reverse-polarity diode) and test for days, not minutes.

26.6 Practice questions

26.6.1 Basic level

Q1 Which board has copper strips that you cut to make circuits?

(a) breadboard
(b) perfboard with isolated pads
(c) PCB with ground plane
(d) stripboard (Veroboard)

Q2 What does a good solder joint look like?

(a) a dull ball on the pad
(b) shiny and cone-shaped, wetting both pad and lead
(c) a bridge between two pads
(d) black and burnt

Q3 Which free, open-source program is widely used for PCB design?

(a) KiCad **(b)** Photoshop **(c)** Excel **(d)** Blender only

Q4 What is a footprint in PCB design?

(a) a schematic symbol
(b) the case design
(c) the copper pad pattern that matches a physical component
(d) the Gerber file

Q5 Which files are sent to a PCB manufacturer?

(a) Gerber files (and the drill file)
(b) .ino files
(c) .stl files
(d) the BOM only

Q6 What frequency crystal does a standard Uno-compatible ATmega328P use?

(a) 8 kHz **(b)** 16 MHz **(c)** 32.768 kHz **(d)** 240 MHz

Q7 Why is a ground plane used on a PCB?

(a) to make the board heavier
(b) to replace all traces
(c) to stop soldering
(d) to reduce noise and simplify routing of ground connections

Q8 Which material is best for a 3D-printed enclosure used outdoors in sunlight?

(a) PLA **(b)** paper **(c)** ASA or PETG **(d)** chocolate

26.6.2 Intermediate level

Q9 What does DRC stand for?

(a) digital resistor code
(b) direct register control
(c) design rules check
(d) device reset circuit

Q10 Where should a 0.1 µF decoupling capacitor be placed?

(a) anywhere on the board

(b) at the battery connector only
(c) inside the enclosure lid
(d) as close as possible to the IC power pin

Q11 Which pull-up resistor is connected to the ATmega328P RESET pin?

(a) 220 Ω to GND (b) 10 kΩ to VCC (c) 1 MΩ to GND (d) none

Q12 What value capacitors are used with a 16 MHz crystal?

(a) two 22 pF (b) two 100 µF (c) one 0.1 µF (d) two 10 nF

Q13 Why must a blank ATmega328P have a bootloader burned before uploading over serial?

(a) to set the clock to 8 MHz
(b) the bootloader is the small program that receives sketches over serial
(c) to erase EEPROM
(d) to enable Wi-Fi

Q14 Which example sketch turns an Uno into a programmer for burning bootloaders?

(a) ArduinoISP (b) Blink
(c) AnalogReadSerial (d) Knob

Q15 What does the ERC check in a schematic?

(a) the board size
(b) the enclosure fit
(c) electrical mistakes such as unconnected pins or two outputs tied together
(d) solder temperature

Q16 Two wires in a schematic carry the same net label. What does that mean?

(a) they must cross (b) they are on different boards
(c) nothing (d) they are electrically connected

26.6.3 Advanced level

Q17 Which feature protects a product from a reversed battery?

(a) a bigger LED
(b) a 74HC595
(c) a ground plane
(d) a reverse-polarity protection diode or MOSFET

Q18 A product must run unattended for months. Which firmware feature is most important for recovering from rare hangs?

(a) a faster baud rate (b) more comments
(c) a watchdog timer (d) larger fonts

Q19 Which file pair lets a manufacturer assemble surface-mount parts?

(a) BOM (bill of materials) and CPL (placement) file
(b) Gerber and STL
(c) INO and HEX
(d) PDF and PNG

Q20 An outdoor sensor enclosure should be rated at least

(a) IP00 **(b)** IP20
(c) no rating needed **(d)** IP65

Q21 What is "burn-in" testing?

(a) soldering at high temperature
(b) running the device continuously for a long period to find failures
(c) testing the bootloader
(d) heating the PCB to melt solder

Q22 Which is typically the minimum signal trace width offered by low-cost PCB services?

(a) about 5 mm **(b)** about 0.15–0.25 mm
(c) about 0.001 mm **(d)** about 2 cm

Q23 Why should power traces be wider than signal traces?

(a) to look better
(b) to reduce cost
(c) signal traces need more copper
(d) they carry more current; wider traces have lower resistance and heat less

Q24 With the internal 8 MHz oscillator, which external parts can a standalone ATmega328P omit?

(a) the crystal and its two 22 pF capacitors
(b) the decoupling capacitors
(c) the power supply
(d) the GND connection

Q25 What should be done before connecting a newly assembled PCB to a costly module?

(a) nothing; plug everything in
(b) paint the board
(c) check supply voltages and current draw first at power-on
(d) remove the decoupling capacitors

26.7 Solutions

Q1 **Answer: (d) stripboard (Veroboard)**

Stripboard.

Q2 **Answer: (b) shiny and cone-shaped, wetting both pad and lead**

A shiny **volcano** shape.

Q3 **Answer: (a) KiCad**

KiCad.

Q4 **Answer: (c) the copper pad pattern that matches a physical component**

The **physical pads**.

Q5 **Answer: (a) Gerber files (and the drill file)**

Gerbers.

Q6 **Answer: (b) 16 MHz**

16 MHz.

Q7 **Answer: (d) to reduce noise and simplify routing of ground connections**

Lower **noise**, easier routing.

Q8 **Answer: (c) ASA or PETG**

PLA softens in heat; **ASA/PETG** resist heat and UV.

Q9 **Answer: (c) design rules check**

Design Rules Check.

Q10 **Answer: (d) as close as possible to the IC power pin**

Close to the **power pin**.

Q11 **Answer: (b) 10 kΩ to VCC**

10 kΩ to VCC.

Q12 **Answer: (a) two 22 pF**

Two **22 pF** load capacitors.

Q13 **Answer: (b) the bootloader is the small program that receives sketches over serial**

Without it, serial upload has **nothing to talk to**.

Q14 **Answer: (a) ArduinoISP**

ArduinoISP.

Q15 **Answer: (c) electrical mistakes such as unconnected pins or two outputs tied together**

Electrical Rules Check.

Q16 **Answer: (d) they are electrically connected**

Net labels **connect** nets.

Q17 **Answer: (d) a reverse-polarity protection diode or MOSFET**

A **reverse-polarity** diode/MOSFET.

Q18 **Answer: (c) a watchdog timer**

The **watchdog** restarts a hung system.

Q19 **Answer: (a) BOM (bill of materials) and CPL (placement) file**

BOM + CPL.

Q20 **Answer: (d) IP65**

IP65: dust-tight and protected against water jets.

Q21 **Answer: (b) running the device continuously for a long period to find failures**

Long-duration **endurance** testing.

Q22 **Answer: (b) about 0.15–0.25 mm**

6–10 mil (0.15–0.25 mm).

Q23 **Answer: (d) they carry more current; wider traces have lower resistance and heat less**

Current capacity scales with **width**.

Q24 **Answer: (a) the crystal and its two 22 pF capacitors**

The internal oscillator replaces the **crystal**.

Q25 **Answer: (c) check supply voltages and current draw first at power-on**

Power-on checks prevent **damaging** other parts.

Move from breadboard to perfboard/stripboard, then to a PCB: schematic (ERC) → footprints → layout with ground plane and close decoupling → DRC → Gerbers → order. A standalone ATmega328P needs 5 V, decoupling, 10 kΩ RESET pull-up and a 16 MHz crystal with 22 pF capacitors; burn the bootloader with ArduinoISP. Design robust enclosures and test power-on, functions, edge cases and endurance.

MODULE 27 · ADVANCED

Projects and careers

Minimum duration **4 days** | Level **Advanced**

The final module brings everything together. It presents seven capstone projects (smart home, IoT weather station, robot car, RFID attendance, smart irrigation, health monitor and gesture-controlled arm) with block diagrams, components and design notes, explains how to document projects with circuit diagrams, GitHub READMEs and demo videos, lists competitions and hackathons, describes embedded and IoT careers and the next steps after this course, and ends with a 25-question mixed final test covering the whole book.

IN THIS MODULE

27.1 Capstone projects
27.2 Documenting projects
27.3 Competitions and hackathons
27.4 Careers
27.5 Next steps
27.6 Final test
27.7 Solutions

27.1 Capstone projects

Choose one project, write a one-page specification (what it must do, inputs, outputs, power, budget), build it in stages, and test each stage before adding the next. Every project below uses only techniques from this book.

27.1.1 Smart home automation system with app control



SMART HOME: MODULES 7, 13, 16 AND 22

Features: switch loads from the phone, automatic lights on motion after dark, temperature-based fan control, status dashboard. Design notes: manual override switches must still work if Wi-Fi fails; use certified relay boxes for mains.

27.1.2 IoT weather station with cloud dashboard



WEATHER STATION: MODULES 13, 22 AND 23

Design notes: radiation shield for the sensor (direct sun reads high), solar panel with Li-ion cell, deep sleep between readings, retry and buffer readings when Wi-Fi is down.

27.1.3 Obstacle-avoiding and line-following robot car

OBSTACLEAVOIDER.INO

```
// Obstacle avoidance core logic (robot car, Modules 13 and 16)
const int TRIG = 9, ECHO = 10;
const int ENA = 5, IN1 = 7, IN2 = 8, ENB = 6, IN3 = 11, IN4 = 12;

void drive(int l, int r) {
  digitalWrite(IN1, l > 0); digitalWrite(IN2, l < 0); analogWrite(ENA, abs(l));
  digitalWrite(IN3, r > 0); digitalWrite(IN4, r < 0); analogWrite(ENB, abs(r));
}

long distanceCm() {
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  long us = pulseIn(ECHO, HIGH, 30000);
  return us ? us / 58 : 999;
}

void setup() {
  Serial.begin(9600);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
  for (int p : {IN1, IN2, IN3, IN4}) pinMode(p, OUTPUT);
}

void loop() {
  long d = distanceCm();
  if (d < 20) { Serial.print(d); Serial.println(" cm: obstacle -> turn"); drive(-150, 150);
delay(300); }
  else        { Serial.print(d); Serial.println(" cm: clear -> forward"); drive(180, 180); }
  delay(100);
}
```

SERIAL MONITOR

```
60 cm: clear -> forward
30 cm: clear -> forward
15 cm: obstacle -> turn
50 cm: clear -> forward
```

A line follower adds two or more IR reflectance sensors under the front: if the left sensor sees the line, steer left; if the right sees it, steer right. A PID controller (proportional–integral–derivative) on the line position gives smooth, fast tracking.

27.1.4 RFID attendance system with data logging

RC522 reader (SPI) + RTC + SD card + LCD: each card UID is looked up in a table of names; a time-stamped row "date,time,UID,name" is appended to ATTEND.CSV, and the LCD greets the person. Extensions: Wi-Fi upload to a Google Sheet, buzzer feedback, duplicate-scan filtering.

27.1.5 Smart agriculture: automatic irrigation with soil sensors

IRRIGATION.INO

```
// Irrigation controller: water when the soil is dry, with a maximum pump time (Modules 8, 12, 16)
const int SOIL = A0, PUMP = 7;
const int DRY = 600, WET = 450;            // calibrated readings (higher = drier)
const unsigned long MAX_ON = 20000;        // safety limit: 20 s
bool pumping = false;
unsigned long startedAt = 0;

void setup() {
  Serial.begin(9600);
  pinMode(PUMP, OUTPUT);
}

void loop() {
  int s = analogRead(SOIL);
  if (!pumping && s > DRY) { pumping = true; startedAt = millis(); digitalWrite(PUMP, HIGH);
Serial.print(s); Serial.println(" dry -> pump ON"); }
  if (pumping && (s < WET || millis() - startedAt > MAX_ON)) {
    pumping = false; digitalWrite(PUMP, LOW);
    Serial.print(s); Serial.println(millis() - startedAt > MAX_ON ? " timeout -> pump OFF (check
water!)" : " wet -> pump OFF");
  }
  delay(1000);
}
```

SERIAL MONITOR

```
640 dry -> pump ON
440 wet -> pump OFF
```

Two thresholds (hysteresis: start above 600, stop below 450) prevent the pump switching on and off rapidly; the time limit protects against a dry tank or a broken sensor.

27.1.6 Health monitor with pulse sensor and OLED

A MAX30102 (heart rate and SpO_2) or analog pulse sensor with an SSD1306 OLED showing BPM and a live waveform, an alert buzzer, and optional BLE to a phone. Clearly label it as an educational device, not a medical instrument.

27.1.7 Gesture-controlled robotic arm

An MPU6050 on a glove measures hand tilt; the controller maps pitch and roll to servo angles and sends them over nRF24L01 or Bluetooth to an arm with 4 servos driven by a PCA9685. A complementary filter smooths the angles; a flex sensor on a finger opens and closes the gripper.

27.2 Documenting projects

27.2.1 Circuit diagrams with Fritzing or KiCad

Draw a clean schematic (KiCad) and, for beginners, a breadboard view (Fritzing or Wokwi). Label every pin and component value.

27.2.2 Code on GitHub with a README

README section	Content
Title and photo	what it is, in one sentence, with a picture
Features	bullet list of what it does
Hardware	bill of materials with part numbers
Wiring	schematic image and pin table
Software	libraries and versions, how to build and upload
Usage and results	how to operate it; graphs or screenshots
Future work and licence	ideas and terms of use

27.2.3 Demo videos and project write-ups

A 1–2 minute video showing the problem, the device working and one technical highlight is the best portfolio item. Publish a write-up on Hackster.io, Instructables or a blog, and link it in your résumé and LinkedIn profile.

27.3 Competitions and hackathons

27.3.1 Where to compete

Event	About
Smart India Hackathon	national hackathon with hardware editions solving problem statements from ministries and industry
e-Yantra (IIT Bombay)	robotics competition and labs; themes based on real problems
College tech fests	robotics races, line followers, IoT challenges
Hackster.io and other online contests	global contests sponsored by chip and board makers

27.4 Careers

27.4.1 Roles

Role	What they do	Key skills beyond this book
Embedded systems engineer	designs firmware and hardware for products	embedded C, RTOS, debugging, datasheets
Firmware engineer	writes low-level software for microcontrollers	C/C++, drivers, bootloaders, testing

Role	What they do	Key skills beyond this book
IoT developer	connects devices to the cloud and apps	networking, MQTT, cloud platforms, security
Robotics engineer	builds and programs robots	control theory, ROS, sensors, mechanics
Hardware design engineer	designs circuits and PCBs	KiCad/Altium, analog electronics, EMC
Test / validation engineer	verifies products work reliably	test automation, instruments, documentation

27.5 Next steps

27.5.1 Raspberry Pi

Learn Linux, Python and camera/AI projects on the Raspberry Pi, and combine it with Arduino boards for real-time control (the Learn Being Forward Raspberry Pi course).

27.5.2 Robotics and ROS

Robot kinematics, PID control and ROS 2 (Robot Operating System) for navigation and mapping (the Learn Being Forward Robotics Fundamentals course).

27.5.3 Embedded C and RTOS in depth

Program microcontrollers in C without the Arduino layer (registers, interrupts, DMA), learn FreeRTOS or Zephyr properly, and study communication stacks and low-power design.

27.6 Final test

27.6.1 Basic level

Q1 Which function runs repeatedly after setup()?

(a) main() **(b)** repeat() **(c)** run() **(d)** loop()

Q2 What resistor limits a red LED (2 V) to 15 mA from 5 V?

(a) 20 Ω **(b)** 200 Ω (use 220 Ω)
(c) 2 kΩ **(d)** 330 kΩ

Q3 With INPUT_PULLUP and a button to GND, a pressed button reads

(a) LOW **(b)** HIGH **(c)** 1023 **(d)** floating

Q4 What does analogRead() return for 2.5 V on an Uno R3 (5 V reference)?

(a) about 255 **(b)** about 1023 **(c)** about 512 **(d)** about 2500

Q5 analogWrite(9, 64) gives what duty cycle?

(a) about 25 % **(b)** about 64 % **(c)** about 50 % **(d)** about 6 %

Q6 Which protocol uses SDA and SCL?

(a) SPI **(b)** I2C **(c)** UART **(d)** One-Wire

Q7 What is the non-blocking timing pattern?

(a) delay(interval)
(b) while (millis() < interval) {}
(c) if (millis() == interval)
(d) if (millis() - last >= interval) { last = millis(); … }

Q8 An HC-SR04 echo of 2900 µs means a distance of about

(a) 29 cm **(b)** 100 cm **(c)** 50 cm **(d)** 5 cm

27.6.2 Intermediate level

Q9 Combine skills: read a potentiometer (A0), set LED brightness on pin 9, and print the percentage only when it changes by at least 5 %.

Q10 Which keyword must a variable shared with an ISR have?

(a) static **(b)** const **(c)** volatile **(d)** extern

Q11 How many steps per revolution does a 1.8° stepper have at 1/8 microstepping?

(a) 200 **(b)** 800 **(c)** 3200 **(d)** 1600

Q12 Which function sets a servo to 90°?

(a) analogWrite(90) **(b)** myServo.write(90)
(c) myServo.angle = 90 **(d)** tone(90)

Q13 What is the typical I2C address of an SSD1306 OLED?

(a) 0x3C **(b)** 0x27 **(c)** 0x68 **(d)** 0x76

Q14 How many write cycles does each EEPROM cell typically survive?

(a) about 100 **(b)** about 100 000 **(c)** unlimited **(d)** about 10

Q15 Which ESP32 mode creates its own Wi-Fi network?

(a) access point (softAP) **(b)** station
(c) deep sleep **(d)** Bluetooth

Q16 Implement hysteresis for a fan: switch on above 30 °C and off below 27 °C, printing changes for readings 28, 31, 29, 26, 28.

27.6.3 Advanced level

Q17 Battery life of a node averaging 2 mA from a 2000 mAh battery (80 % usable)?

(a) about 1000 days **(b)** about 4 days
(c) about 33 days **(d)** about 1 year

Q18 Which MQTT action sends a message to a topic?

(a) subscribe **(b)** connect **(c)** poll **(d)** publish

Q19 Which wireless technology reaches several kilometres at very low power?

(a) Bluetooth Classic **(b)** IR
(c) NFC **(d)** LoRa

Q20 Look up an RFID card UID in a table of names and print a greeting, or "Unknown card".

Q21 Which tool configures multiple board environments in one project?

(a) Serial Monitor **(b)** Fritzing
(c) PlatformIO (platformio.ini) **(d)** Tinkercad

Q22 What are the two parts needed with a 16 MHz crystal on a standalone ATmega328P?

(a) two 22 pF capacitors **(b)** two 10 kΩ resistors
(c) two LEDs **(d)** two diodes

Q23 Why does an irrigation controller need a maximum pump-on time?

(a) to save code space
(b) to make the pump faster
(c) it is not needed
(d) to stop the pump if the tank is empty or the sensor fails

Q24 Which document section of a README lists all the parts?

(a) licence **(b)** hardware / bill of materials
(c) title **(d)** future work

Q25 Final challenge: a non-blocking blink on pin 13 every 500 ms plus a serial command "T" that prints the uptime in seconds.

27.7 Solutions

Q1 **Answer: (d) loop()**

loop().

Q2 **Answer: (b) 200 Ω (use 220 Ω)**

(5 - 2) / 0.015 = **200 Ω**.

Q3 **Answer: (a) LOW**

LOW.

Q4 **Answer: (c) about 512**

2.5 / 5 × 1023 ≈ **512**.

Q5 **Answer: (a) about 25 %**

64 / 255 ≈ **25 %**.

Q6 **Answer: (b) I2C**

I2C.

Q7 **Answer: (d) if (millis() - last >= interval) { last = millis(); … }**

Subtract and compare.

Q8 **Answer: (c) 50 cm**

2900 / 58 = **50 cm**.

Q9 **Answer: Solution program**

Printing only significant changes keeps the output readable. Readings 0, 20, 300, 310, 1023 print 0 %, 29 % and 100 % (the small changes are ignored).

```
int lastPct = -100;
void setup() { Serial.begin(9600); }
void loop() {
  int raw = analogRead(A0);
  analogWrite(9, raw / 4);
  int pct = map(raw, 0, 1023, 0, 100);
  if (abs(pct - lastPct) >= 5) { lastPct = pct; Serial.print(pct); Serial.println(" %"); }
  delay(50);
}
```

SERIAL MONITOR

```
0 %
29 %
100 %
```

Q10 **Answer: (c) volatile**

volatile.

Q11 **Answer: (d) 1600**

200 × 8 = **1600**.

Q12 **Answer: (b) myServo.write(90)**

write(90).

Q13 **Answer: (a) 0x3C**

0x3C.

Q14 **Answer: (b) about 100 000**

≈ 100 000.

Q15 **Answer: (a) access point (softAP)**

softAP.

Q16 **Answer: Solution program**

Between 27 and 30 °C the fan keeps its current state, so it does not chatter on and off around a single threshold.

```
bool fan = false;
void check(float t) {
  if (!fan && t > 30) { fan = true; Serial.print(t, 0); Serial.println(" C: fan ON"); }
  else if (fan && t < 27) { fan = false; Serial.print(t, 0); Serial.println(" C: fan OFF"); }
}
void setup() {
  Serial.begin(9600);
  float temps[] = {28, 31, 29, 26, 28};
  for (float t : temps) check(t);
}
void loop() {}
```

SERIAL MONITOR

```
31 C: fan ON
26 C: fan OFF
```

Q17 **Answer: (c) about 33 days**

1600 / 2 = 800 h ≈ **33 days**.

Q18 **Answer: (d) publish**

publish.

Q19 **Answer: (d) LoRa**

LoRa.

Q20 **Answer: Solution program**

An array of structs acts as a small database; strcmp() compares the C strings. In the full project the UID comes from the RC522 reader.

```
struct Person { const char *uid; const char *name; };
Person people[] = {{"04A1B2C3", "Asha"}, {"049F8E7D", "Ravi"}};
void greet(const char *uid) {
  for (auto &p : people) if (strcmp(p.uid, uid) == 0) { Serial.print("Welcome, ");
Serial.println(p.name); return; }
  Serial.print("Unknown card "); Serial.println(uid);
}
void setup() {
  Serial.begin(9600);
  greet("049F8E7D");
  greet("11223344");
}
void loop() {}
```

SERIAL MONITOR

```
Welcome, Ravi
Unknown card 11223344
```

Q21 **Answer: (c) PlatformIO (platformio.ini)**

PlatformIO.

Q22 **Answer: (a) two 22 pF capacitors**

22 pF load capacitors.

Q23 **Answer: (d) to stop the pump if the tank is empty or the sensor fails**

A **safety limit** for failures.

Q24 **Answer: (b) hardware / bill of materials**

The **BOM**.

Q25 **Answer: Solution program**

Both tasks share loop() without blocking: the essence of responsive Arduino code. (The scripted "T" arrives at start-up, so the uptime printed is 0 s.)

```
unsigned long last = 0; int led = LOW;
void setup() { Serial.begin(9600); pinMode(13, OUTPUT); }
void loop() {
  if (millis() - last >= 500) { last = millis(); led = !led; digitalWrite(13, led); }
  if (Serial.available()) {
    char c = Serial.read();
    if (c == 'T') { Serial.print("uptime "); Serial.print(millis() / 1000); Serial.println(" s"); }
  }
}
```

SERIAL MONITOR

```
uptime 0 s
```

Specify, build in stages, test each stage, and document everything (schematic, README with BOM and versions, demo video). Compete in hackathons to build a portfolio, and continue with Raspberry Pi, robotics and ROS, and embedded C with an RTOS. You can now take an idea from a breadboard to a connected, battery-powered, enclosed product.